

考虑物性变化与径向收缩的药材热湿耦合模型及数值验证

摘要

药材热风烘干同时涉及温度传递、水分迁移及尺寸变化，表面状态难以直接代表内部干燥程度。针对圆柱形药材的四个问题，本文基于题给环境观测和物性关系，建立一维径向有效热湿耦合模型，并以全域最大干基含水率确定烘干结束时间。模型区分有效显热容量与守恒干物质密度，将气固含水率映射、观测窗后环境平台和同比径向收缩作为明确闭合；在环形控制体上采用 Kirchhoff 浓度势处理非线性扩散，配合解析稀疏 Jacobian 与隐式 BDF 求解。

针对问题一，采用常热物性与含水率依赖扩散系数建立预热模型，计算 0—1800 s 内温度和水分径向分布。1800 s 时中心、表面温度分别为 33.5753℃ 和 36.7856℃，干基含水率分别显示 2.5500 和 1.5102 kg/kg，反映表层先响应及内部水分迁移滞后。温度场另与包含时变边界卷积的圆柱解析级数对照，验证简化条件下的数值实现。

针对问题二，从初始状态统一采用附录 3 的变物性，联立计算温度对扩散及水分对热物性的反馈。附件观测区间内进行分段线性插值，4 h 后按 50℃ 与等效平衡含水率 0.05 延拓。3 h 时中心、表面温度为 49.8495℃ 和 49.9664℃，而含水率仍为 1.7662 和 1.0081 kg/kg，说明升温接近完成早于全域脱水完成。

针对问题三，复用全过程轨迹，以全域最大含水率下降穿越 0.15 kg/kg 定位临界根，并区别临界等号与严格可行的报告时刻，得到烘干时长 57.4724 h。针对问题四，在干物质守恒下取材料坐标，将给定半径变化转化为固定区间上的时变几何因子；采用整组附录 4 物性，得到时长 51.0906 h、结束半径约 1.2000 cm，输出中将域外位置与真实表面分别处理。

针对问题四，在干物质守恒条件下引入材料坐标变换，将随半径收缩的移动区域映射到固定区间，并依据半径观测和第四问物性关系更新几何因子、扩散系数及边界通量。计算得到烘干时间为 51.0906 h，结束时半径约为 1.2000 cm。该时间较问题三缩短 6.3818 h，体现径向收缩与物性变化的综合影响；同物性对照进一步表明，径向收缩通过缩短水分迁移距离加快干燥。输出时区分药材外部位置与实际表面，保证收缩过程中的空间对应关系。

通过圆柱解析级数对照、网格加密、时间积分设置检验以及独立 MATLAB 程序比较，验证了模型在所检验范围内的数值一致性。敏感性分析表明，烘干时间对水分扩散参数和表面传质边界较为敏感，所求时长适用于题给物性、环境延拓及收缩假设。

关键词：热湿耦合；有限体积法；Kirchhoff 变换；径向收缩；干燥时间

一、问题重述

1.1 问题背景

中药材干燥是产地初加工的重要环节，干燥方式及工艺参数会影响产品品质、能耗和生产效率。热风干燥是常用方法之一，相关技术与装备的发展现状已有系统综述[1]。数值模拟可用于分析药材干燥过程中的温度、水分分布及工艺条件的影响，为实验设计和参数选择提供参考[2]。本题给定的烘干过程分为预热平衡与恒温干燥两个阶段，本文据此研究圆柱形药材的径向热湿传递及烘干时间。

1.2 问题提出

基于上述背景，要求建立数学模型解决以下问题：

本文以圆柱形中药材的热风烘干过程为研究对象，该药材长为 25cm、半径为 2cm，初始温度为 28℃，初始水分浓度（即干基含水率）为 2.55kg/kg。

问题一：根据烘房的温度以及水分浓度变化情况，建立预热平衡阶段药材温度和水分浓度变化规律的数学模型，据此计算出 1800s 内每隔 1s、到药材中心距离每隔 0.1cm 的药材温度以及水分浓度。

问题二：将研究范围扩展至整个烘干过程，统一采用题给附录 3 的物性关系，建立药材温度与干基含水率的时空变化模型，计算每隔 1 s、径向每隔 0.1 cm 的温度和含水率，并按题定时刻与半径给出结果表。

问题三：在问题二的基础上反求问题，约束烘干要求药材各处水分浓度均低于 0.15kg/kg。要求依据已建立的水分浓度模型，模拟药材内部水分随时间的变化，计算出药材内部水分浓度每隔 60s、到药材中心距离每隔 0.1cm 的药材水分浓度以及烘干结束时间。

问题四：在实际烘干过程中，药材由于水分流失引起的尺寸变化会影响到烘干的最终效果，需要建立含尺寸变化的温度和水分浓度模型，重新确定烘干时长，据此计算出药材内部水分浓度每隔 60s 时间、0.1cm 距离的水分浓度以及烘干结束时间。

二、问题分析

2.1 问题一分析

已知圆柱形药材的几何尺寸、均匀初始状态、烘房温湿度序列及附录 2 参数，问题一要求得到 1800 s 内的径向温度和含水率分布。由傅里叶导热定律与 Fick 扩散定律建立轴对称控制方程，并给定中心零通量和表面对流边界条件。本问热物性为常数，温度方程可独立求解，并可构造圆柱 Bessel 级数基准进行检验；水分扩散系数 $D_1(C)$ 仍随含水率变化，采用守恒型有限体积离散和隐式时间积分求解。问题一的扩散系数不含温度项；问题二至四含 Arrhenius 温度因子的经验式统一以开尔文温度代入。

2.2 问题二分析

问题二区别于上一问，将时间拉长至整个烘干过程，已知烘干过程持续数天，预热平衡与恒温干燥两个阶段的参数不同，附录给出了密度、比热容、导热系数和扩散系数随含水率或温度变化的经验关系，要求得到整个烘干过程的温度与水分浓度时空分布。与问题一相比，本问物性同时依赖含水率与温度，温度方程和水分方程不再解耦，必须联立求解。由于两问采用的物性体系不同，本问从初始时刻重新开始计算，不以问题一的末状态作为初值。环境驱动方面，前一段时间按附件数据分段线性插值，之后取平台温湿度延拓，该延拓作为假设在论文中明确说明。求解方法上仍采用守恒型有限体积离散，时间推进需处理方程刚性与非线性耦合，因此配了解析 Jacobian 并采用隐式 BDF 格式。非线性系数不能移出散度，热容量项不能改写成特定微分形式。

2.3 问题三分析

问题三沿用问题二的变物性耦合模型及同一条数值轨迹，确定药材各处干基含水率均低于 0.15 kg/kg 所需的时间。以全域最大含水率构造阈值函数，通过连续数值解定位其下降穿越阈值的临界时刻，再在指定时间精度下选取并核验满足严格不等式的烘干时长。该判据同时检查全部径向位置，不预先指定中心或表面作为唯一控制点。

2.4 问题四分析

问题四需要处理半径随时间减小引起的移动边界。半径由附件 2 线性插值得到，热容量、导热系数和扩散系数采用附录 4 的经验关系，并随当前含水率与温度更新。

在干物质守恒及径向同比收缩的假设下，引入材料坐标 $x = r/R(t)$ ，将随时间变化的药材区域映射到固定区间 $[0,1]$ 。从组分守恒方程出发进行坐标变换，骨架运动产生的平流项与坐标变化项相消，收缩影响通过时变几何系数及边界条件体现。在此基础上采用有限体积法离散求解。为考察收缩本身的影响，保持附录 4 物性、初始条件、环境条件及数值设置一致，分别计算固定半径与给定半径变化两种情形，比较其含水率分布和烘干时长。问题三与问题四的结果之差则反映物性变化和径向收缩的综合影响。

三、模型假设

假设药材长径比较大，忽略药材两个端面的传热与水分散失，将三维传热传质过程简化为一维径向模型。

假设烘房内热风充分混合，同一时刻药材表面外侧温湿度均匀，对流换热系数、对流传质系数取恒定值。

假设初始时刻药材内部温度与干基含水率均匀分布，不存在初始梯度。

假设药材干物质初始分布均匀，烘干过程中不存在干物质的生成、损失及相对于骨架的迁移。问题一至三忽略药材尺寸变化；问题四假设轴向长度保持不变，干物质骨架仅发生径向同比收缩，即各材料点到轴线的距离均按 $R(t)/R_0$ 的比例变化。

假设不考虑药材内部生化反应，仅研究热传导与水分扩散物理过程。

四、符号说明

符号	说明	单位
t	时间	s
r	到药材中心的径向距离	m
$x=r/R(t)$	材料归一化径向坐标	1
T	药材温度	K
C	干基含水率= m 水/ m 干物质	kg/kg
$w=C/(1+C)$	湿基质量分数	1
$R(t)$	药材当前半径	m
ρ_{eff}	有效密度（用于热容量）	kg/m ³
ρ_d	守恒干物质密度	kg/m ³
c_p	比热容	J/(kg·K)
k	导热系数	W/(m·K)
D	有效水分扩散系数	m ² /s

符号	说明	单位
h	对流换热系数	$\text{W}/(\text{m}^2 \cdot \text{K})$
β	对流传质系数	m/s
$B=\rho_{\text{eff}} \cdot c_p$	有效体积显热容量	$\text{J}/(\text{m}^3 \cdot \text{K})$
q_r	径向导热热流密度（向外为正）	W/m^2
j_w	径向水分质量通量（向外为正）	$\text{kg}/(\text{m}^2 \cdot \text{s})$
$\ell(t)$	每单位初始干物质的累计失水	kg/kg
C_{eq}	环境对应的等效平衡含水率	kg/kg
T_{∞}	烘房温度	K
t_c	全域含水率阈值的临界时刻	s

五、模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 建模思路

本文采用圆柱径向热湿传递框架研究药材内部状态。计算几何按题面取长度 0.25 m、初始半径 0.02 m；温度方程以开尔文为单位，结果表换算为摄氏度。含水率 C 表示单位药材干物质所含的水质量，其与湿基质量分数 w 的关系为

$$C = \frac{m_w}{m_d}, w = \frac{C}{1+C} \quad (1)$$

干基含水率表示水质量与干物质质量的比值。为保持水量计算的量纲一致，引入单位当前体积内的干物质质量 ρ_d ，则水质量密度为 $\rho_d C$ 。问题一至三采用固定且空间均匀的干物质骨架；问题四允许骨架随材料收缩运动，并假设干物质相对于骨架的通量为零。下文据此建立移动区域上的组分守恒方程。

热学和质量方程需要分别说明密度的用途。题给 ρ 与 c_p 的乘积记为 B ，用来描述单位体积温升的有效显热需求； ρ_d 则用于计算随材料守恒的干物质和水质量。若把题给 ρ 同时解释为真实湿密度，再强行令 $\rho_d = \rho/(1+C)$ ，会对固定长度和给定半径施加额外约束。附录 A.2 表明，该约束与所给后期收缩数据不能同时满足。因此本文采用有效容量与独立干骨架的闭合，使两套方程的定义一致；这一处理也限定了结果的物理解释，不能视

为对真实组分密度的验证。

平衡含水率与材料性质、温度及环境湿度有关,通常需由吸附或解吸等温线描述[3]。附件 1 的烘房水分指标仅标注 kg/kg, 未给出其与药材干基含水率之间的转换关系。记该观测数值为 Y_∞ , 基线取 $C_{eq} = Y_\infty$, 作为表面传质的等效平衡值。这一数值映射是本文的边界假设; 文献中的一般平衡关系不构成该映射的标定依据。若进一步将 Y_∞ 解释为空气湿度比, 其参考质量也需另作说明。

5.1.2 变量定义与公式推导

在半径 r 至 $r+dr$ 的圆柱薄壳内, 对向外为正的热流和水分通量采用 *Fourier* 与 *Fick* 本构。薄壳内外表面积不同, 积分守恒后须保留径向几何因子。得到

$$B(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} (rk(C) \frac{\partial T}{\partial r}), 0 < r < R_0 \quad (2)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} (rD(T, C) \frac{\partial C}{\partial r}) \quad (3)$$

式(2)表示有效显热的局部变化由净导热供给; 式(3)由水质量守恒结合 Fick 扩散定律, 在干骨架密度均匀且固定时得到。导热系数和扩散系数保留在散度内, 以反映状态变化引起的通量梯度。相应常系数圆柱问题的 Bessel 函数级数解可参考 Crank 的推导[4], 用于检验简化条件下的数值实现。

中心由于轴对称不存在跨轴通量, 表面与烘房发生对流换热和有效传质。初始条件与边界条件分别为

$$T(r, 0) = 301.15 \text{ K}, C(r, 0) = 2.55, T_r(0, t) = C_r(0, t) = 0 \quad (4)$$

$$-kT_r(R_0, t) = h(T_s - T_\infty), -DC_r(R_0, t) = \beta(C_s - C_{eq}) \quad (5)$$

这里 T_s 与 C_s 是药材真实表面的状态, $h=25 \text{ W}/(\text{m}^2 \cdot \text{K})$, $\beta=8 \times 10^{-7} \text{ m/s}$ 。初始场与表面边界在 $t=0$ 处可能不完全相容, 意味着启动阶段形成边界层, 不应人为修改初始温湿度来消除该现象。采用较小的早期步长处理快速响应, 并在后续检查非负性与通量方向。

问题一采用附录 2: $\rho=820 \text{ kg/m}^3$, $c_p=2600 \text{ J}/(\text{kg} \cdot \text{K})$, $k=0.36 \text{ W}/(\text{m} \cdot \text{K})$, 水分扩散系数为

$$D_1(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right) \text{ m}^2/\text{s} \quad (6)$$

因此本问温度方程是可独立求解的线性热方程, 水方程仍因 $D_1(C)$ 而非线性。式(6)没有温度项; *Arrhenius* 温度因子只在问题二至四的物性关系中出现。不能把“常热物性”扩大解释为“全部物性均为常数”, 也不应给问题一另外加入温度活化项。

5.1.3 求解算法

设归一化坐标 $x = r/R_0$, 在 $0 \leq x \leq 1$ 上划分 N 个等长区间, 节点 $x_i = i/N$ 包含

中心和表面。每个节点对应相邻半节点之间的环形控制体，中心和表面取截断控制体，记其无量纲体积权重为 w_i 。有限体积法先计算相邻控制体共用的面通量，再由净通量建立收支；内部面在全域求和时成对相消[5]。该处理可直接落实轴心零通量条件。

对于随含水率显著变化的扩散系数，参照浓度相关扩散方程的积分变换方法[4]，构造 Kirchhoff 浓度势。将题给系数写为 $D = D_0 A(T) \exp(-a/C)$ ，取

$$\Phi_a(C) = C \exp(-a/C) + a \text{Ei}(-a/C), \Phi'_a(C) = \exp(-a/C) \quad (7)$$

相邻节点之间的水分通量通过势差计算，温度因子在该面单独取值。不能把 $A(T)\Phi(C)$ 整体作差，否则会产生原 Fick 闭合中不存在的温度梯度通量。浓度相近时采用局部展开避免两项接近的势值相减损失有效数字。式(7)使浓度方向的积分得到解析处理，但温度面值、有限网格与时间推进仍有离散误差；它不是对整个偏微分方程的精确求解。

细网格缩短了局部扩散时间尺度，显式推进受到较严格的步长限制。本文将温度与含水率交错排列，利用相邻节点耦合构造解析稀疏雅可比矩阵，并采用适于刚性方程组的 BDF 方法进行时间积分[6]。实际使用 SciPy 的 BDF 实现及其稀疏雅可比接口[7]。状态末项累计失水，用于核对全域水量收支。题定输出时刻由同一连续数值解查询；积分器的内部步长与 1 s 或 60 s 的输出间隔分别设置。

第一步：读取附件观测，统一长度和温度单位，核对初始场与附录 2 参数。第二步：生成环形控制体、初值和相邻面通量。第三步：采用隐式变步长推进，检查求解成功、有限数值及物性正性。第四步：从同一数值轨迹查询指定时刻及物理半径。第五步：导出结果表，并检查累计失水与全域水量变化是否一致。

问题一采用 $N=3200$ ，积分至 1800 s；相对容差为 10^{-10} ，温度与水分绝对容差分别为 10^{-10} K 和 10^{-12} kg/kg。严格容差用于控制时间积分，不能解释为最终物理预测具备相同精度。网格选择依据及时间设置对照见附录 C。

5.1.4 结果分析

表 1、表 2 列出题目要求的 7 个时刻和 5 个半径位置；完整逐秒、0.1 cm 结果由同一轨迹生成。温度和浓度分别保留四位小数，表头中的半径以 cm 表示。

表 1 问题一的径向温度 (° C)

时间/s	$r=0$ cm	$r=0.5$ cm	$r=1$ cm	$r=1.5$ cm	$r=2$ cm
100.0000	28.0001	28.0003	28.0040	28.0327	28.1801
300.0000	28.0408	28.0635	28.1514	28.3681	28.8489
600.0000	28.4533	28.5360	28.8039	29.3159	30.1652
900.0000	29.3243	29.4583	29.8755	30.6161	31.7304

时间/s	$r=0$ cm	$r=0.5$ cm	$r=1$ cm	$r=1.5$ cm	$r=2$ cm
1200.0000	30.5427	30.7098	31.2223	32.1126	33.4276
1500.0000	31.9957	32.1867	32.7660	33.7463	35.1203
1800.0000	33.5753	33.7720	34.3642	35.3621	36.7856

表 2 问题一的径向干基含水率 (kg/kg)

时间/s	$r=0$ cm	$r=0.5$ cm	$r=1$ cm	$r=1.5$ cm	$r=2$ cm
100.0000	2.5500	2.5500	2.5500	2.5500	2.2470
300.0000	2.5500	2.5500	2.5500	2.5492	2.0508
600.0000	2.5500	2.5500	2.5500	2.5353	1.8770
900.0000	2.5500	2.5500	2.5497	2.5045	1.7547
1200.0000	2.5500	2.5500	2.5482	2.4646	1.6586
1500.0000	2.5500	2.5499	2.5445	2.4206	1.5787
1800.0000	2.5500	2.5497	2.5383	2.3755	1.5102

至 1800 s, 中心温度为 33.5753°C, 表面温度为 36.7856°C, 差约 3.2102°C。表层直接受到热风影响, 温度上升先于中心, 随后导热使内部逐渐升温。含水率的空间响应更不均匀: 表面已下降到 1.5102 kg/kg, 中心显示 2.5500 kg/kg。原精度中心值约 2.5499924, 说明其发生了很小的变化; 四位表格中的相同数字不意味着水分完全静止。

表层含水率降低后, 式(6)中的 D 随之减小, 形成较强的干燥阻力。因而即使表面与外界存在传质驱动力, 内部水分也不会立刻均匀释放。只用一个全体平均含水率描述此阶段, 会掩盖径向差异, 无法回答题目要求的各位置浓度。

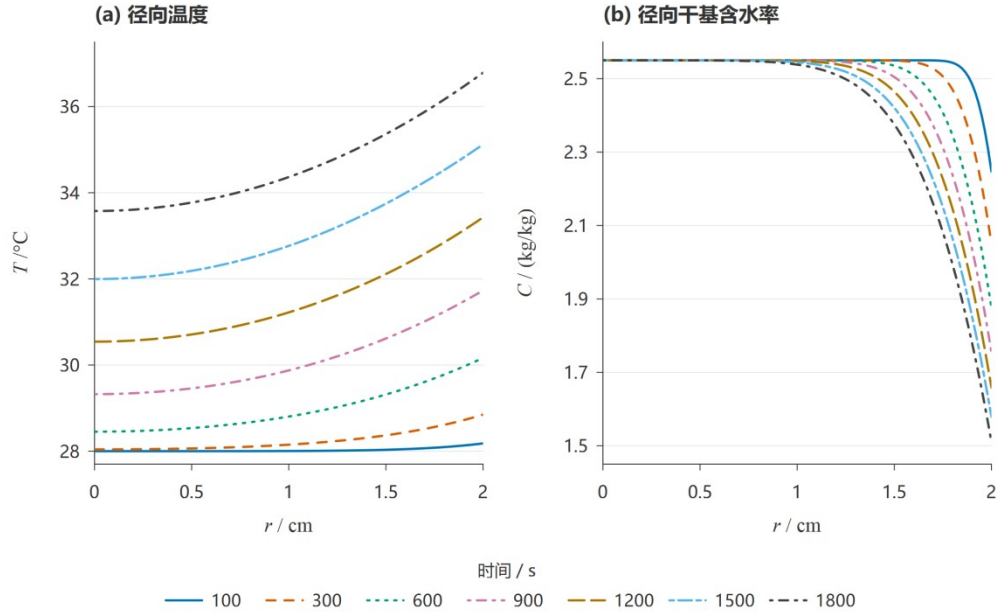


图1 问题一径向温度含水率分布

5.1.5 模型检验或灵敏度分析

在本问常热物性条件下，将变温边界写入圆柱 *Robin* 特征函数展开及时间卷积，得到热场解析基准。它与数值结果在 N3200、1801 个整数秒和 21 个半径点上最大相差约 1.80×10^{-6} K。非线性水方程另用守恒检查与面格式对照核验，不由热场的一致性替代。基准公式与取样范围在附录 C 交代，正文不逐项展开误差矩阵。

5.2 问题二模型的建立与求解

5.2.1 建模思路

问题二沿用式(2)—(5)的几何与边界框架，重新从均匀初始场开始。按附录 3 在整个求解区间使用同一套变物性，不在 1800 s 将附录 2 解拼接到附录 3，也不把问题一末状态用作新的初始条件。这样才能区分两问的物性设定，避免在中途人为改变材料参数。

附录 3 使两个状态之间出现双向反馈： T 进入扩散系数， C 同时进入热容量和导热系数。在一次时间推进中，新的温度会影响水通量，新的含水率又改变温升速度和热通量。因此即使外部环境完全相同，问题二也不能沿用问题一的温度轨迹。联立算法在同一时刻更新 T 、 C 与全部物性，使耦合关系进入隐式残量；先独立计算全程热场再求水分，会改变这里采用的模型。

5.2.2 变量定义与公式推导

令 $w=C/(1+C)$ ，附录 3 规定

$$\rho_{\text{eff}} = 650 + 128C, c_p = 1450 + 2736w, k = 0.21 + 0.38w \quad (8)$$

$$D_3(T, C) = 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C} - \frac{3850}{T}\right) \quad (9)$$

式(8)的密度、比热容、导热系数依次采用 kg/m^3 、 $\text{J}/(\text{kg}\cdot\text{K})$ 、 $\text{W}/(\text{m}\cdot\text{K})$ ，式(9)的单位为 m^2/s ， T 必须是绝对温度。指数中的 $0.45/C$ 和 $3850/T$ 均为无量纲经验组合，系数按题目给定取值，不从当前附件重新拟合。后续采用 $B(C)=\rho_{\text{eff}}(C)c_p(C)$ ，仍是有效显热解释。

附件 1 包含 0—4 h 共 241 组观测，观测间隔 60 s。为了向连续时间求解器提供边界，观测区间内作分段线性插值。 $t > 4$ h 后取 $T_\infty=50^\circ\text{C}$ 、 $C_{\text{eq}}=0.05$ 。这一延拓利用题目恒温干燥的工况描述及末段平台特征，但平台尾段并非持续数天的实测记录。相应的分段表达为

$$(T_\infty, C_{\text{eq}})(t) = \begin{cases} (I_T(t), I_Y(t)), & 0 \leq t \leq 14400 \text{ s}, \\ (323.15, 0.05), & t > 14400 \text{ s}. \end{cases} \quad (10)$$

其中两种插值算子直接连接相邻观测点，不额外平滑或外推趋势。超过观测窗后的输出是指定工况下的条件预测。附录 3 未重给表面换热和传质系数，本文明确沿用 h 和 β ；这也属于全过程边界设置，而不是对不同风速或设备的普遍结论。

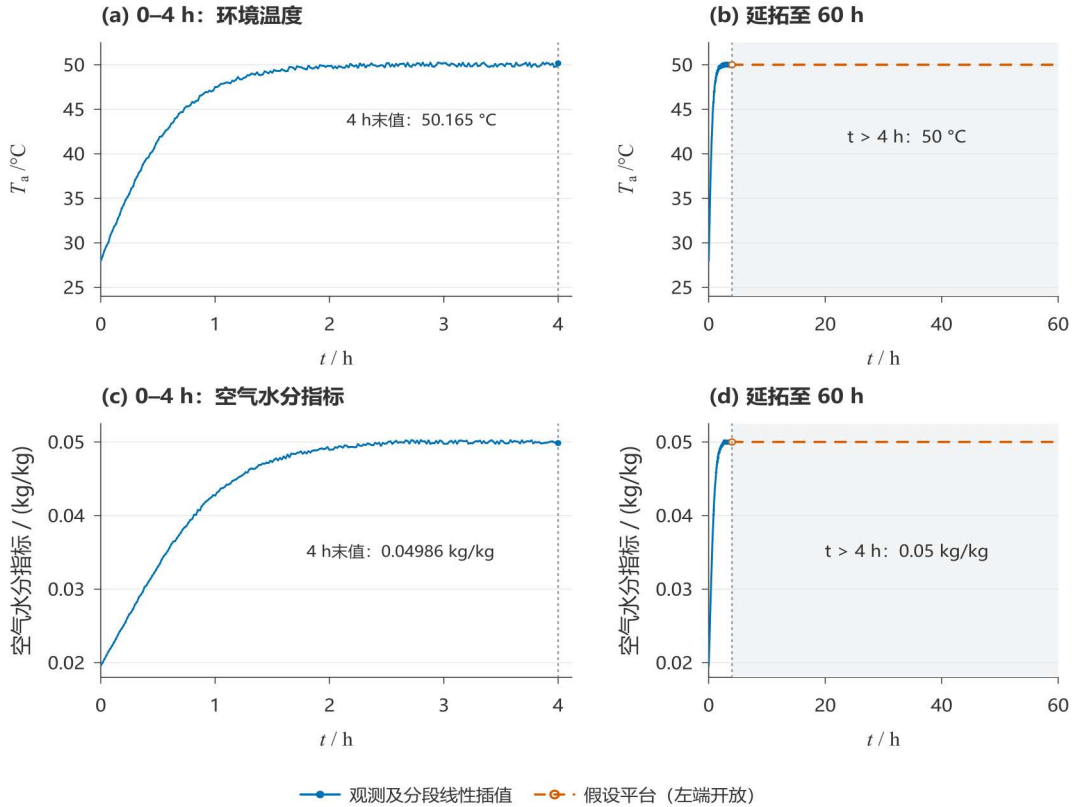


图 2 烘房环境边界条件与延拓

5.2.3 求解算法

沿用问题一的有限体积网格，同时更新 $B(C)$ 、 $k(C)$ 与 $D(T,C)$ 。温度面通量采用导热系数调和平均，水分面通量用 $a=0.45$ 的 *Kirchhoff* 势及面温度因子。解析 *Jacobian* 保留 T 对水通量、 C 对热容量和热通量的导数。中心无通量与表面换热传质直接写入控制体余额，从而保证两类边界与内部离散采用一致的符号。

第一步：从 $t=0$ 读取初始状态并选定附录 3。第二步：每次右端函数求值时查询当前环境，更新各节点物性及面通量。第三步：用隐式 BDF 与稀疏 *Jacobian* 联立推进，早期限制较小步长，后期根据误差控制自动调整。第四步：保存同一条全过程轨迹，查询题目要求的前 3 h 径向表和逐秒全过程结果。第五步：将该轨迹交给问题三进行阈值判断，避免两次独立求解造成终止时间与结果表不一致。

网格采用 N3200，主要容差与问题一一致。模型最长允许计算范围只是程序保护上限；实际输出区间由问题三达标事件和其后校核点确定，不以任意预设天数冒充烘干完成时刻。

5.2.4 结果分析

表 3、表 4 给出题目规定的 6 个时刻。两表由全过程解在同一组时间和半径上查询，因此可以直接比较热响应与水分迁移的时间尺度。

表 3 问题二的径向温度 (°C)

时间/h	$r=0$ cm	$r=0.5$ cm	$r=1$ cm	$r=1.5$ cm	$r=2$ cm
0.5000	32.1892	32.3821	32.9659	33.9608	35.4130
1.0000	40.3816	40.5536	41.0605	41.8782	42.9976
1.5000	45.8468	45.9348	46.1930	46.6051	47.1401
2.0000	48.4502	48.4881	48.5984	48.7736	49.0033
2.5000	49.4670	49.4792	49.5136	49.5653	49.6609
3.0000	49.8495	49.8553	49.8746	49.9101	49.9664

表 4 问题二的径向干基含水率 (kg/kg)

时间/h	$r=0$ cm	$r=0.5$ cm	$r=1$ cm	$r=1.5$ cm	$r=2$ cm
0.5000	2.5499	2.5489	2.5256	2.3257	1.6486
1.0000	2.5257	2.4948	2.3578	2.0230	1.4711
1.5000	2.3861	2.3256	2.1344	1.8020	1.3476
2.0000	2.1709	2.1084	1.9236	1.6259	1.2311
2.5000	1.9566	1.9006	1.7360	1.4720	1.1166
3.0000	1.7662	1.7165	1.5702	1.3333	1.0081

3 h 时中心和表面温度分别为 49.8495°C、49.9664°C，温差已缩小到约 0.1169°C，而中心与表面的含水率分别为 1.7662、1.0081 kg/kg，仍存在明显差异。升温接近完成不代表脱水已经完成；单独根据药材温度接近烘房温度来决定出料，会忽视中心水分的持续迁移。

水分扩散对温度的响应与含水率的响应方向不同。在相同 C 下，升温使式(9)的温度因子增大，有利于扩散；随着药材逐渐变干， $\exp(-0.45/C)$ 减小，使后期扩散变慢。因此全过程通常表现为前期升温推动水分迁移、后期低含水率区域限制传递的组合效应。该解释来自物性公式及场分布，不把时间曲线的形状直接归因为某个未经观测的组织变化。

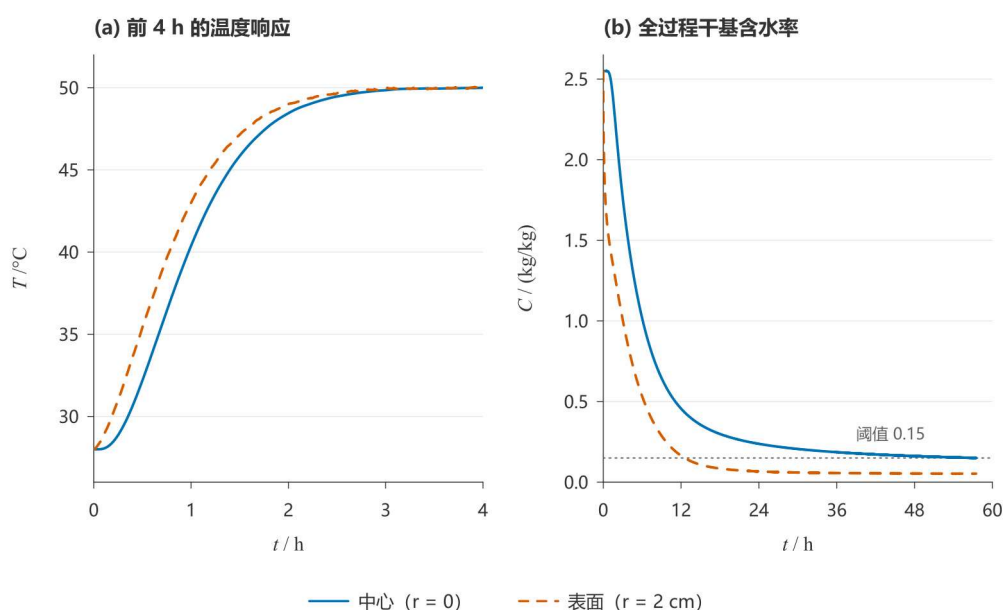


图 3 问题二中心表面温湿响应曲线

5.2.5 模型检验或灵敏度分析

空间加密、收紧时间设置以及不同积分器的对照表明，本组数值解在所检验的采样范围内稳定。表 3 中 1.5 h 表面温度接近四位小数的舍入边界，基线与更严格时间设置可能给出不同末位；表中统一采用基线计算值 47.1401°C。该差异反映计算末位对时间设置的敏感性，详细比较见附录 C。

5.3 问题三模型的建立与求解

5.3.1 建模思路

问题三将给定时刻的温湿状态计算转化为烘干时间求解。沿用问题二的完整数值轨迹，以全域最大含水率构造阈值事件，定位临界时刻，并据此确定满足严格含水率约束的烘干时长。

5.3.2 变量定义与公式推导

问题三要求确定药材各处干基含水率均低于 0.15 kg/kg 所需的烘干时间。沿用问题二得到的含水率场 $C(r, t)$ ，定义时刻 t 的全域最大含水率：

$$M(t) = \max_{0 \leq r \leq R_0} C(r, t) \quad (11)$$

其中， $M(t)$ 的单位仍为 kg/kg。药材各处均达标，等价于全域最大含水率满足

$$M(t) < 0.15 \quad (12)$$

为定位阈值，定义辅助函数

$$g(t) = M(t) - 0.15 \quad (13)$$

当 $g(t) > 0$ 时尚有位置未达标； $g(t) = 0$ 对应临界边界； $g(t) < 0$ 表示全域达标。数值计算对全部径向节点取最大值。

记 $g(t)$ 首次下降穿越零点的临界时刻为 t_c 。在临界附近连续下降的条件下，根时刻对应等号。取时间间隔 $\Delta t = 0.36 \text{ s} = 10^{-4} \text{ h}$ ，在该离散时间网格上选择不早于临界时刻且满足严格阈值的首个时刻：

$$t_r = \min\{k\Delta t: k \in \mathbb{N}_0, k\Delta t \geq t_c, M(k\Delta t) < 0.15\} \quad (14)$$

式中， $\mathbb{N}_0$ 为非负整数集合； t_c 、 t_r 与 Δt 统一以秒计算。达标判定采用未舍入的数值解，最终将 t_r 换算为小时表示。候选点的生成和逐格核验见附录 C.4。

5.3.3 求解算法

第一步：在问题二的积分过程中监测 $g(t)$ 下降穿零。第二步：利用连续数值解在

包围区间内定位临界根，并核对最大含水率与阈值的一致性。第三步：按式(14)选取并核验烘干时长。第四步：从同一轨迹查询每隔 60 s、径向每隔 0.1 cm 的含水率。另在 $N = 800$ 的验证轨迹上采用独立二分法复核事件定位；正式结果由 $N = 3200$ 的计算轨迹给出。

求解阈值方程得到临界时刻 $t_c \approx 57.472302$ h。按式(14)选取并核验未舍入含水率，得到烘干时长为 57.4724 h。

5.3.4 结果分析

表 5 列出每隔 6 h 及最终烘干时刻的径向含水率，用于展示干燥过程中的空间差异；题定 60 s 间隔的完整结果另行导出。

表 5 问题三的径向干基含水率 (kg/kg)

时间/h	$r=0$ cm	$r=0.5$ cm	$r=1$ cm	$r=1.5$ cm	$r=2$ cm
6.0000	1.0170	0.9881	0.9015	0.7550	0.5328
12.0000	0.4561	0.4432	0.4031	0.3297	0.1642
18.0000	0.2988	0.2915	0.2687	0.2253	0.0845
24.0000	0.2378	0.2327	0.2167	0.1855	0.0665
30.0000	0.2058	0.2018	0.1892	0.1641	0.0598
36.0000	0.1859	0.1825	0.1718	0.1504	0.0566
42.0000	0.1721	0.1691	0.1597	0.1407	0.0549
48.0000	0.1618	0.1592	0.1507	0.1334	0.0537
54.0000	0.1539	0.1514	0.1436	0.1277	0.0530
57.4724	0.1500	0.1477	0.1402	0.1249	0.0527

注：含水率保留四位小数。57.4724 h 时的全域最大含水率约为 0.149999897 kg/kg，表中显示为 0.1500；达标判定采用未舍入的计算值。

12 h 时表面已接近 0.15，但中心仍为 0.4561 kg/kg；至 48 h 中心仍为 0.1618 kg/kg。由此可见，表面达标不能代表全域达标。后期中心由 0.1859 下降到 0.1500 经历较长时间，反映低含水率条件下扩散系数降低的影响。结束时中心与表面仍有浓度差，达到产品阈值并不要求内部先达到完全均匀的平衡状态。

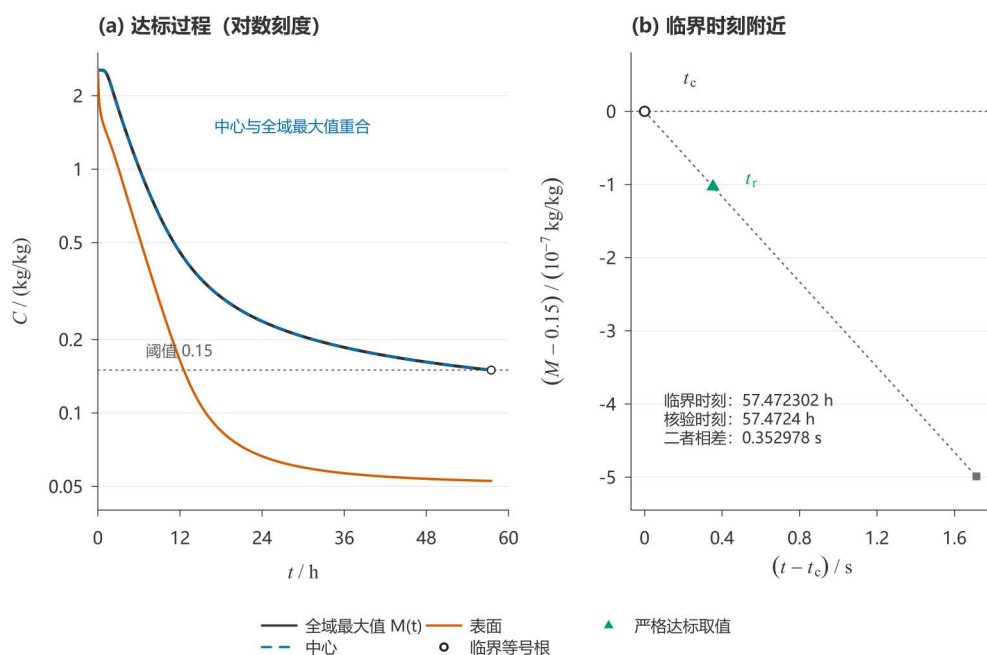


图4 问题三烘干阈值穿越临界时刻

5.3.5 模型检验或灵敏度分析

约 93.04% 的报告时长位于附件 1 的 4 h 观测窗之后，因此 57.4724 h 应理解为规定平台温湿条件持续存在时的模型预测。数值根的复核能检查“给定轨迹上是否找准阈值”，不能消除边界延拓和气固映射的物理不确定性。若后续更换平台温湿度、 β 或平衡映射，必须重新积分并重新确定事件，不能只按经验比例改动最终时长。

5.4 问题四模型的建立与求解

5.4.1 建模思路

问题四同时改变几何与物性。从 $t=0$ 起整组采用附录 4，并由附件 2 查询 $R(t)$ 。附件 2 在 0—72 h 提供 145 个半径观测，间隔 0.5 h，按分段线性插值连接。基线假定轴向长度保持不变，内部材料发生与半径一致的同比例径向收缩；材料速度为 $u(r,t)=rR'(t)/R(t)$ 。该速度场与表面运动相容，是从半径数据推断内部运动的明确假设。

如果只在固定域模型中将 R_0 换成 $R(t)$ ，而不说明水质量和材料速度的含义，就可能重复加入浓缩项或漏掉物质运动。本文先从组分守恒建立物质导数，再进行材料坐标变换，保证控制方程的来源清楚。

5.4.2 变量定义与公式推导

在没有干物质生成或相对迁移的条件下，干质量守恒及水质量守恒为

$$\partial_t \rho_d + \nabla \cdot (\rho_d \mathbf{u}) = 0, \partial_t (\rho_d C) + \nabla \cdot (\rho_d C \mathbf{u} + \mathbf{j}_w) = 0 \quad (15)$$

将水质量方程减去干质量方程的 C 倍, 得到 $\rho_d D_t C = -\nabla \cdot \mathbf{j}_w$ 。取 $\mathbf{j}_w = -\rho_d D \nabla C$, 并利用干骨架密度的空间均匀性消去 ρ_d 。令 $x = r/R(t)$, 定义 $\hat{C}(x, t) = C(R(t)x, t)$ 、 $\hat{T}(x, t) = T(R(t)x, t)$ 。链式法则中的坐标变化项与骨架平流项相消, 得到

$$\frac{\partial \hat{C}}{\partial t} = \frac{1}{R(t)^2 x} \frac{\partial}{\partial x} (x D \frac{\partial \hat{C}}{\partial x}) \quad (16)$$

$$B(\hat{C}) \frac{\partial \hat{T}}{\partial t} = \frac{1}{R(t)^2 x} \frac{\partial}{\partial x} (x k \frac{\partial \hat{T}}{\partial x}) \quad (17)$$

式(17)沿用有效显热闭合, 未显式包含潜热、湿分携焓及收缩功。材料坐标方程中不再出现额外网格平流项; 干基比值也不另加体积浓缩项。收缩引起的干骨架密度变化满足 $\rho_d(t) = \rho_{d0}[R_0/R(t)]^2$ 。坐标变换和干质量守恒推导见附录 A。

在 $x=0$ 保持零通量, 表面边界改为

$$-\frac{k}{R(t)} \hat{T}_x(1, t) = h(T_s - T_\infty), -\frac{D}{R(t)} \hat{C}_x(1, t) = \beta(C_s - C_{eq}) \quad (18)$$

边界中的 $1/R$ 来自空间导数换元, 不能遗漏。附录 4 物性为

$$\rho_{eff} = 760 + 90C, c_p = 1850 + 2150w, k = 0.12 + 0.20w, \quad (19)$$

$$D_4(T, C) = 4.2 \times 10^{-4} \exp(-\frac{0.30}{C} - \frac{3850}{T}) \quad (20)$$

在相同温度与含水率下, 两组扩散系数满足 $D_4/D_3 = 0.175 \exp(0.15/C)$ 。当 $C \approx 0.08606 \text{ kg/kg}$ 时两者相等; 高于该值时 $D_4 < D_3$, 低于该值时大小关系反转。因此, 物性变化的综合作用由全过程数值对照评估, 不能仅由扩散系数前因子判断。

5.4.3 求解算法

在材料坐标中采用 $N = 6400$ 的固定网格。每次右端函数求值时, 根据当前 t 更新 $R(t)$, 再更新式(16)—(20)中的几何因子、物性和边界通量。网格点随材料运动, 输出时转换到题目要求的物理半径 r 。

Step 1: 选定整组附录 4 参数, 读取半径序列并检查其正值与非增性。**Step 2:** 在材料坐标上生成初值、控制体和通量。**Step 3:** 按当前半径联立积分温度与含水率。**Step 4:** 监测材料域内最大 C 的阈值事件并复核严格报告时刻。**Step 5:** 按每 60 s 和 0.1 cm 生成物理距离结果, 同时单列当前真实表面及 $R(t)$ 。

$r > R(t)$ 的物理位置已不在药材内, 不能插值或填成零含水率。本文保留这些固定半径列为空, 并单列“药材表面”, 从而同时保留原距离坐标与移动边界信息。真实表面不一定恰好落在 0.1 cm 网格, 不能取最近固定列冒充。

5.4.4 结果分析

问题四的连续临界时刻约为 51.090575 h，按与问题三相同的时间网格和严格阈值判据，得到烘干时长为 51.0906 h，对应半径约为 1.2000 cm。该时刻位于附件 2 的观测范围内。表 6 与表 7 分别给出含水率和表面半径。

表 6 问题四的径向干基含水率（kg/kg；空白表示域外）

时间/h	$r=0\text{ cm}$	$r=0.5\text{ cm}$	$r=1\text{ cm}$	$r=1.5\text{ cm}$	$r=2\text{ cm}$	药材表面
6.0000	1.7196	1.5377	1.0226			0.4207
12.0000	0.7377	0.6546	0.4083			0.1670
18.0000	0.4087	0.3686	0.2397			0.0892
24.0000	0.2851	0.2611	0.1790			0.0673
30.0000	0.2263	0.2094	0.1493			0.0595
36.0000	0.1928	0.1797	0.1317			0.0560
42.0000	0.1712	0.1604	0.1200			0.0541
48.0000	0.1561	0.1468	0.1116			0.0530
51.0906	0.1500	0.1413	0.1081			0.0526

表 7 问题四各输出时刻的表面半径

时间/h	表面半径/cm
6.0000	1.3740
12.0000	1.2480
18.0000	1.2140
24.0000	1.2040
30.0000	1.2010
36.0000	1.2000
42.0000	1.2000

时间/h	表面半径/cm
48.0000	1.2000
51.0906	1.2000

6 h 时半径已收缩至 1.3740 cm，因此 1.5 cm 和 2 cm 固定位置均位于域外；此时表面含水率为 0.4207，中心仍为 1.7196 kg/kg。至最终时刻，中心和表面分别显示 0.1500 与 0.0526，仍有明显梯度。径向收缩减少了迁移距离，但不会消除由低含水率扩散变慢造成的后期阻力。

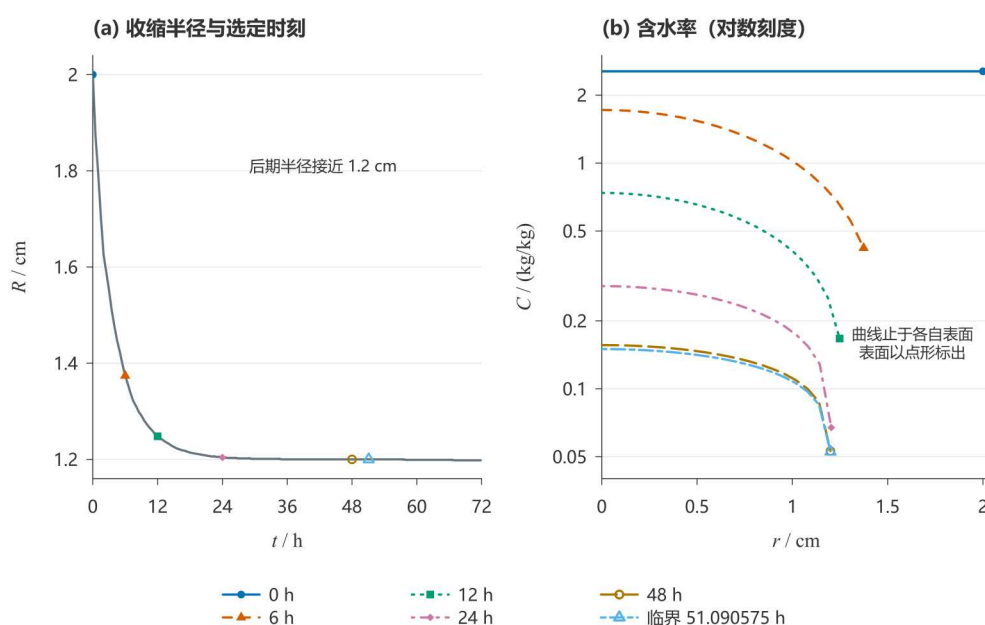


图 5 问题四径向收缩及含水率分布

5.4.5 模型检验或灵敏度分析

问题三与四的报告时长相差约 6.3818 h，但两问物性和几何同时变化，该差值不能全部归因于收缩。为分离几何影响，在相同附录 4 物性、边界和 $N200$ 网格下，分别固定半径及采用给定收缩，得到 129.8452 h 与 51.0910 h；在这个指定对照中，收缩情形时长减少约 60.65%。这里的 $N200$ 结果用于机理比较，不替代 $N6400$ 正式报告。

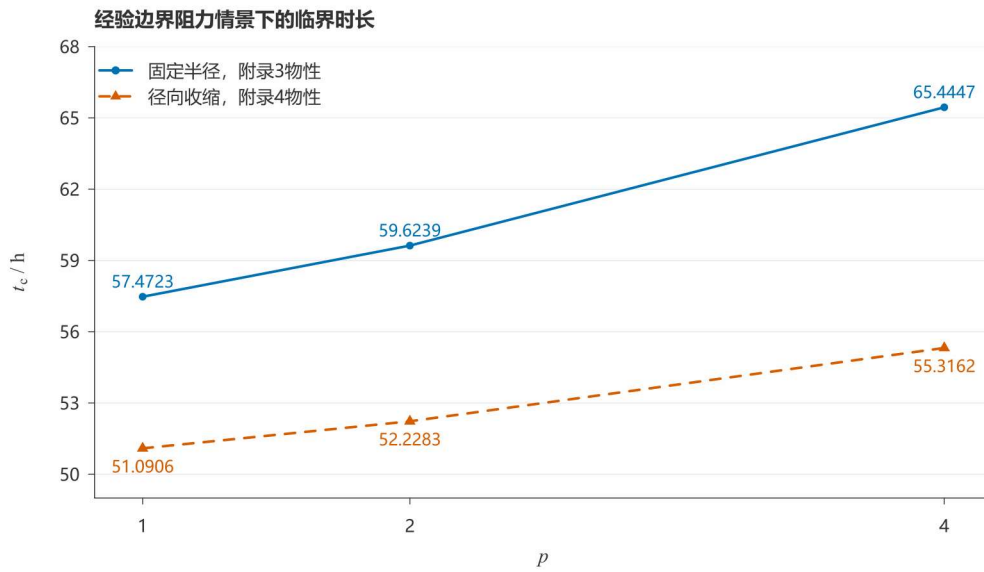
式(16)直接显示收缩为何会加快内部传递：在相同局部物性下，径向扩散项随 R^{-2} 增强；式(18)又表明表面交换与内部传递的相对比例也在变化。 R^2 时标换算与同物性定半径对照相差约 1.37%，支持迁移路径缩短的尺度解释。由于换算积分使用了仿真事件作为上限，它是机理核对而非额外的独立解析时长。几何数据覆盖达标时刻，但环境数据没有：约 92.17% 的时长仍处于式(10)的平台延拓中，两类观测范围必须分清。

5.4.6 边界阻力的补充修正

为考察基线气固映射的不确定性，在保持主答案不变的前提下，构造受水活度启发的经验阻力族。令 $C_{\text{ref}}=0.05$, $p \geq 1$ ，对表面通量增加因子

$$K_p(C_s) = \frac{1-(C_{\text{ref}}/C_s)^{1/p}}{1-C_{\text{ref}}/C_s}, j_w = \rho_d \beta K_p(C_s)(C_s - C_{\text{eq}}) \quad (21)$$

当 C_s 趋于 C_{ref} 时取连续极限 $1/p$ ；对 $C_s \geq C_{\text{ref}}$ 有 $1/p \leq K_p \leq 1$, $p=1$ 恢复基线。该构造通过指定参考点下的活动度差缩放获得启发，实际计算仍是一族经验边界阻力，尚未标定真实吸附等温线。历史 N800 情景下， $p=2$ 时问题二三的临界时长为 59.6239 h，问题四为 52.2283 h； $p=4$ 时分别为 65.4447 h 和 55.3162 h。它们表明所选边界阻力会改变预测时长，不能据此推出真实过程必大于基线，亦不能证明所有 p 下的全局单调性。



统一采用 N = 800、零潜热负荷；该经验参数族未经实测等温线标定。

图6 边界阻力参数灵敏度分析

六、模型检验与灵敏度分析

本题缺少药材内部温度与含水率实测值，因此检验重点是所建方程的数值实现及假设敏感性。解析级数、独立语言程序和不同积分器分别更换求解环节，仍共享题设参数与边界；各方法不能提供独立的物理预测准确率。表8 差值均取未舍入结果，完整计算过程与采样范围见附录C。

表 8 数值检验的对象、范围与结果

检验对象	方法及范围	主要结果	支持的结论
问题一热场	$N3200, 1801$ 整数秒 $\times 21$ 半径：有限体积与 Robin-Bessel 级数	最大温差 1.804×10^{-6} K	常热物性热方程的数值一致性
三条计算轨迹	$N40$ ；Python 与 MATLAB，100 场值及 2 事件	温差 $\leq 2.375 \times 10^{-7}$ K，水分差 $\leq 5.851 \times 10^{-10}$ kg/kg，事件差 $\leq 2.297 \times 10^{-4}$ s	共同采样点的跨实现一致性
问题三、四临界时间	$N800, N1600$ ；BDF 与 Radau	最大事件差约 3.2×10^{-6} s	同一空间离散的积分器一致性
场值分辨率	最细相邻网格；同 N 收敛容差与步长，公共有效域	最大水分差分别 $\leq 4.70 \times 10^{-5}$ 、 $\leq 9.95 \times 10^{-9}$ kg/kg	检验范围内的离散敏感性

空间检查分别采用问题一、二、三的 $N1600$ 与 $N3200$ ，以及问题四的 $N3200$ 与 $N6400$ ；时间检查固定最细网格。比较覆盖全整数秒及 21 个有效物理半径点，问题四排除域外位置，真实表面另作核对。网格差不是连续解的严格误差界，四位输出也不保证全部末位稳定。水分平均值与累计流出量的残差约为 10^{-14} kg/kg，说明共享通量记账自洽；热侧验证的是瞬时加权温度变化与边界通量相符，不能扩大为完整热力学能量守恒。

达标时间由全域最大含水率事件确定，并在 $N = 800$ 的验证轨迹上用独立二分法复核事件定位。最终烘干时刻按式(14)确定，其可行性由未舍入的最大含水率核验；两种定位方法的比较见附录 C.4。

在 $N200$ 单参数情景中，扩散前因子倍率由 0.7 增至 1.4 时，问题三时长由 79.4187 h 降至 43.0796 h，问题四由 70.4059 h 降至 38.3160 h；导热系数在 $\pm 15\%$ 内变化，两问时长跨度均小于 0.011%。这支持所选工况下时长对扩散参数更敏感，不能据此认为潜热或气固边界不重要。

边界阻力族采用 $p=1, 2, 4$ 时，已计算的临界时间随所选参数点增大。 $p=4$ 相对同 N 基线使问题三、四分别延长约 13.87%和 8.27%；这批结果只完成事件积分，不替代正式严格报告值。其用途是展示未标定边界关系的影响，不能据此推出真实时长下界或参数全域单调性。

七、模型评价与推广

7.1 模型优点

模型结构统一：四个问题的解决用同一套守恒框架，只是物性、几何与判据的切换，逻辑一致、便于对比。

针对扩散系数随含水率显著变化的特点，采用 Kirchhoff 变换构造水分扩散的界面通量；针对径向收缩引起的区域变化，采用材料坐标将移动区域映射到固定区间。空间离散后，使用隐式 BDF 进行时间积分，并提供解析构造的稀疏雅可比矩阵，以处理细网格离散和热湿耦合形成的刚性方程组。

检验分层且可追溯：解析基准、离散守恒、空间/时间收敛、跨语言交叉核验四类证据分开报告，且明确各自的适用范围。

本文将四问置于同一守恒框架内，并明确区分各问的物性、几何和输出任务。问题二与三共享全过程轨迹，使状态表与达标时刻具有共同来源；问题四从组分守恒推导材料坐标，避免重复加入网格速度或把干基浓度误当体积浓度。节点包括中心与真实表面，因此中心对称、表面对流和移动域输出能够一致处理。

圆柱控制体统一处理轴心、内部界面和表面收支；磁盘连续输出降低高网格轨迹的内存占用。解析基准、网格加密、时间设置和独立实现分别检查不同求解环节，为结果一致性提供相互补充的证据。

7.2 模型不足

模型的主要限制来自气固平衡映射、有效显热容量和内部同比收缩等假设；这些关系仍需相应实验进一步验证。

计算设置稳定之后，主要不确定性仍来自模型输入的解释。没有内部温湿度或称重曲线，就无法用观测残差评价预测准确性；4 h 之后没有环境记录，就必须依赖长期工况假设；没有药材等温线，就无法从空气指标唯一确定材料平衡状态。此外，半径观测只约束表面，不能单独证明内部同比收缩或轴向长度恒定。这些条件与数值网格差属于不同来源，需要在未来实验中分别识别。

基线采用有效显热容量，没有同时求解完整相变、迁移携焓及收缩功；一维模型忽略端面与轴向差异，也未描述品质损伤或有效成分变化。因此温湿预测不能直接推出最低能耗、最佳药效或最优设备参数。表面潜热负荷和经验阻力只提供指定情景下的响应，尚不构成真实过程的上下界。

7.3 改进方向与适用范围

走向工艺优化：将模型作为约束条件接入优化框架，在品质约束下以能耗或时长为目标，利用参数扫描或启发式算法寻优。

维度扩展：针对长径比较小、端面影响不可忽略的物料，可将一维径向扩展为二维轴对称模型，显式处理端面传热传质。

后续可补充不同温度下的药材吸附、解吸等温线，结合质量变化及内部测温数据标定气固边界和有效扩散参数，并留出独立工况检验。若观察到显著轴向温差、长度变化或非均匀收缩，应扩展为二维轴对称模型，重新建立材料运动和质量守恒。完整能量模型还需蒸发位置、潜热及相关传质参数支持。

现有框架适用于几何可近似圆柱、径向传递占主导且物性形式已给定的条件计算。迁移到其他药材时，应重新确定物性、平衡关系和收缩轨迹，而非直接沿用本文系数。在本文约定条件下，四问结果分别给出了预热分布、全过程耦合变化、固定半径达标时长及收缩修正后的时长，可作为后续实验和工况比较的量化依据。

八、AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于资料检索、模型推导辅助、程序调试、数值结果核对及文字整理，详细使用情况见支撑材料。

AI 参与内容包括文献检索与真实性核查、模型推导辅助、程序调试、计算结果核对、公式与引用整理。本稿中的数值对照用于说明所列计算条件下的一致性。参赛队对采用的模型、源码、引用和结论承担核实责任；核心代码及最终稿的团队人工审查另行记录。

参考文献

- [1] 王乐意, 李长河, 刘明政, 等. 中药材干燥技术与装备研究现状[J]. 农业工程学报, 2024, 40(2): 1-28. DOI: 10.11975/j.issn.1002-6819.202306104.
- [2] 王晓辉, 王学成, 唐培渝, 等. 数值模拟仿真研究现状及其在中药干燥领域应用展望[J]. 中国中药杂志, 2023, 48(13): 3440-3447. DOI: 10.19540/j.cnki.cjcmm.20230331.301.
- [3] MUJUMDAR A S, ed. Handbook of industrial drying[M]. 4th ed. Boca Raton: CRC Press, 2014. DOI: 10.1201/b17208.
- [4] CRANK J. The mathematics of diffusion[M]. 2nd ed. Oxford: Clarendon Press, 1975.
- [5] EYMARD R, GALLOUËT T, HERBIN R. Finite volume methods[M]//CIARLET P G, LIONS J L, eds. Handbook of numerical analysis. Vol. 7. Amsterdam: North-Holland, 2000: 713-1018. DOI: 10.1016/S1570-8659(00)07005-8.
- [6] BYRNE G D, HINDMARSH A C. A polyalgorithm for the numerical solution of ordinary differential equations[J]. ACM Transactions on Mathematical Software, 1975, 1(1): 71-96. DOI: 10.1145/355626.355636.
- [7] THE SCIPY COMMUNITY. scipy.integrate.BDF[EB/OL]. [2026-09-12]. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.BDF.html>.

九、附录

9.0 支撑材料文件列表

支撑材料共 137 个文件。下表覆盖题面与数据、参考资料、生产程序、结果表格、数值检验、绘图资料和 AI 工具使用详情。

表 1 支撑材料分类汇总

目录	文件数
根目录	7
01_赛题与原始数据	10
02_参考文献与网络资料	11
03_程序代码	30
04_结果表格	12
05_数值检验与实验	43
06_绘图程序与数据	24

表 2 根目录

序号	文件名	用途
1	00_可用于论文附录的支撑清单.txt	含简短用途的完整文件清单
2	00_支撑材料总说明.txt	材料组织与复现入口说明
3	00_整理验收说明.txt	材料整理与真实核查范围
4	00_文件清单.txt	全部文件的相对路径清单
5	00_材料来源说明.txt	数据 程序 文献及图表来源
6	AI 工具使用详情.docx	AI 工具与使用环节记录
7	源码去注释与沙盒核查.txt	既有去注释与独立沙盒核查范围

表 3 01_赛题与原始数据

序号	文件名	用途
8	A 题.pdf	A 题题面及参数和结果要求
9	数据说明.txt	原始数据与清洗字段说明

表 4 01_赛题与原始数据/清洗后数据

序号	文件名	用途
----	-----	----

序号	文件名	用途
10	A_environment_observed.csv	环境观测数据 时间 温度和湿度
11	A_radius_observed.csv	药材半径观测数据

表 5 01_赛题与原始数据/附件

序号	文件名	用途
12	附件 1.xlsx	原始环境温湿度观测数据
13	附件 2.xlsx	原始药材半径观测数据

表 6 01_赛题与原始数据/附件/附件 3

序号	文件名	用途
14	result1.xlsx	问题 1 空白结果模板
15	result2.xlsx	问题 2 空白结果模板
16	result3.xlsx	问题 3 空白结果模板
17	result4.xlsx	问题 4 空白结果模板

表 7 02_参考文献与网络资料

序号	文件名	用途
18	Handbook of Industrial Drying.pdf	工业干燥手册第 4 版 5 页节选
19	Handbook of numerical analysis.pdf	有限体积法 共 254 页
20	中药材干燥技术与装备研究现状.pdf	王乐意等 2024 年综述全文 28 页
21	参考文献文件说明.txt	参考文献版本及实际阅读范围
22	数值模拟仿真研究现状及其在中药干燥领域应用展望_王晓辉.pdf	王晓辉等 2023 年综述全文 8 页

表 8 02_参考文献与网络资料/补充网络资料

序号	文件名	用途
23	FAO_干燥原理_官方网页.html	FAO 干燥原理网页存档
24	FAO_干燥原理_官方网页_离线文本.html	FAO 干燥原理可离线阅读文本
25	SciPy_solve_ivp_官方 API 网页.html	SciPy 积分接口网页存档
26	SciPy_solve_ivp_官方 API 网页_离线文本.html	SciPy 积分接口可离线阅读文本

序号	文件名	用途
27	Shampine1997_BYU 原始扫描_35页.pdf	常微分方程求解补充资料 35 页扫描
28	补充资料说明.txt	补充文献和网页资料来源

表 9 03_程序代码

序号	文件名	用途
29	A_CodeReview.pyproj	Visual Studio Python 工程配置
30	A_CodeReview.sln	Visual Studio 方案入口
31	analyticJacobian.py	正式模型解析稀疏 Jacobian
32	diskDense.py	BDF 连续输出多项式存储
33	dryingCore.py	热湿耦合方程离散与 BDF 求解
34	exportOutputs.py	四份结果工作簿导出与回读
35	input_manifest.csv	正式输入 模板和参照校验清单
36	numerical_design.txt	离散方程与阈值事件说明
37	q1Model.py	问题一模型配置
38	q2Model.py	问题二与三共同轨迹配置
39	q3Model.py	问题三严格达标时刻判定
40	q4Model.py	问题四收缩模型配置
41	README.txt	目录运行与依赖说明
42	requirements.txt	Python 依赖及固定版本
43	runDelivery.py	四问生产计算与输入预检入口
44	runLogged.ps1	生产运行监督与退出码记录

表 10 03_程序代码/docs

序号	文件名	用途
45	源码变更说明.txt	生产源码来源和变更说明
46	运行验收说明.txt	生产运行证据与适用范围

表 11 03_程序代码/inputs/cleaned

序号	文件名	用途
47	A_environment_observed.csv	环境观测数据 时间 温度和湿度
48	A_radius_observed.csv	药材半径观测数据

表 12 03_程序代码/inputs/original

序号	文件名	用途
49	附件 1.xlsx	原始环境温湿度观测数据
50	附件 2.xlsx	原始药材半径观测数据

表 13 03_程序代码/inputs/templates

序号	文件名	用途
51	result1.xlsx	问题 1 空白结果模板
52	result2.xlsx	问题 2 空白结果模板
53	result3.xlsx	问题 3 空白结果模板
54	result4.xlsx	问题 4 空白结果模板

表 14 03_程序代码/reference

序号	文件名	用途
55	reference_data.py	冻结数值与精度参照常量

表 15 03_程序代码/reference/Q1

序号	文件名	用途
56	sampld_solution.npz	Q1 冻结采样数组

表 16 03_程序代码/reference/Q23

序号	文件名	用途
57	sampld_solution.npz	Q23 冻结采样数组

表 17 03_程序代码/reference/Q4

序号	文件名	用途
58	sampld_solution.npz	Q4 冻结采样数组

表 18 04_结果表格

序号	文件名	用途
----	-----	----

序号	文件名	用途
59	result1.xlsx	问题 1 完整计算结果
60	result2.xlsx	问题 2 完整计算结果
61	result3.xlsx	问题 3 完整计算结果
62	result4.xlsx	问题 4 完整计算结果
63	结果表说明.txt	完整工作簿与正文表格说明

表 19 04_结果表格/Reference Table Files

序号	文件名	用途
64	q1_paper_moisture.csv	正文问题一含水率结果表
65	q1_paper_temperature.csv	正文问题一温度结果表
66	q2_paper_moisture.csv	正文问题二含水率结果表
67	q2_paper_temperature.csv	正文问题二温度结果表
68	q3_paper_moisture.csv	正文问题三含水率结果表
69	q4_paper_moisture.csv	正文问题四含水率结果表
70	q4_paper_radius.csv	正文问题四半径结果表

表 20 05_数值检验与实验

序号	文件名	用途
71	数值检验说明.txt	检验程序用途及运行前提
72	本轮补件与运行核验.txt	本轮路径适配与实际复现说明
73	源码文件与校验值.txt	检验源码用途 来源与 SHA256
74	独立沙盒运行矩阵.csv	既有沙盒逐任务运行状态
75	独立沙盒逐源码核查.txt	既有逐源码实算及失败范围
76	论文采用检验_独立运行说明.txt	本轮路径适配与实际复现说明

表 21 05_数值检验与实验/MATLAB 对照证据

序号	文件名	用途
77	compareMatlab.mjs	MATLAB 与 Python 结果比较程序
78	runCrossCheck.m	MATLAB 独立热湿模型对照实现

表 22 05_数值检验与实验/Python 检验源码

序号	文件名	用途
79	requirements.txt	Python 依赖及固定版本
80	run_checks.py	网格 时间和方法检验统一入口

表 23 05_数值检验与实验/Python 检验源码/paper_output/code/modeling

序号	文件名	用途
81	analytic_jacobian.py	检验模型解析 Jacobian
82	axisymmetric_check.py	轴对称假设与轴向影响检验
83	compare_analytic.py	数值解与圆柱解析基准比较
84	disk_dense.py	检验模型 BDF 连续输出存储
85	drying_core.py	检验所用热湿模型核心
86	export_outputs.py	检验所用工作簿导出模块
87	production_provenance.py	运行监督 输入源码版本记录
88	publication_plots.py	既有绘图辅助程序
89	q1_model.py	检验所用问题一配置
90	q2_model.py	检验所用问题二配置
91	q3_model.py	检验所用问题三达标判定
92	q4_model.py	检验所用问题四配置
93	run_experiments.py	既有实验任务入口
94	run_modeling.py	旧模型流程入口 保留原运行前提
95	validate_bessel.py	圆柱 Robin 与 Bessel 解析基准
96	verify_convergence.py	空间网格加密误差检验
97	verify_dense_storage.py	BDF 连续输出存储一致性检验
98	verify_time_accuracy.py	时间容差与步长敏感性检验

表 24 05_数值检验与实验/Python 检验源码/paper_output/code/verification

序号	文件名	用途
99	analytic_metric_check.py	解析误差指标历史诊断
100	crossvalidate_series.py	描述序列对照及历史趋势诊断
101	crossvalidate_solver.py	不同积分器的数值对照
102	energy_balance_check.py	模型能量收支检查

序号	文件名	用途
103	energy_balance_diagnose.py	能量闭合历史诊断
104	energy_balance_diagnose2.py	能量闭合进一步历史诊断
105	isotherm_activity_closure.py	历史等温线闭合试验
106	isotherm_diagnose.py	历史等温线接口诊断
107	isotherm_jacobian_probe.py	历史 Jacobian 注入探测
108	latent_heat_scenarios.py	潜热情景计算
109	method_comparison.py	两种守恒通量格式对照
110	sensitivity_analysis.py	参数扰动与敏感性计算
111	threshold_and_scaling_checks.py	阈值事件和标度检验

表 25 05_数值检验与实验/Python 检验源码/paper_output/data_cleaned

序号	文件名	用途
112	A_environment_observed.csv	环境观测数据 时间 温度和湿度
113	A_radius_observed.csv	药材半径观测数据

表 26 06_绘图程序与数据

序号	文件名	用途
114	draw_figures.R	六幅论文图的 PNG 绘制程序
115	图像校验.csv	文件来源与校验清单
116	数据来源与校验.csv	文件来源与校验清单
117	绘图复现说明.txt	本轮路径适配与实际复现说明

表 27 06_绘图程序与数据/CSV/图 1

序号	文件名	用途
118	moisture.csv	图 1 径向含水率曲线数据
119	temperature.csv	图 1 径向温度曲线数据

表 28 06_绘图程序与数据/CSV/图 2

序号	文件名	用途
120	assumed_platform_4_60h.csv	图 2 后续环境平台假设数据
121	observed_0_4h.csv	图 2 前 4 小时环境实测点

表 29 06_绘图程序与数据/CSV/图 3

序号	文件名	用途
122	moisture_to_event.csv	图 3 中心和表面含水率数据
123	temperature_0_4h.csv	图 3 中心和表面温度数据

表 30 06_绘图程序与数据/CSV/图 4

序号	文件名	用途
124	centre_near_event.csv	图 4 临界附近中心含水率数据
125	drying_main.csv	图 4 全域含水率与达标曲线
126	event_markers.csv	图 4 连续事件与严格报告点
127	maximum_near_event.csv	图 4 临界附近全域最大含水率

表 31 06_绘图程序与数据/CSV/图 5

序号	文件名	用途
128	moisture_profiles_xy_pairs.csv	图 5 收缩域内含水率 XY 数据
129	radius_observed.csv	图 5 原始半径观测数据
130	selected_times_and_radii.csv	图 5 所选时刻及对应半径

表 32 06_绘图程序与数据/CSV/图 6

序号	文件名	用途
131	scenario_comparison.csv	图 6 经验边界阻力情景数据

表 33 06_绘图程序与数据/PNG

序号	文件名	用途
132	图 1_径向温度与含水率.png	图 1_径向温度与含水率成图
133	图 2_环境观测与平台延拓.png	图 2_环境观测与平台延拓成图
134	图 3_中心与表面热湿响应.png	图 3_中心与表面热湿响应成图
135	图 4_全域达标与临界放大.png	图 4_全域达标与临界放大成图
136	图 5_收缩半径与域内含水率.png	图 5_收缩半径与域内含水率成图
137	图 6_经验边界阻力情景.png	图 6_经验边界阻力情景成图

9.1 连续模型、坐标变换与数值方法的完整推导

本附录给出正文采用的有效热湿模型及其数值实现的推导，补充公式的成立条件、变量含义和计算检查范围。题给物性、环境观测及模型闭合分别说明；数学推导不改变正文已冻结的模型和结果。计算统一使用 m、s、K；含水率以 kg 水/kg 干物质计。题定长度为 $L_z = 0.25$ m，初始半径为 $R_0 = 0.02$ m，初温为 $T_0 = 301.15$ K，初始干基含水率为 $C_0 = 2.55$ 。为避免与长度混淆，累计平均失水量统一记为 ℓ 。文献序号沿用正文参考文献；扩散与圆柱基准的理论出处为 Crank^[4]，有限体积的一般构造见 Eymard 等^[5]，隐式多步求解及具体实现见文献^{[6] [7]}。以下推导同时列明这些一般方法在当前题设中的使用条件。

9.1.1 干基含水率、湿基含水率与参考质量

设某一材料微团内干物质质量为 m_d 、水质量为 m_w 。干基含水率 C 与湿基水质量分数 w 定义为

$$C = \frac{m_w}{m_d}, w = \frac{m_w}{m_d + m_w} = \frac{C}{1+C}, C = \frac{w}{1-w} \quad (\text{A.1})$$

这里的 C 是质量比，可以大于 1； w 是总湿质量中的水质量分数，应在 0 与 1 之间。由式 (A.1)， $C_0 = 2.55$ 对应 $w_0 = 2.55/3.55 \approx 0.7183098592$ ，即初始水分约占湿质量的 71.83%。不能将 $C_0 = 2.55$ 直接写成 2.55% 或 255% 的湿基质量分数。

在当前物理体积内，分别记干物质质量密度为 ρ_d 、水质量密度为 ρ_w 。微团两组分占用同一统计体积时，干基定义等价于

$$\rho_w = \rho_d C, m_d = \int_{\Omega(t)} \rho_d dV, m_w = \int_{\Omega(t)} \rho_d C dV \quad (\text{A.2})$$

$\Omega(t)$ 为当前药材区域， ρ_d 与 ρ_w 的单位均为 kg/m^3 。水质量的计算必须有相应的干物质权重，不能把含水率节点的算术平均直接当作体积总水质量。

附件的烘房水分指标仅标注 kg/kg ，记其观测数值为 $Y_\infty(t)$ 。基线采用 $C_{\text{eq}}(t) = Y_\infty(t)$ 的数值映射，构造表面有效平衡含水率。这是一项边界闭合假设。若进一步把 Y_∞ 解释为空气湿度比，则其分母是干空气质量，与药材 C 的干物质参考质量不同；两者同写 kg/kg 不能证明数值可直接相减。材料真实平衡含水率一般还依赖温度与吸附、解吸等温线，当前附件没有给出这种标定关系。

9.1.2 从组分质量平衡得到干基扩散方程

设干物质骨架速度为 $\mathbf{v}_s$ ，水相对骨架的质量通量为 $\mathbf{j}_w$ ，其单位为 $\text{kg}/(\text{m}^2\text{s})$ 。无干物质生成、损失及相对于骨架迁移时，干物质连续性方程为

$$\frac{\partial \rho_d}{\partial t} + \nabla \cdot (\rho_d \mathbf{v}_s) = 0 \quad (\text{A.3})$$

水的实验室坐标总通量由随骨架运输的 $\rho_d C \mathbf{v}_s$ 与相对通量 $\mathbf{j}_w$ 组成。无内部水源项时，水质量守恒为

$$\frac{\partial(\rho_d C)}{\partial t} + \nabla \cdot (\rho_d C \mathbf{v}_s + \mathbf{j}_w) = 0 \quad (\text{A.4})$$

将式 (A.4) 展开，得到

$$\rho_d \left(\frac{\partial C}{\partial t} + \mathbf{v}_s \cdot \nabla C \right) + C \left[\frac{\partial \rho_d}{\partial t} + \nabla \cdot (\rho_d \mathbf{v}_s) \right] = - \nabla \cdot \mathbf{j}_w \quad (\text{A.5})$$

方括号内为干物质连续性方程的左端，因而为零。定义随骨架运动的物质导数 $D_s/Dt = \partial_t + \mathbf{v}_s \cdot \nabla$ ，即可得到干基含水率的输运方程

$$\rho_d \frac{D_s C}{Dt} = - \nabla \cdot \mathbf{j}_w \quad (\text{A.6})$$

式 (A.6) 显示，骨架压缩导致的干密度变化已经在质量比的求导过程中相消。若没有相对水通量，则 $D_s C/Dt = 0$ ；纯几何收缩不使同一材料微团的干基质量比自行增加。

采用相对骨架的有效 Fick 闭合。扩散通量与浓度梯度关系的一般理论可参见 Crank[4]；这里对干骨架参考系和有效扩散率的选取仍是本模型的具体闭合：

$$\mathbf{j}_w = - \rho_d D(T, C) \nabla C, \quad \rho_d \frac{D_s C}{Dt} = \nabla \cdot [\rho_d D(T, C) \nabla C] \quad (\text{A.7})$$

D 的单位为 m^2/s ，负号表示水沿含水率下降方向迁移。将散度展开后，若 ρ_d 空间不均匀，一般有

$$\frac{D_s C}{Dt} = \nabla \cdot (D \nabla C) + D \nabla (\ln \rho_d) \cdot \nabla C \quad (\text{A.8})$$

因此只有在干骨架密度空间均匀时，才能从散度内约去 ρ_d 。本文以初始干物质均匀分布、固定骨架或同比收缩保证这一条件；若采用非均匀收缩，须回到式 (A.7)，不能沿用已经约去密度的简式。

9.1.3 圆柱薄壳收支与固定域方程

忽略轴向、周向差异及两端面的传递，所有场量仅随径向位置 r 和时间 t 变化。固定半径时 $\mathbf{v}_s = 0$ 。半径 r 至 $r + dr$ 、长度 L_z 的薄壳体积为 $2\pi r L_z dr$ ，两侧传递面积分别为 $2\pi r L_z$ 与 $2\pi(r + dr)L_z$ 。将水的净流入量除以薄壳体积并令 $dr \rightarrow 0$ ，有

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[r D(T, C) \frac{\partial C}{\partial r} \right], \quad 0 < r < R_0 \quad (\text{A.9})$$

这里 $1/r$ 来自圆柱薄壳面积随半径的变化。 D 随状态变化时，应保留在径向导数内；把右端写成 $D(C_{rr} + C_r/r)$ 会遗漏系数梯度贡献。

热侧采用 Fourier 通量 $\mathbf{q} = -k(C) \nabla T$ ，并以题给物性构造有效显热容量 $B(C) = \rho_{\text{eff}}(C) c_p(C)$ 。选择 $B(C) D_s T/Dt$ 作为本模型的局部温升需求，薄壳导热收支给出

$$\begin{aligned} B(C) \frac{D_s T}{Dt} &= \nabla \cdot [k(C) \nabla T], \\ B(C) \frac{\partial T}{\partial t} &= \frac{1}{r} \frac{\partial}{\partial r} [r k(C) \frac{\partial T}{\partial r}] \quad (v_s = 0) \end{aligned} \quad (\text{A.10})$$

k 的单位为 $\text{W}/(\text{m K})$, c_p 为 $\text{J}/(\text{kg K})$, ρ_{eff} 为 kg/m^3 , 故 B 为 $\text{J}/(\text{m}^3 \text{K})$ 。式 (A.10) 是本文采用的有效显热闭合; 由 Fourier 定律只能确定导热通量, 不能单独推出完整多组分能量方程恰好等于该容量形式。

特别地, ρ_{eff} 不自动等于质量守恒中的 ρ_d 。比热、密度依赖含水率时, 有

$$\frac{D_s}{Dt} [B(C)T] = B(C) \frac{D_s T}{Dt} + T B'(C) \frac{D_s C}{Dt} \quad (\text{A.11})$$

因此不能把式 (A.10) 的左端改写为 $D_s(BT)/Dt$ 而省略第二项。基线未逐项建立水分迁移携焓、相变热及机械功, 后面的有效热通量恒等式只检查所采用的方程, 不证明真实完整过程的总能量平衡已经闭合。

9.1.4 四问物性与驱动条件

问题一采用题给附录 2 的常热物性及含水率相关扩散率:

$$\rho_{\text{eff}} = 820, c_p = 2600, k = 0.36, D_1(C) = 7 \times 10^{-9} \exp(-0.89/C) \quad (\text{A.12})$$

密度、比热、导热率和扩散率依次使用 kg/m^3 、 $\text{J}/(\text{kg K})$ 、 $\text{W}/(\text{m K})$ 和 m^2/s 。问题一的热场可独立求解, 水分扩散仍是非线性的; D_1 不含温度因子。

问题二与问题三从同一均匀初态开始, 在整个过程采用题给附录 3:

$$\begin{aligned} \rho_{\text{eff}}(C) &= 650 + 128C, \\ c_p(C) &= 1450 + 2736 \frac{C}{1+C}, \\ k(C) &= 0.21 + 0.38 \frac{C}{1+C}, \\ D_{23}(T, C) &= 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C} - \frac{3850}{T}\right) \end{aligned} \quad (\text{A.13})$$

问题四从初始时刻整组使用题给附录 4, 并同时引入半径变化:

$$\begin{aligned} \rho_{\text{eff}}(C) &= 760 + 90C, \\ c_p(C) &= 1850 + 2150 \frac{C}{1+C}, \\ k(C) &= 0.12 + 0.20 \frac{C}{1+C}, \\ D_4(T, C) &= 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C} - \frac{3850}{T}\right) \end{aligned} \quad (\text{A.14})$$

式 (A.13)、式 (A.14) 的单位与式 (A.12) 相同。 T 必须以 K 代入, 3850 带有使 $3850/T$ 无量纲的温度尺度; a/C 中参数 a 与干基含水率采用同一数值口径。这些系数来自题给经验关系, 不是由本次附件重新拟合, 也不是从第一原理推导的常数。问题二、三不拼接问题一的末态, 问题四也不在某一阶段才切换附录 4。

相同 T, C 下, 两组全过程扩散率之比为

$$\frac{D_4(T,C)}{D_{23}(T,C)} = 0.175 \exp\left(\frac{0.15}{C}\right), C_{\times} = \frac{0.15}{\ln(1/0.175)} \approx 0.08606 \quad (\text{A.15})$$

其中 $C_{\times}$ 是两扩散率相等的含水率。 $C > C_{\times}$ 时 $D_4 < D_{23}$ ， $0 < C < C_{\times}$ 时顺序反转。因此不能写“附录4 扩散系数在全程整体更小”；收缩和整组物性变化的净作用需由保持条件明确的数值对照判断。

环境表有241个0—4 h记录，半径表有145个0—72 h记录。对相邻时间节点 $t_j \leq t \leq t_{j+1}$ ，以 f_j 表示同一观测变量，分段线性插值为

$$I_f(t) = f_j + \frac{f_{j+1} - f_j}{t_{j+1} - t_j} (t - t_j) \quad (\text{A.16})$$

温度观测先换成K；半径换成m。正文基线环境为

$$(T_{\infty}, C_{\text{eq}})(t) = \begin{cases} (I_T(t), I_Y(t)), & 0 \leq t \leq 14400 \text{ s}, \\ (323.15, 0.05), & t > 14400 \text{ s}. \end{cases} \quad (\text{A.17})$$

式(A.17)后一段是平台延拓假设，不是持续数天的观测。收缩半径在附件2的观测区间内使用 $R(t) = I_R(t)$ 。正式问题四的达标时刻位于72 h内，未用到半径观测窗外延拓；对其他长时情景则必须另行说明。

9.1.5 初始条件、表面边界与量纲

三条正式轨迹均从均匀初态开始，轴心由对称性给零径向梯度：

$$T(r, 0) = T_0, C(r, 0) = C_0, T_r(0, t) = C_r(0, t) = 0 \quad (\text{A.18})$$

记 $T_s = T(R(t), t)$ 、 $C_s = C(R(t), t)$ 为真实表面状态。向外为正时，表面热、水通量分别为

$$q_{\text{out}} = -kT_r = h(T_s - T_{\infty}), j_{w,\text{out}} = -\rho_d DC_r = \rho_d \beta (C_s - C_{\text{eq}}) \quad (\text{A.19})$$

$h = 25 \text{ W}/(\text{m}^2\text{K})$ ， $\beta = 8 \times 10^{-7} \text{ m/s}$ 。热边界两侧单位均为 W/m^2 ；水质量边界两侧单位均为 $\text{kg}/(\text{m}^2\text{s})$ 。将水边界约去同一表面干密度得到源码使用的形式

$$-DC_r = \beta(C_s - C_{\text{eq}}) \quad (\text{A.20})$$

式(A.20)的两侧是以干密度归一化的传递量，其长度—时间单位为 m/s ，尚不是质量通量。材料较空气冷时 $T_s < T_{\infty}$ ，向外热通量为负，热量进入材料； $C_s > C_{\text{eq}}$ 时向外水通量为正，材料失水。该符号规则贯穿边界与控制体离散。

初值与瞬时表面 Robin 条件未必在 $t = 0$ 完全相容。这表示启动阶段形成快速边界层，不要求人为改动题给初态。数值求解采用较小的早期步长，并另行检查接受状态和早期场误差。

9.1.6 干物质密度、有效密度与收缩的一致性

问题一至三令骨架固定；问题四采用定长、径向同比收缩。每个初始材料半径 r_0 随时间变为 $r = r_0 R(t)/R_0$ ，相应骨架速度为

$$v_s(r, t) = \frac{\partial r}{\partial t} \Big|_{r_0} = \frac{r \dot{R}(t)}{R(t)}, \nabla \cdot \mathbf{v}_s = \frac{1}{r} \frac{\partial(r v_s)}{\partial r} = 2 \frac{\dot{R}(t)}{R(t)} \quad (\text{A.21})$$

$\dot{R} < 0$ 表示收缩；轴向速度取零。式 (A.3) 因而给出

$$\begin{aligned} \frac{D_s \rho_d}{Dt} &= -2 \frac{\dot{R}}{R} \rho_d, \\ \rho_d(t) &= \rho_{d0} \left(\frac{R_0}{R(t)} \right)^2, \\ M_d &= \pi L_z R(t)^2 \rho_d(t) = \pi L_z R_0^2 \rho_{d0} \end{aligned} \quad (\text{A.22})$$

这里 M_d 为恒定总干质量， ρ_{d0} 为初始均匀干物质密度。同比收缩使每个材料体积均按 R^2/R_0^2 缩小，因此初始均匀干密度保持空间均匀而随时间变化。半径的表面观测不能单独证明内部运动同比，式 (A.21) 是用于闭合内部运动的明确假设。

若同时强行把附录 4 的 760 + 90C 解释为真实湿密度，则非负含水率下应有

$$\rho_d = \frac{760+90C}{1+C} = 90 + \frac{670}{1+C} \leq 760 \quad (\text{A.23})$$

以初始关系 $\rho_{d0} = (760 + 90C_0)/(1 + C_0) \approx 278.7323943662$ 归一化干质量，再与式 (A.22) 联立，得到

$$R(t) \geq R_0 \sqrt{\frac{\rho_{d0}}{760}} \approx 1.2112029565 \text{ cm} \quad (\text{A.24})$$

附件 2 在 72 h 给出 1.198 cm，小于上述下界。因此真实湿密度解释、非负含水率、固定长度和给定收缩轨迹不能在所给全时段同时成立。本文将 $\rho_{\text{eff}} c_p$ 用作有效热容量， ρ_d 独立守恒；初始归一化不意味着随时重新设置 $\rho_d = \rho_{\text{eff}}(C)/(1 + C)$ 。基线温湿方程约去均匀干密度后不依赖其绝对标度；只有换算绝对水量或添加指定潜热负荷时才需要该标度。

9.1.7 一般移动坐标与网格速度项

为明确网格运动的来源，先不假定网格速度等于骨架速度。取 $r = R(t)x$ ， $0 \leq x \leq 1$ ；定义 $\hat{C}(x, t) = C(R(t)x, t)$ ， $\hat{T}(x, t) = T(R(t)x, t)$ 。固定 x 的网格速度为 $v_g = x\dot{R}$ 。链式法则给出

$$\frac{\partial C}{\partial t} \Big|_r = \frac{\partial \hat{C}}{\partial t} \Big|_x - \frac{x \dot{R}}{R} \frac{\partial \hat{C}}{\partial x}, \frac{\partial C}{\partial r} = \frac{1}{R} \frac{\partial \hat{C}}{\partial x} \quad (\text{A.25})$$

把物理方程中的骨架运输项同时变换，得到

$$\begin{aligned}\frac{D_s C}{Dt} &= \frac{\partial \hat{C}}{\partial t} + \frac{v_s - v_g}{R} \frac{\partial \hat{C}}{\partial x}, \\ \frac{D_s T}{Dt} &= \frac{\partial \hat{T}}{\partial t} + \frac{v_s - v_g}{R} \frac{\partial \hat{T}}{\partial x}\end{aligned}\quad (\text{A.26})$$

在干密度空间均匀的条件下，完整变换后的水分方程为

$$\frac{\partial \hat{C}}{\partial t} + \frac{v_s - x\dot{R}}{R} \frac{\partial \hat{C}}{\partial x} = \frac{1}{R^2 x} \frac{\partial}{\partial x} (xD \frac{\partial \hat{C}}{\partial x}) \quad (\text{A.27})$$

式 (A.27) 保留了网格相对骨架运动的项。如果假设骨架在空间固定而只有计算边界改变，则 $v_s = 0$ ，左侧必须保留 $-x\dot{R}\hat{C}_x/R$ ；不能直接套用材料随动形式。这样的假设是否与边界处物质流失、干质量守恒相容，还需重新说明，不能只换坐标而省略物理解释。

本文问题四采用式 (A.21)，故 $v_s = v_g = x\dot{R}$ 。坐标变化项与骨架平流项恰好相消，材料坐标控制方程为

$$\begin{aligned}\frac{\partial \hat{C}}{\partial t} &= \frac{1}{R(t)^2 x} \frac{\partial}{\partial x} (xD(\hat{T}, \hat{C})\hat{C}_x), \\ B(\hat{C}) \frac{\partial \hat{T}}{\partial t} &= \frac{1}{R(t)^2 x} \frac{\partial}{\partial x} (xk(\hat{C})\hat{T}_x)\end{aligned}\quad (\text{A.28})$$

固定半径的前三问是 $R(t) = R_0$ 、 $v_s = v_g = 0$ 的特例。材料坐标形式不再重复添加网格平流，也不增加 $-2\dot{R}\hat{C}/R$ 的体积浓缩项；后者只适用于相应体积密度的收支，不能施加在已由组分方程消去压缩项的干基比值上。

表面导数换元后，边界条件变为

$$\begin{aligned}\hat{T}_x(0, t) &= \hat{C}_x(0, t) = 0, \\ -\frac{k_s}{R} \hat{T}_x(1, t) &= h(T_s - T_\infty), \\ -\frac{D_s}{R} \hat{C}_x(1, t) &= \beta(C_s - C_{eq})\end{aligned}\quad (\text{A.29})$$

k_s, D_s 为当前表面物性。 $1/R$ 来自空间导数换元，内部方程中的 $1/R^2$ 来自两次长度缩放。半径折线的 $\dot{R}$ 在观测节点处可不连续，但相消后的求解方程只需连续的 $R(t)$ ；各开区间上的推导可拼接为相容的分段解，不需对折线半径作数值微分。

9.1.8 节点控制体、轴心及表面权重

有限体积法以控制体积分收支和共用面通量建立离散方程，一般框架见 Eymard 等 [5]；本题的具体圆柱权重和节点配置如下推导。取 N 个均匀区间， $\Delta x = 1/N$ ，节点 $x_i = i\Delta x$ ， $i = 0, \dots, N$ 。对偶控制体的内部边界为相邻节点中点，左、右外边界分别截在 0 和 1。记第 i 个控制体为 $[a_i, b_i]$ ，其圆柱权重为

$$\begin{aligned}
w_i &= \int_{a_i}^{b_i} x \, dx = \frac{b_i^2 - a_i^2}{2}, \\
w_0 &= \frac{\Delta x^2}{8}, \quad w_i = x_i \Delta x (1 \leq i < N), \\
w_N &= \frac{\Delta x}{2} - \frac{\Delta x^2}{8}
\end{aligned} \tag{A.30}$$

中心控制体为 $[0, \Delta x/2]$ ，表面控制体为 $[1 - \Delta x/2, 1]$ 。由区间拼接可得 $\sum_i w_i = 1/2$ 。单位轴向长度的实际控制体积为 $2\pi R^2 w_i$ ，完整控制体体积为 $2\pi L_z R^2 w_i$ 。

未知量 T_i, C_i 位于节点，节点包括轴心与真实表面。用 $w_i C_i$ 近似积分 $\int_{a_i}^{b_i} x \hat{C} dx$ ，用 $w_i B(C_i) \hat{T}_i$ 近似有效热容量积分。这个集总近似不将节点值变成精确单元平均；点值基准与环体平均基准应分别比较。

定义便于装配的梯度面量

$$g^T = x k \hat{T}_x, \quad g^C = x D \hat{C}_x \tag{A.31}$$

g^T 单位为 W/m， g^C 为 m^2/s ，它们与向外物理热流、水流的符号相反。对式 (A.28) 在控制体内乘 x 积分，并采用上述节点集总，得到半离散方程

$$\dot{T}_i = \frac{g_{i+1/2}^T - g_{i-1/2}^T}{R^2 w_i B_i}, \quad \dot{C}_i = \frac{g_{i+1/2}^C - g_{i-1/2}^C}{R^2 w_i}, \quad B_i = B(C_i) \tag{A.32}$$

这里 $i - 1/2, i + 1/2$ 表示第 i 个控制体的两侧面；中心外侧面和表面外侧面的坐标按0、1截断。内部面必须由相邻控制体共用同一个数值，左控制体取正贡献，右控制体取负贡献。

中心面直接令 $g_{-1/2}^T = g_{-1/2}^C = 0$ ，因此程序不计算 $1/x_0$ 。例如常系数热方程的第一行是

$$\dot{T}_0 = \frac{4k(T_1 - T_0)}{BR^2 \Delta x^2} \tag{A.33}$$

若光滑轴对称场在中心展开为 $T(x) = T(0) + \frac{1}{2} T_{xx}(0) x^2 + O(x^4)$ ，式 (A.33) 趋于 $2kT_{xx}(0)/(BR^2)$ ，与圆柱算子 $k(T_{xx} + T_x/x)/(BR^2)$ 的轴心极限一致。

由式 (A.29)，表面外侧面直接取

$$g_{N+1/2}^T = hR(T_\infty - T_N), \quad g_{N+1/2}^C = -\beta R(C_N - C_{eq}) \tag{A.34}$$

最后一个未知量本身位于实际表面，无须从末单元中心再外推一个表面值，也不额外附加半单元传质内阻。若改成单元中心网格，未知量位置与边界离散必须同步重建。

9.1.9 导热调和平均与 Kirchhoff 水分面量

对内部面 $i + 1/2$ ，记 $\gamma_i = x_{i+1/2}/\Delta x$ 。两侧各半间距的导热阻相加，等效导热系数与热面量为

$$\begin{aligned}\frac{\Delta x}{k_H} &= \frac{\Delta x/2}{k_i} + \frac{\Delta x/2}{k_{i+1}}, \\ k_H &= \frac{2k_i k_{i+1}}{k_i + k_{i+1}}, \\ g_{i+1/2}^T &= \gamma_i k_H (T_{i+1} - T_i)\end{aligned}\tag{A.35}$$

这里的等效阻力解释针对面附近两段系数近似，圆柱面位置也按 $x_{i+1/2}$ 近似；不是对整个有限厚圆柱壳稳态导热的无误差积分。

三组扩散率可统一写成 $D = D_0 A(T) f_a(C)$ ，其中 $A(T) = \exp(-b/T)$ 、 $f_a(C) = \exp(-a/C)$ 。参数 (a, b, D_0) 依次为 $(0.89, 0.7 \times 10^{-9})$ 、 $(0.45, 3850, 2.4 \times 10^{-3})$ 和 $(0.30, 3850, 4.2 \times 10^{-4})$ 。对 $C > 0$ 定义 Kirchhoff 浓度势

$$\Phi_a(C) = C \exp(-a/C) + a \text{Ei}(-a/C)\tag{A.36}$$

Ei 为指数积分函数。利用 $d\text{Ei}(z)/dz = e^z/z$ 和链式法则，分别有

$$\begin{aligned}\frac{d}{dC}[C e^{-a/C}] &= e^{-a/C} \left(1 + \frac{a}{C}\right), \\ \frac{d}{dC}[a \text{Ei}(-a/C)] &= -\frac{a}{C} e^{-a/C}, \\ \Phi_a'(C) &= e^{-a/C}\end{aligned}\tag{A.37}$$

因此 $\Phi_a(C_{i+1}) - \Phi_a(C_i) = \int_{C_i}^{C_{i+1}} e^{-a/c} dc$ 。在面上用当前两侧温度平均 $\bar{T}_i = (T_i + T_{i+1})/2$ ，水分面量为

$$g_{i+1/2}^C = \gamma_i D_0 \exp(-\frac{b}{T_i}) [\Phi_a(C_{i+1}) - \Phi_a(C_i)]\tag{A.38}$$

式(A.38)对浓度因子的积分采用解析势差；温度面值、空间重构、几何面值及时间积分仍有数值误差。Q1的 $b = 0$ ；其余两组的温度因子在一个面内单独取值。“面温度冻结”指这一步空间近似，不表示 Newton 迭代中温度不更新。

不能对 $A(T)\Phi_a(C)$ 整体作差。因为其空间导数为

$$\frac{\partial}{\partial x}[A(T)\Phi_a(C)] = A(T)e^{-a/C}C_x + A(T)\frac{b}{T^2}\Phi_a(C)T_x\tag{A.39}$$

第二项不在本题所用 Fick 闭合内。当两侧含水率相同、温度不同时，正确浓度梯度通量应为零，整体势差却会产生非零值，因而不符合原模型。

相邻浓度很接近时，两项势值相减存在消减误差。令 $\Delta C = C_{i+1} - C_i$ 、 $\bar{C}_i = (C_i + C_{i+1})/2$ ，则中点积分展开给出

$$\Delta \Phi = f_a(\bar{C}_i)\Delta C + \frac{f_a'(\bar{C}_i)}{24}(\Delta C)^3 + O((\Delta C)^5)\tag{A.40}$$

现行源码在 $|\Delta C| < 10^{-7} \max(\bar{C}_i, 10^{-3})$ 时使用第一项，其局部差为 $O((\Delta C)^3)$ 。一般分支保留指数积分势差。系数求值采用 $C^+ = \max(C, 10^{-12})$ 保护 Newton 试探点，实际比较

与势差分支以 C^* 为准；它不裁剪求得的状态。正值保护是数值定义，不是干基含水率的物理下限。

9.1.10 干基质量与有效热收支的离散恒等式

全体水方程乘 $2w_i$ 求和，内部面两两相消，只有表面项保留。定义加权平均含水率 $\bar{C}_N = 2\sum_i w_i C_i$ 和累计平均失水量 $\ell(0) = 0$ ，有

$$\begin{aligned}\frac{d\bar{C}_N}{dt} &= \frac{2g_{N+1/2}^C}{R^2} = -\frac{2\beta}{R}(C_N - C_{eq}), \\ \dot{\ell} &= \frac{2\beta}{R}(C_N - C_{eq}), \\ \bar{C}_N(t) + \ell(t) &= C_0\end{aligned}\tag{A.41}$$

连续材料域的对应平均为 $\bar{C} = 2\int_0^1 x \hat{C} dx$ 。由 $\rho_d R^2$ 恒定，水质量为 $M_d \bar{C}$ ；故式(A.41)与移动体积下的干质量守恒相容。 ℓ 单位为 kg 水/kg 干物质， $M_d \ell$ 才是累计水质量。当前基线以失水为主；若边界条件改变而出现吸湿， $\dot{\ell}$ 可以为负，仍是带符号的净流出量。

热方程乘 $2\pi R^2 w_i B_i$ 求和，同样得到瞬时有有效热通量恒等式

$$2\pi R^2 \sum_i w_i B_i \dot{T}_i = 2\pi R h (T_\infty - T_N)\tag{A.42}$$

式(A.42)两侧均为单位轴向长度的功率，单位 W/m。若另定义单位长度的容量加权温升量 $H_{\text{eff}} = 2\pi R^2 \sum_i w_i B_i (T_i - T_{\text{ref}})$ ，其中 T_{ref} 为固定参考温度，则乘积法则给出

$$\begin{aligned}\frac{dH_{\text{eff}}}{dt} &= 2\pi R h (T_\infty - T_N) + 2\pi R^2 \sum_i w_i (T_i - T_{\text{ref}}) B_i' \dot{C}_i \\ &\quad + 4\pi R \dot{R} \sum_i w_i B_i (T_i - T_{\text{ref}})\end{aligned}\tag{A.43}$$

第二项来自容量变化，第三项来自物理体积变化。二者是这一数学量的乘积求导项，未经额外热力学本构不能认定为真实机械功或潜热。Q1 的 B, R 恒定时，后两项为零，可对式(A.42)积分核验相应常物性显热收支；Q23/Q4 不能省略它们后把残差解释为数值积分误差。

9.1.11 解析 Jacobian：面量、容量、边界与累计量

将状态交错排列为 $\mathbf{y} = (T_0, C_0, T_1, C_1, \dots, T_N, C_N, \ell)^T$ ，维数为 $2N+3$ ，半离散系统写为 $\dot{\mathbf{y}} = \mathbf{F}(t, \mathbf{y})$ 。Jacobian 为 $\mathbf{J} = \partial \mathbf{F} / \partial \mathbf{y}$ 。固定求值时刻时， R, T_∞, C_{eq} 是外部驱动，不对状态求导。

对内部热面，记 $\Delta T = T_{i+1} - T_i$ 。调和系数偏导为

$$\frac{\partial k_H}{\partial k_i} = \frac{2k_{i+1}^2}{(k_i + k_{i+1})^2}, \quad \frac{\partial k_H}{\partial k_{i+1}} = \frac{2k_i^2}{(k_i + k_{i+1})^2}\tag{A.44}$$

由此，热面量对四个相邻状态的偏导是

$$\begin{aligned}\partial_{T_i} g^T &= -\gamma_i k_H, \partial_{T_{i+1}} g^T = \gamma_i k_H, \\ \partial_{C_i} g^T &= \gamma_i \Delta T \frac{2k_{i+1}^2 k_i'}{(k_i + k_{i+1})^2}, \partial_{C_{i+1}} g^T = \gamma_i \Delta T \frac{2k_i^2 k_{i+1}'}{(k_i + k_{i+1})^2}\end{aligned}\quad (\text{A.45})$$

其中 $k_i' = dk(C_i)/dC_i$ 。本题 k 不显式依赖温度，故热通量的温度导数来自温差，浓度导数来自导热系数。

对水面量定义 $G_i = \gamma_i D_0 e^{-b/\bar{T}_i}$ 、 $\Delta\Phi = \Phi_a(C_{i+1}) - \Phi_a(C_i)$ 。常规正值势差分支下

$$\begin{aligned}\partial_{C_i} g^C &= -G_i e^{-a/C_i}, \partial_{C_{i+1}} g^C = G_i e^{-a/C_{i+1}}, \\ \partial_{T_i} g^C &= g^C \frac{b}{2T_i^2}, \partial_{T_{i+1}} g^C = g^C \frac{b}{2T_i^2}\end{aligned}\quad (\text{A.46})$$

温度偏导不能因采用面温度近似而省去；两侧温度各以 $1/2$ 进入 $\bar{T}_i$ 。Q1 的 $b = 0$ ，相应热对水耦合偏导才为零。

近等浓度分支必须对实际用到的 $f_a(\bar{C}_i)\Delta C$ 求导，得到

$$\begin{aligned}\partial_{C_i} \Delta\Phi &= \frac{1}{2} f_a'(\bar{C}_i) \Delta C - f_a(\bar{C}_i), \\ \partial_{C_{i+1}} \Delta\Phi &= \frac{1}{2} f_a'(\bar{C}_i) \Delta C + f_a(\bar{C}_i), \\ f_a'(C) &= \frac{a}{C^2} e^{-a/C}\end{aligned}\quad (\text{A.47})$$

在系数保护区，势差与物性关于浓度的偏导还需乘对应节点的活跃标记 $\mathbf{1}_{C_i > 10^{-12}}$ 。表面驱动使用原状态 $C_N - C_{eq}$ ，其下述边界偏导不因此变为零。保护阈值处属于分段数值规则，不是处处光滑的物理本构；Jacobian 采用与右端相同的分支，不能用另一表达式的导数替代。

热方程除面量外还有依赖 C_i 的分母 B_i 。设 $H_i = g_{i+1/2}^T - g_{i-1/2}^T$ ，则

$$\partial_{C_i} \dot{T}_i = \frac{\partial_{C_i} H_i}{R^2 w_i B_i} - \dot{T}_i \frac{B_i'}{B_i}, B_i' = \rho_{\text{eff}}'(C_i) c_p(C_i) + \rho_{\text{eff}}(C_i) c_p'(C_i) \quad (\text{A.48})$$

物性导数由题给表达式直接求得：

$$\begin{aligned}\text{Q1:} & (\rho_{\text{eff}}' c_p', k') = (0, 0, 0), \\ \text{Q23:} & (\rho_{\text{eff}}' c_p', k') = (128, \frac{2736}{(1+C)^2}, \frac{0.38}{(1+C)^2}), \\ \text{Q4:} & (\rho_{\text{eff}}' c_p', k') = (90, \frac{2150}{(1+C)^2}, \frac{0.20}{(1+C)^2})\end{aligned}\quad (\text{A.49})$$

将每个面偏导向左、右控制体按正、负号加入，再乘各自的 $1/(R^2 w_i B_i)$ 或 $1/(R^2 w_i)$ ；热方程同时添加式 (A.48) 的容量项。由此得到邻近节点的 2×2 块带结构。

基线表面与累计量导数为

$$\partial_{T_N} g_s^T = -hR, \partial_{C_N} g_s^C = -\beta R, \partial_{C_N} \dot{\ell} = \frac{2\beta}{R} \quad (\text{A.50})$$

累计量 ℓ 不反馈温湿场，因此 Jacobian 对 ℓ 的整列为零。令行向量 $\mathbf{m}^T$ 在每个 C_i 分量取 $2w_i$ 、在 ℓ 分量取 1、其余取零，则式 (A.41) 的微分结构要求

$$\mathbf{m}^T \mathbf{F} = 0, \mathbf{m}^T \mathbf{J} = 0^T \quad (\text{A.51})$$

前者检查共享通量的质量装配，后者检查其导数装配。方向差分检验采用 $[\mathbf{F}(\mathbf{y} + \varepsilon \mathbf{v}) - \mathbf{F}(\mathbf{y} - \varepsilon \mathbf{v})]/(2\varepsilon)$ 与 $\mathbf{J}\mathbf{v}$ 对照；必须同时覆盖浓度、温度、边界、累计量及近等浓度分支，不能只检查对角项。

9.1.12 隐式 BDF 与连续数值输出

空间加密使扩散算子的快速尺度约随 $(R\Delta x)^{-2}$ 增大，状态相关扩散率又带来不同的局部时间尺度。系统因此采用隐式 BDF 推进。以当前状态和若干历史状态的插值多项式在当前时刻求导，可得一般后向差分残量

$$\mathbf{G}(\mathbf{y}_n) = \alpha_0 \mathbf{y}_n + \sum_{j=1}^q \alpha_j \mathbf{y}_{n-j} - \Delta t_n \beta_q \mathbf{F}(t_n, \mathbf{y}_n) = 0 \quad (\text{A.52})$$

q 为阶数， α_j, β_q 是该历史时间布局对应的系数。等步长二阶仅作为原理示例：

$$\frac{3\mathbf{y}_n - 4\mathbf{y}_{n-1} + \mathbf{y}_{n-2}}{2\Delta t} = \mathbf{F}(t_n, \mathbf{y}_n) \quad (\text{A.53})$$

对式 (A.52) 作 Newton 线性化，每次修正求解

$$(\alpha_0 \mathbf{I} - \Delta t_n \beta_q \mathbf{J}) \delta \mathbf{y} = -\mathbf{G} \quad (\text{A.54})$$

实际采用 SciPy BDF 的 1—5 阶自适应实现及 NDF 精度修正；式 (A.53) 不表示全程固定二阶。隐式多步常微分方程求解的算法背景见 Byrne 与 Hindmarsh[6]，当前 BDF 的阶数、误差控制与 Jacobian 接口依 SciPy 官方说明[7]及本次使用版本源码核对。变步长、变阶时的历史差分更新、误差估计和失败重试由库实现，不能将固定等步长系数用于任意不等间距历史点。

时间局部误差按分量尺度归一化：

$$s_j = \text{atol}_j + \text{rtol} |y_j| \quad (\text{A.55})$$

正式参数为 $\text{rtol} = 10^{-10}$ ，温度绝对容差 10^{-10}K ，水分与累计失水绝对容差 10^{-12} 。前 4 h 最大步长 2 s，之后 120 s；程序在 14400 s 处切段，并用上一段接受末态作为下一段初态。这些参数控制时间积分局部误差，不能当作全时空场误差的严格上界。

BDF 每个接受步带有连续输出多项式。以一个接受步的右端 t_n 、多项式步长尺度 h_p 、阶数 q 和后向差分系数向量 $\mathbf{D}_0, \dots, \mathbf{D}_q$ 表示，其求值形式为

$$\mathbf{P}_n(t) = \mathbf{D}_0 + \sum_{j=1}^q \mathbf{D}_j \prod_{k=0}^{j-1} \frac{t - (t_n - kh_p)}{(k+1)h_p} \quad (\text{A.56})$$

式 (A.56) 中的差分系数和尺度是求解器为该步输出对象保存的量，不应另从稀疏采样反推。`diskDense.py` 保存原始双精度系数与接受状态，并使用同一求值公式；只

改变存储位置。BDF 内部断点恰好落在查询点时，保留与 SciPy 原 BDF 一致的右侧多项式选择；跨显式分段边界则保持 `Run.state` 的段选择规则。

题定每 1 s 或 60 s 的数值直接从尚未关闭的连续数值轨迹查询。保存的 60 s 等审查用 NPZ 并不是逐秒输出的插值来源。连续输出多项式是离散 ODE 解的数值表示，不是偏微分方程的解析解。

9.1.13 全域最大含水率、临界根与严格报告时刻

对连续模型定义

$$M(t) = \max_{0 \leq r \leq R(t)} C(r, t) = \max_{0 \leq x \leq 1} \hat{C}(x, t) \quad (\text{A.57})$$

$M(t)$ 表示时刻 t 最湿位置的干基含水率，单位仍为 kg/kg。题目要求各处含水率均低于 $C_* = 0.15$ ，等价于 $M(t) < C_*$ ；平均含水率低于阈值不能替代这一条件，也不应在求解前只指定中心为唯一检查点。

数值计算实际采用全部 $N + 1$ 个节点的最大值

$$M_N(t) = \max_{0 \leq i \leq N} C_i(t), \quad g_N(t) = M_N(t) - C_* \quad (\text{A.58})$$

节点间作分段线性重构时，每段值都是两个端点的凸组合，因此重构场的最大值恰等于 M_N 。这使数值判据覆盖整个重构空间域，但不等于证明真实连续场与重构场的最大值完全相同。二者差异仍需空间、时间误差检查；若需要真实连续场的严格可行保证，还需可用的统一误差界。

以 g_N 从正到负的第一次穿零定位临界时刻 $t_{c,N}$ 。在临界附近连续下降时， $M_N(t_{c,N}) = C_*$ 对应等号，严格可行集合的下确界为该根，严格集合本身未必具有最小时间点。最大值函数在控制节点切换时可以不可微，但连续性仍使包围区间上的标量事件定位有意义；不要求对最大值求解析导数。

正文式(14)对应指定报告网格上求严格可行的首点。本附录以离散数值模型明确写为

$$\begin{aligned} \mathcal{K}_N &= \{k \in N_0: k\Delta t_r \geq t_{c,N}, M_N(k\Delta t_r) < C_*\}, \\ t_{r,N} &= \Delta t_r \min \mathcal{K}_N, \Delta t_r = 0.36 \text{ s} = 10^{-4} \text{ h} \end{aligned} \quad (\text{A.59})$$

N_0 为非负整数集合， $\mathcal{K}_N$ 为满足时间与严格含水率条件的整数指标集合。所有集合内比较使用未舍入的数值解， $t_{c,N}$, $t_{r,N}$, Δt_r 统一以秒计算。该定义在所搜索区间内有可行点时才成立；未触发事件不能把预设计算上限当作烘干时长。

程序先从临界状态实际续算至

$$t_{\text{end}} = \Delta t_s \lceil \lceil \frac{t_{c,N}}{\Delta t_s} \rceil + 1 \rceil, \Delta t_s = 1 \text{ s}, \quad (\text{A.60})$$

再生成报告候选首点

$$t_r^{(0)} = \Delta t_r \lceil \frac{t_{c,N}}{\Delta t_r} \rceil, t_r^{(j)} = t_r^{(0)} + j\Delta t_r \quad (\text{A.61})$$

从 $j = 0$ 起逐点查询 M_N ，第一个严格小于 0.15 的点作为 $t_{r,N}$ ；若查询超出已实际积分的 t_{end} ，程序报错，不外推状态。 t_{end} 是保守的事件后整数秒核对点，不宣称是最早严格可行整数秒； $t_{r,N}$ 才是按 0.0001 h 精度报告的时长。若根恰在报告格点且该点仍为等号，必须继续到下一格点。

现行正式轨迹给出问题三 $t_{c,N}/3600 = 57.47230195056044$ h、 $t_{r,N}/3600 = 57.4724$ h；问题四相应为 51.09057478683054 h 和 51.0906 h。四位最近舍入得到的临界根显示值，不能替代式 (A.59) 的严格可行时刻。表格中含水率显示 0.1500 也不直接否定未舍入值小于 0.15；不通过手改末位改变判定。

9.1.14 固定物理位置查询与缩域输出

给定物理半径 r_j ，先在每个查询时刻计算 $x_j(t) = r_j/R(t)$ 。若 $0 \leq x_j \leq 1$ 且 $x_i \leq x_j \leq x_{i+1}$ ，使用

$$\hat{C}_N(x_j, t) = (1 - \eta)C_i(t) + \eta C_{i+1}(t), \eta = \frac{x_j - x_i}{\Delta x} \quad (\text{A.62})$$

温度查询采用同样的空间重构，时间值先由式 (A.56) 所在轨迹查询。 $r_j > R(t)$ 时该位置已不在药材内，输出留空，不能填零或外推。真实表面另查 $x = 1$ ，不能用最近的 0.1 cm 固定列替代。代码对仅由浮点边界换算造成的约 10^{-12} m 容差内归属差异按表面值一致处理，不把它扩展成物理域外预测。

完整逐秒或逐分钟工作簿、正文采样表与达标时刻均源自同一条轨迹。最后才转换 $K \rightarrow ^\circ\text{C}$ 、 $s \rightarrow \text{h}$ ，并进行四位显示舍入。问题二和问题三复用 Q23 的同一个 Run，避免场表和事件时间来源不同。

9.1.15 圆柱 Robin - Bessel 解析基准

常系数圆柱扩散的特征函数方法可参见 Crank[4]。本节只对问题一常热物性热方程展开当前时变边界下的推导，令 $\alpha = k/B$ 、 $x = r/R_0$ 、 $v(x, t) = T(x, t) - T_\infty(t)$ 。由于外界温度在空间均匀，有

$$\begin{aligned} v_t &= \frac{\alpha}{R_0^2} \frac{1}{x} (xv_x)_x - \dot{T}_\infty(t), \\ v_x(0, t) &= 0, v_x(1, t) + Bi_h v(1, t) = 0, \\ Bi_h &= \frac{hR_0}{k} \end{aligned} \quad (\text{A.63})$$

Bi_h 为该常系数热问题的 Biot 数。空间特征方程为 $(xX')' + \mu^2 xX = 0$ ，轴心有界的解取第一类零阶 Bessel 函数 $X(x) = J_0(\mu x)$ ；第二类函数在轴心发散而舍去。利用 $J_0'(z) = -J_1(z)$ ，表面 Robin 条件给出特征根

$$\mu_n J_1(\mu_n) = Bi_h J_0(\mu_n), \lambda_n = \frac{\alpha \mu_n^2}{R_0^2}, n = 1, 2, \dots \quad (\text{A.64})$$

λ_n 为正的模态衰减率，单位 s^{-1} 。

本题 Bi_h 有限且大于零，设 $j_{v,n}$ 为 J_v 的第 n 个正零点，根的计算可在下列区间分别包围并求解：

$$0 < \mu_1 < j_{0,1}, j_{1,n-1} < \mu_n < j_{0,n} (n \geq 2) \quad (\text{A.65})$$

该区间选择与现行解析基准源码一致，避免用等距扫描漏根。绝热 $Bi_h = 0$ 时包含零模态，均匀初态保持不变，应单独处理；不能把这一极限直接代入只列正根的展开流程。对两个不同正根，将特征方程交叉相乘、相减并积分，边界项因同一 Robin 条件消失，得到带圆柱权重 x 的正交性：

$$\int_0^1 x J_0(\mu_m x) J_0(\mu_n x) dx = 0 (m \neq n) \quad (\text{A.66})$$

均匀函数的投影系数需要以下两项积分：

$$\int_0^1 x J_0(\mu_n x) dx = \frac{J_1(\mu_n)}{\mu_n}, \int_0^1 x J_0(\mu_n x)^2 dx = \frac{J_0(\mu_n)^2 + J_1(\mu_n)^2}{2} \quad (\text{A.67})$$

因此常数 1 的加权展开系数为

$$b_n = \frac{2J_1(\mu_n)}{\mu_n [J_0(\mu_n)^2 + J_1(\mu_n)^2]} \quad (\text{A.68})$$

令 $v = \sum_n a_n(t) J_0(\mu_n x)$ ，将式 (A.63) 投影到每一特征函数，得

$$\dot{a}_n + \lambda_n a_n = -b_n \dot{T}_\infty(t), a_n(0) = b_n [T_0 - T_\infty(0)] \quad (\text{A.69})$$

使用积分因子求解该一阶常微分方程，得到时变边界的圆柱热场基准

$$T(x, t) = T_\infty(t) + \sum_{n=1}^{\infty} b_n J_0(\mu_n x) \left[[T_0 - T_\infty(0)] e^{-\lambda_n t} - \int_0^t e^{-\lambda_n(t-\tau)} \dot{T}_\infty(\tau) d\tau \right] \quad (\text{A.70})$$

这里的解析基准包含附件给定时变温度的卷积，不是恒温阶跃公式。外界温度在一段 $[a, b]$ 内线性变化、斜率为 s 时，该段在 $t \geq b$ 的卷积贡献为

$$\int_a^b e^{-\lambda_n(t-\tau)} s d\tau = \frac{s}{\lambda_n} [e^{-\lambda_n(t-b)} - e^{-\lambda_n(t-a)}] \quad (\text{A.71})$$

若 t 落在当前段，则把上限截为 $b = t$ ；尚未开始的段不贡献。分段线性导数在节点处的跳变不影响该积分；若要推广到边界值本身跳变的时刻，必须另外处理对应阶跃，不能只积普通斜率。

对固定半径且常扩散率 D_* 的水分验证特例，同样令 $Bi_m = \beta R_0/D_*$ 、 $\lambda_n = D_* \mu_n^2/R_0^2$ ，将 T, T_∞, T_0 替换为 C, C_{eq}, C_0 即可构造相同类型的线性基准。这个特例只用于检验算法，不替代正式 $D_1(C)$ 非线性水分结果，也不能验证 Q23/Q4 的变物性收缩问题。

9.1.16 解析比较的点值、单元平均及误差范围

本文生产未知量按节点点值解释，主要比较 T_i 与 $T(x_i, t)$ 。若为了诊断另外比较解析环体平均，应明确使用

$$\begin{aligned}\langle J_0 \rangle_i &= \frac{1}{w_i} \int_{a_i}^{b_i} x J_0(\mu x) dx \\ &= \frac{b_i J_1(\mu b_i) - a_i J_1(\mu a_i)}{\mu w_i}\end{aligned}\quad (\text{A.72})$$

式 (A.72) 与节点点值 $J_0(\mu x_i)$ 不是同一个对象。尤其表面控制体为单侧区间，若梯度非零，平均值与端点值之差可为 $O(\Delta x)$ ；不能把这种表示差异直接称为物理误差。

对指定公共采样集合 $\mathcal{S}$ ，定义点值最大差和均方根差

$$\begin{aligned}E_\infty &= \max_{(t,r) \in \mathcal{S}} |U_N(t, r) - U_{\text{ref}}(t, r)|, \\ E_{\text{RMS}} &= \sqrt{\frac{1}{|\mathcal{S}|} \sum_{(t,r) \in \mathcal{S}} [U_N(t, r) - U_{\text{ref}}(t, r)]^2}\end{aligned}\quad (\text{A.73})$$

U 可表示温度或含水率， U_{ref} 是明确指定的解析解或更细数值轨迹。解析级数实际截为有限项，需另比较 120、240、480 等截断项数；截断差估计不自动成为全空间全时间的严格上界。正文 N3200 常热物性温度与解析基准在 1801 个整数秒、21 个半径点上的最大差约为 $1.804 \times 10^{-6} \text{K}$ ，只支持该比较范围内的实现一致性。

早期及本轮独立复核中，冻结扩散率的 N400 密集样点最大水分误差为 $2.875083008730961 \times 10^{-4}$ ，超过当时 10^{-4} 门槛；N800 对应约 $7.158659870798445 \times 10^{-5}$ 。程序外层退出 0 不改变该内部不通过结果。另一个历史脚本得到的点值/环体平均差随 N800→6400 逐次约减半，而其静态解释称“不随细化减少”；本附录按实际数值与式 (A.72) 解释，不采纳该错误文字结论。

9.1.17 方法比较、收缩尺度与参数敏感性

在不同空间网格间比较时，采用相同物性、环境、边界、初态及时间判据，在共同物理位置上求差；问题四排除较小当前域之外的位置并单列真实表面。若观察到幂律误差 $E_N \approx KN^{-p}$ ，两倍加密下可计算观察阶

$$p_{\text{obs}} = \frac{\ln(E_N/E_{2N})}{\ln 2} \quad (\text{A.74})$$

该式仅在误差度量一致且进入渐近区时具有通常的阶数解释；若只有两条未知误差数值解，则它们的差本身并非真实误差。正式正文分别采用 Q1/Q23 的 N1600/N3200 和 Q4 的 N3200/N6400 比较，并在最细网格保持空间设置不变、同时收紧容差和最大步长检查时间敏感性。本轮没有为本附录新增正式 PDE 重算；旧全秒空间投影未全部保留的限制继续保留，不将摘要差值冒充重新全量回放。

对另一守恒水分面格式，可将相邻 D_i, D_{i+1} 的调和平均乘浓度差，与式 (A.38) 对比。常 D 时两式退化为同一个线性面量；变量 D 时差别属于空间离散。现行

method_comparison.py--quick 的 8 条 Q1 轨迹实际完成，变量 D 的 N100 与 N200 两格式最大差约为 1.4891×10^{-5} 和 3.7288×10^{-6} ，观察阶约 1.9977。历史粗网格调和格式严重偏离时长的记录不能用来否定调和平均在所有问题中的适用性，但足以说明本题不能采用未加密检查的粗网格答案。

从式 (A.28) 看，在给定参考扩散率 D_{ref} 下，内部扩散自然累积尺度为

$$\tau(t) = \int_0^t \frac{D_{\text{ref}}}{R(s)^2} ds, \tilde{\tau}(t) = R_0^2 \int_0^t \frac{1}{R(s)^2} ds \quad (\text{A.75})$$

τ 无量纲， $\tilde{\tau}$ 的单位为秒。该换算只揭示几何缩短路径的尺度，Robin 边界中的 R 、随状态变化的物性和温湿反馈同时存在，所以不能由 R^2 比例直接得出精确全过程时长。使用仿真事件作积分上限的尺度核对也不构成独立求得的解析烘干时间。

分离收缩影响时，应固定附录 4 的全部物性、初态、环境和数值设置，只改变 $R(t)$ 。正文已有 N200 对照：定半径临界时长约 129.8452 h，给定收缩约 51.0910 h。问题三与问题四的正式时长差约 6.3818 h 同时含物性与几何变化，不能全部归因于收缩。对照网格 N200 用于所列机理比较，不替代 Q4 N6400 正式结果。

参数敏感性必须真正进入正在使用的面通量。以扩散前因子倍率 s_D 、导热率倍率 s_k 为例，应分别将式 (A.38) 中的 D_0 和热面中的 k 同步改变。只修改未被 Kirchhoff 分支读取的节点 D 数组，可能使所谓扩散参数扰动实际没有进入 RHS，不能作为有效敏感性结果。已采用的修复后 N200 单参数扫描表明， s_D 从 0.7 增至 1.4 时，问题三临界时长由约 79.4187 h 降至 43.0796 h，问题四由 70.4059 h 降至 38.3160 h； k 在 $\pm 15\%$ 范围内变化时，两问时长跨度均小于 0.011%。这些是指定工况的一维参数扫描，不能扩大为完整全局敏感性排序，未完成的 Sobol 和旧无效 Morris 注入不采纳。

9.1.18 经验边界阻力及潜热情景的推导范围

为展示气固映射未标定的影响，可在 $C_s \geq C_{\text{ref}} = 0.05$ 、 $p \geq 1$ 的限定下定义如下经验关系。分式适用于 $C_s > C_{\text{ref}}$ ；在 $C_s = C_{\text{ref}}$ 处按连续延拓定义 $K_p(C_{\text{ref}}) = 1/p$ ，不直接计算 0/0：

$$\begin{aligned} K_p(C_s) &= \frac{1 - (C_{\text{ref}}/C_s)^{1/p}}{1 - C_{\text{ref}}/C_s} (C_s > C_{\text{ref}}), \\ K_p(C_{\text{ref}}) &= \frac{1}{p}, \\ j_{w,\text{out}} &= \rho_d \beta K_p(C_s) (C_s - C_{\text{eq}}) \end{aligned} \quad (\text{A.76})$$

p 为无量纲经验参数。取 $z = C_{\text{ref}}/C_s \in (0,1]$ 、 $q = 1/p \in (0,1]$ 。由 $z^q \geq z$ 与凹函数 z^q 在 1 处的切线不等式 $z^q \leq 1 + q(z - 1)$ ，得到

$$\begin{aligned} q(1 - z) &\leq 1 - z^q \leq 1 - z, \\ \frac{1}{p} &\leq K_p(C_s) \leq 1, K_1(C_s) = 1, \\ \lim_{C_s \downarrow C_{\text{ref}}} K_p(C_s) &= \frac{1}{p} \end{aligned} \quad (\text{A.77})$$

这个族包含原边界并可控地减弱所设传质驱动，但不是由实测材料等温线推得的唯一边界。界限仅在上述 C_s, p 范围成立。正文历史 N800 的 $p = 1, 2, 4$ 情景保留为条件数值比较，不据几个参数点证明所有 p 下时长单调，也不能作为真实烘干时长下界。当前旧经验闭合源码存在参数接口和结果数组方向问题，本轮 `sandbox` 没有把该批历史情景重新完整跑通； $p=1$ 代数恒等性检查不能替代其原入口复现。

若另指定比例 χ 的外流水在表面汽化，汽化潜热为 L_v ，则热边界梯度面量改为

$$g_s^T = hR(T_\infty - T_s) - R\chi L_v \rho_d \beta (C_s - C_{eq}) \quad (\text{A.78})$$

$\chi = 0$ 恢复正式基线， L_v 单位为 J/kg；加入 K_p 情景时，蒸发质量项也须一致乘 $K_p(C_s)$ 。固定求值时刻下，不含 K_p 的潜热项对 C_s 的偏导为 $-R\chi L_v \rho_d \beta$ ，应加入表面热行，累计失水方程保持与实际水边界一致。这类情景约定了蒸发位置和比例，但没有识别真实气膜传递、凝结、吸附热或内部相变，不能声称补入一个潜热项就完成总热力学闭合。

9.2 程序源码

以下列出核心计算、数值检验及 MATLAB 对照程序。核心计算程序位于支撑材料的“03_程序代码”；Python 检验程序位于“05_数值检验与实验/Python 检验源码/paper_output/code”；MATLAB 对照程序位于“05_数值检验与实验/MATLAB 对照证据”。相同计算逻辑的核心模块仅列出一份。

表 9 核心计算程序及用途

编号	程序文件	主要用途
1	runDelivery.py	统一调用各问模型并导出计算结果。
2	q1Model.py	设置第一问参数并调用热湿耦合求解器。
3	q2Model.py	设置第二问参数并计算变物性干燥过程。
4	q3Model.py	按含水率阈值计算满足要求的干燥时间。
5	q4Model.py	设置收缩参数并计算移动边界干燥过程。
6	dryingCore.py	离散热湿耦合方程并求解温度和含水率。
7	analyticJacobian.py	计算耦合方程雅可比矩阵，辅助隐式求解。
8	diskDense.py	缓存连续数值解，支持任意时刻插值读取。
9	exportOutputs.py	整理计算数据并生成各问所需结果表格。
10	reference/reference_data.py	保存参考数值，供程序对照检查计算输出。
11	runLogged.ps1	启动求解程序并记录运行过程和退出状态。

表 10 Python 数值检验程序及用途

编号	程序文件	主要用途
12	axisymmetric_check.py	建立轴对称模型，考察端面传递的影响。
13	compare_analytic.py	比较数值解与贝塞尔级数解的误差。
14	production_provenance.py	记录计算来源并管理结果文件的统一输出。
15	publication_plots.py	绘制温湿分布、干燥曲线和检验图。
16	run_experiments.py	批量设置网格、时间和物理参数对照实验。
17	run_modeling.py	组织各问计算并汇总数值结果与图表。
18	validate_bessel.py	计算圆柱扩散问题的贝塞尔级数参考解。
19	verify_convergence.py	比较不同网格的温湿分布和干燥时间。
20	verify_dense_storage.py	核对内存与磁盘存储的连续解一致性。
21	verify_time_accuracy.py	调整时间步长与容差，检验时间离散误差。
22	analytic_metric_check.py	比较解析节点值与控制体平均值的差异。
23	crossvalidate_series.py	分析环境和干燥序列的趋势与阶段变化。

续表 10 Python 数值检验程序及用途

编号	程序文件	主要用途
24	crossvalidate_solver.py	比较求解算法、网格及边界处理的影响。
25	energy_balance_check.py	检验离散热量平衡及累计能量残差。
26	energy_balance_diagnose.py	逐项核对热通量与温度变化率的对应关系。
27	energy_balance_diagnose2.py	对照热量变化与边界积分，分析能量残差。
28	isotherm_activity_closure.py	引入水分活度关系，考察边界闭合的影响。
29	isotherm_diagnose.py	检查活度边界下的方程变化率及局部响应。
30	isotherm_jacobian_probe.py	改变差分步长，检查活度模型求解稳定性。
31	latent_heat_scenarios.py	改变表面潜热比例，比较干燥过程变化。
32	method_comparison.py	比较水分通量离散方法及其数值差异。
33	sensitivity_analysis.py	扰动模型参数，分析干燥时间的敏感性。
34	threshold_and_scaling_checks.py	检验干燥阈值、尺度关系与求解稳定性。

表 11 MATLAB 对照程序及用途

编号	程序文件	主要用途
35	runCrossCheck.m	用 MATLAB 独立实现热湿模型并进行对照。
36	compareMatlab.mjs	对照 MATLAB 与 Python 的温湿场和时间。

9.2.1 runDelivery.py

```
from __future__ import annotations
import argparse
import csv
from contextlib import redirect_stderr, redirect_stdout
from datetime import datetime, timezone
import gc
import hashlib
import importlib
import importlib.metadata
import json
import os
from pathlib import Path
import re
import sys
import time
import traceback
import warnings

ROOT = Path(__file__).resolve().parent
os.environ['OPENBLAS_NUM_THREADS'] = '1'
os.environ['OMP_NUM_THREADS'] = '1'
sys.dont_write_bytecode = True
for key in ['TMPDIR', 'TEMP', 'TMP']:
    os.environ[key] = str(ROOT/'tmp/cache/runtime')
(ROOT/'tmp/cache/runtime').mkdir(parents=True, exist_ok=True)

def utcNow():
    return datetime.now(timezone.utc).isoformat()

def writeJson(path, value):
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False)+'\n',
encoding='utf-8')

def sha256(path):
    digest = hashlib.sha256()
    with path.open('rb') as stream:
        for block in iter(lambda: stream.read(1024*1024), b''):
            digest.update(block)
    return digest.hexdigest()

def fileRecord(path):
    return {'path': path.relative_to(ROOT).as_posix(), 'bytes': path.stat().st_size, 'sha256':
sha256(path)}

class Tee:
    def __init__(self, screen, log):
        self.screen, self.log = screen, log
    def write(self, text):
        self.screen.write(text)
        self.log.write(text)
        self.flush()
        return len(text)
    def flush(self):
        self.screen.flush()
```

```

        self.log.flush()

def preflight():
    if sys.maxsize <= 2**32:
        raise RuntimeError('Use a 64-bit Python interpreter.')
    with (ROOT/'input_manifest.csv').open('r', encoding='utf-8-sig', newline='') as stream:
        manifest = list(csv.DictReader(stream))
    checked = []
    for expected in manifest:
        path = (ROOT/expected['path']).resolve()
        if not path.is_relative_to(ROOT) or not path.is_file():
            raise FileNotFoundError(expected['path'])
        actual = fileRecord(path)
        if actual['sha256'] != expected['sha256']:
            raise AssertionError('Input/reference checksum mismatch: '+expected['path'])
        checked.append(actual)
    import dryingCore
    import exportOutputs
    env, rad = dryingCore.loadInputs()
    if len(env) != 241 or len(rad) != 145:
        raise AssertionError('Observed input row counts differ from audited attachments')
    templates = {qid: exportOutputs.loadTemplate(qid)['sheet_names'] for qid in ['Q1', 'Q2',
'Q3', 'Q4']}
    return {'status': 'PASS', 'observedRows': {'environment':len(env), 'radius':len(rad)},
        'templates': templates, 'checkedFiles': checked}

def compareReference(run, question, summary, output):
    import numpy as np
    from reference.reference_data import REFERENCE_DATA
    reference = REFERENCE_DATA['questions'][question]
    tolerances = {'T_K':5e-6, 'C':5e-7, 'mean_C':5e-7, 'cumulative_loss':5e-7,
        'radius_m':1e-10, 'event_s':0.005}
    checks = []

    with np.load(ROOT/'reference'/question/'sampled_solution.npz', allow_pickle=False) as
frozen:
        ts = frozen['times_s']
        event = reference['diagnostics']['event_s']
        mask = ts < min(run.endS, reference['diagnostics']['end_s'])-1e-5
        if event is not None:
            mask &= np.abs(ts-event) > 1e-5
        selected = ts[mask]
        values = {key: [] for key in ['T_K', 'C', 'mean_C', 'cumulative_loss', 'radius_m']}
        for first in range(0, len(selected), 128):
            times = selected[first:first+128]
            T, C = run.fields(times, materialX=frozen['material_x'])
            raw = run.state(times)
            values['T_K'].append(T)
            values['C'].append(C)
            values['mean_C'].append(2*run.model.w@raw[1:-1:2])
            values['cumulative_loss'].append(raw[-1])
            values['radius_m'].append(run.model.radius(times))
        for key, arrays in values.items():
            actual = np.concatenate(arrays, axis=0)
            expected = frozen[key][mask]
            difference = float(np.max(np.abs(actual-expected)))
            if not np.isfinite(actual).all() or difference > tolerances[key]:
                raise AssertionError(f'Frozen reference mismatch {question}/{key}:

```

```

{difference}')
    checks.append({'array':key, 'valuesCompared':int(actual.size),
'maximumAbsoluteDifference':difference,
                  'absoluteTolerance':tolerances[key], 'status':'PASS'})
    with np.load(output/'sampled_solution.npz', allow_pickle=False) as actual_npz:
        exact = {key:bool(actual_npz[key].shape == frozen[key].shape and
                           np.array_equal(actual_npz[key], frozen[key], equal_nan=True)) for key in
frozen.files}
    sample = reference['reproduction_samples']
    T, C = run.fields(sample['times_s'], materialX=sample['material_x'])
    for key, actual in [(('T_K', T), ('C', C))]:
        difference = float(np.max(np.abs(actual-np.asarray(sample[key]))))
        if difference > tolerances[key]:
            raise AssertionError('Fixed reproduction sample mismatch: '+question+'/'+key)
        checks.append({'array':'reproduction_samples/'+key,
'maximumAbsoluteDifference':difference,
                  'absoluteTolerance':tolerances[key], 'status':'PASS'})
    if question != 'Q1':
        difference = abs(run.eventS-reference['diagnostics']['event_s'])
        if difference > tolerances['event_s']:
            raise AssertionError('Continuous event mismatch')
        completion = summary['completion']
        if completion['reported_drying_time_h'] !=
reference['completion']['reported_drying_time_h']:
            raise AssertionError('Strict report time moved to a different 0.0001-hour grid point')
        if not completion['max_C_at_reported_time'] < .15:
            raise AssertionError('Reported state does not satisfy the strict threshold')
        checks.append({'array':'event_s', 'maximumAbsoluteDifference':difference,
                  'absoluteTolerance':tolerances['event_s'], 'status':'PASS',
                  'strictReportGridMatches':True, 'strictUnroundedThreshold':True})
    return {'status':'PASS', 'source':'final_v6a', 'checks':checks,
            'supplementaryExactEqualityOfAllSavedArrays':exact,
            'scope':'Numerical reproducibility, not physical prediction accuracy'}

def runQuestion(question, intervals, output, profile):
    import dryingCore
    import exportOutputs
    import q3Model
    name = {'Q1':'q1Model', 'Q23':'q2Model', 'Q4':'q4Model'}[question]
    run = None
    try:
        print(f'{utcNow()} SOLVE {question} N={intervals}', flush=True)
        with warnings.catch_warnings(record=True) as caught:
            warnings.simplefilter('always')
            run = importlib.import_module(name).solve(intervals)
        if caught:
            raise RuntimeError('Solver warning: '+'; '.join(str(w.message) for w in caught))
        directory = output/question
        summary = dryingCore.saveRun(run, directory)
        summary['solverWarningCount'] = 0
        if question != 'Q1':
            summary['completion'] = q3Model.completion(run)
            if not run.diagnostics()['strictly_dry_at_end']:
                raise AssertionError('Post-event state is not dry')
        writeJson(directory/'summary.json', summary)
        comparison = compareReference(run, question, summary, directory) if profile == 'final'
    else {
        'status':'NOT_APPLICABLE', 'reason':'N40 quick checks are not the fine-grid numerical
result'}
    exports = []
    for qid in (['Q2', 'Q3'] if question == 'Q23' else [question]):
        print(f'{utcNow()} EXPORT_AND_FULL_READBACK {qid}', flush=True)

```

```

        artifact = exportOutputs.exportQuestion(run, qid, output/'outputs')
        validation = exportOutputs.validateExports([artifact], runs={qid:run})
        if validation['status'] != 'PASS' or not validation['fully_verified_with_live_Run']:
            raise AssertionError('Workbook/archive/live Run verification failed')
        exports.append(validation)
        print(f'{utcNow()} VERIFIED {qid}')
    cells={validation["exports"][0]["checked_cells"]}, flush=True)
    checks = {'referenceComparison':comparison, 'exports':exports}
    writeJson(directory/'comparison.json', checks)
    return summary, checks
finally:
    if run is not None:
        run.close()
    gc.collect()
    print(f'{utcNow()} RELEASED {question}', flush=True)

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--profile', choices=['quick', 'final'], default='quick')
    parser.add_argument('--run-id', default=None)
    parser.add_argument('--preflight-only', action='store_true')
    args = parser.parse_args()
    run_id = args.run_id or
args.profile+'_'+datetime.now(timezone.utc).strftime('%Y%m%dT%H%M%SZ')
    if not re.fullmatch(r'[A-Za-z0-9][A-Za-z0-9_-]{0,79}', run_id):
        parser.error('run-id may contain only letters, digits, underscore, hyphen')
    output = ROOT/'results'/run_id
    output.mkdir(parents=True, exist_ok=False)
    started = time.perf_counter()
    sources = [fileRecord(path) for path in sorted(ROOT.glob('*.py'))]
    report = {'status':'RUNNING', 'profile':args.profile, 'runId':run_id,
'startedAtUtc':utcNow(),
        'guiReproduced':'NOT_PERFORMED', 'humanReview':'PENDING_USER_REVIEW',
        'environment':{'python':sys.version,
'libraries':{'n:importlib.metadata.version(n)
            for n in ['numpy', 'scipy', 'openpyxl']}, 'cpuThreads':1},
        'sources':sources, 'randomSeed':None, 'deterministic':True,

'historical107FileRegression':'NOT_RUN_NOT_REQUIRED_FOR_INDEPENDENT_REPRODUCTION'}
    with (output/'stdout.log').open('w', encoding='utf-8') as log:
        with redirect_stdout(Tee(sys.stdout, log)), redirect_stderr(Tee(sys.stderr, log)):
            try:
                report['preflight'] = preflight()
                print(f'{utcNow()} PREFLIGHT PASS profile={args.profile}', flush=True)
                if not args.preflight_only:
                    report['summaries'], report['comparisons'] = {}, {}
                    for q, n in [('Q1', 3200), ('Q23', 3200), ('Q4', 6400)]:
                        report['summaries'][q], report['comparisons'][q] = runQuestion(
                            q, 40 if args.profile == 'quick' else n, output, args.profile)
                    if sources != [fileRecord(path) for path in sorted(ROOT.glob('*.py'))]:
                        raise AssertionError('Source changed during run')
                    report['status'] = 'PASS'
                    print(f'{utcNow()} ALL_REQUESTED_CHECKS_COMPLETED', flush=True)
            except BaseException as error:
                report.update(status='FAIL', error=repr(error))
                traceback.print_exc()
                raise
            finally:
                report.update(finishedAtUtc=utcNow(),
elapsedSeconds=time.perf_counter()-started)
                writeJson(output/'runResult.json', report)

    return 0

```

```
if __name__ == '__main__':
    raise SystemExit(main())
```

9.2.2 q1Model.py

```
from dryingCore import Settings, solveCase
```

```
def solve(intervals=3200):
    return solveCase(Settings(question='Q1', intervals=intervals, faceScheme='kirchhoff',
        rtol=1e-10, atolTemperature=1e-10, atolMoisture=1e-12,
        earlyMaxStepS=2., maxStepS=120.))
```

9.2.3 q2Model.py

```
from dryingCore import Settings, solveCase
```

```
def solve(intervals=3200):
    return solveCase(Settings(question='Q23', intervals=intervals, faceScheme='kirchhoff',
        rtol=1e-10, atolTemperature=1e-10, atolMoisture=1e-12,
        earlyMaxStepS=2., maxStepS=120.))
```

9.2.4 q3Model.py

```
import math
import numpy as np
```

```
def completion(run):
    if run.eventS is None:
        raise ValueError('No full-domain drying event within the solved horizon')

    count = math.ceil(run.eventS / 3600. * 10000.)

    while True:
        reportH = count/10000.
        t = reportH*3600.
        if t > run.endS:
            raise RuntimeError('Four-decimal reporting time is outside the verified trajectory')
        values = run.state([t])[1:-1:2,0]
        if float(np.max(values)) < .15:
            break
        count += 1
    return {'critical_event_s':run.eventS,'critical_event_h':run.eventS/3600.,
        'reported_drying_time_h':reportH,'reported_time_s':t,
        'max_C_at_reported_time':float(np.max(values)),
        'slowest_material_coordinate':float(run.model.x[np.argmax(values)]),
        'conservative_post_verification_s':run.endS,
        'rounding_convention':'upward on 0.0001 h grid, followed by actual strict threshold
check',
        'interpretation':'A conditional numerical event, not a confidence bound on physical
drying time.'}
```

9.2.5 q4Model.py

```
from dryingCore import Settings, solveCase
```

```
def solve(intervals=6400):
    return solveCase(Settings(question='Q4', intervals=intervals, faceScheme='kirchhoff',
        shrink=True, rtol=1e-10, atolTemperature=1e-10, atolMoisture=1e-12,
        earlyMaxStepS=2., maxStepS=120.))
```

9.2.6 dryingCore.py

```
from __future__ import annotations

from dataclasses import asdict, dataclass
from pathlib import Path
import hashlib
import io
import json
import time

import numpy as np
from scipy.integrate import solve_ivp
from scipy.sparse import lil_matrix
from scipy.special import expi
import analyticJacobian
import diskDense

ROOT = Path(__file__).resolve().parent
LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
C0, T0, R0, LENGTH = 2.55, 301.15, 0.02, 0.25

def loadInputs(withRecords=False):
    arrays, records = [], []
    for name in ['A_environment_observed.csv', 'A_radius_observed.csv']:
        path = ROOT / 'inputs/cleaned' / name
        content = path.read_bytes()
        arrays.append(np.genfromtxt(io.StringIO(content.decode('utf-8-sig')),
            delimiter=',', names=True))
        records.append({'path':path.relative_to(ROOT).as_posix(), 'bytes':len(content),
            'sha256':hashlib.sha256(content).hexdigest(), 'exists':True})
    return (*arrays, records) if withRecords else tuple(arrays)

@dataclass(frozen=True)

class Settings:
    question: str = 'Q23'
    intervals: int = 100
    rtol: float = 1e-7
    atolTemperature: float = 1e-7
    atolMoisture: float = 1e-9
    maxStepS: float = 600.0
    earlyMaxStepS: float = 30.0
    horizonH: float = 240.0
    shrink: bool = False
    boundaryExtension: str = 'nominal'
    tailTemperatureC: float = 50.0
    tailEquilibrium: float = 0.05
    h: float = 25.0
    beta: float = 8e-7
    equilibriumScale: float = 1.0
    surfaceLatentFraction: float = 0.0
    latentJkg: float = 2.4e6
    constantD: float | None = None
    constantThermal: bool = False
    method: str = 'BDF'
```

```

faceScheme: str = 'harmonic'
jacobianMode: str = 'analytic'
denseStorage: str = 'disk'

```

```

class RadialModel:
    def __init__(self, settings: Settings):
        self.settings = settings
        if settings.intervals < 2 or settings.jacobianMode not in ('analytic',
'finite_difference'):
            raise ValueError('At least two intervals and a supported Jacobian mode are required')
        self.env, self.rad, self.inputRecords = loadInputs(withRecords=True)

        self.x = np.linspace(0., 1., settings.intervals + 1)
        self.dx = 1. / settings.intervals
        faces = np.r_[0., (self.x[1:] + self.x[:-1]) / 2., 1.]
        self.w = np.diff(faces ** 2) / 2.
        self.internalFaces = faces[1:-1]
        self.n = len(self.x)
        self.rhoD0 = (760 + 90 * C0) / (1 + C0) if settings.question == 'Q4' else (
            820. / (1 + C0) if settings.question == 'Q1' else (650 + 128 * C0) / (1 + C0))
        self evaluations = 0
        self.jacPattern = self.buildSparsity()

    def radius(self, t):
        if self.settings.shrink:
            return np.interp(t, self.rad['time_s'], self.rad['radius_m'])
        return np.asarray(t) * 0. + R0

    def environment(self, t):
        s = self.settings
        tair = np.interp(t, self.env['time_s'], self.env['temperature_K'])
        ceq = np.interp(t, self.env['time_s'], self.env['air_moisture_kg_per_kg'])
        after = np.asarray(t) > self.env['time_s'][-1]
        if s.boundaryExtension == 'nominal':
            tair = np.where(after, s.tailTemperatureC + 273.15, tair)
            ceq = np.where(after, s.tailEquilibrium, ceq)
        elif s.boundaryExtension == 'tail_mean':
            tail = self.env['time_s'] >= 10800
            tair = np.where(after, self.env['temperature_K'][tail].mean(), tair)
            ceq = np.where(after, self.env['air_moisture_kg_per_kg'][tail].mean(), ceq)
        elif s.boundaryExtension != 'last':
            raise ValueError('Unknown boundary extension')
        return tair, ceq * s.equilibriumScale

    def properties(self, T, C):
        s = self.settings

        positiveC = np.maximum(C, 1e-12)
        if np.any(T <= 0):
            raise FloatingPointError('Nonpositive absolute temperature')
        wet = positiveC / (1. + positiveC)
        if s.question == 'Q1':
            rho = np.full_like(C, 820.)
            cp = np.full_like(C, 2600.)
            k = np.full_like(C, .36)
            D = 7e-9 * np.exp(-.89 / positiveC)
        elif s.question in ('Q2', 'Q3', 'Q23'):
            rho, cp, k = 650 + 128 * positiveC, 1450 + 2736 * wet, .21 + .38 * wet
            D = 2.4e-3 * np.exp(-.45 / positiveC - 3850 / T)
        elif s.question == 'Q4':

```

```

        rho, cp, k = 760 + 90 * positiveC, 1850 + 2150 * wet, .12 + .20 * wet
        D = 4.2e-4 * np.exp(-.30 / positiveC - 3850 / T)
    else:
        raise ValueError(s.question)
    if s.constantD is not None:
        D = np.full_like(C, s.constantD)
    if s.constantThermal:
        rho, cp, k = np.full_like(C, 820.), np.full_like(C, 2600.), np.full_like(C, .36)
    return rho, cp, k, D

@staticmethod
def harmonic(a):
    return 2 * a[:-1] * a[1:] / np.maximum(a[:-1] + a[1:], np.finfo(float).tiny)

def buildSparsity(self):
    p = lil_matrix((2 * self.n + 1, 2 * self.n + 1), dtype=int)
    for i in range(self.n):
        for j in range(max(0, i - 1), min(self.n, i + 2)):
            p[2*i:2*i+2, 2*j:2*j+2] = 1
    p[-1, 2 * (self.n - 1) + 1] = 1
    return p.tocsr()

def waterInternalFlux(self, T, C, D):
    if self.settings.faceScheme == 'harmonic' or self.settings.constantD is not None:
        return self.internalFaces * self.harmonic(D) * np.diff(C) / self.dx
    if self.settings.faceScheme != 'kirchhoff':
        raise ValueError('Unknown nonlinear face scheme')
    a, D0 = {'Q1':(.89,7e-9), 'Q23':(.45,2.4e-3), 'Q2':(.45,2.4e-3),
            'Q3':(.45,2.4e-3), 'Q4':(.30,4.2e-4)}[self.settings.question]
    cc = np.maximum(C, 1e-12)

    potential = cc * np.exp(-a/cc) + a * expi(-a/cc)
    difference = np.diff(potential)
    small = np.abs(np.diff(cc)) < 1e-7 * np.maximum((cc[:-1]+cc[1:])/2, 1e-3)
    difference[small] = (np.exp(-a/((cc[:-1][small]+cc[1:][small])/2)) *
np.diff(cc)[small])

    thermalFactor = 1. if self.settings.question == 'Q1' else
np.exp(-3850/((T[:-1]+T[1:])/2))

    return self.internalFaces * D0 * thermalFactor * difference / self.dx

def rhs(self, t, state):
    self.evaluations += 1
    T, C = state[:-1:2], state[1:-1:2]
    rho, cp, k, D = self.properties(T, C)
    radius = float(self.radius(t))
    tair, ceq = self.environment(t)
    heatG = np.zeros(self.n + 1)
    waterG = np.zeros(self.n + 1)
    heatG[1:-1] = self.internalFaces * self.harmonic(k) * np.diff(T) / self.dx
    waterG[1:-1] = self.waterInternalFlux(T, C, D)

    waterG[-1] = -self.settings.beta * radius * (C[-1] - ceq)
    heatG[-1] = -self.settings.h * radius * (T[-1] - tair)
    if self.settings.surfaceLatentFraction:
        rhoD = self.rhoD0 * (R0 / radius) ** 2
        jEvap = rhoD * self.settings.beta * (C[-1] - ceq)
        heatG[-1] -= (radius * self.settings.surfaceLatentFraction *

```

```

        self.settings.latentJKg * jEvap)
    derivative = np.empty_like(state)

    derivative[:-1:2] = np.diff(heatG) / (radius ** 2 * self.w * rho * cp)
    derivative[1:-1:2] = np.diff(waterG) / (radius ** 2 * self.w)
    derivative[-1] = 2 * self.settings.beta / radius * (C[-1] - ceq)
    return derivative

def initial(self):
    state = np.empty(2 * self.n + 1)
    state[:-1:2], state[1:-1:2], state[-1] = T0, C0, 0.
    return state

class Run:
    def __init__(self, model, pieces, elapsed, eventS, cache=None):
        self.model, self.pieces = model, pieces
        self.cache = cache
        self.elapsedS, self.eventS = elapsed, eventS
        self.endS = float(pieces[-1].t[-1])

    def close(self):
        self.pieces.clear()
        if self.cache is not None:
            self.cache.close()

    def state(self, times):
        tt = np.atleast_1d(np.asarray(times, dtype=float))
        if np.min(tt) < -1e-10 or np.max(tt) > self.endS + 1e-7:
            raise ValueError('Requested time outside solved interval')
        out = np.empty((2 * self.model.n + 1, len(tt)))
        remaining = np.ones(len(tt), dtype=bool)
        for result in self.pieces:
            select = remaining & (tt >= result.t[0]-1e-7) & (tt <= result.t[-1]+1e-7)
            if np.any(select):
                out[:, select] = result.sol(tt[select])
                remaining[select] = False
        if np.any(remaining):
            raise RuntimeError('Missing dense solution segment')
        return out

    def fields(self, times, radiiM=None, materialX=None):
        tt = np.atleast_1d(np.asarray(times, dtype=float))
        state = self.state(tt)
        Ts, Cs = state[:-1:2].T, state[1:-1:2].T
        if materialX is not None:
            points = np.asarray(materialX, dtype=float)
            if np.any(~np.isfinite(points)) or np.any((points < 0.) | (points > 1.)):
                raise ValueError('Material coordinates must be finite and within [0,1]')
            return np.array([np.interp(points, self.model.x, row) for row in Ts]), np.array([
                np.interp(points, self.model.x, row) for row in Cs])
        if radiiM is None:
            return Ts, Cs
        radial = np.asarray(radiiM)
        Tout, Cout = [], []
        for i, t in enumerate(tt):
            xx = radial / self.model.radius(t)
            Tout.append(np.interp(xx, self.model.x, Ts[i], left=np.nan, right=np.nan))
            Cout.append(np.interp(xx, self.model.x, Cs[i], left=np.nan, right=np.nan))
        return np.asarray(Tout), np.asarray(Cout)

```

```

def diagnostics(self):

    minimumC = minimumT = minimumD = minimumProperty = np.inf
    maximumC = maximumT = maximumD = maximumRadialIncrease = -np.inf
    maximumMassResidual = 0.
    for piece in self.pieces:
        for first in range(0, len(piece.t), 256):
            raw = piece.y[:, first:first+256]
            T, C = raw[:-1:2], raw[1:-1:2]
            residual = 2*self.model.w*C + raw[-1] - C0
            rho, cp, k, D = self.model.properties(T.ravel(), C.ravel())
            minimumC, maximumC = min(minimumC, C.min()), max(maximumC, C.max())
            minimumT, maximumT = min(minimumT, T.min()), max(maximumT, T.max())
            minimumD = min(minimumD, D.min())
            maximumD = max(maximumD, D.max())
            minimumProperty = min(minimumProperty, rho.min(), cp.min(), k.min(), D.min())
            maximumRadialIncrease = max(maximumRadialIncrease, np.max(np.diff(C, axis=0)))
            maximumMassResidual = max(maximumMassResidual, np.max(np.abs(residual)))
    final = self.state([self.endS])[:, 0]
    finalC = final[1:-1:2]
    return {
        'event_s': self.eventS, 'event_h': None if self.eventS is None else self.eventS/3600,
        'end_s': self.endS, 'elapsed_s': self.elapsedS,
        'end_time_convention': 'ceil(critical_event_s)+1: conservative post-crossing
verification second; not claimed earliest integer second',
        'max_mass_balance_abs_kg_per_kg': float(maximumMassResidual),
        'min_C': float(minimumC), 'max_C': float(maximumC),
        'min_T_K': float(minimumT), 'max_T_K': float(maximumT),
        'min_D': float(minimumD), 'max_D': float(maximumD),
        'positive_properties': bool(minimumProperty > 0),
        'max_radial_C_increase': float(maximumRadialIncrease),
        'final_max_C': float(finalC.max()), 'final_surface_C': float(finalC[-1]),
        'strictly_dry_at_end': bool(finalC.max() < .15),
        'radius_end_m': float(self.model.radius(self.endS)),
        'radius_extrapolation_used': bool(self.model.settings.shrink and self.endS >
259200),
        'rhs_evaluations': self.model.evaluations,
        'accepted_time_points': sum(len(p.t) for p in self.pieces),
        'nfev': sum(p.nfev for p in self.pieces),
        'njev': sum(p.njev for p in self.pieces), 'nlu': sum(p.nlu for p in self.pieces),
        'solver_success': all(p.success for p in self.pieces),
        'dense_storage': self.model.settings.denseStorage,
        'dense_coefficient_bytes': 0 if self.cache is None else self.cache.bytesWritten,
        'dense_polynomial_count': 0 if self.cache is None else self.cache.polynomialCount,
    }

def solveCase(settings: Settings) -> Run:
    started = time.perf_counter()
    if settings.denseStorage not in ('memory', 'disk'):
        raise ValueError('Dense storage must be memory or disk')
    if settings.denseStorage == 'disk' and settings.method != 'BDF':
        raise ValueError('Exact disk dense storage currently supports BDF only')
    cache = diskDense.DenseCache(ROOT) if settings.denseStorage == 'disk' else None
    try:
        return solveCaseImpl(settings, cache, started)
    except BaseException as error:
        if cache is not None:
            try:
                cache.close()
            except BaseException as cleanup_error:

```

```

        error.add_note('Private cache cleanup also failed: '+repr(cleanup_error))
    raise

def solveCaseImpl(settings, cache, started):
    model = RadialModel(settings)
    method = diskDense.DiskBDF if cache is not None else settings.method
    jacobianOptions = ({'jac': lambda t, y: analyticJacobian.jacobian(model, t, y)}
        if settings.jacobianMode == 'analytic' else {'jac_sparsity': model.jacPattern})
    if cache is not None:
        jacobianOptions['dense_cache'] = cache

    def dryEvent(t, y):
        return float(np.max(y[1:-1:2]) - .15)
    dryEvent.terminal, dryEvent.direction = True, -1
    atol = np.empty(2 * model.n + 1)
    atol[1:-1:2], atol[1:-1:2], atol[-1] = settings.atolTemperature, settings.atolMoisture,
    settings.atolMoisture
    horizon = 1800. if settings.question == 'Q1' else settings.horizonH * 3600.
    pieces, initial, eventS = [], model.initial(), None

    endpoints = [0., min(14400., horizon)]
    if horizon > 14400.:
        endpoints.append(horizon)
    for left, right in zip(endpoints[:-1], endpoints[1:]):
        piece = solve_ivp(model.rhs, (left, right), initial, method=method,
            rtol=settings.rtol, atol=atol, **jacobianOptions,
            max_step=settings.earlyMaxStepS if left < 14400. else settings.maxStepS,
            events=None if settings.question == 'Q1' else dryEvent, dense_output=True)
        if cache is not None:
            diskDense.alignBdfSegments(piece)
        pieces.append(piece)
        if not piece.success:
            raise RuntimeError(piece.message)
        initial = piece.y[:, -1].copy()
        if cache is not None:
            piece.y = cache.storeAccepted(piece.y)
        if piece.t_events is not None and len(piece.t_events[0]):
            eventS = float(piece.t_events[0][0])

        end = float(np.ceil(eventS) + 1)
        tail = solve_ivp(model.rhs, (eventS, end), initial, method=method,
            rtol=settings.rtol, atol=atol, **jacobianOptions,
            max_step=1., dense_output=True)
        if cache is not None:
            diskDense.alignBdfSegments(tail)
        if not tail.success:
            raise RuntimeError(tail.message)
        if cache is not None:
            tail.y = cache.storeAccepted(tail.y)
        pieces.append(tail)
        break
    run = Run(model, pieces, time.perf_counter() - started, eventS, cache)
    diagnostic = run.diagnostics()
    if diagnostic['min_C'] < -1e-8 or not diagnostic['positive_properties']:
        raise FloatingPointError('Physical range/positive property check failed')
    if diagnostic['max_mass_balance_abs_kg_per_kg'] > 1e-6:
        raise FloatingPointError('Dry-basis mass balance failed')
    return run

def fileRecord(path):

```

```

p = Path(path)
return {'path': p.relative_to(ROOT).as_posix(), 'bytes': p.stat().st_size,
        'sha256': hashlib.sha256(p.read_bytes()).hexdigest(), 'exists': True}

def saveRun(run: Run, directory: Path):
    codeRecord = fileRecord(Path(__file__))
    if codeRecord['sha256'] != LOADED_CODE_SHA256:
        raise RuntimeError('Solver file changed after import; restart to obtain valid
provenance')
    jacobianRecord = fileRecord(Path(analyticJacobian.__file__))
    if jacobianRecord['sha256'] != analyticJacobian.LOADED_CODE_SHA256:
        raise RuntimeError('Jacobian file changed after import; restart to obtain valid
provenance')
    storageRecord = fileRecord(Path(diskDense.__file__))
    if storageRecord['sha256'] != diskDense.LOADED_CODE_SHA256:
        raise RuntimeError('Dense storage file changed after import; restart for valid
provenance')
    for record in run.model.inputRecords:
        if fileRecord(ROOT/record['path'])['sha256'] != record['sha256']:
            raise RuntimeError('Input changed after being loaded; retain failure and rerun')
    directory.mkdir(parents=True, exist_ok=True)
    times = np.unique(np.r_[np.arange(0., run.endS, 60.),
        [t for t in [100., 300., 600., 900., 1200., 1500., 1800., 3600., 5400., 7200.,
9000., 10800.] if t <= run.endS],
        run.endS, [] if run.eventS is None else [run.eventS]])
    x = np.linspace(0., 1., 21)
    temperature, moisture, means, losses = [], [], [], []
    for first in range(0, len(times), 128):
        blockTimes = times[first:first+128]
        Tb, Cb = run.fields(blockTimes, materialX=x)
        raw = run.state(blockTimes)
        temperature.append(Tb); moisture.append(Cb)
        means.append(2*run.model.w@raw[1:-1:2]); losses.append(raw[-1].copy())
    T, C = np.vstack(temperature), np.vstack(moisture)
    np.savez_compressed(directory/'sampled_solution.npz', times_s=times, material_x=x,
        T_K=T, C=C, radius_m=run.model.radius(times), mean_C=np.concatenate(means),
        cumulative_loss=np.concatenate(losses))
    summary = {'settings': asdict(run.model.settings), 'diagnostics': run.diagnostics(),
        'code': codeRecord,
        'jacobian_code': jacobianRecord,
        'dense_storage_code': storageRecord,
        'inputs': run.model.inputRecords,
        'human_review_status': 'pending', 'gui_reproduced': False}
    (directory/'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2),
encoding='utf-8')
    return summary

```

9.2.7 analyticJacobian.py

```

from __future__ import annotations

import argparse
from dataclasses import asdict
from datetime import datetime, timezone
import hashlib
import json
from pathlib import Path
import sys
import time
import warnings

import numpy as np

```

```

from scipy.integrate import solve_ivp
from scipy.sparse import coo_matrix
from scipy.special import expi

LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

def harmonicPartials(left, right):

    total = left + right
    tiny = np.finfo(float).tiny
    denominator = np.maximum(total, tiny)
    ordinary = total > tiny
    dl = np.where(ordinary, 2 * (right / denominator) ** 2, 2 * right / denominator)
    dr = np.where(ordinary, 2 * (left / denominator) ** 2, 2 * left / denominator)
    return dl, dr

def jacobian(model, t, y):

    state = np.asarray(y, dtype=float)
    n = model.n
    if state.ndim != 1 or state.size != 2*n + 1:
        raise ValueError("Jacobian requires interleaved 1D state of length 2*n+1")
    s = model.settings
    T, C = state[:-1:2], state[1:-1:2]
    cc = np.maximum(C, 1e-12)
    active = (C > 1e-12).astype(float)
    rho, cp, k, D = model.properties(T, C)
    if s.question == 'Q1':
        a, d0, tempConstant = .89, 7e-9, 0.
        rhoC, cpC, kC = np.zeros(n), np.zeros(n), np.zeros(n)
    elif s.question in ('Q2', 'Q3', 'Q23'):
        a, d0, tempConstant = .45, 2.4e-3, 3850.
        rhoC = np.full(n, 128.) * active
        cpC = 2736. / (1. + cc)**2 * active
        kC = .38 / (1. + cc)**2 * active
    elif s.question == 'Q4':
        a, d0, tempConstant = .30, 4.2e-4, 3850.
        rhoC = np.full(n, 90.) * active
        cpC = 2150. / (1. + cc)**2 * active
        kC = .20 / (1. + cc)**2 * active
    else:
        raise ValueError(s.question)
    if s.constantThermal:
        rhoC, cpC, kC = np.zeros(n), np.zeros(n), np.zeros(n)
    capacity = rho * cp
    capacityC = rhoC * cp + rho * cpC
    if s.constantD is None:
        dC = D * (a / cc**2) * active
        dT = D * tempConstant / T**2
    else:
        dC, dT = np.zeros(n), np.zeros(n)

    radius = float(model.radius(t))
    if radius <= 0:
        raise ValueError("Nonpositive radius")
    tair, ceq = model.environment(t)
    geom = model.internalFaces / model.dx
    deltaT, deltaC = np.diff(T), np.diff(C)

```

```

kHarm = model.harmonic(k)
khL, khR = harmonicPartials(k[:-1], k[1:])

heatDeriv = np.column_stack((
    -geom * kHarm,
    geom * khL * kC[:-1] * deltaT,
    geom * kHarm,
    geom * khR * kC[1:] * deltaT,
))
heatG = np.zeros(n + 1)
heatG[1:-1] = geom * kHarm * deltaT
heatG[-1] = -s.h * radius * (T[-1] - tair)

if s.faceScheme == 'harmonic' or s.constantD is not None:
    dh = model.harmonic(D)
    dhL, dhR = harmonicPartials(D[:-1], D[1:])
    waterDeriv = np.column_stack((
        geom * dhL * dT[:-1] * deltaC,
        geom * (dhL * dC[:-1] * deltaC - dh),
        geom * dhR * dT[1:] * deltaC,
        geom * (dhR * dC[1:] * deltaC + dh),
    ))
elif s.faceScheme == 'kirchhoff':
    primitive = cc * np.exp(-a / cc) + a * expi(-a / cc)
    primitiveDelta = np.diff(primitive)
    meanC = (cc[:-1] + cc[1:]) / 2.
    meanT = (T[:-1] + T[1:]) / 2.
    deltaCc = np.diff(cc)
    small = np.abs(deltaCc) < 1e-7 * np.maximum(meanC, 1e-3)
    expLeft, expRight = np.exp(-a/cc[:-1]), np.exp(-a/cc[1:])
    primitiveL = -expLeft * active[:-1]
    primitiveR = expRight * active[1:]

    if np.any(small):
        fMid = np.exp(-a / meanC[small])
        fPrimeMid = fMid * a / meanC[small]**2
        primitiveDelta[small] = fMid * deltaCc[small]
        primitiveL[small] = (0.5*fPrimeMid*deltaCc[small]-fMid)*active[:-1][small]
        primitiveR[small] = (0.5*fPrimeMid*deltaCc[small]+fMid)*active[1:][small]
        temperatureFactor = np.exp(-tempConstant / meanT)
        coefficient = geom * d0 * temperatureFactor
        waterG = coefficient * primitiveDelta
        waterT = waterG * tempConstant / (2 * meanT**2)
        waterDeriv = np.column_stack((waterT, coefficient*primitiveL,
                                         waterT, coefficient*primitiveR))
    else:
        raise ValueError('Unknown nonlinear face scheme')

heatScale = 1. / (radius**2 * model.w * capacity)
waterScale = 1. / (radius**2 * model.w)
surfaceHeatC = 0.
if s.surfaceLatentFraction:
    rhoD = model.rhoD0 * (.02 / radius)**2
    surfaceHeatC = -radius*s.surfaceLatentFraction*s.latentJKg*rhoD*s.beta
    heatG[-1] += surfaceHeatC * (C[-1] - ceq)
tempDerivative = np.diff(heatG) * heatScale

faceIndex = np.arange(n-1)
faceColumns = np.column_stack((2*faceIndex, 2*faceIndex+1,
                                2*faceIndex+2, 2*faceIndex+3)).ravel()
rows, columns, values = [], [], []

```

```

def addFace(rowIndex, derivatives, factor):
    rows.append(np.repeat(rowIndex, 4))
    columns.append(faceColumns)
    values.append((derivatives * factor[:, None]).ravel())

addFace(2*faceIndex, heatDeriv, heatScale[:-1])
addFace(2*faceIndex+2, heatDeriv, -heatScale[1:])
addFace(2*faceIndex+1, waterDeriv, waterScale[:-1])
addFace(2*faceIndex+3, waterDeriv, -waterScale[1:])
cellIndex = np.arange(n)
rows.append(2*cellIndex)
columns.append(2*cellIndex+1)

values.append(-tempDerivative * capacityC / capacity)
rows.append(np.array([2*n-2, 2*n-2, 2*n-1, 2*n]))
columns.append(np.array([2*n-2, 2*n-1, 2*n-1, 2*n-1]))
values.append(np.array([-s.h*radius*heatScale[-1],
                        surfaceHeatC*heatScale[-1],
                        -s.beta*radius*waterScale[-1], 2*s.beta/radius]))

matrix = coo_matrix((np.concatenate(values),
                    (np.concatenate(rows), np.concatenate(columns))),
                    shape=(2*n+1, 2*n+1)).tocsc()
matrix.sum_duplicates()
matrix.eliminate_zeros()
return matrix

def selfTest(outputDirectory):

    import scipy
    from dryingCore import ROOT, Settings, RadialModel, LOADED_CODE_SHA256 as
CORE_LOADED_CODE_SHA256

    started = time.perf_counter()
    rng = np.random.default_rng(20260910)
    relativeTolerance, absoluteTolerance = 5e-6, 5e-10
    cases = []
    for question in ['Q1', 'Q23', 'Q4']:
        for scheme in ['harmonic', 'kirchhoff']:
            for latent in [0., 1.]:
                cases.append(Settings(question=question, intervals=8, faceScheme=scheme,
                                    shrink=(question=='Q4'), surfaceLatentFraction=latent))
    for question in ['Q1', 'Q23', 'Q4']:
        for constantDiffusivity, constantThermal in [(2e-9, False), (None, True), (2e-9, True)]:
            cases.append(Settings(question=question, intervals=8, faceScheme='kirchhoff',
                                shrink=(question=='Q4'), constantD=constantDiffusivity,
                                constantThermal=constantThermal, surfaceLatentFraction=1.))
    records, failures = [], []
    for settings in cases:
        model = RadialModel(settings)
        x = model.x
        states = {
            'initial': (0., model.initial()),
            'nonuniform': (18000., model.initial()),
            'late_dry': (150000., model.initial()),
            'near_uniform_small_branch': (14401., model.initial()),
        }
        states['nonuniform'][1][:-1:2] = 303. + 17.*x**2
        states['nonuniform'][1][1:-1:2] = 2.4 - 2.0*x**2
        states['late_dry'][1][:-1:2] = 321. + 2.0*x**2
        states['late_dry'][1][1:-1:2] = .175 - .115*x**2
        states['near_uniform_small_branch'][1][:-1:2] = 303. + 17.*x**2

```

```

states['near_uniform_small_branch'][1][1:-1:2] = .15 + 1e-10*x
for stateName, (t, y) in states.items():
    with warnings.catch_warnings(record=True) as captured:
        warnings.simplefilter('always')
        matrix = jacobian(model, t, y)
        vectors = []
        for _ in range(4):
            direction = rng.normal(size=y.size)
            direction[:-1:2] *= 1.0
            direction[1:-1:2] *= .02
            direction[-1] = .3
            vectors.append(direction)
        directional = []
        for direction in vectors:
            predicted = matrix @ direction
            steps = []
            for step in [1e-4, 3e-5, 1e-5]:
                finiteDifference = (model.rhs(t, y+step*direction)-
                                   model.rhs(t, y-step*direction))/(2*step)
                absoluteError = float(np.max(np.abs(predicted-finiteDifference)))
                scale = max(float(np.max(np.abs(predicted))),
                           float(np.max(np.abs(finiteDifference))), 1e-30)
                scaled = float(np.max(np.abs(predicted-finiteDifference)/
                                         (absoluteTolerance + relativeTolerance*np.maximum(
                                             np.abs(predicted), np.abs(finiteDifference)))))
                steps.append({'step':step, 'max_abs_error':absoluteError,
                             'relative_inf_error':absoluteError/scale,
                             'max_component_tolerance_ratio':scaled})
            best = min(steps, key=lambda item:item['max_component_tolerance_ratio'])
            directional.append({'all_step_errors':steps, 'best':best})
        passive = np.zeros(y.size); passive[-1] = 1.
        passiveAnalytic = float(np.max(np.abs(matrix @ passive)))
        passiveFd = float(np.max(np.abs(model.rhs(t,
y+passive)-model.rhs(t,y-passive))))
        massWeights = np.zeros(y.size)
        massWeights[1:-1:2] = 2*model.w
        massWeights[-1] = 1.
        conservation = float(np.max(np.abs(np.asarray(massWeights @ matrix))))
        warningMessages = [str(w.message) for w in captured]
        passed = (all(step['max_component_tolerance_ratio'] <= 1
                      for d in directional for step in d['all_step_errors'])
                  and passiveAnalytic == 0 and passiveFd == 0 and conservation < 1e-11
                  and np.isfinite(matrix.data).all() and not warningMessages)
        record = {'settings':asdict(settings), 'state':stateName, 'time_s':t,
                  'status':'PASS' if passed else 'FAIL', 'matrix_shape':matrix.shape,
                  'matrix_nnz':matrix.nnz, 'directional_checks':directional,
                  'passive_column_analytic_abs':passiveAnalytic,
                  'passive_column_finite_difference_abs':passiveFd,
                  'mass_balance_derivative_abs':conservation, 'warnings':warningMessages}
        records.append(record)
    if not passed:
        failures.append(f"{settings.question}/{settings.faceScheme}/latent={settings.surfaceLatentF
raction}/{stateName}/constant_D={settings.constantD}/constant_thermal={settings.constantThe
rmal}")
    smokeRecords = []
    for scheme in ['harmonic', 'kirchhoff']:
        for question in ['Q23', 'Q4']:
            settings = Settings(question=question, intervals=8, faceScheme=scheme,
                               shrink=(question=='Q4'), surfacelLatentFraction=1.)
            model = RadialModel(settings)
            with warnings.catch_warnings(record=True) as captured:
                warnings.simplefilter('always')

```

```

        result = solve_ivp(model.rhs, (0., 2.), model.initial(), method='BDF',
                           jac=lambda t,y:jacobian(model,t,y), rtol=1e-10,
                           atol=1e-12, max_step=.2)
    msgs = [str(w.message) for w in captured]
    passed = bool(result.success and np.isfinite(result.y).all() and not msgs)
    smokeRecords.append({'question':question, 'face_scheme':scheme,
'interval_s':[0,2],
                        'status':'PASS' if passed else 'FAIL', 'warnings':msgs,
                        'nfev':result.nfev, 'njev':result.njev, 'nlu':result.nlu,
                        'purpose':'Short Jacobian/BDF wiring check, not production
accuracy'}})
    if not passed:
        failures.append(f"BDF_smoke/{question}/{scheme}")
        currentSourceHash = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
        currentCoreHash = hashlib.sha256((ROOT/'dryingCore.py').read_bytes()).hexdigest()
        if currentSourceHash != LOADED_CODE_SHA256 or currentCoreHash != CORE_LOADED_CODE_SHA256:
            failures.append('Source file changed after module load during validation')
        report = {'schema_version':'1.0', 'created_utc':datetime.now(timezone.utc).isoformat(),
                  'status':'PASS' if not failures else 'FAIL', 'failures':failures,
                  'source_sha256':LOADED_CODE_SHA256,
                  'core_sha256':CORE_LOADED_CODE_SHA256,
                  'source_hash_policy':'SHA-256 frozen when each module is imported; files checked
unchanged before report save.',

'runtime':{'python':sys.version,'executable':sys.executable,'numpy':np.__version__,'scipy':
scipy.__version__},
            'seed':20260910,'relative_component_tolerance':relativeTolerance,
            'absolute_component_tolerance':absoluteTolerance,
            'finite_difference_policy':'Central directional differences, three decreasing
steps; every step must satisfy the mixed absolute/relative component tolerance. Best comparison
is supplementary only.',
            'case_state_count':len(records), 'direction_count':4*len(records),
            'checks':records,'short_BDF_checks':smokeRecords,
            'elapsed_s':time.perf_counter()-started,
            'limitations':['No full production run or grid convergence performed here.',
                           'Tiny denominator/underflow extensions at nonphysical Newton probes
are not calibrated physical data.',
                           'Only this module and the short tests use analytic Jacobian until main
agent hooks core.',
                           'Visual Studio reproduction and human review remain pending.']}
        outputDirectory.mkdir(parents=True, exist_ok=True)
        (outputDirectory/'analytic_jacobian_selftest.json').write_text(
            json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
        brief = {key:report[key] for key in
['status','failures','case_state_count','direction_count','elapsed_s']}
        brief['max_best_component_tolerance_ratio'] = max(
            d['best']['max_component_tolerance_ratio'] for r in records for d in
r['directional_checks'])
        brief['max_all_steps_component_tolerance_ratio'] = max(
            step['max_component_tolerance_ratio'] for r in records
            for d in r['directional_checks'] for step in d['all_step_errors'])
        brief['max_mass_balance_derivative_abs'] = max(r['mass_balance_derivative_abs'] for r in
records)
        brief['warning_count'] = sum(len(r['warnings']) for r in records+smokeRecords)
        print(json.dumps(brief,ensure_ascii=False,indent=2))
        return 0 if not failures else 1

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--self-test', action='store_true')
    parser.add_argument('--output-dir', type=Path,
                        default=Path(__file__).resolve().parent/'results/jacobianValidation')

```

```

arguments = parser.parse_args()
if not arguments.self_test:
    parser.error('Use --self-test, or import jacobian(model,t,y) from this module.')
raise SystemExit(selfTest(arguments.output_dir))

```

9.2.8 diskDense.py

```

from __future__ import annotations
import hashlib
from pathlib import Path
import tempfile
import numpy as np
from scipy.integrate import OdeSolution
from scipy.integrate._ivp.bdf import BDF, BdfDenseOutput
from scipy.integrate._ivp.base import DenseOutput

LOADED_CODE_SHA256 = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

class DenseCache:
    def __init__(self, root):
        self.parent = (Path(root)/'tmp/cache/solver_runs').resolve()
        if not self.parent.is_relative_to(Path(root).resolve()):
            raise ValueError('Private solver cache must remain in this project')
        self.parent.mkdir(parents=True, exist_ok=True)
        self.directory = Path(tempfile.mkdtemp(prefix='bdf_', dir=self.parent)).resolve()
        try:
            self.handle = (self.directory/'polynomials.bin').open('w+b', buffering=0)
        except BaseException as error:
            try:
                self.directory.rmdir()
            except OSError as cleanup_error:
                error.add_note('Private cache directory cleanup failed: '+repr(cleanup_error))
            raise
        self.bytesWritten = 0
        self.polynomialCount = 0
        self.acceptedArrays = []
        self.closed = False

    def append(self, values):
        if self.closed:
            raise RuntimeError('Dense solution cache is closed')
        array = np.ascontiguousarray(values, dtype=np.float64)
        offset = self.bytesWritten
        self.handle.seek(offset)
        array.tofile(self.handle)
        self.bytesWritten += array.nbytes
        self.polynomialCount += 1
        return offset, array.shape

    def read(self, offset, shape):
        if self.closed:
            raise RuntimeError('Dense solution cache is closed')
        count = int(np.prod(shape))
        self.handle.seek(offset)
        values = np.fromfile(self.handle, dtype=np.float64, count=count)
        if values.size != count:
            raise IOError('Incomplete BDF coefficient cache')
        return values.reshape(shape)

    def storeAccepted(self, values):
        path = self.directory/f'accepted_{len(self.acceptedArrays):03d}.npy'

```

```

        mapped = np.lib.format.open_mmap(path, mode='w+', dtype=values.dtype,
shape=values.shape)

        self.acceptedArrays.append(mapped)
        mapped[:] = values
        mapped.flush()
        return mapped

def close(self):
    if self.closed:
        return
    self.handle.close()
    for array in self.acceptedArrays:
        array._mmap.close()
    self.acceptedArrays.clear()

    if self.directory.parent != self.parent or not self.directory.name.startswith('bdf_'):
        raise RuntimeError('Unexpected private cache path; cleanup refused')
    for path in self.directory.iterdir():
        if not path.is_file() or path.is_symlink():
            raise RuntimeError('Unexpected cache entry; cleanup refused')
        path.unlink()
    self.directory.rmdir()
    self.closed = True

class FileBdfDenseOutput(DenseOutput):
    def __init__(self, original, cache):
        super().__init__(original.t_old, original.t)
        self.order = original.order
        self.t_shift = original.t_shift.copy()
        self.denom = original.denom.copy()
        self.cache = cache
        self.offset, self.shape = cache.append(original.D)

    def _call_impl(self, t):

        dense = object.__new__(BdfDenseOutput)
        dense.D = self.cache.read(self.offset, self.shape)
        dense.t_shift, dense.denom = self.t_shift, self.denom
        return dense._call_impl(t)

class DiskBDF(BDF):
    def __init__(self, *args, dense_cache, **kwargs):
        self.dense_cache = dense_cache
        super().__init__(*args, **kwargs)

    def _dense_output_impl(self):
        return FileBdfDenseOutput(super()._dense_output_impl(), self.dense_cache)

def alignBdfSegments(result):

    if result.sol is not None:
        result.sol = OdeSolution(result.sol.ts, result.sol.interpolants, alt_segment=True)

```

9.2.9 exportOutputs.py

```

from __future__ import annotations

```

```

import argparse
import csv
from dataclasses import asdict
from datetime import datetime, timezone
import gzip
import hashlib
import inspect
import json
import math
from pathlib import Path
import re
import sys
import tempfile
import time

import numpy as np
from openpyxl import Workbook, load_workbook
from openpyxl.cell import WriteOnlyCell
from openpyxl.styles import Alignment, Font, PatternFill
from openpyxl.utils import get_column_letter

ROOT = Path(__file__).resolve().parent
SOURCE = Path(__file__).resolve()
LOADED_EXPORTER_SHA256 = hashlib.sha256(SOURCE.read_bytes()).hexdigest()
TEMPLATE_DIR = ROOT / 'inputs/templates'
RADII_M = np.arange(21, dtype=float)*0.001
RADII_CM = [i/10 for i in range(21)]
NUMBER_FORMAT = '0.0000'
SIZE_LIMIT_BYTES = 20_000_000

def sha256File(path):
    h=hashlib.sha256()
    with Path(path).open('rb') as fh:
        for block in iter(lambda:fh.read(1024*1024),b''):
            h.update(block)
    return h.hexdigest()

def exportFileRecord(path, role=None):
    path=Path(path).resolve()
    result={'path':path.relative_to(ROOT).as_posix(),'absolute_path':str(path),
           'bytes':path.stat().st_size,'sha256':sha256File(path),'exists':True}
    if role:
        result['role']=role
    return result

def writeJson(path,value):
    path.write_text(json.dumps(value,ensure_ascii=False,indent=2,allow_nan=False)+'\n',encoding
='utf-8')

def normalizeQuestionId(questionId):
    value=str(questionId).upper()
    if value in ['1','2','3','4']:
        value='Q'+value
    if value not in ['Q1','Q2','Q3','Q4']:
        raise ValueError('Export requires Q1, Q2, Q3 or Q4')
    return value

```

```

def outputTimes(questionId,end,event):
    if questionId=='Q1':
        if end<1800-1e-7:
            raise ValueError('Q1 requires a solved interval covering 0..1800s')
        return np.arange(1801,dtype=float)
    if questionId=='Q2':
        if abs(end-round(end))>1e-7:
            raise ValueError('Q2 needs an integer post-crossing verification endpoint')
        return np.arange(int(round(end))+1,dtype=float)
    if event is None:
        raise ValueError('Q3/Q4 final export requires an actual critical drying event')
    if not 0<=event<=end:
        raise ValueError('Critical event must lie inside the solved interval')
    return np.unique(np.r_[np.arange(math.floor(end/60)+1,dtype=float)*60,event,end])

def roundFourDecimals(x):
    if x is None:
        return None
    if not math.isfinite(float(x)):
        raise ValueError('A non-finite value cannot be exported as a numerical result')
    return float(round(float(x),4))

def loadTemplate(questionId):
    path=TEMPLATE_DIR/f'result{questionId[-1]}.xlsx'
    wb=load_workbook(path,read_only=True,data_only=False)
    try:
        header=next(wb.worksheets[0].iter_rows(values_only=True))
        names=wb.sheetnames
        return
    except:
        {'record':exportFileRecord(path,'original_result_template'),'time_header':header[0],
         'sheet_names':names,'surface_header':header[-1] if questionId=='Q4' else None}
    finally:
        wb.close()

def runProvenance(run):
    module=sys.modules[run.model.__class__.__module__]
    core=Path(inspect.getfile(run.model.__class__)).resolve()
    loaded=getattr(module,'LOADED_CODE_SHA256',None)
    if loaded is not None and loaded!=sha256File(core):
        raise RuntimeError('Core file changed after this Run implementation was imported')
    if sha256File(SOURCE)!=LOADED_EXPORTER_SHA256:
        raise RuntimeError('Exporter changed after import; restart for consistent provenance')
    inputRecords=list(getattr(run.model,'inputRecords',[]))
    for rec in inputRecords:
        if sha256File(ROOT/rec['path'])!=rec['sha256']:
            raise RuntimeError('Observed input changed after solve: '+rec['path'])
    return
    {'core':exportFileRecord(core,'solver_source'),'exporter':exportFileRecord(SOURCE,'exporter_source'),
     'inputs':inputRecords,'settings':asdict(run.model.settings),
     'event_s':None if run.eventS is None else float(run.eventS),
     'end_s':float(run.endS),'source_kind':'live_Run_dense_solution',
     'per_second_values_are_not_interpolated_from_60s_npz':True}

def fieldBlock(run,times,qid):
    T,C=run.fields(times,radiiM=RADII_M)
    radius=np.asarray(run.model.radius(times),dtype=float)
    if radius.ndim==0:

```

```

        radius=np.full(len(times),radius)
    surfaceC=None
    if qid=='Q4':
        _,surface=run.fields(times,materialX=np.array([1.0]))
        surfaceC=surface[:,0]
        inside=RADII_M[None,:] <= radius[:,None]+1e-12

        nearBoundary=inside & (~np.isfinite(C))
        if np.any(nearBoundary):
            C=np.where(nearBoundary,surfaceC[:,None],C)
            T=np.where(inside,T,np.nan)
            C=np.where(inside,C,np.nan)
            if not np.all(np.isfinite(C[inside])) or not np.all(np.isfinite(surfaceC)):
                raise ValueError('Non-finite moisture inside Q4 domain')
    else:
        inside=np.ones_like(C,dtype=bool)
        if not np.all(np.isfinite(T)) or not np.all(np.isfinite(C)):
            raise ValueError('Non-finite output inside fixed domain')
    return T-273.15,C,radius,surfaceC,inside

def rawHeaders(qid):
    h=['time_s','radius_m']
    if qid in ['Q1','Q2']:
        h += [f'T_C_r_{i/10:.1f}_cm' for i in range(21)]
    h += [f'C_r_{i/10:.1f}_cm' for i in range(21)]
    if qid=='Q4':
        h += ['C_surface']
    return h

def rawRow(qid,t,T,C,R,surface,inside):
    row=[float(t),float(R)]
    if qid in ['Q1','Q2']:
        row+=list(map(float,T))
    row += [float(c) if valid else None for c,valid in zip(C,inside)]
    if qid=='Q4':
        row.append(float(surface))
    return row

def roundedSheets(qid,row):
    t,R=row[:2]
    if qid in ['Q1','Q2']:
        return {'温度':[roundFourDecimals(t)]+[roundFourDecimals(v) for v in row[2:23]],
                '水分浓度':[roundFourDecimals(t)]+[roundFourDecimals(v) for v in row[23:44]]}
    if qid=='Q3':
        return {'Sheet1':[roundFourDecimals(t)]+[roundFourDecimals(v) for v in row[2:23]]}
    return {'Sheet1':[roundFourDecimals(t)]+[roundFourDecimals(v) for v in row[2:24]],
            '半径':[roundFourDecimals(t),roundFourDecimals(R*100)]}

def createSheet(wb,name,header):
    ws=wb.create_sheet(name)
    ws.freeze_panes='B2'
    ws.sheet_view.showGridLines=False
    ws.column_dimensions['A'].width=30
    for j in range(2,len(header)+1):
        ws.column_dimensions[get_column_letter(j)].width=13 if j<len(header) else 16
    ws.row_dimensions[1].height=34
    cells=[]
    for value in header:
        cell=WriteOnlyCell(ws,value=value)

```

```

        cell.font=Font(name='Arial',size=10,bold=True,color='FFFFFF')
        cell.fill=PatternFill(fill_type='solid',fgColor='334155')
        cell.alignment=Alignment(horizontal='center',vertical='center',wrap_text=True)
        if isinstance(value,(int,float)):
            cell.number_format='0.0'
        cells.append(cell)

    cache=ROOT/'tmp/cache/openpyxl_exports'
    cache.mkdir(parents=True,exist_ok=True)
    previousTempdir=tempfile.tempdir
    try:
        tempfile.tempdir=str(cache)
        ws.append(cells)
    finally:
        tempfile.tempdir=previousTempdir
    return ws

def appendNumeric(ws,row):
    cells=[]
    for value in row:
        if value is None:
            cells.append(None)
        else:
            cell=WriteOnlyCell(ws,value=value)
            cell.number_format=NUMBER_FORMAT
            cells.append(cell)
    ws.append(cells)

def paperTables(run,qid,directory):
    if qid=='Q1':
        ts=np.array([100,300,600,900,1200,1500,1800],dtype=float)
    elif qid=='Q2':
        ts=np.arange(1,7,dtype=float)*1800
        if run.endS<10800-1e-7:
            raise ValueError('Q2 paper tables require 3h of actual solution')
    else:
        from q3Model import completion
        reportedEnd=completion(run)['reported_time_s']
        ts=np.unique(np.r_[np.arange(21600.,reportedEnd+1e-8,21600.),reportedEnd])
    rs=np.array([0.,.005,.01,.015,.02])
    T,C=run.fields(ts,radiiM=rs)
    T=T-273.15
    R=np.asarray(run.model.radius(ts))
    headers=['时间/s' if qid=='Q1' else '时间/h']+[0,0.5,1,1.5,2]
    if qid=='Q4':
        headers += ['药材表面']
        _,Cs=run.fields(ts,materialX=[1.])
    outputs=[]
    fields=[('temperature',T),('moisture',C)] if qid in ['Q1','Q2'] else [('moisture',C)]
    for kind,value in fields:
        path=directory/f'{qid.lower()}_paper_{kind}.csv'
        with path.open('w',encoding='utf-8-sig',newline='') as fh:
            w=csv.writer(fh);w.writerow(headers)
            for i,t in enumerate(ts):
                row=[f'{t if qid=="Q1" else t/3600:.4f}']
                for j,r in enumerate(rs):
                    outside=qid=='Q4' and r>R[i]+1e-12
                    value=values[i,j]
                    if not outside and not np.isfinite(value) and qid=='Q4' and
abs(r-R[i])<=1e-12:

```

```

        value=Cs[i,0]
        row.append('' if outside else f'{float(value):.4f}')
    if qid=='Q4':
        row += [f'{Cs[i,0]:.4f}']
    w.writerow(row)
    outputs.append(exportFileRecord(path, 'paper_table'))
if qid=='Q4':
    path=directory/'q4_paper_radius.csv'
    with path.open('w',encoding='utf-8-sig',newline='') as fh:
        w=csv.writer(fh);w.writerow(['时间/h','表面半径/cm'])
        w.writerows([[f'{t/3600:.4f}',f'{r*100:.4f}'] for t,r in zip(ts,R)])
    outputs.append(exportFileRecord(path, 'paper_surface_coordinates'))
return outputs

def validatePaperTables(manifest,run):

    qid=manifest['question_id']
    if run is None:
        return {'status':'NOT_REQUESTED','checked_cells':0,'files':[],
                'scope':'Paper CSV hashes only; no independent live Run available.'},[]
    errors=[];records=manifest['paper_tables'];completionInfo=None
    if qid=='Q1':
        times=np.asarray([100.,300.,600.,900.,1200.,1500.,1800.])
        expectedKinds=['temperature','moisture']
    elif qid=='Q2':
        times=1800.*np.arange(1,7)
        expectedKinds=['temperature','moisture']
    else:
        from q3Model import completion
        completionInfo=completion(run)
        finalS=float(completionInfo['reported_time_s'])

        regular=[21600.*k for k in range(1,math.floor(finalS/21600.)+1)
                 if 21600.*k<finalS-1e-8]
        times=np.asarray([*regular,finalS],dtype=float)
        expectedKinds=['moisture']+(['radius'] if qid=='Q4' else [])
        stored=manifest.get('paper_table_time_contract',{})
        for key in ['reported_time_s','reported_drying_time_h']:
            if key not in stored or not numericEqual(stored[key],completionInfo[key]):
                errors.append('Paper CSV report-time metadata differs from independent
completion: '+key)
            if not
numericEqual(stored.get('reported_time_h'),completionInfo['reported_drying_time_h']):
                errors.append('Paper CSV reported_time_h alias differs from independent completion')
        if times[-1]>run.endS+1e-7:
            errors.append('Paper CSV contract exceeds the live solution horizon')
            return {'status':'FAIL','checked_cells':0,'files':[]},errors
        expectedNames={f'{qid.lower()}_paper_{kind}.csv':kind for kind in expectedKinds}
        actualNames=[Path(r['path']).name for r in records]
        if set(actualNames)!=set(expectedNames) or len(actualNames)!=len(expectedNames):
            errors.append('Missing, duplicate, or unexpected paper CSV artifact')
        files=[];totalCells=0
        for rec in records:
            path=ROOT/rec['path'];kind=expectedNames.get(path.name)
            if kind is None:continue
            beforeErrors=len(errors)
            with path.open('r',encoding='utf-8-sig',newline='') as fh:
                reader=csv.reader(fh);header=next(reader,None);rows=list(reader)
                desiredTime='时间/s' if qid=='Q1' else '时间/h'
                if kind=='radius':

```

```

correctHeader=['时间/h','表面半径/cm']
if header!=correctHeader:
    errors.append(path.name+': radius/time units or header mismatch')
    continue
radii=None
expected=np.asarray(run.model.radius(times),dtype=float).reshape(-1,1)*100.
inside=np.ones_like(expected,dtype=bool)
unit='cm';surfaceColumn=None
else:
    wantedColumns=7 if qid=='Q4' else 6
    if header is None or len(header)!=wantedColumns or header[0]!=desiredTime:
        errors.append(path.name+': time/column header mismatch');continue
    try:
        radii=np.asarray([float(v)*.01 for v in header[1:6]])
    except ValueError:
        errors.append(path.name+': nonnumeric physical-radius header');continue
    if not np.all(np.isfinite(radii)) or not
np.allclose(radii,[0.,.005,.01,.015,.02],rtol=0,atol=1e-12):
        errors.append(path.name+': physical-radius grid must be 0,0.5,1,1.5,2
cm');continue
    if qid=='Q4' and header[-1]!='药材表面':
        errors.append(path.name+': distinct material surface column missing');continue
    temperature,moisture=run.fields(times,radiiM=radii)
    expected=np.asarray(temperature)-273.15 if kind=='temperature' else
np.asarray(moisture)
    inside=np.ones_like(expected,dtype=bool);surfaceColumn=None
    if qid=='Q4':
        physicalR=np.asarray(run.model.radius(times),dtype=float)
        inside=radii[None,:] <= physicalR[:,None]+1e-12
        _,surfaceValues=run.fields(times,materialX=np.asarray([1.]))

        coincidence=np.abs(radii[None,:]-physicalR[:,None])<=1e-12
        expected=np.where(coincidence & ~np.isfinite(expected),surfaceValues,expected)
        expected=np.column_stack([expected,surfaceValues[:,0]])
        inside=np.column_stack([inside,np.ones(len(times),dtype=bool)])
        surfaceColumn=7
        unit='degC' if kind=='temperature' else 'kg/kg'
if len(rows)!=len(times):
    errors.append(f'{path.name}: expected {len(times)} data rows, read {len(rows)}')
checked=0;blanks=0;maxFieldError=0.;maxTimeError=0.
for i,(row,t) in enumerate(zip(rows,times)):
    if len(row)!=len(header):
        errors.append(f'{path.name}: column count mismatch at row {i+2}');continue
    timeValue=float(t if qid=='Q1' else t/3600.)
    if row[0]!=format(timeValue,.4f'):
        errors.append(f'{path.name}: time label mismatch at row {i+2}; query time is
{t:.17g}s')
    else:maxTimeError=max(maxTimeError,abs(float(row[0])-timeValue))
    checked+=1
    for j,cell in enumerate(row[1:]):
        checked+=1
        if not inside[i,j]:
            blanks+=1
            if cell!='':errors.append(f'{path.name}: domain outside must be blank at row
{i+2}, column {j+2}')
            continue
        value=float(expected[i,j])
        if not math.isfinite(value):
            errors.append(f'{path.name}: live value is not finite inside
material');continue
        if not re.fullmatch(r'-?\d+\.\d{4}',cell):
            errors.append(f'{path.name}: finite four-decimal number required at row {i+2},

```

```

column {j+2}')
        continue
        actual=float(cell)
        if not math.isfinite(actual) or abs(actual-float(format(value, '.4f')))>1e-10:
            errors.append(f'{path.name}: independently queried value mismatch at row
{i+2}, column {j+2}')
```

maxFieldError=max(maxFieldError,abs(actual-value))
 totalCells+=checked
 files.append({'artifact':exportFileRecord(path),'kind':kind,'status':'PASS' if
len(errors)==beforeErrors else 'FAIL',

'header':header,'data_rows':len(rows),'expected_data_rows':len(times),'checked_cells':check
ed,
 'source_query_times_s':times.tolist(),'display_time_unit':'s' if qid=='Q1' else
'h',
 'max_display_time_rounding_error':maxTimeError,'field_unit':unit,
 'max_field_rounding_error_from_live_value':maxFieldError,'outside_domain_blank_cells':blank
s,
 'physical_radii_m':None if radii is None else radii.tolist(),
 'surface_column_1based':surfaceColumn,'surface_query':'material_x=1; never fixed 2
cm' if surfaceColumn else None})
 return {'status':'FAIL' if errors else 'PASS','checked_cells':totalCells,'files':files,
 'completion':completionInfo,'completion_source':exportFileRecord(SOURCE.with_name('q3Model.
py')) if completionInfo else None,
 'scope':'Every paper CSV cell independently re-evaluated from live Run and fixed question
contracts; no generation arrays or 60s archives reused.',
 'rounding_limit':'CSV stores four decimals; matching rounded values cannot distinguish
sub-rounding perturbations of the original source value.'},errors

def exportQuestion(run,questionId,outputDir,*,chunkRows=1000,overwrite=False):

started=time.perf_counter()
 qid=normalizeQuestionId(questionId)
 directory=Path(outputDir).resolve()
 if not directory.is_relative_to(ROOT) or directory.is_relative_to(ROOT/'inputs'):
 raise ValueError('Outputs must be inside the extracted code package, outside inputs')
 if not 1<=int(chunkRows)<=2000:
 raise ValueError('chunk_rows must be between 1 and 2000')
 directory.mkdir(parents=True,exist_ok=True)
 stem='result'+qid[-1]
 bookPath=directory/(stem+'.xlsx')
 rawPath=directory/(stem+'_unrounded.csv.gz')
 manifestPath=directory/(stem+'.export.json')
 if not overwrite and any(p.exists() for p in [bookPath,rawPath,manifestPath]):
 raise FileExistsError('Export target already exists; use a new output version')
 provenance=runProvenance(run)
 allowed={'Q1':{'Q1'},'Q2':{'Q2','Q23'},'Q3':{'Q2','Q3','Q23'},'Q4':{'Q4'}}
 if run.model.settings.question not in allowed[qid]:
 raise ValueError('Run physics/question do not match the requested output question')
 if qid=='Q4' and not run.model.settings.shrink:
 raise ValueError('Formal Q4 output requires the shrinking-domain Run, not its
fixed-radius control')
 template=loadTemplate(qid)
 times=outputTimes(qid,run.endS,run.eventS)
 if len(times)+1>1_048_576:
 raise ValueError('Required full time grid exceeds the XLSX worksheet row limit; do not
truncate')
 workbook=Workbook(write_only=True)
 workbook.properties.creator=''

```

workbook.properties.lastModifiedBy=''
workbook.properties.title=f'问题{qid[-1]}结果'
headers={name:[template['time_header']] + RADII_CM for name in template['sheet_names']}
if qid=='Q4':
    headers['Sheet1'].append(template['surface_header'])
    headers['半径']=['时间/s', '药材表面半径/cm']
sheets={name:createSheet(workbook,name,header) for name,header in headers.items()}

counts={name:{'data_rows':0,'total_rows':1,'columns':len(header),'outside_domain_blank_cells':0,
              'first_time_s':None,'last_time_s':None,'header':header} for name,header in
headers.items()}
samplesIdx=set(np.linspace(0,len(times)-1,min(13,len(times)),dtype=int).tolist())
samplesIdx.update([0,min(1,len(times)-1),len(times)-1])
if run.eventS is not None:
    samplesIdx.update(np.flatnonzero(times==run.eventS).tolist())
samples=[]
with gzip.open(rawPath,'wt',encoding='utf-8',newline='',compresslevel=6) as fh:
    rawWriter=csv.writer(fh);rawWriter.writerow(rawHeaders(qid))
    for start in range(0,len(times),int(chunkRows)):
        ts=times[start:start+int(chunkRows)]
        T,C,R,Cs,inside=fieldBlock(run,ts,qid)
        for i,t in enumerate(ts):
            raw=rawRow(qid,t,T[i],C[i],R[i],None if Cs is None else Cs[i],inside[i])
            rawWriter.writerow(['' if v is None else format(v,'.17g') for v in raw])
            values=roundedSheets(qid,raw)
            for name,row in values.items():
                appendNumeric(sheets[name],row)
                count=counts[name]
                count['data_rows']+=1;count['total_rows']+=1
                count['outside_domain_blank_cells']+=sum(v is None for v in row)
                if count['first_time_s'] is None:
                    count['first_time_s']=row[0]
                count['last_time_s']=row[0]
            if start+i in samplesIdx:
                samples.append({'data_row_index':start+i,'source_time_s':float(t),
                              'raw_values':raw,'rounded_sheets':values})

workbook.save(bookPath)
paperRecords=paperTables(run,qid,directory)
paperTimeContract={'unit':'s' if qid=='Q1' else 'h','stored_decimals':4,
                  'query_uses_unrounded_seconds':True}
if qid in ['Q3','Q4']:
    from q3Model import completion
    completed=completion(run)
    paperTimeContract.update({key:completed[key] for key in

['reported_time_s','reported_drying_time_h','critical_event_s',

'max_C_at_reported_time','conservative_post_verification_s']})
    paperTimeContract['reported_time_h']=completed['reported_drying_time_h']
    paperTimeContract['regular_step_s']=21600.
    paperTimeContract['endpoint_convention']='6-hour samples plus upward-reported
strict-drying time; distinct from workbook post-verification endpoint'
    endProvenance=runProvenance(run)
    if endProvenance!=provenance or
sha256File(ROOT/template['record']['path'])!=template['record']['sha256']:
        raise RuntimeError('Source, settings or template changed during export')
    bookRecord=exportFileRecord(bookPath,'contest_result_workbook')
    archiveRecord=exportFileRecord(rawPath,'internal_unrounded_output_grid_archive')
    manifest={
        'schema_version':'1.0','question_id':qid,'generated_by':'exportOutputs.py',

'generated_at':datetime.now(timezone.utc).isoformat(),'status':'EXPORTED_PENDING_STREAM_REA

```

```

DBACK',
    'workbook':bookRecord,'unrounded_archive':archiveRecord,'paper_tables':paperRecords,
    'paper_table_time_contract':paperTimeContract,
    'provenance':provenance,'template':template['record'],'sheets':counts,
    'time_grid':{'start_s':0.,'end_s':float(times[-1]),'regular_step_s':1 if qid in
['Q1','Q2'] else 60,
        'critical_event_s':provenance['event_s'] if qid in ['Q3','Q4'] else None,
        'include_post_verification_endpoint':qid!='Q1',

    'count':len(times),'rounding_duplicate_time_count':int(np.sum(np.diff(np.round(times,4))==0)
)),

    'radius_grid':{'fixed_radii_cm':RADII_CM,'surface_column':qid=='Q4','surface_coordinate_she
et':'半径' if qid=='Q4' else None},
    'rounding':{'decimal_places':4,'storage':'numeric rounded to four
decimals','number_format':NUMBER_FORMAT,
        'raw_archive':'17 significant digits, source times in seconds and radius in
metres',
        'threshold_judgement':'use original full precision event and max C, never
rounded 0.1500'},
    'domain_rule':{'outside_values':'blank (None); never
zero','comparison_tolerance_m':1e-12,
        'Q4_surface_is_not_fixed_2cm':True},
    'samples':samples,'elapsed_s':time.perf_counter()-started,

    'size':{'workbook_bytes':bookRecord['bytes'],'workbook_MiB':bookRecord['bytes']/2**20,

    'unrounded_archive_bytes':archiveRecord['bytes'],'warning_limit_bytes':SIZE_LIMIT_BYTES,
        'workbook_exceeds_20_decimal_MB':bookRecord['bytes']>SIZE_LIMIT_BYTES,
        'archive_is_internal_not_automatically_in_submission_zip':True,
        'complete_support_archive_still_requires_actual_size_check':True,
        'no_required_time_or_space_values_removed':True},
    'validation_required':'validate_exports([record], runs={question_id: run}); full
workbook/archive comparison, live Run workbook samples, and every paper CSV cell independently
queried from live Run',
    'visual_studio_gui':'pending','human_review':'pending',
    }
    writeJson(manifestPath,manifest)
    return
{'question_id':qid,'workbook_path':str(bookPath),'manifest_path':str(manifestPath),

    'artifacts':[bookRecord,archiveRecord]+paperRecords+[exportFileRecord(manifestPath,'export_
manifest')],
        'sheets':counts,'size':manifest['size'],'validation_status':'pending'}

def resolveManifestPath(item):
    if isinstance(item,dict):
        return Path(item['manifest_path'])
    path=Path(item)
    return path.with_suffix('.export.json') if path.suffix=='.xlsx' else path

def numericEqual(a,b,tolerance=1e-10):
    return a is None and b is None or (a is not None and b is not None and
        isinstance(a,(int,float)) and not isinstance(a,bool) and math.isfinite(float(a)) and
        abs(float(a)-float(b))<=tolerance)

def validateExports(paths,runs=None):

    if isinstance(paths,(str,Path,dict)):

```

```

        paths=[paths]
runs={} if runs is None else runs
results=[]
for item in paths:
    started=time.perf_counter()
    manifestPath=resolveManifestPath(item).resolve()
    manifest=json.loads(manifestPath.read_text(encoding='utf-8'))
    qid=manifest['question_id'];errors=[];checkedCells=0;checkedSheets={}
    for rec in
[manifest['workbook'],manifest['unrounded_archive'],manifest['template']] + manifest['paper_t
ables']:
        if not (ROOT/rec['path']).exists() or sha256File(ROOT/rec['path'])!=rec['sha256']:
            errors.append('Hash mismatch or missing artifact: '+rec['path'])
    if errors:
        raise RuntimeError('; '.join(errors))

times=outputTimes(qid,manifest['provenance']['end_s'],manifest['provenance']['event_s'])
wb=load_workbook(ROOT/manifest['workbook']['path'],read_only=True,data_only=False)
if wb.sheetnames!=list(manifest['sheets']):
    errors.append('Sheet names/order differ from export contract')
try:
    for name,spec in manifest['sheets'].items():
        ws=wb[name]
        rows=ws.iter_rows(min_row=1,max_col=spec['columns'])
        header=[cell.value for cell in next(rows)]
        if header!=spec['header']:
            errors.append(name+": header/physical radius grid mismatch")
            count=0;blanks=0;first=None;last=None;maxRoundingError=0.
            with
gzip.open(ROOT/manifest['unrounded_archive']['path'],'rt',encoding='utf-8',newline='') as
rawfh:
                sourceRows=csv.reader(rawfh)
                if next(sourceRows)!=rawHeaders(qid):
                    errors.append('Unrounded archive schema mismatch')
                for index,sourceRow in enumerate(sourceRows):
                    try:
                        cells=next(rows)
                    except StopIteration:
                        errors.append(name+": workbook ends before raw archive");break
                    raw=[None if v=='' else float(v) for v in sourceRow]
                    if index>=len(times) or abs(raw[0]-times[index])>1e-8:
                        errors.append(name+": source time schedule mismatch at "+str(index))
                    if qid=='Q4':
                        for j,r in enumerate(RADII_M):
                            outside=r>raw[1]+1e-12
                            if (raw[2+j] is None)!=outside:
                                errors.append(name+": incorrect moving-domain mask at
'+str(index))

                                elif any(v is None for v in raw):
                                    errors.append(name+": unexpected blank in fixed domain")
                                expected=roundedSheets(qid,raw)[name]
                                for j,(cell,value) in enumerate(zip(cells,expected)):
                                    if not numericEqual(cell.value,value):
                                        errors.append(f'{name}: value mismatch at row {index+2}, column
{j+1}')

                                    if value is not None:
                                        if cell.number_format!=NUMBER_FORMAT:
                                            errors.append(f'{name}: four-decimal format missing at row
{index+2}, column {j+1}')

                                        if not math.isfinite(float(cell.value)) or
abs(float(cell.value)*1e4-round(float(cell.value)*1e4))>1e-5:
                                            errors.append(f'{name}: non-finite or non-four-decimal stored
number')

```

```

maxRoundingError=max(maxRoundingError,abs(float(cell.value)-value))
        else:
            blanks+=1
            checkedCells+=len(cells);count+=1
            if first is None:first=cells[0].value
            last=cells[-1].value
            if len(errors)>30:
                raise RuntimeError('Export validation failed: '+';
'.join(errors[:30]))
            if next(rows,None) is not None:
                errors.append(name+': workbook has uncontracted extra rows')
            if count!=spec['data_rows'] or count!=len(times):
                errors.append(name+': row count differs from time grid')
            if blanks!=spec['outside_domain_blank_cells']:
                errors.append(name+': blank count mismatch')

checkedSheets[name]={'data_rows':count,'total_rows':count+1,'first_time_s':first,
'last_time_s':last,'outside_domain_blank_cells':blanks,'value_comparison_max_abs_error':max
RoundingError}
    finally:
        wb.close()
    run=runs.get(qid)
    live={'status':'NOT_REQUESTED','samples':0,'max_abs_unrounded_difference':None}
    if run is not None:
        current=runProvenance(run)
        if current!=manifest['provenance']:
            errors.append('Live Run provenance differs from the exported Run')
        samples=manifest['samples'];ts=np.array([s['source_time_s'] for s in samples])
        T,C,R,Cs,inside=fieldBlock(run,ts,qid)
        difference=0.
        for i,sample in enumerate(samples):
            actual=rawRow(qid,ts[i],T[i],C[i],R[i],None if Cs is None else Cs[i],inside[i])
            for a,b in zip(actual,sample['raw_values']):
                if a is None or b is None:
                    if not (a is None and b is None):errors.append('Live Run sample domain
mismatch')
                else:
                    difference=max(difference,abs(a-b))
            if difference>1e-10:
                errors.append('Independent live Run sample mismatch exceeds 1e-10')
        live={'status':'PASS' if difference<=1e-10 else 'FAIL','samples':len(samples),
'max_abs_unrounded_difference':difference}
    paperValidation,paperErrors=validatePaperTables(manifest,run)
    errors.extend(paperErrors)
    report={'schema_version':'1.0','question_id':qid,'generated_by':'exportOutputs.py',
'generated_at':datetime.now(timezone.utc).isoformat(),
'status':'FAIL' if errors else 'PASS','fully_verified_with_live_Run':not errors
and run is not None,

'checked_cells':checkedCells,'sheets':checkedSheets,'live_Run_sample_validation':live,
'paper_CSV_live_Run_validation':paperValidation,
'errors':errors,'elapsed_s':time.perf_counter()-started,

'workbook':manifest['workbook'],'export_manifest':exportFileRecord(manifestPath),
'scope':'Full workbook/raw-archive validation; independent workbook live Run
samples and every paper CSV time/position/value/surface/radius cell when live Run is provided.',
'four_decimals_are_storage_and_display_not_physical_accuracy_claim':True,

'visual_render':'pending','visual_studio_gui':'pending','human_review':'pending'}

reportPath=manifestPath.with_name(manifestPath.name.replace('.export.json','.validation.jso

```

```

n'))

writeJson(reportPath,report);report['report_path']=str(reportPath);results.append(report)
    if errors:
        raise RuntimeError('Export validation failed: '+'; '.join(errors[:30]))
    return {'status': 'PASS' if all(r['status']=='PASS' for r in results) else 'FAIL',
            'fully_verified_with_live_Run': all(r['fully_verified_with_live_Run'] for r in
results), 'exports': results}

def paperCsvSelfcheck(directory):

    from dryingCore import Settings,solveCase
    import copy
    directory=Path(directory).resolve()
    directory.mkdir(parents=True,exist_ok=True)
    summaries=[];negativeChecks=[]
    for qid in ['Q1','Q2','Q3','Q4']:
        settings={'question': 'Q23' if qid in ['Q2','Q3'] else qid,
                  'intervals': 40, 'faceScheme': 'kirchhoff', 'shrink': qid=='Q4',
                  'rtol': 1e-8, 'atolTemperature': 1e-8, 'atolMoisture': 1e-10,
                  'earlyMaxStepS': 5.}
        if qid=='Q2':
            settings.update(constantD=2e-8,beta=8e-6,horizonH=24.)
        print('PAPER_CSV_SELFCHECK '+qid+' : actual N40 solve',flush=True)
        run=solveCase(Settings(**settings))
        try:
            if qid=='Q2' and (run.eventS is None or run.endS<10800.):
                raise RuntimeError('Accelerated export test must still cover 3h and a real drying
event')
            exported=exportQuestion(run,qid,directory/qid)
            checked=validateExports([exported],runs={qid:run})
            manifest=json.loads(Path(exported['manifest_path']).read_text(encoding='utf-8'))
            defects={'Q1': ['temperature_value'], 'Q2': ['moisture_value'],
'Q3': ['last_time_label'], 'Q4': ['outside_nonblank', 'surface_missing', 'radius_value']}[qid]
            for defect in defects:
                changed=copy.deepcopy(manifest)
                kind='temperature' if defect=='temperature_value' else 'radius' if
defect=='radius_value' else 'moisture'
                selected=next(r for r in changed['paper_tables'] if
Path(r['path']).name==f'{qid.lower()}_{paper}_{kind}.csv')
                originalPath=ROOT/selected['path']
                with originalPath.open('r',encoding='utf-8-sig',newline='') as
fh:rows=list(csv.reader(fh))
                if defect in ['temperature_value','moisture_value']:
                    rows[1][1]=format(float(rows[1][1])+.01, '.4f')
                elif defect=='last_time_label':
                    rows[-1][0]=format(float(rows[-1][0])-.001, '.4f')
                elif defect=='outside_nonblank':
                    ri,ci=next((i,j) for i in range(1,len(rows)) for j in range(1,6) if
rows[i][j]=='')
                    rows[ri][ci]='0.0000'
                elif defect=='surface_missing':rows[-1][-1]=''
                elif defect=='radius_value':rows[-1][1]=format(float(rows[-1][1])+.01, '.4f')
                target=directory/'deliberate_defects'/defect/originalPath.name
                target.parent.mkdir(parents=True,exist_ok=True)
                with target.open('w',encoding='utf-8-sig',newline='') as
fh:csv.writer(fh).writerows(rows)
                selected.update(exportFileRecord(target,selected.get('role')))
                result,errors=validatePaperTables(changed,run)
                if not errors or result['status']!='FAIL':
                    raise AssertionError('Independent paper validator missed deliberate defect:

```

```

'+defect)

negativeChecks.append({'question_id':qid,'defect':defect,'correctly_rejected':True,
                      'errors':errors,'artifact':exportFileRecord(target)})

summaries.append({'question_id':qid,'settings':asdict(run.model.settings),'export':exported
,
                  'validation':checked,'event_s':run.eventS,'end_s':run.endS,
                  'Q2_is_accelerated_constant_D_export_test_only':qid=='Q2'})
print(json.dumps({'question_id':qid,'status':'PASS','end_s':run.endS,

'paper_cells':checked['exports'][0]['paper_CSV_live_Run_validation']['checked_cells'],ensu
re_ascii=False),flush=True)
    finally:
        run.close()
    report={'status':'PASS','generated_at':datetime.now(timezone.utc).isoformat(),
           'scope':'N40 exporter fidelity only. Q2 uses constant_D=2e-8 and beta=8e-6 solely
to shorten the real full-event export test; these are not formal physical results. Q1/Q3/Q4 use
their nominal physics.',

'source':exportFileRecord(SOURCE),'exports':summaries,'negative_checks':negativeChecks,
          'negative_test_scope':'Copied paper CSVs only; six deliberate defects must be
rejected even after their recorded hashes are updated.',
          'visual_studio_gui':'pending','human_review':'pending'}
    writeJson(directory/'paper_csv_selfcheck_report.json',report)

print(json.dumps({'status':'PASS','questions':4,'deliberate_defects_rejected':len(negativeC
hecks),

'report':str(directory/'paper_csv_selfcheck_report.json')},ensure_ascii=False),flush=True)
    return 0

def main():
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--selfcheck',action='store_true')
    parser.add_argument('--paper-selfcheck',action='store_true')
    parser.add_argument('--output-dir',default='results/exportSelfcheck')
    args=parser.parse_args()
    if not args.selfcheck and not args.paper_selfcheck:
        parser.error('Use the Python API with final Runs, or --selfcheck for a real Q1 validation
export')
    if Path.cwd().resolve() != ROOT:
        raise RuntimeError('Use the extracted code package as cwd')
    if args.paper_selfcheck:
        return paperCsvSelfcheck(ROOT/args.output_dir)
    from dryingCore import Settings,solveCase

    run=solveCase(Settings(question='Q1',intervals=40,faceScheme='kirchhoff',

rtol=1e-8,atolTemperature=1e-8,atolMoisture=1e-10,earlyMaxStepS=5.))
    record=exportQuestion(run,'Q1',ROOT/args.output_dir)
    checks=validateExports([record],runs={'Q1':run})
    output=Path(args.output_dir).resolve()
    note={'status':checks['status'],'scope':'Actual Q1 export selfcheck only, not final
production or numerical-accuracy acceptance',
          'settings':asdict(run.model.settings),'export':record,'validation':checks}
    writeJson(output/'selfcheck_summary.json',note)

print(json.dumps({'status':checks['status'],'fully_verified_with_live_Run':checks['fully_ve
rified_with_live_Run'],

```



```

        'verification second; not claimed '
        'earliest integer second',
        'max_mass_balance_abs_kg_per_kg':
1.1546319456101628e-14,
        'min_C': 0.05265944808510597,
        'max_C': 2.5500000000000001,
        'min_T_K': 301.15,
        'max_T_K': 323.18986600675885,
        'min_D': 3.124541898624993e-12,
        'max_D': 1.252185966946026e-08,
        'positive_properties': True,
        'max_radial_C_increase': 3.1086244689504383e-15,
        'final_max_C': 0.14999950103529133,
        'final_surface_C': 0.05265944808510597,
        'strictly_dry_at_end': True,
        'radius_end_m': 0.02,
        'radius_extrapolation_used': False,
        'rhs_evaluations': 32705,
        'accepted_time_points': 14283,
        'nfev': 32705,
        'njev': 618,
        'nlu': 3552,
        'solver_success': True,
        'dense_storage': 'disk',
        'dense_coefficient_bytes': 3754872872,
        'dense_polynomial_count': 14280},
    'completion': {'critical_event_s': 206900.28702201758,
        'critical_event_h': 57.47230195056044,
        'reported_drying_time_h': 57.4724,
        'reported_time_s': 206900.64,
        'max_C_at_reported_time': 0.14999989718225315,
        'slowest_material_coordinate': 0.0,
        'conservative_post_verification_s': 206902.0,
        'rounding_convention': 'upward on 0.0001 h grid, followed by
        'actual strict threshold check',
        'interpretation': 'A conditional numerical event, not a '
        'confidence bound on physical drying '
        'time.'},
    'reproduction_samples': {'times_s': [0.0,
        1800.0,
        10800.0,
        206900.64,
        206902.0],
        'material_x': [0.0, 0.25, 0.5, 0.75, 1.0],
        'T_K': [[301.15, 301.15, 301.15, 301.15, 301.15],
        [305.3392446508569,
        305.532114218488,
        306.1159401214376,
        307.11077899334816,
        308.56302049753884],
        [322.9994956060796,
        323.0053407098511,
        323.02461224281694,
        323.0601477811811,
        323.1164123947858],
        [323.15,
        323.15,
        323.15,
        323.15,
        323.1499999999999],
        [323.15, 323.15, 323.15, 323.15, 323.15]],
        'C': [[2.55, 2.55, 2.55, 2.55, 2.55],

```

```

[2.549934098712396,
 2.5488838430623373,
 2.5255580144516143,
 2.3257039999977334,
 1.648617763296951],
[1.7661771002440243,
 1.716533013166519,
 1.5701979936396047,
 1.3332634188779369,
 1.0081231118401026],
[0.14999989718225315,
 0.14768651296460814,
 0.1402086276135209,
 0.12491406907205477,
 0.0526594803435617],
[0.14999950103529133,
 0.14768612746407359,
 0.14020827567351044,
 0.12491378167987972,
 0.05265944808510597]]}],
'Q4': {'diagnostics': {'event_s': 183926.06923258994,
  'event_h': 51.09057478683054,
  'end_s': 183928.0,
  'elapsed_s': 164.5545338999982,
  'end_time_convention': 'ceil(critical_event_s)+1: '
    'conservative post-crossing '
    'verification second; not claimed '
    'earliest integer second',
  'max_mass_balance_abs_kg_per_kg': 7.993605777301127e-15,
  'min_C': 0.05261996970729463,
  'max_C': 2.5500000000000025,
  'min_T_K': 301.15,
  'max_T_K': 323.2000976421016,
  'min_D': 9.398366254447737e-12,
  'max_D': 2.477122820067253e-09,
  'positive_properties': True,
  'max_radial_C_increase': 4.440892098500626e-15,
  'final_max_C': 0.14999900856092843,
  'final_surface_C': 0.052619969707294635,
  'strictly_dry_at_end': True,
  'radius_end_m': 0.012,
  'radius_extrapolation_used': False,
  'rhs_evaluations': 36934,
  'accepted_time_points': 15996,
  'nfev': 36934,
  'njev': 737,
  'nlu': 4099,
  'solver_success': True,
  'dense_storage': 'disk',
  'dense_coefficient_bytes': 8327788168,
  'dense_polynomial_count': 15993},
  'completion': {'critical_event_s': 183926.06923258994,
  'critical_event_h': 51.09057478683054,
  'reported_drying_time_h': 51.0906,
  'reported_time_s': 183926.16,
  'max_C_at_reported_time': 0.1499995339076624,
  'slowest_material_coordinate': 0.0,
  'conservative_post_verification_s': 183928.0,
  'rounding_convention': 'upward on 0.0001 h grid, followed by
    'actual strict threshold check',
  'interpretation': 'A conditional numerical event, not a '
    'confidence bound on physical drying

```

```

time.'},
      'reproduction_samples': {'times_s': [0.0,
      1800.0,
      10800.0,
      183926.16,
      183928.0],
      'material_x': [0.0, 0.25, 0.5, 0.75, 1.0],
      'T_K': [[301.15, 301.15, 301.15, 301.15, 301.15],
      [304.18781907151936,
      304.46728468564174,
      305.3224885001255,
      306.80252608201823,
      309.05980848026536],
      [322.97124085294246,
      322.9784486845918,
      323.0028781283762,
      323.0496413187221,
      323.12744861066466],
      [323.15, 323.15, 323.15, 323.15, 323.15],
      [323.15, 323.15, 323.15, 323.15, 323.15]],
      'C': [[2.55, 2.55, 2.55, 2.55, 2.55],
      [2.55,
      2.5499999999995815,
      2.549998436080619,
      2.5309421443741127,
      1.1313425016195608],
      [2.4547252277379927,
      2.3815382147789257,
      2.112246351658363,
      1.5585872624122925,
      0.6843727284423174],
      [0.14999995339076624,
      0.14692749737405153,
      0.13718379398934455,
      0.11813224940015873,
      0.05262002809955053],
      [0.14999900856092843,
      0.14692658503537287,
      0.13718298178958888,
      0.11813162008386399,
      0.052619969707294635]]]}}

```

9.2.11 runLogged.ps1

```

param(
    [string]$Python = 'python',
    [ValidateSet('quick', 'final')][string]$Profile = 'quick',
    [string]$RunId = ''
)
$ErrorActionPreference = 'Stop'
if ([string]::IsNullOrEmpty($RunId)) {
    $RunId = $Profile + '_' + [DateTime]::UtcNow.ToString('yyyyMMddTHHmssZ')
}
if ($RunId -notmatch '^[A-Za-z0-9][A-Za-z0-9_-]{0,79}$') {
    throw 'Invalid run identifier.'
}
$packageDirectory = $PSScriptRoot
$entryPath = Join-Path $packageDirectory 'runDelivery.py'
$resultDirectory = Join-Path (Join-Path $packageDirectory 'results') $RunId
if (Test-Path -LiteralPath $resultDirectory) {
    throw 'The run directory already exists; use a new run identifier.'
}
$startedUtc = [DateTime]::UtcNow.ToString('o')

```

```

$timer = [Diagnostics.Stopwatch]::StartNew()
$entryHash = (Get-FileHash -LiteralPath $entryPath -Algorithm SHA256).Hash.ToLowerInvariant()
& $Python -X utf8 -B $entryPath --profile $Profile --run-id $RunId
$processExit = $LASTEXITCODE
$timer.Stop()
if (Test-Path -LiteralPath $resultDirectory) {
    $record = [ordered]@{
        runId = $RunId
        profile = $Profile
        startedAtUtc = $startedUtc
        finishedAtUtc = [DateTime]::UtcNow.ToString('o')
        elapsedSeconds = $timer.Elapsed.TotalSeconds
        actualExitCode = $processExit
        entrySha256AtStart = $entryHash
        entrySha256AtEnd = (Get-FileHash -LiteralPath $entryPath -Algorithm
SHA256).Hash.ToLowerInvariant()
        visualStudioGui = 'NOT_OBSERVED_BY_THIS_SCRIPT'
        humanReview = 'PENDING_USER_REVIEW'
    }
    $record | ConvertTo-Json -Depth 5 | Set-Content -LiteralPath (Join-Path $resultDirectory
'processExit.json') -Encoding UTF8
}
exit $processExit

```

9.2.12 axisymmetric_check.py

```

from __future__ import annotations

import argparse
from dataclasses import asdict
from datetime import datetime, timezone
import hashlib
import json
import os
from pathlib import Path
import time
import traceback

for _name in ("OMP_NUM_THREADS", "OPENBLAS_NUM_THREADS", "MKL_NUM_THREADS"):
    os.environ.setdefault(_name, "1")

import numpy as np
from scipy.integrate import solve_ivp
from scipy.sparse import coo_matrix
from scipy.special import expi

from drying_core import ROOT, C0, T0, R0, LENGTH, RadialModel, Settings, solve_case

CODE_PATH = Path(__file__).resolve()
CORE_PATH = CODE_PATH.with_name("drying_core.py")
CODE_HASH = hashlib.sha256(CODE_PATH.read_bytes()).hexdigest()
CORE_HASH = hashlib.sha256(CORE_PATH.read_bytes()).hexdigest()

def harmonic_pair(a, b):
    return 2 * a * b / np.maximum(a + b, np.finfo(float).tiny)

def concentration_potential(c):
    cc = np.maximum(c, 1e-12)
    return cc * np.exp(-0.45 / cc) + 0.45 * expi(-0.45 / cc)

```

```

class AxisymmetricModel:
    def __init__(self, nr=30, nz=30, end_faces=True):
        self.nr, self.nz = nr, nz
        self.shape = (nr + 1, nz + 1)
        self.nodes = (nr + 1) * (nz + 1)
        self.settings = Settings(question="Q23", intervals=nr,
                                   face_scheme="kirchhoff", horizon_h=100.)
        self.reference = RadialModel(self.settings)
        self.x, self.wr = self.reference.x, self.reference.w
        self.xr_faces = self.reference.internal_faces
        self.y = np.linspace(0., 1., nz + 1)
        faces_z = np.r_[0., (self.y[:-1] + self.y[1:]) / 2., 1.]
        self.wz = np.diff(faces_z)
        self.dx, self.dy = 1. / nr, 1. / nz
        self.half_length = LENGTH / 2.
        self.end_faces = bool(end_faces)
        self.mean_weights = 2 * self.wr[:, None] * self.wz[None, :]
        self.calls = 0
        self.started = 0.
        self.budget_s = 900.
        self.jac_pattern = self._sparsity()

    def _sparsity(self):
        idx = np.arange(self.nodes).reshape(self.shape)
        pairs = [(idx.ravel(), idx.ravel())]
        for a, b in [(idx[:-1], idx[1:]), (idx[:, :-1], idx[:, 1:]):
            pairs += [(a.ravel(), b.ravel()), (b.ravel(), a.ravel())]
        rows, cols = [], []
        for a, b in pairs:
            for i in (0, 1):
                for j in (0, 1):
                    rows.extend((2 * a + i).tolist())
                    cols.extend((2 * b + j).tolist())
        surface = np.unique(np.r_[idx[-1, :], idx[:, -1]])
        rows.extend([2 * self.nodes] * len(surface))
        cols.extend((2 * surface + 1).tolist())
        size = 2 * self.nodes + 1
        return coo_matrix((np.ones(len(rows)), (rows, cols)),
                           shape=(size, size)).tocsr()

    def initial(self):
        out = np.empty(2 * self.nodes + 1)
        out[:-1:2], out[1:-1:2], out[-1] = T0, C0, 0.
        return out

    def fields(self, state):
        return state[:-1:2].reshape(self.shape), state[1:-1:2].reshape(self.shape)

    @staticmethod
    def face_water(Ta, Tb, Ca, Cb, Fa, Fb):
        cc_a, cc_b = np.maximum(Ca, 1e-12), np.maximum(Cb, 1e-12)
        dc = cc_b - cc_a
        dc_potential = Fb - Fa
        small = np.abs(dc) < 1e-7 * np.maximum((cc_a + cc_b) / 2, 1e-3)
        dc_potential = np.where(small, np.exp(-.45 / ((cc_a + cc_b) / 2)) * dc,
                                dc_potential)
        return 2.4e-3 * np.exp(-3850 / ((Ta + Tb) / 2)) * dc_potential

    def rhs(self, t, state):
        self.calls += 1
        if self.calls % 50 == 0 and time.perf_counter() - self.started > self.budget_s:
            raise TimeoutError("Axisymmetric per-run CPU/wall budget reached")

```

```

T, C = self.fields(state)
rho, cp, k, _ = self.reference.properties(T, C)
tair, ceq = self.reference.environment(t)
F = concentration_potential(C)
gr_t = np.zeros((self.nr + 2, self.nz + 1))
gr_c = np.zeros_like(gr_t)
gz_t = np.zeros((self.nr + 1, self.nz + 2))
gz_c = np.zeros_like(gz_t)
gr_t[1:-1] = (self.xr_faces[:, None] *
    harmonic_pair(k[:-1], k[1:]) * np.diff(T, axis=0) / self.dx)
gr_c[1:-1] = (self.xr_faces[:, None] *
    self.face_water(T[:-1], T[1:], C[:-1], C[1:], F[:-1], F[1:])) / self.dx)
gz_t[:, 1:-1] = harmonic_pair(k[:, :-1], k[:, 1:]) * np.diff(T, axis=1) / self.dy
gz_c[:, 1:-1] = self.face_water(T[:, :-1], T[:, 1:], C[:, :-1], C[:, 1:],
    F[:, :-1], F[:, 1:])) / self.dy
gr_t[-1] = -self.settings.h * R0 * (T[-1] - tair)
gr_c[-1] = -self.settings.beta * R0 * (C[-1] - ceq)
if self.end_faces:
    gz_t[:, -1] = -self.settings.h * self.half_length * (T[:, -1] - tair)
    gz_c[:, -1] = -self.settings.beta * self.half_length * (C[:, -1] - ceq)
heat_div = (np.diff(gr_t, axis=0) / (R0**2 * self.wr[:, None]) +
    np.diff(gz_t, axis=1) / (self.half_length**2 * self.wz[None, :]))
mass_div = (np.diff(gr_c, axis=0) / (R0**2 * self.wr[:, None]) +
    np.diff(gz_c, axis=1) / (self.half_length**2 * self.wz[None, :]))
derivative = np.empty_like(state)
derivative[:-1:2] = (heat_div / (rho * cp)).ravel()
derivative[1:-1:2] = mass_div.ravel()
lateral_loss = 2 * self.settings.beta / R0 * (self.wz @ (C[-1] - ceq))
end_loss = (self.settings.beta / self.half_length *
    (2 * self.wr @ (C[:, -1] - ceq))) if self.end_faces else 0.
derivative[-1] = lateral_loss + end_loss
return derivative

```

```

class AxisymmetricRun:
    def __init__(self, model, pieces, elapsed, event):
        self.model, self.pieces, self.elapsed_s, self.event_s = model, pieces, elapsed, event
        self.end_s = float(pieces[-1].t[-1])

    def state(self, times):
        tt = np.atleast_1d(np.asarray(times, float))
        result = np.empty((2 * self.model.nodes + 1, len(tt)))
        pending = np.ones(len(tt), bool)
        for p in self.pieces:
            take = pending & (tt >= p.t[0] - 1e-8) & (tt <= p.t[-1] + 1e-8)
            if np.any(take):
                result[:, take] = p.sol(tt[take])
                pending[take] = False
        if pending.any():
            raise ValueError("Requested time outside computed axisymmetric solution")
        return result

    def diagnostics(self):
        m = self.model
        mass_error, cmin, cmax, tmin, tmax, axial_spread = 0., C0, C0, T0, T0, 0.
        for p in self.pieces:
            T, C = p.y[:-1:2], p.y[1:-1:2]
            means = m.mean_weights.ravel() @ C
            mass_error = max(mass_error, float(np.max(np.abs(means + p.y[-1] - C0))))
            cmin, cmax = min(cmin, float(C.min())), max(cmax, float(C.max()))
            tmin, tmax = min(tmin, float(T.min())), max(tmax, float(T.max()))
            CC = C.reshape(*m.shape, -1)
            axial_spread = max(axial_spread, float(np.max(np.ptp(CC, axis=1))))

```

```

event_state = self.state([self.event_s if self.event_s is not None else self.end_s][:,
0]
_, finalC = m.fields(event_state)
index = np.unravel_index(np.argmax(finalC), m.shape)
result = {"event_s":self.event_s,
"event_h":None if self.event_s is None else self.event_s/3600.,
"end_s":self.end_s, "elapsed_s":self.elapsed_s,
"max_mass_residual_kg_per_kg":mass_error, "min_C":cmin, "max_C":cmax,
"min_T_K":tmin, "max_T_K":tmax, "max_axial_C_spread":axial_spread,
"slowest_point_at_event_m":{"r":float(m.x[index[0]]*R0),
"z_from_midplane":float(m.y[index[1]]*m.half_length)},
"max_C_at_event":float(finalC.max()),
"mean_C_at_event":float(np.sum(m.mean_weights*finalC)),
"nfev":sum(p.nfev for p in self.pieces), "njev":sum(p.njev for p in self.pieces),
"nlu":sum(p.nlu for p in self.pieces),
"rhs_calls_including_jacobian":m.calls,
"accepted_time_points":sum(len(p.t) for p in self.pieces),
"solver_success":all(p.success for p in self.pieces)}
if mass_error > 1e-6 or cmin < -1e-8:
    raise FloatingPointError(f"Axisymmetric physical check failed: {result}")
return result

def solve_axisymmetric(nr=30, nz=30, end_faces=True, budget_s=600.):
    started = time.perf_counter()
    model = AxisymmetricModel(nr, nz, end_faces)
    model.started, model.budget_s = started, budget_s
    def event(t, y):
        return np.max(y[1:-1:2]) - .15
    event.terminal, event.direction = True, -1
    atol = np.empty(2*model.nodes+1)
    atol[1:-1:2], atol[1:-1:2], atol[-1] = 1e-7, 1e-9, 1e-9
    pieces, initial, event_s = [], model.initial(), None
    for left, right in [(0.,14400.), (14400.,360000.)]:
        p = solve_ivp(model.rhs, (left,right), initial, method="BDF",
            rtol=1e-7, atol=atol, max_step=30. if left==0 else 600.,
            jac_sparsity=model.jac_pattern, events=event, dense_output=True)
        pieces.append(p)
        if not p.success:
            raise RuntimeError(p.message)
        initial = p.y[:,-1]
        if len(p.t_events[0]):
            event_s = float(p.t_events[0][0])
            break
    return AxisymmetricRun(model,pieces,time.perf_counter()-started,event_s)

def compare(run, radial):
    end=min(run.end_s,radial.end_s)
    times=np.unique(np.r_[np.arange(0.,end,21600.),1800.,10800.,end])
    times=times[times<=end]
    state2=run.state(times)
    T2=state2[1:-1:2].reshape(*run.model.shape,-1)
    C2=state2[1:-1:2].reshape(*run.model.shape,-1)
    state1=radial.state(times)
    T1,C1=state1[1:-1:2],state1[1:-1:2]
    mean2=run.model.mean_weights.ravel()@state2[1:-1:2]
    mean1=2*radial.model.w@C1
    delta_t=None if run.event_s is None or radial.event_s is None else
(run.event_s-radial.event_s)/3600
    comparison={"same_radial_intervals":run.model.nr,
"one_dimensional_event_h":None if radial.event_s is None else radial.event_s/3600,
"axisymmetric_event_h":None if run.event_s is None else run.event_s/3600,

```

```

        "event_difference_h_2d_minus_1d":delta_t,
        "event_relative_difference":None if delta_t is None else delta_t/(radial.event_s/3600),
        "max_midplane_C_difference":float(np.max(np.abs(C2[:,0,:]-C1))),
        "max_midplane_T_difference_K":float(np.max(np.abs(T2[:,0,:]-T1))),
        "max_mean_C_difference":float(np.max(np.abs(mean2-mean1))),
        "times_s":times.tolist(),
        "mean_C_2d":mean2.tolist(),"mean_C_1d":mean1.tolist(),
        "center_C_2d":C2[0,0].tolist(),"center_C_1d":C1[0].tolist(),
        "center_T_K_2d":T2[0,0].tolist(),"center_T_K_1d":T1[0].tolist()}
    sample={"times_s":times,"x":run.model.x,"y":run.model.y,
        "T_K_2d":T2,"C_2d":C2,"T_K_1d":T1,"C_1d":C1,
        "mean_C_2d":mean2,"mean_C_1d":mean1}
    return comparison,sample

def main():
    p=argparse.ArgumentParser()
    p.add_argument("--nr",type=int,default=30)
    p.add_argument("--nz",type=int,default=30)
    p.add_argument("--end-faces",choices=["on","off"],default="on")
    p.add_argument("--budget-s",type=float,default=600.)
    p.add_argument("--tag",default="")
    a=p.parse_args()

    folder=ROOT/"paper_output/results/axisymmetric_check"/f"Nr{a.nr}_Nz{a.nz}_end_{a.end_faces}"
    {a.tag}"
    if folder.exists():
        raise FileExistsError(f"Refusing to overwrite {folder}")
    folder.mkdir(parents=True)
    start=datetime.now(timezone.utc).isoformat()
    print(f"START axisymmetric {folder.name} {start}",flush=True)
    try:
        run=solve_axisymmetric(a.nr,a.nz,a.end_faces=="on",a.budget_s)
        diagnostic=run.diagnostics()
        print(json.dumps({"axisymmetric_complete":diagnostic}),flush=True)

    radial=solve_case(Settings(question="Q23",intervals=a.nr,face_scheme="kirchhoff",horizon_h=
    100.))
    comparison,samples=compare(run,radial)
    np.savez_compressed(folder/"comparison_fields.npz",**samples)
    if hashlib.sha256(CODE_PATH.read_bytes()).hexdigest()!=CODE_HASH:
        raise RuntimeError("axisymmetric source changed during execution")
    if hashlib.sha256(CORE_PATH.read_bytes()).hexdigest()!=CORE_HASH:
        raise RuntimeError("shared core source changed during execution")

    summary={"started_at_utc":start,"finished_at_utc":datetime.now(timezone.utc).isoformat(),
        "settings":{"nr":a.nr,"nz":a.nz,"end_faces":a.end_faces=="on",
            "R_m":R0,"half_length_m":LENGTH/2,
            "reference":asdict(run.model.settings)},
        "diagnostics":diagnostic,"one_dimensional_diagnostics":radial.diagnostics(),
        "comparison":comparison,
        "provenance":{"axisymmetric_sha256":CODE_HASH,"core_sha256":CORE_HASH,
            "inputs":run.model.reference.input_records},
        "gui_reproduced":False,"human_review_status":"pending",
        "limits":"Same-physics numerical comparison, not experimental validation. Grid/time
    convergence evaluated separately."}

    (folder/"summary.json").write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding
    ="utf-8")
    print(json.dumps({"folder":str(folder),"comparison":comparison}),flush=True)
    except Exception:
        (folder/"failure.txt").write_text(traceback.format_exc(),encoding="utf-8")
        raise

```

```
if __name__=="__main__":
    main()
```

9.2.13 compare_analytic.py

```
from __future__ import annotations

import csv
import hashlib
import json
import os
from dataclasses import asdict
from datetime import datetime, timezone
from pathlib import Path
import platform
import sys
import time

import numpy as np

from drying_core import Settings, solve_case, load_inputs, LOADED_CODE_SHA256
from validate_bessel import bessel_temperature, bessel_moisture

ROOT=Path(__file__).resolve().parents[3]
OUT=ROOT/'paper_output/results/analytic_comparison'
SOURCE=Path(__file__).resolve()
CORE=SOURCE.with_name('drying_core.py')
REFERENCE=SOURCE.with_name('validate_bessel.py')

def sha(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()

def file_record(path):
    p=Path(path)
    return {'path':p.relative_to(ROOT).as_posix(),'bytes':p.stat().st_size,'sha256':sha(p)}

def write_json(path,data):
    path.write_text(json.dumps(data,ensure_ascii=False,indent=2,allow_nan=False)+'\n',encoding='utf-8')

def norm_summary(actual,reference,times,radii):
    err=actual-reference
    idx=np.unravel_index(np.argmax(np.abs(err)),err.shape)
    return {
        'max_abs_error':float(np.max(np.abs(err))),
        'rms_error':float(np.sqrt(np.mean(err**2))),
        'mean_signed_error':float(np.mean(err)),
        'max_error_time_s':float(times[idx[0]]),
        'max_error_radius_m':float(radii[idx[1]]),
        'error_at_max_location':float(err[idx]),
        'centre_max_abs_error':float(np.max(np.abs(err[:,0]))),
        'surface_max_abs_error':float(np.max(np.abs(err[:,-1]))),
        'finite_values':bool(np.all(np.isfinite(actual)) and np.all(np.isfinite(reference))),
    }
```

```

def write_comparison_csv(path,times,radii,T,Tref,C,Cref):
    with path.open('w',encoding='utf-8',newline='') as fh:
        writer=csv.writer(fh)
        writer.writerow(['time_s','radius_m','fvm_temperature_K','analytic_temperature_K',
                        'temperature_error_K','fvm_moisture_kg_per_kg',
                        'analytic_frozen_D_moisture_kg_per_kg','moisture_error_kg_per_kg'])
        for i,t in enumerate(times):
            for j,r in enumerate(radii):
                numeric=[t,r,T[i,j],Tref[i,j],T[i,j]-Tref[i,j],C[i,j]]
                values=[format(float(v),'.17g') for v in numeric]
                values += ['', ''] if Cref is None else
[format(Cref[i,j],'.17g'),format(C[i,j]-Cref[i,j],'.17g')]
                writer.writerow(values)

def main():
    started=time.perf_counter()
    if Path.cwd().resolve() != ROOT:
        raise RuntimeError('Run from the 2026CUMCM workspace root')
    OUT.mkdir(parents=True,exist_ok=True)
    source_records=[file_record(p) for p in [SOURCE,CORE,REFERENCE]]
    if sha(CORE) != LOADED_CODE_SHA256:
        raise RuntimeError('Solver file changed after import')
    env,_,inputs=load_inputs(with_records=True)
    env_t,env_T,env_C=env['time_s'],env['temperature_K'],env['air_moisture_kg_per_kg']
    paper_times=np.array([100.,300.,600.,900.,1200.,1500.,1800.])
    paper_radii=np.array([0.,.005,.01,.015,.02])
    dense_times=np.arange(0.,1800.+1,10.)
    dense_radii=np.linspace(0.,.02,21)
    D=float(7e-9*np.exp(-.89/2.55))
    analytic={}
    for grid,ts,rs in [('paper',paper_times,paper_radii),('dense',dense_times,dense_radii)]:
        exact_T=bessel_temperature(ts,rs,env_t,env_T,n_terms=240)
        exact_C=bessel_moisture(ts,rs,env_t,env_C,D,n_terms=240)
        exact_T_refined=bessel_temperature(ts,rs,env_t,env_T,n_terms=480)
        exact_C_refined=bessel_moisture(ts,rs,env_t,env_C,D,n_terms=480)
        analytic[grid]=(exact_T,exact_C,{
            'heat_240_480_max_difference_K':float(np.max(np.abs(exact_T-exact_T_refined))),
            'frozen_water_240_480_max_difference_kg_per_kg':float(np.max(np.abs(exact_C-exact_C_refined)
        )))
    case_results=[]
    plot_payload=[]
    for mode,N in
[('nonlinear_Q1',100),('nonlinear_Q1',200),('nonlinear_Q1',400),('frozen_D',100),('frozen_D',
',400),('frozen_D',800)]:
        name=f'{mode}_N{N}'
        setting=Settings(question='Q1',intervals=N,rtol=1e-10,
            atol_temperature=1e-10,atol_moisture=1e-11,
            early_max_step_s=2.,max_step_s=2.,
            face_scheme='kirchhoff',constant_D=D if mode=='frozen_D' else None)
    print(json.dumps({'event':'starting_case','case':name,'elapsed_total_s':time.perf_counter()
-started}),flush=True)
        run=solve_case(setting)
        for rec in source_records+inputs:
            if sha(ROOT/rec['path']) != rec['sha256']:
                raise RuntimeError('Source/input changed during comparison: '+rec['path'])
            case={'case_id':name,'settings':asdict(setting),'diagnostics':run.diagnostics(),
                'comparison_scope':'temperature only; nonlinear C has no Bessel reference' if
mode=='nonlinear_Q1'
                    else 'constant-property heat and frozen-D linear moisture',
                'comparisons':{},'output_artifacts':[]}

```

```

        for grid,ts,rs in
[('paper',paper_times,paper_radii),('dense',dense_times,dense_radii)]:
            T,C=run.fields(ts,radii_m=rs)
            Tref,Cref,truncation=analytic[grid]
            if mode!='frozen_D':
                Cref=None

comparisons={'temperature':norm_summary(T,Tref,ts,rs),'analytic_truncation_sensitivity':truncation}

        if Cref is not None:
            comparisons['frozen_D_moisture']=norm_summary(C,Cref,ts,rs)
            case['comparisons'][grid]=comparisons
            path=OUT/f'{name}_{grid}.csv'
            write_comparison_csv(path,ts,rs,T,Tref,C,Cref)
            case['output_artifacts'].append(file_record(path))
            if grid=='dense':
                plot_payload.append((mode,N,ts,np.max(np.abs(T-Tref),axis=1),
                    None if Cref is None else np.max(np.abs(C-Cref),axis=1)))

        node_T,node_C=run.fields(paper_times)
        node_r=run.model.x*.02
        edges=np.r_[0.,(node_r[1:]+node_r[:-1])/2.,.02]

mean_T=bessel_temperature(paper_times,node_r,env_t,env_T,n_terms=240,cell_edges_m=edges)
point_T=bessel_temperature(paper_times,node_r,env_t,env_T,n_terms=240)
case['representation_diagnostic']={
    'primary_interpretation':'collocated nodal values with surrounding dual
control-volume balances',
    'strict_cell_average_interpretation_is_not_primary':True,
    'heat_node_max_abs_error_K':float(np.max(np.abs(node_T-point_T))),
    'heat_vs_annular_average_max_abs_error_K':float(np.max(np.abs(node_T-mean_T))),

'analytic_point_vs_annular_average_max_difference_K':float(np.max(np.abs(point_T-mean_T))),
    'note':'Endpoint dual cells are one-sided, so point-vs-average differences can be
first order; they are not physical model error.')}
        if mode=='frozen_D':
            early_times=np.array([1.,2.,5.,10.,20.,30.,60.])
            early_radii=np.array([0.,.019,.02])
            early_T,early_C=run.fields(early_times,radii_m=early_radii)
            early_Tref=bessel_temperature(early_times,early_radii,env_t,env_T,n_terms=480)
            early_Cref=bessel_moisture(early_times,early_radii,env_t,env_C,D,n_terms=480)
            early_Cref240=bessel_moisture(early_times,early_radii,env_t,env_C,D,n_terms=240)
            case['early_boundary_layer']={
                'times_s':early_times.tolist(),'radii_m':early_radii.tolist(),
                'moisture':norm_summary(early_C,early_Cref,early_times,early_radii),
                'surface_errors_kg_per_kg':(early_C[:-1]-early_Cref[:-1]).tolist(),

'analytic_240_480_max_difference':float(np.max(np.abs(early_Cref240-early_Cref)))}
            early_path=OUT/f'{name}_early.csv'

write_comparison_csv(early_path,early_times,early_radii,early_T,early_Tref,early_C,early_Cref)

            case['output_artifacts'].append(file_record(early_path))
            case_results.append(case)
            write_json(OUT/f'{name}_report.json',case)
            print(json.dumps({'event':'completed_case','case':name,'elapsed_s':run.elapsed_s,

'paper_T_error':case['comparisons']['paper']['temperature']['max_abs_error'],

'dense_T_error':case['comparisons']['dense']['temperature']['max_abs_error'],

'dense_C_error':case['comparisons']['dense'].get('frozen_D_moisture',{}).get('max_abs_error

```

```

    ')), flush=True)
    heat=[c for c in case_results if c['case_id'].startswith('nonlinear_Q1')]
    water=[c for c in case_results if c['case_id'] in ['frozen_D_N100','frozen_D_N400']]
    refined_water=[c for c in case_results if c['case_id']=='frozen_D_N800'][0]
    convergence={}
    for grid in ['paper','dense']:
        eT=[c['comparisons'][grid]['temperature']['max_abs_error'] for c in heat]
        eC=[c['comparisons'][grid]['frozen_D_moisture']['max_abs_error'] for c in water]
        convergence[grid]={
            'heat_intervals':[100,200,400], 'heat_max_abs_errors_K':eT,
            'heat_observed_orders':[float(np.log(eT[i]/eT[i+1])/np.log(2.)) for i in [0,1]],
            'water_intervals':[100,400], 'water_max_abs_errors_kg_per_kg':eC,
            'water_observed_order_100_400':float(np.log(eC[0]/eC[1])/np.log(4.)),
            'both_refinements_reduce_heat_error':bool(eT[0]>eT[1]>eT[2]),
            'water_refinement_reduces_error':bool(eC[0]>eC[1]),
            'N400_heat_max_error_below_0_0001_K':bool(eT[-1]<1e-4),
            'N400_frozen_water_max_error_below_0_0001':bool(eC[-1]<1e-4),

            'N800_frozen_water_max_abs_error':refined_water['comparisons'][grid]['frozen_D_moisture']['max_abs_error'],

            'water_observed_order_400_800':float(np.log(eC[-1]/refined_water['comparisons'][grid]['frozen_D_moisture']['max_abs_error']/np.log(2.)),
        }
    os.environ.setdefault('MPLCONFIGDIR',str(ROOT/'tmp/cache/matplotlib'))
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    fig,axes=plt.subplots(1,2,figsize=(11,4.3),constrained_layout=True)
    for mode,N,ts,eT,eC in plot_payload:
        if mode=='nonlinear_Q1':
            axes[0].semilogy(ts[1:],np.maximum(eT[1:],1e-16),label=f'N={N}')
        if mode=='frozen_D':
            axes[1].semilogy(ts[1:],np.maximum(eC[1:],1e-16),label=f'N={N}')
    for ax in axes:
        ax.set_xlabel('Time (s)');ax.grid(alpha=.25);ax.legend()
    axes[0].set(title='Q1 heat: FVM vs Bessel',ylabel='Maximum radial absolute error (K)')
    axes[1].set(title='Frozen-D moisture benchmark',ylabel='Maximum radial absolute error (kg/kg)')
    plot_path=OUT/'analytic_error_history.png'
    fig.savefig(plot_path,dpi=180);plt.close(fig)
    for rec in source_records+inputs:
        if sha(ROOT/rec['path'])!=rec['sha256']:
            raise RuntimeError('Source/input changed before final report: '+rec['path'])
        validation_pass=all(v['both_refinements_reduce_heat_error'] and
v['water_refinement_reduces_error']
            and v['N400_heat_max_error_below_0_0001_K'] and
v['N400_frozen_water_max_error_below_0_0001']
                for v in convergence.values())
        report={

'schema_version':'1.0','question_id':'Q1','generated_by':'paper_output/code/modeling/compar
e_analytic.py',
'generated_at':datetime.now(timezone.utc).isoformat(),'status':'PASS' if
validation_pass else 'REVIEW_REQUIRED',
'validation_criteria':'Both heat refinements and 100-to-400 frozen-water refinement
reduce max error; N400 sampled max errors below 1e-4 in physical units.',
'scope':'Numerical verification against the selected constant-coefficient PDE; no
internal experimental accuracy claim.',
'point_value_comparison':'Nodal-dual state is compared to exact point values at the same
physical radii. Separate representation diagnostics quantify strict annular averages.',

'primary_space_time_samples':{'paper_times_s':paper_times.tolist(),'paper_radii_m':paper_ra

```

```

dii.tolist(),

        'dense_time_step_s':10.,'dense_time_range_s':[0.,1800.],

'dense_radius_step_m':.001,'dense_radius_range_m':[0.,.02]],
    'nonlinear_moisture_note':'The nonlinear Q1 C solution is stored for provenance only,
not compared with constant-D Bessel or labeled analytic truth.',
    'constant_diffusion_coefficient_m2_s':D,
    'core_static_review':{
        'kirchhoff_primitive':'F(C)=C*exp(-a/C)+a*Ei(-a/C), so F_prime(C)=exp(-a/C)',
        'water_face_flux':'Internal face radius times
D0*exp(-3850/T_face)*(F(C_right)-F(C_left))/dx, with no thermal factor in the differenced
potential.',
        'Q1_temperature_factor':'Exactly one, since D1 has no temperature dependence.',
        'no_spurious_soret_term':True,
        'internal_flux_cancellation':'Each face occurs with opposite signs in adjacent
control-volume balances; weighted sum leaves only the surface flux.',
        'density_interpretation':'rho*cp is effective thermal capacity; independent
dry-matter weighting is used for conserved moisture.',
        'scope_limitation':'Static consistency and linear benchmark do not alone validate
all nonlinear long-time or shrinkage scenarios.'},
    'convergence':convergence,'cases':case_results,
    'total_elapsed_s':time.perf_counter()-started,
    'visual_studio_gui':'pending','human_review':'pending',
    'execution_provenance':{'run_command':sys.executable+' -B '+' '.join(sys.argv),
        'run_exit_code':0,'python_version':platform.python_version(),
        'source_files':source_records,'input_artifacts':inputs,
        'output_artifacts':[file_record(p) for p in
sorted(OUT.glob('*.csv'))]+[file_record(plot_path)]},
    }
    write_json(OUT/'analytic_comparison_report.json',report)

print(json.dumps({'status':report['status'],'convergence':convergence,'total_elapsed_s':rep
ort['total_elapsed_s']},indent=2,flush=True)
    return 0

if __name__=='__main__':
    raise SystemExit(main())

```

9.2.14 production_provenance.py

```

from __future__ import annotations

from contextlib import contextmanager
from datetime import datetime, timezone
import copy
import hashlib
import importlib.metadata
import json
import os
from pathlib import Path
import platform
import subprocess
import sys
import uuid

SOURCE = Path(__file__).resolve()
ROOT = SOURCE.parents[3]
LOADED_CODE_SHA256 = hashlib.sha256(SOURCE.read_bytes()).hexdigest()

def utc_now():
    return datetime.now(timezone.utc).isoformat()

```

```

def sha256(path):
    digest = hashlib.sha256()
    with Path(path).open('rb') as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b''):
            digest.update(block)
    return digest.hexdigest()

def within(root, path):
    root, path = Path(root).resolve(), Path(path).resolve()
    if not path.is_relative_to(root):
        raise ValueError('Path is outside the intended workspace: ' + str(path))
    return path

def file_record(path, root=ROOT):
    path = within(root, path)
    before = path.stat()
    digest = sha256(path)
    after = path.stat()
    if (before.st_size, before.st_mtime_ns) != (after.st_size, after.st_mtime_ns):
        raise RuntimeError('File changed while hashing: ' + str(path))
    return {'path': path.relative_to(Path(root).resolve()).as_posix(),
            'bytes': after.st_size, 'sha256': digest, 'exists': True}

def assert_records(records, root=ROOT):
    for expected in records:
        path = within(root, Path(root) / expected['path'])
        actual = file_record(path, root)
        for key in ('sha256', 'bytes'):
            if key in expected and actual[key] != expected[key]:
                raise RuntimeError('Artifact/input changed: ' + expected['path'])

def json_bytes(value):
    return (json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) +
            '\n').encode('utf-8')

def atomic_bytes(path, data):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = path.with_name(path.name + '.tmp-' + uuid.uuid4().hex)
    try:
        with temporary.open('xb') as stream:
            stream.write(data)
            stream.flush()
            os.fsync(stream.fileno())
        os.replace(temporary, path)
    finally:
        if temporary.exists():
            temporary.unlink()

def write_json(path, value):
    atomic_bytes(path, json_bytes(value))

def exclusive_json(path, value):
    path = Path(path)

```

```

path.parent.mkdir(parents=True, exist_ok=True)
with path.open('xb') as stream:
    stream.write(json_bytes(value))
    stream.flush()
    os.fsync(stream.fileno())

def runtime_record():
    versions = {}
    for name in ('numpy', 'scipy', 'openpyxl', 'matplotlib'):
        versions[name] = importlib.metadata.version(name)
    return {'executable': sys.executable, 'implementation': platform.python_implementation(),
            'version': platform.python_version(), 'platform': platform.platform(),
            'installed_library_versions': versions,
            'thread_environment': {name: os.environ.get(name) for name in
('OPENBLAS_NUM_THREADS', 'OMP_NUM_THREADS')},
            'scope': 'Installed distributions; actual imported module paths are recorded by
worker.'}

def pointer(document, json_pointer):
    current = document
    if not json_pointer.startswith('/'):
        raise ValueError('Expected an absolute JSON pointer')
    for token in json_pointer[1:].split('/'):
        token = token.replace('~1', '/').replace('~0', '~')
        current = current[int(token)] if isinstance(current, list) else current[token]
    return current

def validate_metric_evidence(metrics, root=ROOT):
    cache = {}
    for metric in metrics:
        path = within(root, Path(root) / metric['evidence_path'])
        if path not in cache:
            cache[path] = json.loads(path.read_text(encoding='utf-8'))
        actual = pointer(cache[path], metric['evidence_json_pointer'])
        if actual != metric['value']:
            raise ValueError('Metric does not equal its saved evidence: ' + metric['metric_name'])
    return {'status': 'PASS', 'checked_metric_count': len(metrics),
            'scope': 'Exact equality to the identified saved JSON value, not physical accuracy.'}

def figure_artifact_records(figure, root=ROOT):
    candidates = []
    primary = figure.get('path') or figure.get('output_path')
    if primary:
        candidates.append({key: figure[key] for key in ('path', 'bytes', 'sha256') if key in
figure})
    if figure.get('path') else {'path': primary})
    artifacts = figure.get('artifacts', {})
    candidates.extend(artifacts.values() if isinstance(artifacts, dict) else artifacts)
    for name in ('plot_data', 'metadata'):
        if isinstance(figure.get(name), dict) and figure[name].get('path'):
            candidates.append(figure[name])
    checked = {}
    for expected in candidates:
        if not isinstance(expected, dict) or not expected.get('path'):
            raise ValueError('Invalid figure artifact record')
        actual = file_record(Path(root) / expected['path'], root)
        if not actual['bytes']:
            raise ValueError('Empty figure artifact: ' + actual['path'])
        if expected.get('sha256') and actual['sha256'] != expected['sha256']:

```

```

        raise RuntimeError('Existing figure hash mismatch: ' + actual['path'])
    checked[actual['path']] = actual
    if not checked:
        raise ValueError('Computed figure has no usable artifact')
    return list(checker.values())

def merge_figure_index(existing, new_figures, root=ROOT):
    merged, omitted, replaced, preserved = {}, [], [], []
    for figure in (existing or {}).get('figures', []):
        status = str(figure.get('status', '')).lower()
        if (figure.get('placeholder') or figure.get('planned') or
            status in ('planned', 'pending', 'not_started') or figure.get('ok') is False):
            omitted.append({'figure_id': figure.get('figure_id'), 'reason': 'not an existing
usable computed figure'})
            continue
        figure_artifact_records(figure, root)
        if not figure.get('figure_id'):
            raise ValueError('Existing computed figure needs figure_id')
        merged[figure['figure_id']] = copy.deepcopy(figure)
        preserved.append(figure['figure_id'])
    seen = set()
    for figure in new_figures:
        identity = figure['figure_id']
        if identity in seen:
            raise ValueError('Duplicate new figure_id: ' + identity)
        seen.add(identity)
        figure_artifact_records(figure, root)
        if identity in merged:
            replaced.append(identity)
            merged[identity] = copy.deepcopy(figure)
    return list(merged.values()), {'preserved_existing_ids': [v for v in preserved if v not in
replaced],
    'replaced_same_ids': replaced, 'omitted_noncomputed_entries': omitted,
    'prior_index_retained_in_publication_backup': True}

@contextmanager
def publication_lock(root, run_id):
    path = within(root, Path(root) / 'paper_output/results/.production_publish.lock')
    identity = {'run_id': run_id, 'pid': os.getpid(), 'created_at': utc_now(), 'token':
uuid.uuid4().hex}
    exclusive_json(path, identity)
    try:
        yield
    finally:
        if path.exists() and json.loads(path.read_text(encoding='utf-8')) == identity:
            path.unlink()

def publish_transaction(root, directory, updates, manifest):

    root, directory = Path(root).resolve(), within(root, directory)
    if manifest['runs'][0]['returncode'] != 0:
        raise ValueError('Cannot publish a nonzero worker return code')
    manifest_name = 'paper_output/results/run_manifest.json'
    if manifest_name in updates:
        raise ValueError('The manifest must be committed last')
    paths = [*updates, manifest_name]
    previous = {}
    backup = directory / 'publication_previous'
    for relative in paths:

```

```

        target = within(root, root / relative)
        previous[relative] = target.read_bytes() if target.exists() else None
        if previous[relative] is not None:
            atomic_bytes(backup / relative, previous[relative])
        write_json(backup / 'inventory.json', {'saved_at': utc_now(),
        'files': [{'path': p, 'previously_existed': b is not None} for p, b in previous.items()]})
        intended = {relative: json_bytes(document) for relative, document in updates.items()}
        changed = False
    try:

        write_json(root / manifest_name, {'status': 'PUBLISHING', 'run_id':
manifest['runs'][0]['run_id'],
        'generated_at': utc_now(), 'version_directory':
directory.relative_to(root).as_posix()})
        changed = True
        for relative, content in intended.items():
            atomic_bytes(root / relative, content)
        for relative, content in intended.items():
            if sha256(root / relative) != hashlib.sha256(content).hexdigest():
                raise RuntimeError('Published contract bytes differ: ' + relative)

        assert_records(manifest['runs'][0]['output_artifacts'], root)
        assert_records(manifest['runs'][0].get('input_files', []), root)
        font = manifest['runs'][0].get('font')
        if font and sha256(font['path']) != font['sha256']:
            raise RuntimeError('Font changed before publication commit')
        write_json(directory / 'run_manifest.json', manifest)
        write_json(root / manifest_name, manifest)
    except Exception as error:
        rollback_errors = []
        if changed:

            for relative in updates:
                try:
                    target = root / relative
                    if previous[relative] is None:
                        if target.exists():
                            target.unlink()
                    else:
                        atomic_bytes(target, previous[relative])
                except Exception as rollback_error:
                    rollback_errors.append({'path': relative, 'error': repr(rollback_error)})
            if not rollback_errors:
                try:
                    target = root / manifest_name
                    if previous[manifest_name] is None:
                        if target.exists():
                            target.unlink()
                    else:
                        atomic_bytes(target, previous[manifest_name])
                except Exception as rollback_error:
                    rollback_errors.append({'path': manifest_name, 'error':
repr(rollback_error)})
            if rollback_errors:
                write_json(root / manifest_name, {'status': 'FAILED_PUBLICATION', 'error':
repr(error),
        'rollback_errors': rollback_errors, 'generated_at': utc_now()})
            write_json(directory / 'publication_failure.json', {'status': 'FAIL', 'error':
repr(error),
        'rollback_errors': rollback_errors, 'prior_publication_restored': changed and not
rollback_errors,

```

```

        'generated_at': utc_now()})
    raise

def measured_process(command, cwd, log_path):

    started = utc_now()
    stream_error = None
    with Path(log_path).open('x', encoding='utf-8') as log:
        process = subprocess.Popen(command, cwd=cwd, stdout=subprocess.PIPE,
stderr=subprocess.STDOUT,
            text=True, encoding='utf-8', errors='replace')
        try:
            for line in process.stdout:
                log.write(line)
                log.flush()
                print(line, end='', flush=True)
        except BaseException as error:
            stream_error = error

        for unused in process.stdout:
            pass
    finally:

        code = process.wait()
        process.stdout.close()
    result = {'pid': process.pid, 'returncode': code, 'started_at': started,
        'finished_at': utc_now(), 'command': subprocess.list2cmdline(command),
        'argv': command, 'cwd': str(Path(cwd).resolve())}
    if stream_error is not None:
        result['stdout_capture_error'] = repr(stream_error)

    write_json(Path(log_path).with_name('process_result.json'), result)
    if stream_error is not None:
        raise RuntimeError('Child exited, but stdout capture failed') from stream_error
    return result

def entry_sequence_selfcheck(directory):

    from types import SimpleNamespace
    from unittest.mock import patch
    import numpy as np
    import run_modeling as entry

    source = file_record(entry.SOURCE)
    start = {'input_files': [source], 'runtime': {}, 'font': {}, 'worker_command': 'fixture only;
no model solve'}
    global_paths = [ROOT / p for p in entry.CONTRACT_TARGETS.values()]
    global_before = {str(p): sha256(p) if p.exists() else None for p in global_paths}
    active, events, identities = set(), [], {}
    class FixtureRun:
        def __init__(self, name, intervals):
            if active:
                raise AssertionError('Two Runs are retained simultaneously')
            self.name, self.intervals = name, intervals
            self.end_s = 1800. if name == 'Q1' else 15002.
            active.add(id(self))
            events.append('solve:' + name)
        def fields(self, times, material_x):

```

```

        shape = (len(times), len(material_x))
        return np.full(shape, 310.), np.full(shape, .1499)
    def close(self):
        events.append('close:' + self.name)
        active.remove(id(self))
    def save_fixture(run, output):
        return {'settings': {'intervals': run.intervals},
                'diagnostics': {'max_mass_balance_abs_kg_per_kg': 0.}}
    def fixture_completion(run):
        return {'reported_time_s': 15000.12, 'reported_drying_time_h': 15000.12 / 3600.,
                'critical_event_h': 15000. / 3600., 'max_C_at_reported_time': .1499}
    def export_fixture(run, qid, output):
        events.append('export:' + qid)
        identities[qid] = id(run)
        path = output / (qid + '_fixture.json')
        write_json(path, {'fixture_only': True, 'qid': qid})
        return {'question_id': qid, 'artifacts': [file_record(path)], 'size': {},
                'manifest_path': str(path)}
    def validate_fixture(exports, runs):
        export = exports[0]
        path = Path(export['manifest_path']).with_suffix('.validation.json')
        write_json(path, {'fixture_only': True})
        return {'status': 'PASS', 'fully_verified_with_live_Run': True, 'exports':
[{'report_path': str(path)}]}
    def plot_fixture(run, qid, output):
        events.append('plot:' + qid)
        path = output / (qid + '_fixture.bin')
        path.parent.mkdir(parents=True, exist_ok=True)
        path.write_bytes(b'fixture bytes; not a scientific figure')
        return {'figure_id': 'fixture_' + qid, 'question_id': qid, **file_record(path)}
    mappings = {'drying_core': SimpleNamespace(completion=fixture_completion),
                'q3_model': SimpleNamespace(completion=fixture_completion),
                'export_outputs': SimpleNamespace(export_question=export_fixture,
validate_exports=validate_fixture),
                'publication_plots': SimpleNamespace(make_question_plot=plot_fixture)}
    for key, name in [('Q1', 'q1_model'), ('Q23', 'q2_model'), ('Q4', 'q4_model')]:
        mappings[name] = SimpleNamespace(solve=lambda n, key=key: FixtureRun(key, n))
    original_import = importlib.import_module
    def importer(name, *args, **kwargs):
        return mappings[name] if name in mappings else original_import(name, *args, **kwargs)
    args = SimpleNamespace(version='fixture_sequence', n1=2, n23=2, n4=2, review=False)
    with patch.object(entry, 'assert_dependencies'), patch.object(entry,
'imported_model_records', return_value=[]), \
        patch.object(entry.importlib, 'import_module', side_effect=importer):
        entry.worker(args, directory / 'entry_fixture_success', start)
        assert not active and identities['Q2'] == identities['Q3']
        assert events == ['solve:Q1', 'export:Q1', 'plot:Q1', 'close:Q1',
                'solve:Q23', 'export:Q2', 'plot:Q2', 'export:Q3', 'plot:Q3', 'close:Q23',
                'solve:Q4', 'export:Q4', 'plot:Q4', 'close:Q4']
    def fail_q3(run, qid, output):
        if qid == 'Q3':
            raise RuntimeError('injected fixture Q3 export failure')
        return export_fixture(run, qid, output)
    mappings['export_outputs'].export_question = fail_q3
    try:
        entry.worker(args, directory / 'entry_fixture_failure', start)
    except RuntimeError as error:
        assert 'injected fixture' in str(error)
    else:
        raise AssertionError('Fixture failure was not propagated')
    assert not active and events[-1] == 'close:Q23'
    global_after = {str(p): sha256(p) if p.exists() else None for p in global_paths}
    assert global_after == global_before

```

```

    return ['entry_one_run_at_a_time_and_Q2_Q3_same_identity',
            'entry_closes_run_after_export_failure',
            'worker_does_not_publish_global_contracts']

def selfcheck():

    import tempfile
    from unittest.mock import patch

    base = ROOT / 'notes/A-modeling/2026-09-10/data/production_entry_contract_selfcheck'
    base.mkdir(parents=True, exist_ok=True)
    directory = Path(tempfile.mkdtemp(prefix='check-', dir=base))
    fixture = directory / 'fixture'
    fixture.mkdir()
    checks = []
    original = fixture / 'input.txt'
    original.write_text('observed bytes', encoding='utf-8')
    rec = file_record(original, fixture)
    assert_records([rec], fixture)
    original.write_text('changed bytes', encoding='utf-8')
    try:
        assert_records([rec], fixture)
    except RuntimeError:
        checks.append('hash_mutation_rejected')
    else:
        raise AssertionError('Changed input was accepted')
    lock = fixture / 'launch.json'
    exclusive_json(lock, {'version': 'one'})
    try:
        exclusive_json(lock, {'version': 'two'})
    except FileExistsError:
        checks.append('duplicate_version_marker_rejected')
    else:
        raise AssertionError('Existing marker overwritten')
    evidence = fixture / 'summary.json'
    write_json(evidence, {'metric_samples': {'Q3': {'C': [1.1499], 'time_s': 123.48}}})
    metric = {'metric_name': 'center_C', 'evidence_path': 'summary.json',
              'evidence_json_pointer': '/metric_samples/Q3/C/0/0', 'value': 1.1499}
    validate_metric_evidence([metric], fixture)
    metric['value'] = .1
    try:
        validate_metric_evidence([metric], fixture)
    except ValueError:
        checks.append('metric_mismatch_rejected')
    else:
        raise AssertionError('Wrong evidence accepted')
    image_path = fixture / 'observed.png'
    image_path.write_bytes(b'fixture-artifact-only-not-a-real-figure')
    figure = {'figure_id': 'observed', 'path': 'observed.png', 'status': 'computed',
              'sha256': sha256(image_path), 'placeholder': False}
    merged, note = merge_figure_index({'figures': [figure, {'figure_id': 'planned', 'planned':
True}], [], fixture)
    assert len(merged) == 1 and note['preserved_existing_ids'] == ['observed']
    checks.append('existing_artifact_preserved_planned_not_promoted')
    old_contract = {'generation': 'old'}
    write_json(fixture / 'paper_output/results/model_results.json', old_contract)
    write_json(fixture / 'paper_output/results/run_manifest.json', {'status': 'PASS', 'old':
True})
    version = fixture / 'paper_output/results/production/test'
    version.mkdir(parents=True)
    updates = {'paper_output/results/model_results.json': {'generation': 'new'}}
    intended_bytes = json_bytes(updates['paper_output/results/model_results.json'])

```

```

intended_record = {'path': 'paper_output/results/model_results.json', 'bytes':
len(intended_bytes),
                    'sha256': hashlib.sha256(intended_bytes).hexdigest()}
manifest = {'status': 'PASS', 'runs': [{'run_id': 'test', 'returncode': 0,
    'input_files': [file_record(original, fixture)], 'output_artifacts':
[intended_record]]}]
real_write = atomic_bytes
fired = False
def fail_contract_once(path, data):
    nonlocal fired
    if Path(path) == fixture / 'paper_output/results/model_results.json' and not fired:
        fired = True
        raise OSError('injected fixture publication failure')
    real_write(path, data)
with patch.object(sys.modules[__name__], 'atomic_bytes', fail_contract_once):
    try:
        publish_transaction(fixture, version, updates, manifest)
    except OSError:
        pass
    else:
        raise AssertionError('Injected failure was swallowed')
assert json.loads((fixture / 'paper_output/results/model_results.json').read_text()) ==
old_contract
assert json.loads((fixture / 'paper_output/results/run_manifest.json').read_text()) ==
{'status': 'PASS', 'old': True}
checks.append('publication_failure_rolls_back_old_contract_and_manifest')
publish_transaction(fixture, version, updates, manifest)
assert json.loads((fixture / 'paper_output/results/model_results.json').read_text()) ==
{'generation': 'new'}
checks.append('successful_publication_commits_manifest_last')
exits = [measured_process([sys.executable, '-c', 'import sys; print("tiny-process");
sys.exit(' + str(code) + ')'],
                        fixture, directory / ('exit-' + str(code) + '.log')) for code in (0,
7)]
assert [p['returncode'] for p in exits] == [0, 7]
checks.append('actual_process_zero_and_nonzero_exits_observed')
checks.extend(entry_sequence_selfcheck(directory))
report = {'status': 'PASS', 'scope': 'Isolated tiny-file contracts; no model solve or real
production publication.',
        'checks': checks, 'processes': exits, 'source': file_record(SOURCE), 'generated_at':
utc_now(),
        'visual_studio_gui': 'pending', 'human_review': 'pending'}
write_json(directory / 'selfcheck.json', report)
print(json.dumps({'status': report['status'], 'checks': len(checks), 'report':
str(directory / 'selfcheck.json')}))

if __name__ == '__main__':
    if sys.argv[1:] != ['--selfcheck']:
        raise SystemExit('Use --selfcheck for isolated contract validation only')
    if Path.cwd().resolve() != ROOT:
        raise SystemExit('Use the competition workspace as cwd')
    selfcheck()

```

9.2.15 publication_plots.py

```

from __future__ import annotations

import argparse
from dataclasses import asdict
from datetime import datetime, timezone
import hashlib
import inspect

```

```

import json
import os
from pathlib import Path
import sys

import numpy as np

ROOT=Path(__file__).resolve().parents[3]
SOURCE=Path(__file__).resolve()
LOADED_PLOT_SHA256=hashlib.sha256(SOURCE.read_bytes()).hexdigest()
os.environ.setdefault('MPLCONFIGDIR',str(ROOT/'tmp/cache/matplotlib'))
import matplotlib
matplotlib.use('Agg')
from matplotlib import font_manager
from matplotlib.colors import Normalize
from matplotlib.ticker import NullLocator
import matplotlib.pyplot as plt

FONT_PATH=Path('C:/Windows/Fonts/msyh.ttc')
if not FONT_PATH.exists():
    raise RuntimeError('Verified Chinese font Microsoft YaHei is unavailable')
font_manager.fontManager.addfont(str(FONT_PATH))
FONT_NAME=font_manager.FontProperties(fname=str(FONT_PATH)).get_name()
STYLE={'font.family':FONT_NAME,'font.size':10,'axes.titlesize':11,'axes.labelsize':10,
        'axes.unicode_minus':False,'axes.spines.top':False,'axes.spines.right':False,
        'legend.frameon':False,'pdf.fonttype':42,'ps.fonttype':42,'svg.fonttype':'path',
        'savefig.facecolor':'white','figure.facecolor':'white','axes.facecolor':'white'}
COLOURS={'centre':'#2563A6','mean':'#C76829','surface':'#218573','threshold':'#51555D'}

def _sha(path):
    h=hashlib.sha256()
    with Path(path).open('rb') as f:
        for b in iter(lambda:f.read(1024*1024),b''):h.update(b)
    return h.hexdigest()

def _record(path):
    p=Path(path).resolve()
    return
{'path':p.relative_to(ROOT).as_posix(),'bytes':p.stat().st_size,'sha256':_sha(p),'exists':True}

def _directory(directory):
    p=Path(directory).resolve()
    if not p.is_relative_to(ROOT) or p.is_relative_to(ROOT/'problem_files'):
        raise ValueError('Write figures only inside the competition workspace outside original inputs')
    p.mkdir(parents=True,exist_ok=True)
    return p

def _run_source(run):
    module=sys.modules[run.model.__class__.__module__]
    path=Path(inspect.getfile(run.model.__class__)).resolve()
    if getattr(module,'LOADED_CODE_SHA256',_sha(path))!=_sha(path):
        raise RuntimeError('Core changed after import; regenerate figure from a consistent Run')
    records=[_record(path)]
    jac=path.with_name('analytic_jacobian.py')
    if jac.exists():records.append(_record(jac))
    for rec in run.model.input_records:
        if _sha(ROOT/rec['path'])!=rec['sha256']:

```

```

        raise RuntimeError('Run input changed: '+rec['path'])
    records.append(rec)
    return {'kind':'live_Run','settings':asdict(run.model.settings),'files':records,
            'event_s':None if run.event_s is None else
            float(run.event_s),'end_s':float(run.end_s)}

def _unchanged(sources):
    if _sha(SOURCE)!=LOADED_PLOT_SHA256:
        raise RuntimeError('Plot module changed after import')
    for source in sources:
        for rec in source.get('files',[]):
            if _sha(ROOT/rec['path'])!=rec['sha256']:
                raise RuntimeError('Source changed during figure generation: '+rec['path'])

def _save(fig,figure_id,qid,title,caption,directory,sources,data,extra=None):
    directory=_directory(directory)
    _unchanged(sources)
    data_path=directory/(figure_id+'_data.npz')
    np.savez_compressed(data_path,**data)
    artifacts={}
    for ext in ['png','svg','pdf']:
        path=directory/(figure_id+'_'+ext)
        fig.savefig(path,dpi=210 if ext=='png' else 160,bbox_inches='tight',pad_inches=.14)
        artifacts[ext]=_record(path)
    plt.close(fig)
    _unchanged(sources)
    record={'figure_id':figure_id,'question_id':qid,'title':title,'caption':caption,
            'purpose':caption,'path':artifacts['png']['path'],
            'bytes':artifacts['png']['bytes'],'sha256':artifacts['png']['sha256'],
            'status':'computed','ok':True,'placeholder':False,'exists':True,
            'artifacts':artifacts,'plot_data':_record(data_path),'sources':sources,
            'generated_by':'paper_output/code/modeling/publication_plots.py',
            'source_code_sha256':LOADED_PLOT_SHA256,
            'generated_at':datetime.now(timezone.utc).isoformat(),
            'font':{'name':FONT_NAME,'file':str(FONT_PATH)},

    'visual_review':'pending','human_review':'pending','visual_studio_gui':'pending'}
    if extra:record.update(extra)
    metadata=directory/(figure_id+'.json')

    metadata.write_text(json.dumps(record,ensure_ascii=False,indent=2,allow_nan=False)+'\n',encoding='utf-8')
    record['metadata']=_record(metadata)
    return record

def _footer(fig,text):

    fig.text(.5,.015,text,ha='center',va='bottom',fontsize=8.1,color='#52565E',linespacing=1.45
    )

def _environment_note(run):
    s=run.model.settings
    if s.boundary_extension=='nominal':
        return f'4 h 后环境延拓为 {s.tail_temperature_C:g} °C、等效平衡含水率 {s.tail_equilibrium*s.equilibrium_scale:g} kg/kg'
    if s.boundary_extension=='tail_mean':return '4 h 后环境按末 1 h 观测均值延拓'
    return '4 h 后环境按最后观测值延拓'

```

```

def _profiles(run,qid,directory):
    source=_run_source(run)
    if qid=='Q1':
        ts=np.array([100.,300.,600.,900.,1200.,1500.,1800.])
        labels=[f'{t:g} s' for t in ts]
        title='第一问: 预热阶段的径向温湿分布'
    else:
        ts=np.arange(1,7,dtype=float)*1800
        labels=[f'{t/3600:g} h' for t in ts]
        title='第二问: 前三小时的径向温湿分布'
    if run.end_s<ts[-1]-1e-7:
        raise ValueError('Run does not cover all question-specific plot times')
    rs=np.linspace(0,.02,121)
    T,C=run.fields(ts,radii_m=rs);T=T-273.15
    with plt.rc_context(STYLE):
        fig,axes=plt.subplots(1,2,figsize=(11.8,4.8))
        fig.subplots_adjust(left=.075,right=.97,bottom=.245,top=.82,wspace=.28)
        colours=plt.get_cmap('viridis')(np.linspace(.08,.9,len(ts)))
        handles=[]
        for i,(label,colour) in enumerate(zip(labels,colours)):
            handle,=axes[0].plot(rs*100,T[i],color=colour,lw=1.7,label=label)
            handles.append(handle)
            axes[1].plot(rs*100,C[i],color=colour,lw=1.7,label=label)
        for ax in axes:
            ax.set_xlim(0,2);ax.set_xticks([0,.5,1,1.5,2]);ax.set_xlabel('到药材中心的距离 /
cm')
            ax.grid(alpha=.2,lw=.7)
        axes[0].set(title='(a) 温度',ylabel='温度 / °C')
        axes[1].set(title='(b) 干基含水率',ylabel='干基含水率 / (kg/kg)')
        fig.suptitle(title,y=.97,fontsize=14,fontweight='bold')
        fig.legend(handles,labels,loc='lower
center',bbox_to_anchor=(.5,.075),ncol=len(labels),fontsize=9)
        _footer(fig,'一维径向有效模型的条件性数值结果: 忽略端面与显式潜热, 环境水分按等效平衡边界处
理。')
    return _save(fig,f'fig_{qid.lower()}_profiles',qid,title,
        '各曲线对应题目指定时刻。温度采用摄氏度, 水分采用干基; 曲线来自同一 live Run 的物理径向点
值。',
        directory,[source],{'times_s':ts,'radii_m':rs,'temperature_C':T,'C':C},
        {'sample_time_count':len(ts),'sample_radius_count':len(rs)})

def _drying(run,qid,directory):
    source=_run_source(run)
    if run.event_s is None:
        raise ValueError('Full drying figure requires an actual threshold event')
    if qid=='Q4' and not run.model.settings.shrink:
        raise ValueError('Q4 primary figure must show the shrinking-domain run')
    radius_knots=run.model.rad['time_s'] if qid=='Q4' else np.array([])
    radius_knots=radius_knots[radius_knots<=run.end_s]
    ts=np.unique(np.r_[np.linspace(0,run.end_s,241),
        np.arange(0,min(run.end_s,14400)+1,600),
        radius_knots,run.event_s,run.end_s])
    rs=np.linspace(0,.02,121)
    _,C=run.fields(ts,radii_m=rs)
    R=np.asarray(run.model.radius(ts))
    inside=rs[None,:]<=R[:,None]+1e-12
    C=np.where(inside,C,np.nan)
    _,ends=run.fields(ts,material_x=[0.,1.])
    state=run.state(ts)
    mean=2*run.model.w@state[1:-1:2]
    max_C=np.max(state[1:-1:2],axis=0)
    del state
    t_h=ts/3600;event_h=run.event_s/3600

```

```

title=('第三问：固定半径下的全程水分迁移' if qid=='Q3' else '第四问：收缩域内的全程水分迁移')
with plt.rc_context(STYLE):
    fig,axes=plt.subplots(1,2,figsize=(12,5.1),gridspec_kw={'width_ratios':[1.08,1]})
    fig.subplots_adjust(left=.065,right=.965,bottom=.21,top=.83,wspace=.31)
    cmap=plt.get_cmap('YlGnBu').copy();cmap.set_bad('white')
    mesh=axes[0].pcolormesh(t_h,rs*100,np.ma.masked_invalid(C.T),shading='auto',
                           cmap=cmap,norm=Normalize(vmin=.05,vmax=2.55),rasterized=True)
    cb=fig.colorbar(mesh,ax=axes[0],pad=.025,fraction=.05)
    cb.set_label('干基含水率 / (kg/kg)',fontsize=9)
    if qid=='Q4':

        axes[0].fill_between(t_h,R*100,2,color='white',zorder=3)
        axes[0].plot(t_h,R*100,color='#202631',lw=1.8,label='药材表面 R(t)',zorder=4)
        axes[0].text(.65*t_h[-1],1.88,'域外（非药材）',ha='center',va='center',fontsize=9,color='#62666D')
        axes[0].legend(loc='lower
right',fontsize=8.5,facecolor='white',framealpha=.85,frameon=True)
        axes[0].set(xlim=(0,t_h[-1]),ylim=(0,2),xlabel='烘干时间 / h',
                   ylabel='物理径向距离 / cm',title='（a）物理空间中的含水率')
        for series,name,colour in [(ends[:,0],'中心',COLOURS['centre']),
                                   (mean,'干质量加权均值',COLOURS['mean']),
                                   (ends[:,1],'表面',COLOURS['surface'])]:
            axes[1].plot(t_h,series,lw=1.8,label=name,color=colour)
        axes[1].axhline(.15,color=COLOURS['threshold'],ls='--',lw=1.25,label='阈值 0.15
kg/kg')
        axes[1].axvline(event_h,color='#6D6473',ls=':',lw=1.2)
        axes[1].annotate(f'全域临界时刻\n{event_h:.3f} h',xy=(event_h,.15),
xytext=(.67*t_h[-1],.64),arrowprops={'arrowstyle':'->','color': '#625A6A'},
                           fontsize=9,ha='center',va='center',color='#514956')
        axes[1].set(xlim=(0,t_h[-1]*1.015),ylim=(0,2.66),xlabel='烘干时间 / h',
                   ylabel='干基含水率 / (kg/kg)',title='（b）中心、均值与表面的变化')
        axes[1].legend(loc='upper right',fontsize=8.8);axes[1].grid(alpha=.18)
        if t_h[-1]>4:
            axes[0].axvline(4,color='#666A73',ls=':',lw=.85,alpha=.8)
            axes[0].text(4+.025*t_h[-1],.13,'4 h',fontsize=8,color='#52545A')
            fig.suptitle(title,y=.97,fontsize=14,fontweight='bold')
            footer='一维径向有效模型：均值按干物质质量加权，达标条件取全域最大值。'
\n'+_environment_note(run)+'。'
        if qid=='Q4' and run.end_s>259200:
            footer+=' 半径超过 72 h 后采用末值延拓。'
        _footer(fig,footer)
        return _save(fig,f'fig_{qid.lower()}_drying',qid,title,
                    '左图为物理半径与时间的水分场；第四问域外留白，黑线为实测半径轨迹的插值。右图显
示中心、干质量加权均值、表面及全域阈值事件。',
                    directory,[source],{'times_s':ts,'radii_m':rs,'radius_m':R,'C':C,
'centre_C':ends[:,0],'mean_C':mean,'surface_C':ends[:,1],'max_C':max_C},
                    {'event_s':float(run.event_s),'post_verification_s':float(run.end_s),
                    'masked_outside_count':int(np.count_nonzero(~inside)),
                    'sample_time_count':len(ts),'sample_radius_count':len(rs),
                    'mean_definition':'2*sum(material_dual_cell_weights*C); dry-solid mass
weighting'})

def make_plots(runs,directory):

    run1=runs['Q1'];run23=runs.get('Q23',runs.get('Q2'));run4=runs['Q4']
    if run23 is None:raise ValueError('Provide the shared Q2/Q3 Run under Q23 or Q2')
    return [_profiles(run1,'Q1',directory),_profiles(run23,'Q2',directory),
            _drying(run23,'Q3',directory),_drying(run4,'Q4',directory)]

```

```

def _saved_summary(path):
    path=Path(path).resolve()
    content=path.read_bytes()
    rec=_record(path)
    if hashlib.sha256(content).hexdigest()!=rec['sha256']:
        raise RuntimeError('Evidence file changed while being read')
    data=json.loads(content)
    diag=data['diagnostics']
    if not diag.get('solver_success') or not diag.get('strictly_dry_at_end') or
diag.get('event_h') is None:
        raise ValueError('Comparison requires a completed successful drying run')

    src={'kind':'saved_completed_run_summary','files':[rec],
        'recorded_solver_code':data.get('code'),'settings':data['settings'],
        'current_solver_equivalence_not_assumed':True}
    return data,src

def make_shrinkage_comparison(fixed_summary_path,shrunk_summary_path,directory):
    fixed,sf=_saved_summary(fixed_summary_path);shrunk,ss=_saved_summary(shrunk_summary_path)
    a,b=fixed['settings'],shrunk['settings']
    if a['question']!='Q4' or b['question']!='Q4' or a['shrink'] or not b['shrink']:
        raise ValueError('Require matched appendix-4 fixed and shrinking runs')
    differences={k for k in set(a)|set(b) if a.get(k)!=b.get(k)}
    if differences!={'shrink'} or
fixed.get('code',{}).get('sha256')!=shrunk.get('code',{}).get('sha256'):
        raise ValueError('A pure shrinkage comparison needs identical settings/source except
shrink')
    values=np.array([fixed['diagnostics']['event_h'],shrunk['diagnostics']['event_h']])
    N=a['intervals'];reduction=100*(1-values[1]/values[0])
    title='同附录4物性下的固定半径与收缩对照'
    with plt.rc_context(STYLE):

fig,ax=plt.subplots(figsize=(8.2,4.7));fig.subplots_adjust(left=.2,right=.92,bottom=.22,top
=.81)
    ax.barh([1,0],values,color=['#8894A3','#218573'],height=.5)
    ax.set_yticks([1,0],['固定半径 2 cm','采用附件2收缩'])
    ax.set_xlim(0,values.max()*1.25);ax.set_xlabel('全域临界烘干时间 /
h');ax.grid(axis='x',alpha=.18)
    for y,v in zip([1,0],values):ax.text(v+values.max()*0.02,y,f'{v:.3f}
h',va='center',fontsize=10)
    ax.text(.98,.12,f'本对照缩短
{reduction:.2f}%',transform=ax.transAxes,ha='right',color='#246653',fontsize=11)
    fig.suptitle(title,y=.95,fontsize=14,fontweight='bold')
    _footer(fig,f'同一已保存计算版本, N={N}, 除收缩开关外设置一致: 这是有效模型中的尺寸效应。\\n
不可把第三、四问同时改变物性的时长差直接归因于收缩。')
    return _save(fig,'fig_q4_shrinkage_control','Q4',title,
        f'已保存同物性、同参数、同源码版本 N={N}对照; 读取来源原始哈希, 未假设与后续细网
格正式结果相同。',
        directory,[sf,ss],{'event_hours':values},

{'intervals':N,'same_property_control':True,'reduction_percent':float(reduction)})

def make_convergence_plot(report_paths,directory):

    sources=[];groups=[]
    for path in report_paths:

path=Path(path).resolve();content=path.read_bytes();data=json.loads(content);rec=_record(pa

```

```

th)
    if hashlib.sha256(content).hexdigest()!=rec['sha256']:raise RuntimeError('Changing
convergence report')
    if not data.get('finished_at') or not
str(data.get('status','')).startswith('computed'):
        raise ValueError('Select only finished convergence reports; in-progress reports are
not plotted')

groups.append(data);sources.append({'kind':'finished_convergence_report','files':[rec]})
    if not groups:raise ValueError('At least one finished convergence report is required')
    labels={'Q1':'第一问','Q23':'第二、三问','Q4':'第四问'}
    colours={'Q1':'#2563A6','Q23':'#C76829','Q4':'#218573'}
    title='相邻网格结果差异与临界时刻收敛'
    plotted={}
    with plt.rc_context(STYLE):

fig,axes=plt.subplots(1,2,figsize=(11.5,4.8));fig.subplots_adjust(left=.08,right=.965,botto
m=.21,top=.82,wspace=.32)
    for group in groups:
        q=group['question'];comp=group['comparisons']
        x=np.array([c['fine_N'] for c in comp],dtype=float)
        y=np.array([c['max_absolute_difference'] ['C'] for c in comp],dtype=float)
        axes[0].loglog(x,y,'o-',color=colours.get(q),label=labels.get(q,q),lw=1.6)
        timed=[c for c in comp if c.get('event_difference_s') is not None]
        if timed:
            axes[1].loglog([c['fine_N'] for c in timed],[abs(c['event_difference_s']) for c
in timed],
                        'o-',color=colours.get(q),label=labels.get(q,q),lw=1.6)
            plotted[q+'_fine_N']=x;plotted[q+'_C_difference']=y
            axes[0].set(title='(a) 共同物理采样点的最大含水率差',xlabel='较细网格的区间数 N',ylabel='
相邻网格最大差 / (kg/kg)')
            axes[1].set(title='(b) 全域临界时刻的变化',xlabel='较细网格的区间数 N',ylabel='相邻网格
临界时刻差的绝对值 / s')
            all_ticks=sorted({c['fine_N'] for g in groups for c in g['comparisons']})
            time_ticks=sorted({c['fine_N'] for g in groups for c in g['comparisons'] if
c.get('event_difference_s') is not None})
            axes[0].set_xticks(all_ticks,[str(n) for n in all_ticks])
            if time_ticks:axes[1].set_xticks(time_ticks,[str(n) for n in time_ticks])
            for ax in axes:
                ax.xaxis.set_minor_locator(NullLocator())
                ax.grid(which='both',alpha=.2);ax.legend(fontsize=9)
            fig.suptitle(title,y=.97,fontsize=14,fontweight='bold')
            _footer(fig,'读取已完成检验报告并记录来源哈希;相邻网格差用于数值收敛检验,不是实测预测误差。
')
    return _save(fig,'fig_model_grid_convergence','Q3',title,
                '横轴为相邻比较中较细网格的区间数;所画为实际计算的最大含水率差和阈值时刻差,不
把相邻差直接称作严格误差上界。',
                directory,sources,plotted,{'question_ids':[g['question'] for g in
groups]})

def make_sensitivity_plot(summary_paths,directory,question_id='Q4',baseline_label='基准'):

    if baseline_label not in summary_paths:raise ValueError('An explicit baseline label is
required')
    labels=list(summary_paths);rows=[];sources=[]
    for label,path in summary_paths.items():
        data,source=_saved_summary(path);rows.append(data);sources.append(source)
    if len({r['settings']['intervals'] for r in rows})!=1 or len({r['settings']['question'] for
r in rows})!=1:
        raise ValueError('Sensitivity cases must have a common question and grid size')
    expected={'Q4'} if question_id=='Q4' else {'Q2','Q3','Q23'}
    if rows[0]['settings']['question'] not in expected:

```

```

        raise ValueError('Sensitivity figure label does not match the saved physical question')
    if len({r.get('code',{}).get('sha256') for r in rows})!=1:
        raise ValueError('Sensitivity cases must use the same saved source version')
    values=np.array([r['diagnostics']['event_h'] for r in
rows]);baseline=values[labels.index(baseline_label)]
    delta=100*(values/baseline-1);N=rows[0]['settings']['intervals']
    title=('第四问' if question_id=='Q4' else '第二、三问')+'：物理假设与参数情景比较'
    with plt.rc_context(STYLE):
        fig,ax=plt.subplots(figsize=(9.4,max(4.7,.48*len(labels)+1.8)))
        fig.subplots_adjust(left=.25,right=.93,bottom=.2,top=.82)
        pos=np.arange(len(labels));colours=['#8A949F' if x==baseline_label else '#218573' for
x in labels]

ax.barh(pos,values,color=colours,height=.58);ax.set_yticks(pos,labels);ax.invert_yaxis()
    ax.set_xlim(0,values.max()*1.33);ax.set_xlabel('全域临界烘干时间 /
h');ax.grid(axis='x',alpha=.18)
    for y,v,d in zip(pos,values,delta):ax.text(v+values.max()*0.015,y,f'{v:.3f} h
({d:+.2f}%)',va='center',fontsize=9)
    fig.suptitle(title,y=.95,fontsize=14,fontweight='bold')
    _footer(fig,f'同一已保存求解版本，N={N}；百分比相对“{baseline_label}”。这些是指定情景的模
型变化，不是统计置信区间。')
    return
_save(fig,f'fig_{question_id.lower()}_physical_sensitivity',question_id,title,
        '读取明确列出的已完成情景，并记录每份摘要哈希、设置和原计算版本；不将情景差异当
作观测误差。',

directory,sources,{ 'event_hours':values,'relative_change_percent':delta},

{'scenario_labels':labels,'baseline_label':baseline_label,'intervals':N})

def make_question_plot(run, question_id, directory):

    if question_id in ('Q1', 'Q2'):
        return _profiles(run, question_id, _directory(directory))
    if question_id in ('Q3', 'Q4'):
        return _drying(run, question_id, _directory(directory))
    raise ValueError('A question plot requires Q1, Q2, Q3 or Q4')

def main():

    parser=argparse.ArgumentParser(description=__doc__);parser.add_argument('--selfcheck',actio
n='store_true')

    parser.add_argument('--output-dir',default='paper_output/results/plot_selfcheck');args=pars
er.parse_args()
    if not args.selfcheck:parser.error('Use make_plots with live Runs or --selfcheck')
    if Path.cwd().resolve()!=ROOT:raise RuntimeError('Run from the competition workspace')
    from drying_core import Settings,solve_case
    runs={}
    for q in ['Q1','Q23','Q4']:
        print('Selfcheck: solving '+q+' with N=40',flush=True)

runs[q]=solve_case(Settings(question=q,intervals=40,face_scheme='kirchhoff',shrink=q=='Q4',
                        rtol=1e-7,atol_temperature=1e-7,atol_moisture=1e-9))
    output=_directory(ROOT/args.output_dir)
    records=make_plots(runs,output)
    fixed=ROOT/'paper_output/results/experiments/physical_Q4_fixed_N200_K/summary.json'
    shrunk=ROOT/'paper_output/results/experiments/grids_Q4_N200_K/summary.json'
    if fixed.exists() and
shrunk.exists():records.append(make_shrinkage_comparison(fixed,shrunk,output))

```

```

reports=[ROOT/f'paper_output/results/convergence/{q}_K_analytic_J_v3/convergence_report.json'
for q in ['Q1','Q23']]
    if all(p.exists() for p in reports):records.append(make_convergence_plot(reports,output))
    scenario_root=ROOT/'paper_output/results/experiments'
    scenarios={'基准':shrunk,'4 h 后环境
49 °C':scenario_root/'physical_Q4_T49_N200_K/summary.json',
              '4 h 后环境 51 °C':scenario_root/'physical_Q4_T51_N200_K/summary.json',
              '潜热能量负荷压力测试':scenario_root/'physical_Q4_latent_N200_K/summary.json'}
    if all(p.exists() for p in
scenarios.values()):records.append(make_sensitivity_plot(scenarios,output))
    report={'scope':'N40 live-run plot selfcheck; primary plots are not final numerical results.
Optional comparisons preserve their independent N200/finished-report sources.',
           'status':'GENERATED_PENDING_VISUAL_REVIEW','figures':records}

(output/'plot_selfcheck_report.json').write_text(json.dumps(report,ensure_ascii=False,indent=2,allow_nan=False)+'\n',encoding='utf-8')

print(json.dumps({'status':report['status'],'figure_count':len(records),'output':str(output)},ensure_ascii=False),flush=True)
    return 0

```

```

if __name__=='__main__':raise SystemExit(main())

```

9.2.16 run_experiments.py

```

from __future__ import annotations
import argparse
from dataclasses import replace
from datetime import datetime, timezone
import json
from pathlib import Path
import sys
import traceback

from drying_core import ROOT, Settings, solve_case, save_run

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--batch', choices=['baseline', 'grids', 'temporal', 'physical'],
default='baseline')
    parser.add_argument('--n', type=int, default=100)
    parser.add_argument('--tag', default='')
    parser.add_argument('--face-scheme', choices=['harmonic','kirchhoff'],
default='harmonic')
    args = parser.parse_args()
    base = Settings(intervals=args.n, face_scheme=args.face_scheme)
    if args.batch in ('baseline', 'grids'):
        candidates = [('Q1', replace(base, question='Q1')), ('Q23', base),
('Q4', replace(base, question='Q4', shrink=True))]
    elif args.batch == 'temporal':
        candidates = [('Q23_tight', replace(base, rtol=1e-9, atol_moisture=1e-11,
atol_temperature=1e-9, max_step_s=150., early_max_step_s=10.)),
('Q4_tight', replace(base, question='Q4', shrink=True, rtol=1e-9,
atol_moisture=1e-11, atol_temperature=1e-9, max_step_s=150.,
early_max_step_s=10.))]
    else:
        candidates = [('Q4_fixed', replace(base, question='Q4', shrink=False)),
('Q23_last', replace(base, boundary_extension='last')),
('Q23_mean', replace(base, boundary_extension='tail_mean')),
('Q23_T49', replace(base, tail_temperature_C=49.)),
('Q23_T51', replace(base, tail_temperature_C=51.)),

```

```

        ('Q23_eq045', replace(base, tail_equilibrium=.045)),
        ('Q23_eq055', replace(base, tail_equilibrium=.055)),
        ('Q4_T49', replace(base, question='Q4', shrink=True,
tail_temperature_C=49.)),
        ('Q4_T51', replace(base, question='Q4', shrink=True,
tail_temperature_C=51.)),
        ('Q23_beta08', replace(base, beta=6.4e-7)),
        ('Q23_beta12', replace(base, beta=9.6e-7)),
        ('Q23_latent', replace(base, surface_latent_fraction=1.)),
        ('Q4_latent', replace(base, question='Q4', shrink=True,
surface_latent_fraction=1.))]
    out = ROOT/'paper_output/results/experiments'
    out.mkdir(parents=True, exist_ok=True)
    failed = 0
    for name, settings in candidates:
        trial_id = f'{args.batch}_{name}_N{args.n}_{args.tag}'
        folder = out/trial_id
        if (folder/'summary.json').exists():
            raise FileExistsError(f'Refusing to overwrite previous trial: {trial_id}')
        folder.mkdir(parents=True, exist_ok=True)
        start = datetime.now(timezone.utc).isoformat()
        print(f'START {trial_id} {start}', flush=True)
        try:
            run = solve_case(settings)
            summary = save_run(run, folder)
            record = {'trial_id':trial_id, 'started_at':start,
                    'finished_at':datetime.now(timezone.utc).isoformat(),
                    'status':'computed', 'exit_code':0, **summary['diagnostics']}
        except Exception as exc:
            failed += 1
            (folder/'failure.txt').write_text(traceback.format_exc(), encoding='utf-8')
            record = {'trial_id':trial_id, 'started_at':start, 'status':'failed',
                    'exit_code':1, 'error':str(exc)}
            (folder/'trial_record.json').write_text(json.dumps(record, ensure_ascii=False,
indent=2), encoding='utf-8')
            with (out/'trials.jsonl').open('a',encoding='utf-8') as stream:
                stream.write(json.dumps(record, ensure_ascii=False)+'\n')
            print(json.dumps(record, ensure_ascii=False), flush=True)
    return 1 if failed else 0

if __name__ == '__main__':
    sys.exit(main())

```

9.2.17 run_modeling.py

```

from __future__ import annotations

import gc
import hashlib
import importlib
import json
import os
from pathlib import Path
import subprocess
import sys
import time
import traceback
import uuid
import warnings

SOURCE = Path(__file__).resolve()
ROOT = SOURCE.parents[3]

```

```

LOADED_CODE_SHA256 = hashlib.sha256(SOURCE.read_bytes()).hexdigest()
import production_provenance as provenance

ASSUMPTIONS = [
    '一维径向有效模型，忽略轴向/端面基线，以二维独立程序检验端面影响。',
    '题给  $\rho \cdot c_p$  作为有效体积显热容量；不强制它等于守恒骨架的真实湿质量密度。',
    '附件烘房 kg/kg 数值直接作为材料干基等效  $C_{eq}$ ；这不是已证明的气固吸附平衡换算。',
    '基线不显式加入潜热；给定等效驱动力的潜热试验仪是能量负荷压力测试。',
    '前 4h 采用原始线性插值，4h 后  $T_{air}=50$  摄氏度、 $C_{eq}=0.05$  作为长期延拓假设。',
    'Q4 固定长度并采用同比径向材料收缩，半径轨迹来自附件 2 而非模型预测。',
    'Q2/Q3 从  $t=0$  统一附录 3；Q4 从  $t=0$  整组改用附录 4；h 和  $\beta$  沿用题给值。',
]

LIMITATIONS = [
    '没有内部温度/水分实测真值，数值校核不构成实测预测精度认证。',
    '密度、气固湿度映射、潜热闭合和外生收缩是模型解释边界，需团队主导确认。',
    '四位小数为输出格式；网格比较是采样差值，不能冒称连续解严格误差上界。',
    'Visual Studio 复现与用户人工代码审查分别留有独立状态。',
]

CONTRACT_TARGETS = {
    'model_results.json': 'paper_output/results/model_results.json',
    'metrics.json': 'paper_output/results/metrics.json',
    'conclusions.json': 'paper_output/results/conclusions.json',
    'table_index.json': 'paper_output/tables/table_index.json',
    'figure_index.json': 'paper_output/figure_index.json',
}

def dependencies():

    paths = list(SOURCE.parent.glob('*.py'))
    paths += [ROOT / 'paper_output/data_cleaned' / name for name in
               ('A_environment_observed.csv', 'A_radius_observed.csv')]
    paths += [ROOT / 'paper_output/plan/model_route.json']
    paths += list((ROOT / 'problem_files/CUMCM2026Problems/A 题').rglob('*.xlsx'))
    return [provenance.file_record(path) for path in sorted(set(paths))]

def font_record():
    font = Path('C:/Windows/Fonts/msyh.ttc')
    return {'path': str(font), 'bytes': font.stat().st_size, 'sha256': provenance.sha256(font)}

def assert_dependencies(start):
    if dependencies() != start['input_files']:
        raise RuntimeError('Source/input inventory changed after launch')
    if font_record() != start['font']:
        raise RuntimeError('Rendering font changed after launch')
    if provenance.runtime_record() != start['runtime']:
        raise RuntimeError('Runtime environment changed after launch')
    expected = {rec['path']: rec['sha256'] for rec in start['input_files']}
    for path, loaded in ((SOURCE, LOADED_CODE_SHA256), (provenance.SOURCE,
        provenance.LOADED_CODE_SHA256)):
        if loaded != expected[path.relative_to(ROOT).as_posix()]:
            raise RuntimeError('Loaded entry/provenance module differs from launch snapshot')

def imported_model_records():
    paths = set()
    for module in list(sys.modules.values()):
        filename = getattr(module, '__file__', None)
        if filename:
            path = Path(filename).resolve()

```

```

        if path.is_relative_to(SOURCE.parent) and path.suffix == '.py':
            paths.add(path)
    return [provenance.file_record(path) for path in sorted(paths)]

def question_plot(plots, run, qid, directory):
    public = getattr(plots, 'make_question_plot', None)
    if callable(public):
        return public(run, qid, directory)
    name = '_profiles' if qid in ('Q1', 'Q2') else '_drying'
    function = getattr(plots, name, None)
    if not callable(function):
        raise RuntimeError('Publication plot API is missing: ' + name)
    return function(run, qid, directory)

def describe_question(qid, key, summary, export, figure, args, directory, start):
    sample = summary['metric_samples'][qid]
    T, C, sampletime = sample['T_K'][0], sample['C'][0], sample['time_s']
    result_text = (f'在 t={sampletime/3600:.4f}h, 中心/表面温度分别为'
        f'{T[0]-273.15:.4f}/{T[1]-273.15:.4f}摄氏度, 干基含水率为'
        f'{C[0]:.4f}/{C[1]:.4f}kg/kg。')
    if qid in ('Q3', 'Q4'):
        done = summary['completion']
        result_text = (f'连续临界时刻{done["critical_event_h"]:.9f}h; 向上到四位小数并复核后'
            f'报告{done["reported_drying_time_h"]:.4f}h, 原精度最大含水率'
            f'{done["max_C_at_reported_time"]:.12f}<0.15kg/kg。')
        artifacts = [rec['path'] for rec in export['artifacts']] +
[export['validation_report']['path']]
        artifacts += [rec['path'] for rec in provenance.figure_artifact_records(figure)]
        artifacts = sorted(set(artifacts))
        source = next(rec for rec in start['input_files'] if rec['path'] ==
SOURCE.relative_to(ROOT).as_posix())
        question = {'question_id': qid, 'status': 'computed', 'source_run_id': args.version,
            'source_trajectory_id': args.version + ':' + key,
            'model_name': '圆柱有效显热与干基扩散; 守恒有限体积、Kirchhoff 面通量、解析 Jacobian BDF',
            'result_summary': result_text, 'result_files': artifacts,
            'execution_provenance': {'source_code_path': source['path'], 'source_code_sha256':
source['sha256'],
            'run_command': start['worker_command'], 'run_exit_code': None,
            'process_status': 'awaiting_parent_observed_exit', 'output_artifacts': artifacts},
            'assumptions_used': ASSUMPTIONS, 'limitations': LIMITATIONS, 'validation_summary':
summary}
        values = [
            ('mass_balance_absolute_residual',
summary['diagnostics']['max_mass_balance_abs_kg_per_kg'], 'kg/kg',
            '离散干物质基准下, 平均 C 与累计流出之和减初值。',
            '/diagnostics/max_mass_balance_abs_kg_per_kg'),
            ('radial_intervals', summary['settings']['intervals'], '1', '实际求解空间分辨率。',
            '/settings/intervals'),
            ('center_C_at_report_time', C[0], 'kg/kg', '已记录指定时刻的中心含水率。',
f'/metric_samples/{qid}/C/0/0'),
        ]
        if qid in ('Q3', 'Q4'):
            values.append(('reported_drying_time',
summary['completion']['reported_drying_time_h'], 'h',
            '向上取四位小数并用原精度最大值验证严格阈值。', '/completion/reported_drying_time_h'))
        evidence = (directory / key / 'summary.json').relative_to(ROOT).as_posix()
        metrics = [{'question_id': qid, 'metric_name': name, 'metric_role': 'numerical_result',
            'value': value, 'unit': unit, 'status': 'computed', 'evidence_path': evidence,
            'evidence_json_pointer': pointer, 'question_report_time_s': sampletime,
            'interpretation': interpretation}
            for name, value, unit, interpretation, pointer in values]

```

```

conclusion = {'question_id': qid, 'conclusion_text': result_text, 'status': 'computed',
             'supporting_metrics': [value[0] for value in values], 'supporting_artifacts': artifacts,
             'assumptions': ASSUMPTIONS, 'limitations': LIMITATIONS}
return question, metrics, conclusion

def worker(args, directory, start):
    assert_dependencies(start)
    import numpy as np
    core = importlib.import_module('drying_core')
    modules = {key: importlib.import_module(name) for key, name in
               [('Q1', 'q1_model'), ('Q23', 'q2_model'), ('Q4', 'q4_model')]}
    completion = importlib.import_module('q3_model').completion
    solve_only_review = args.review and not getattr(args, 'review_exports', False)
    exporter = None if solve_only_review else importlib.import_module('export_outputs')
    plots = None if solve_only_review else importlib.import_module('publication_plots')
    if core.ROOT.resolve() != ROOT:
        raise RuntimeError('Core workspace differs from production workspace')

    assert_dependencies(start)
    imported = imported_model_records()
    started, timer = provenance.utc_now(), time.perf_counter()
    summaries, exports, tables, figures, export_checks = {}, [], [], [], []
    questions, metrics, conclusions = [], [], []
    for name, n, qids in [('Q1', args.n1, ['Q1']), ('Q23', args.n23, ['Q2', 'Q3']),
                          ('Q4', args.n4, ['Q4'])]:
        run = None
        try:
            print(f'SOLVE {name} N={n} UTC={provenance.utc_now()}', flush=True)
            with warnings.catch_warnings(record=True) as caught:
                warnings.simplefilter('always')
                run = modules[name].solve(n)
            if caught:
                raise RuntimeError('Solver warning: ' + '; '.join(str(item.message) for item in
caught))
            summary = core.save_run(run, directory / name)
            summary['solver_warning_count'] = len(caught)
            summary['source_run_id'] = args.version
            summary['source_trajectory_id'] = args.version + ':' + name
            if name != 'Q1':
                summary['completion'] = completion(run)
            times = np.unique(np.array([0., min(1800., run.end_s), min(10800., run.end_s),
run.end_s] +
                ([[] if name == 'Q1' else [summary['completion']['reported_time_s']]))
            T, C = run.fields(times, material_x=np.array([0., .25, .5, .75, 1.]))
            summary['reproduction_samples'] = {'times_s': times.tolist(),
                'material_x': [0., .25, .5, .75, 1.], 'T_K': T.tolist(), 'C': C.tolist()}
            summary['metric_samples'] = {}
            for qid in qids:
                samptime = 1800. if qid == 'Q1' else 10800. if qid == 'Q2' else
summary['completion']['reported_time_s']
                T, C = run.fields([samptime], material_x=[0., 1.])
                summary['metric_samples'][qid] = {'time_s': samptime, 'material_x': [0., 1.],
                    'T_K': T.tolist(), 'C': C.tolist(), 'source': 'live_Run_dense_solution'}
            provenance.write_json(directory / name / 'summary.json', summary)
            summaries[name] = summary
            print(json.dumps({'question': name, **summary['diagnostics']}, ensure_ascii=False),
flush=True)
            if solve_only_review:
                continue
            for qid in qids:
                print('EXPORT ' + qid, flush=True)
                export = exporter.export_question(run, qid, directory / 'outputs')

```

```

        check = exporter.validate_exports([export], runs={qid: run})
        if check.get('status') != 'PASS' or not
check.get('fully_verified_with_live_Run'):
            raise RuntimeError('Export did not pass live Run validation: ' + qid)
        export['validation_report'] =
provenance.file_record(check['exports'][0]['report_path'])
        export['validation_status'] = 'PASS'
        export_checks.append(check)
        exports.append(export)
        for rec in export['artifacts']:
            if rec.get('role') in ('paper_table', 'paper_surface_coordinates'):
                tables.append({'table_id': Path(rec['path']).stem, 'question_id': qid,
                    'title': Path(rec['path']).stem, 'purpose': '题目指定正文位置/时间采
样',
                        **provenance.file_record(ROOT / rec['path']), 'status': 'computed',
                        'placeholder': False, 'ok': True, 'source_run_id': args.version,
                        'source_trajectory_id': args.version + ':' + name})
                print('PLOT ' + qid, flush=True)
            figure = question_plot(plots, run, qid, directory / 'figures')
            figure.update({'status': 'computed', 'placeholder': False, 'ok': True,
                'source_run_id': args.version, 'source_trajectory_id': args.version + ':' +
name})

            figures.append(figure)
            question, question_metrics, conclusion = describe_question(
                qid, name, summary, export, figure, args, directory, start)
            questions.append(question)
            metrics.extend(question_metrics)
            conclusions.append(conclusion)
            print(json.dumps({'question': qid, 'export_size': export['size'], 'validation':
check['status']}},
                        ensure_ascii=False), flush=True)
        finally:
            if run is not None:
                close = getattr(run, 'close', None)
                try:
                    if callable(close):
                        close()
                finally:
                    close = None
                    run = None
                    gc.collect()
            print('RELEASED ' + name, flush=True)
        assert_dependencies(start)
        provenance.assert_records(imported)
        provenance.write_json(directory / 'numerical_summaries.json', summaries)
        if args.review:
            provenance.write_json(directory / 'review_result.json', {
                'status': 'COMPUTED_PENDING_GUI_OBSERVATION',
                'mode': 'independent_actual_resolve_and_full_exports' if args.review_exports else
'independent_actual_resolve',
                'source': provenance.file_record(SOURCE),
                'inputs': start['input_files'], 'runtime': start['runtime'],
'imported_model_modules': imported,
                'started_at': started, 'finished_at': provenance.utcnow(), 'summaries': summaries,
                'exports': exports, 'export_checks': export_checks, 'figures': figures,
                'elapsed_s': time.perf_counter() - timer,
                'visual_studio_gui': 'Requires separate observed GUI evidence', 'human_review':
'pending'})
            print('REPRODUCTION_SOLVES_COMPLETED', flush=True)
        else:
            provenance.write_json(directory / 'metric_evidence_validation.json',
provenance.validate_metric_evidence(metrics))
            provenance.write_json(directory / 'export_validation.json', {'status': 'PASS', 'checks':

```

```

export_checks})
    meta = {'schema_version': '1.0', 'generated_by': SOURCE.relative_to(ROOT).as_posix(),
            'generated_at': provenance.utc_now(), 'source_run_id': args.version,
            'publication_state': 'version_only_awaiting_parent_observed_exit'}
    contracts = {'model_results.json': {'**meta', 'questions': questions},
                'metrics.json': {'**meta', 'items': metrics}, 'conclusions.json': {'**meta', 'items':
conclusions},
                'table_index.json': {'**meta', 'tables': tables}, 'figure_index.json': {'**meta',
'figures': figures}}
    for filename, content in contracts.items():
        provenance.write_json(directory / 'contracts' / filename, content)
    assert_dependencies(start)
    output_paths = [p for p in directory.rglob('*') if p.is_file() and p.name not in
                    ('stdout.log', 'worker_success.json', 'process_result.json')]
    provenance.write_json(directory / 'worker_success.json', {
        'started_at': started, 'finished_at': provenance.utc_now(), 'elapsed_seconds':
time.perf_counter() - timer,
        'input_files': start['input_files'], 'runtime': start['runtime'], 'font':
start['font'],
        'imported_model_modules': imported, 'output_artifacts': [provenance.file_record(p) for
p in sorted(output_paths)],
        'question_ids': ['Q1', 'Q2', 'Q3', 'Q4'], 'summaries': summaries, 'exports': exports,
        'mode': 'review' if args.review else 'production', 'human_review': 'pending',
'visual_studio_gui': 'pending'})
    print('ALL_SOLVES_AND_REQUESTED_VALIDATIONS_COMPLETED', flush=True)

def publish(args, directory, start, success, process):
    assert_dependencies(start)
    provenance.assert_records(success['output_artifacts'])
    with provenance.publication_lock(ROOT, args.version):
        contracts = {name: json.loads((directory / 'contracts' /
name).read_text(encoding='utf-8'))
                    for name in CONTRACT_TARGETS}
        for data in contracts.values():
            data['publication_state'] = 'parent_verified_actual_worker_exit_zero'
        for question in contracts['model_results.json']['questions']:
            question['execution_provenance'].update({'run_exit_code': process['returncode'],
            'process_status': 'parent_observed_exit_zero',
            'process_evidence_path': (directory /
'process_result.json').relative_to(ROOT).as_posix())}
        index_path = ROOT / 'paper_output/figure_index.json'
        prior = json.loads(index_path.read_text(encoding='utf-8')) if index_path.exists() else
None
        merged, merge_note = provenance.merge_figure_index(prior,
contracts['figure_index.json']['figures'])
        contracts['figure_index.json']['figures'] = merged
        contracts['figure_index.json']['merge_record'] = merge_note
        provenance.validate_metric_evidence(contracts['metrics.json']['items'])
        published = directory / 'published_contracts'
        for name, content in contracts.items():
            provenance.write_json(published / name, content)
        records = {rec['path']: rec for rec in success['output_artifacts']}
        for path in [directory / 'worker_success.json', directory / 'stdout.log', directory /
'process_result.json']:
            rec = provenance.file_record(path)
            records[rec['path']] = rec
        for name in CONTRACT_TARGETS:
            rec = provenance.file_record(published / name)
            records[rec['path']] = rec

        target = {'**rec', 'path': CONTRACT_TARGETS[name]}
        records[target['path']] = target

```

```

    for figure in merged:
        for rec in provenance.figure_artifact_records(figure):
            records[rec['path']] = rec
            assert_dependencies(start)
            source = next(rec for rec in start['input_files'] if rec['path'] ==
SOURCE.relative_to(ROOT).as_posix())
            record = {'run_id': args.version, 'script': source['path'], 'script_sha256':
source['sha256'],
                    'command': process['command'], 'question_ids': success['question_ids'],
                    'returncode': process['returncode'], 'cwd': str(ROOT), 'process': process,
                    'started_at': process['started_at'], 'finished_at': process['finished_at'],
                    'worker_started_at': success['started_at'], 'worker_finished_at':
success['finished_at'],
                    'elapsed_seconds': success['elapsed_seconds'], 'python': start['runtime'], 'font':
start['font'],
                    'input_files': start['input_files'], 'imported_model_modules':
success['imported_model_modules'],
                    'output_artifacts': [records[path] for path in sorted(records)],
                    'version_contract_directory': published.relative_to(ROOT).as_posix(),
                    'human_review': 'pending', 'visual_studio_gui': 'pending'}
            manifest = {'schema_version': '1.0', 'generated_by':
SOURCE.relative_to(ROOT).as_posix(),
                    'generated_at': provenance.utcnow(), 'status': 'PASS', 'runs': [record],
                    'scope': 'Actual worker exit zero, source/input/output hashes and export checks; not
human model acceptance.'}
            provenance.publish_transaction(ROOT, directory,
                {CONTRACT_TARGETS[name]: data for name, data in contracts.items()}, manifest)

def main():
    import argparse
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--version', required=True)
    parser.add_argument('--n1', type=int, default=3200)
    parser.add_argument('--n23', type=int, default=3200)
    parser.add_argument('--n4', type=int, default=6400)
    parser.add_argument('--blas-threads', type=int, default=1)
    parser.add_argument('--worker', action='store_true', help=argparse.SUPPRESS)
    parser.add_argument('--launch-token', help=argparse.SUPPRESS)
    parser.add_argument('--review', action='store_true')
    parser.add_argument('--review-exports', action='store_true',
        help='With --review, also reproduce all Excel/CSV readbacks and figures in the GUI
process')
    args = parser.parse_args()
    if args.review_exports and not args.review:
        raise ValueError('--review-exports requires --review')
    if args.blas_threads < 1:
        raise ValueError('BLAS thread count must be positive')

    os.environ['OPENBLAS_NUM_THREADS'] = str(args.blas_threads)
    os.environ['OMP_NUM_THREADS'] = str(args.blas_threads)
    if Path.cwd().resolve() != ROOT:
        raise RuntimeError('Run with the competition workspace as cwd')
    if not args.version or any(ch not in
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789_- ' for ch in args.version):
        raise ValueError('Use a simple version label')
    if min(args.n1, args.n23, args.n4) < 2:
        raise ValueError('All radial interval counts must be at least 2')
    directory = ROOT / ('paper_output/results/gui_reproduction' if args.review else
        'paper_output/results/production') / args.version
    if args.worker:
        if args.review:

```

```

        raise ValueError('--review uses a single process so Visual Studio can hit solver
breakpoints')
    start = json.loads((directory / 'launch.json').read_text(encoding='utf-8'))
    config = {'n1': args.n1, 'n23': args.n23, 'n4': args.n4, 'review': args.review,
              'review_exports': args.review_exports,
              'blas_threads': args.blas_threads}
    if not args.launch_token or args.launch_token != start['launch_token'] or config !=
start['configuration']:
        raise RuntimeError('Worker must be launched by its matching supervising parent')
    provenance.exclusive_json(directory / 'worker_started.json', {'pid': os.getpid(),
'started_at': provenance.utcnow()})
    print('WORKER_PID=' + str(os.getpid()), flush=True)
    try:
        worker(args, directory, start)
    except BaseException as error:
        provenance.write_json(directory / 'worker_failure.json', {'status': 'FAIL', 'error':
repr(error),
                        'traceback': traceback.format_exc(), 'generated_at': provenance.utcnow(),
                        'returncode': None, 'returncode_note': 'Supervising parent records the actual
process exit.'})
        raise
    return 0
if args.launch_token:
    raise ValueError('--launch-token is reserved for the supervised worker')
directory.parent.mkdir(parents=True, exist_ok=True)
directory.mkdir(exist_ok=False)
token = uuid.uuid4().hex
provenance.exclusive_json(directory / 'launch_claim.json', {'parent_pid': os.getpid(),
'run_id': args.version, 'claimed_at': provenance.utcnow(), 'launch_token': token})
process = None
try:
    command = [sys.executable, '-B', str(SOURCE), '--version', args.version,
               '--n1', str(args.n1), '--n23', str(args.n23), '--n4', str(args.n4),
               '--blas-threads', str(args.blas_threads)]
    if args.review:
        command.append('--review')
        if args.review_exports:
            command.append('--review-exports')
    else:
        command += ['--worker', '--launch-token', token]
    start = {'run_id': args.version, 'parent_pid': os.getpid(), 'launch_token': token,
'created_at': provenance.utcnow(), 'configuration': {'n1': args.n1, 'n23':
args.n23,
                'n4': args.n4, 'review': args.review, 'review_exports': args.review_exports,
                'blas_threads': args.blas_threads}, 'input_files': dependencies(),
'runtime': provenance.runtime_record(), 'font': font_record(),
'worker_command': subprocess.list2cmdline(command), 'worker_argv': command,
'dependency_scope': 'Local module inventory plus A inputs/templates and route;
inventory does not claim execution.'}
    assert_dependencies(start)
    provenance.exclusive_json(directory / 'launch.json', start)
    if args.review:

        provenance.exclusive_json(directory / 'review_started.json', {
            'pid': os.getpid(), 'started_at': provenance.utcnow(), 'mode':
'single_process_VS_review'})
        worker(args, directory, start)
        assert_dependencies(start)
        success = json.loads((directory /
'worker_success.json').read_text(encoding='utf-8'))
        provenance.assert_records(success['output_artifacts'])
        provenance.write_json(directory / 'review_evidence_validation.json', {

```

```

        'status': 'COMPUTED_PENDING_GUI_OBSERVATION', 'input_files':
start['input_files'],
        'worker_success': provenance.file_record(directory / 'worker_success.json'),
        'scope': 'Direct solve function and saved hashes checked; no process exit code
is self-certified.',
        'visual_studio_gui': 'pending', 'human_review': 'pending'})
    print('REVIEW_FUNCTION_COMPLETED;
GUI_OBSERVATION_AND_ACTUAL_EXIT_NOT_SELF_CERTIFIED', flush=True)
    return 0
    process = provenance.measured_process(command, ROOT, directory / 'stdout.log')
    provenance.write_json(directory / 'process_result.json', process)
    if process['returncode'] != 0:
        raise RuntimeError('Worker failed with actual return code ' +
str(process['returncode']))
    assert_dependencies(start)
    success = json.loads((directory / 'worker_success.json').read_text(encoding='utf-8'))
    if success['input_files'] != start['input_files'] or success['runtime'] !=
start['runtime']:
        raise RuntimeError('Worker success differs from launch snapshot')
    provenance.assert_records(success['output_artifacts'])
    publish(args, directory, start, success, process)
    print('ACTUAL_WORKER_RETURN_CODE=0; VERIFIED_RUN_MANIFEST_PUBLISHED', flush=True)
    return 0
except BaseException as error:
    if process is None and (directory / 'process_result.json').exists():
        process = json.loads((directory /
'process_result.json').read_text(encoding='utf-8'))
        provenance.write_json(directory / 'parent_failure.json', {'status': 'FAIL',
'generated_at': provenance.utcnow(), 'error': repr(error), 'traceback':
traceback.format_exc(),
'actual_worker_returncode': None if process is None else process['returncode'],
'process': process, 'version_is_retained': True,
'publication_note': 'Worker does not publish global contracts; publication failure
records rollback.'})
        raise

if __name__ == '__main__':
    try:
        raise SystemExit(main())
    except Exception:
        traceback.print_exc()
        raise SystemExit(1)

```

9.2.18 validate_bessel.py

```

from __future__ import annotations

import argparse
import csv
import hashlib
import json
import platform
import sys
from datetime import datetime, timezone
from functools import lru_cache
from pathlib import Path

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy.integrate import quad
from scipy.optimize import brentq
from scipy.special import j0, j1, jn_zeros

```

```
FloatArray = NDArray[np.float64]
```

```
def _vector(value: ArrayLike, name: str) -> FloatArray:
    a = np.atleast_1d(np.asarray(value, dtype=float))
    if a.ndim != 1 or a.size == 0 or not np.all(np.isfinite(a)):
        raise ValueError(f"{name} must be a nonempty finite 1-D array")
    return a
```

```
@lru_cache(maxsize=24)
```

```
def _eigen_cached(biot: float, n_terms: int) -> tuple[FloatArray, FloatArray]:
    if not np.isfinite(biot) or biot <= 0:
        raise ValueError("Biot must be finite and positive")
    if not isinstance(n_terms, (int, np.integer)) or n_terms < 1:
        raise ValueError("n_terms must be a positive integer")

    upper = jn_zeros(0, n_terms)
    lower = np.r_[0.0, jn_zeros(1, n_terms-1)] if n_terms > 1 else np.array([0.0])
    roots = np.array([
        brentq(lambda z: z*j1(z)-biot*j0(z), lo, hi, xtol=1e-13, rtol=1e-14)
        for lo, hi in zip(lower, upper)
    ])
    a = 2*j1(roots) / (roots*(j0(roots)**2+j1(roots)**2))
    roots.setflags(write=False)
    a.setflags(write=False)
    return roots, a
```

```
def robin_eigenpairs(biot: float, n_terms: int = 120) -> tuple[FloatArray, FloatArray]:

    mu, a = _eigen_cached(float(biot), int(n_terms))
    return mu.copy(), a.copy()
```

```
def _validate_inputs(times_s, env_t, env_values, radius, diffusivity, biot):
    t = _vector(times_s, "times_s")
    knots = _vector(env_t, "env_t")
    values = _vector(env_values, "env_values")
    if len(knots) != len(values) or len(knots) < 2:
        raise ValueError("Boundary needs matching arrays with at least two knots")
    if abs(knots[0]) > 1e-12 or np.any(np.diff(knots) <= 0):
        raise ValueError("Boundary knots must start at t=0 and strictly increase")
    if np.any(t < 0):
        raise ValueError("Negative evaluation times are not permitted")
    if np.max(t) > knots[-1] + 1e-10:
        raise ValueError("Evaluation exceeds boundary observations: supply an explicit extension")
    if not np.isfinite(radius) or radius <= 0 or not np.isfinite(diffusivity) or diffusivity <= 0:
        raise ValueError("radius and diffusivity must be finite and positive")
    if not np.isfinite(biot) or biot < 0:
        raise ValueError("Biot must be finite and nonnegative")
    return t, knots, values
```

```
def _modal_history(
    times: FloatArray, knots: FloatArray, values: FloatArray,
    rates: FloatArray, initial_value: float,
) -> FloatArray:
```

```

z = (initial_value-values[0])*np.exp(-times[:, None]*rates[None, :])
slopes = np.diff(values)/np.diff(knots)
for lo, hi, slope in zip(knots[:-1], knots[1:], slopes):
    if slope == 0:
        continue
    selected = times > lo
    ts = times[selected]
    end = np.minimum(ts, hi)
    duration = end-lo
    contribution = (
        np.exp(-(ts-end)[:, None]*rates[None, :])
        * (-np.expm1(-duration[:, None]*rates[None, :]))
        / rates[None, :]
    )
    z[selected] -= slope*contribution
return z

def _space_modes(mu: FloatArray, radii_m: ArrayLike, radius: float, cell_edges_m=None) ->
FloatArray:
    if cell_edges_m is None:
        x = _vector(radii_m, "radii_m")/radius
        if np.any(x < 0) or np.any(x > 1+1e-12):
            raise ValueError("radii must lie inside the cylinder")
        return j0(mu[:, None]*x[None, :])
    edges = _vector(cell_edges_m, "cell_edges_m")/radius
    if len(edges) < 2 or np.any(np.diff(edges) <= 0) or edges[0] < 0 or edges[-1] > 1+1e-12:
        raise ValueError("cell edges must strictly increase inside [0,radius]")
    lo, hi = edges[:-1], edges[1:]
    return 2*(hi[None, :]*j1(mu[:, None]*hi[None, :])
        - lo[None, :]*j1(mu[:, None]*lo[None, :])) / (mu[:, None]*(hi*hi-lo*lo)[None, :])

def bessell_diffusion(
    times_s: ArrayLike, radii_m: ArrayLike, env_t: ArrayLike,
    env_values: ArrayLike, *, radius: float, diffusivity: float,
    biot: float, initial_value: float, n_terms: int = 120,
    cell_edges_m: ArrayLike | None = None,
) -> FloatArray:
    t, knots, values = _validate_inputs(times_s, env_t, env_values, radius, diffusivity, biot)
    if not np.isfinite(initial_value):
        raise ValueError("initial_value must be finite")
    if not isinstance(n_terms, (int, np.integer)) or n_terms < 1:
        raise ValueError("n_terms must be a positive integer")
    if biot == 0:
        nspace = len(_vector(cell_edges_m, "cell_edges_m"))-1 if cell_edges_m is not None else
len(_vector(radii_m, "radii_m"))
        _space_modes(np.array([1.0]), radii_m, radius, cell_edges_m)
        return np.full((len(t), nspace), initial_value)
    mu, a = _eigen_cached(float(biot), int(n_terms))
    rates = diffusivity*mu*mu/(radius*radius)
    spatial = _space_modes(mu, radii_m, radius, cell_edges_m)
    history = _modal_history(t, knots, values, rates, initial_value)
    result = np.interp(t, knots, values)[:, None] + (history*a[None, :]) @ spatial

    result[t == 0] = initial_value
    return result

```

```

def bessell_temperature(
    times_s: ArrayLike, radii_m: ArrayLike, env_t: ArrayLike,
    env_T: ArrayLike, radius: float = 0.02, h: float = 25.0,
    k: float = 0.36, rho: float = 820.0, cp: float = 2600.0,
    n_terms: int = 120, initial_temperature_K: float = 301.15,
    cell_edges_m: ArrayLike | None = None,
) -> FloatArray:

    if any(not np.isfinite(v) or v <= 0 for v in (k,rho,cp)) or h < 0:
        raise ValueError("k,rho,cp must be positive and h nonnegative")
    return bessell_diffusion(
        times_s, radii_m, env_t, env_T, radius=radius,
        diffusivity=k/(rho*cp), biot=h*radius/k,
        initial_value=initial_temperature_K, n_terms=n_terms,
        cell_edges_m=cell_edges_m,
    )

def bessell_moisture(
    times_s: ArrayLike, radii_m: ArrayLike, env_t: ArrayLike,
    env_Ceq: ArrayLike, diffusion_coefficient: float,
    radius: float = 0.02, beta: float = 8e-7,
    initial_moisture: float = 2.55, n_terms: int = 240,
    cell_edges_m: ArrayLike | None = None,
) -> FloatArray:

    if diffusion_coefficient <= 0 or beta < 0:
        raise ValueError("Frozen diffusion coefficient must be positive, beta nonnegative")
    return bessell_diffusion(
        times_s, radii_m, env_t, env_Ceq, radius=radius,
        diffusivity=diffusion_coefficient, biot=beta*radius/diffusion_coefficient,
        initial_value=initial_moisture, n_terms=n_terms,
        cell_edges_m=cell_edges_m,
    )

def _sha256(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()

def selfcheck(env_t: FloatArray, env_T: FloatArray) -> dict:

    bi = 25*.02/.36
    mu, a = robin_eigenpairs(bi, 120)
    residual = mu*j1(mu)-bi*j0(mu)
    relative_eigen_residual =
    np.max(np.abs(residual)/(1+np.abs(mu*j1(mu))+np.abs(bi*j0(mu))))

    knots = np.array([0.,17.,52.,130.])
    boundary = np.array([301.15,305.,303.,303.])
    targets = np.array([8.,17.,40.,78.,130.])
    rates = np.array([1e-6,.002,.2])
    z = _modal_history(targets,knots,boundary,rates,299.)
    independent = np.empty_like(z)
    for i,t in enumerate(targets):
        for j,lam in enumerate(rates):
            integral = 0.
            for lo,hi,gl,gh in zip(knots[:-1],knots[1:],boundary[:-1],boundary[1:]):
                if t > lo:

```

```

        slope = (gh-gl)/(hi-lo)
        integral += quad(lambda s:
slope*np.exp(-lam*(t-s)),lo,min(t,hi),epsabs=1e-12,epsrel=1e-12)[0]
        independent[i,j] = (299.-boundary[0])*np.exp(-lam*t)-integral
        convolution_error = float(np.max(np.abs(z-independent)))

edges = np.array([0.,.001,.006,.02])
averaged = _space_modes(mu[:8], [0.], .02,edges)
quad_average = np.empty_like(averaged)
for n,m in enumerate(mu[:8]):
    for i,(lo,hi) in enumerate(zip(edges[:-1]/.02,edges[1:]/.02)):
        quad_average[n,i] = 2*quad(lambda x:
x*j0(m*x),lo,hi,epsabs=1e-12)[0]/(hi*hi-lo*lo)
        average_error = float(np.max(np.abs(averaged-quad_average)))

alpha=.36/(820*2600)
lam=alpha*mu**2/.02**2
e=np.exp(-100*lam)
mean_basis=2*j1(mu)/mu
dmean=np.sum(a*(-lam)*(-20)*e*mean_basis)
surface=321.15+np.sum(a*(-20)*e*j0(mu))
influx=2*25/(820*2600*.02)*(321.15-surface)
energy_error=abs(dmean-influx)

ts=np.array([0.,1.,10.,100.,300.,600.,900.,1200.,1500.,1800.])
rs=np.linspace(0,.02,41)
t120=bessel_temperature(ts,rs,env_t,env_T,n_terms=120)
t240=bessel_temperature(ts,rs,env_t,env_T,n_terms=240)
t480=bessel_temperature(ts,rs,env_t,env_T,n_terms=480)
error120_240=float(np.max(np.abs(t120-t240)))
error240_480=float(np.max(np.abs(t240-t480)))
equilibrium=bessel_temperature([0.,1.,100.],[0.,.01,.02],[0.,100.],[301.15,301.15])
equilibrium_error=float(np.max(np.abs(equilibrium-301.15)))
insulating=bessel_temperature([0.,50.,100.],[0.,.01,.02],[0.,100.],[301.15,323.15],h=0)
insulation_error=float(np.max(np.abs(insulating-301.15)))

checks = [
    ("robin_eigen_relative_residual",float(relative_eigen_residual),1e-10,"1"),
    ("piecewise_convolution_vs_adaptive_quadrature",convolution_error,1e-10,"K"),
    ("annular_average_vs_adaptive_quadrature",average_error,1e-10,"1"),
    ("constant_boundary_step_energy_identity",float(energy_error),1e-10,"K/s"),
    ("Q1_observed_boundary_120_vs_240_terms",error120_240,1e-4,"K"),
    ("Q1_observed_boundary_240_vs_480_terms",error240_480,1e-4,"K"),
    ("uniform_equilibrium",equilibrium_error,1e-12,"K"),
    ("insulating_boundary",insulation_error,1e-12,"K"),
]
return {
    "status":"PASS" if all(v<=limit for _,v,limit,_ in checks) else "FAIL",
    "scope":"analytic benchmark implementation selfcheck; production FVM comparison is
separate",
    "checks":[{"name":name,"value":v,"tolerance":limit,"unit":unit,"passed":v<=limit}
for name,v,limit,unit in checks],
    "series_note":"120/240/480 differences estimate truncation sensitivity at sampled
locations and times, not a rigorous uniform error bound.",
    "no_internal_experimental_validation":True,
    "visual_studio_gui":"pending",
    "human_review":"pending",
}

def main() -> int:

```

```

parser=argparse.ArgumentParser(description=__doc__)

parser.add_argument("--environment-csv",default="paper_output/data_cleaned/A_environment_observed.csv")
parser.add_argument("--output-dir",default="paper_output/results/bessel_validation")
args=parser.parse_args()
root=Path.cwd().resolve()
if root != Path(r"D:\Document\数学建模\2026CUMCM").resolve():
    raise RuntimeError("Run only from the 2026CUMCM workspace root")
source=(root/args.environment_csv).resolve()
with source.open(encoding="utf-8-sig",newline="") as fh:
    rows=list(csv.DictReader(fh))
env_t=np.array([float(r["time_s"]) for r in rows])
env_T=np.array([float(r["temperature_K"]) for r in rows])
env_C=np.array([float(r["air_moisture_kg_per_kg"]) for r in rows])
report=selfcheck(env_t,env_T)
out=(root/args.output_dir).resolve()
if not out.is_relative_to(root):
    raise RuntimeError("Output must remain inside the competition workspace")
out.mkdir(parents=True,exist_ok=True)
paper_t=np.array([100.,300.,600.,900.,1200.,1500.,1800.])
paper_r=np.array([0.,.005,.01,.015,.02])
temperature=bessel_temperature(paper_t,paper_r,env_t,env_T,n_terms=240)
frozen_D=7e-9*np.exp(-.89/2.55)
frozen_water=bessel_moisture(paper_t,paper_r,env_t,env_C,float(frozen_D),n_terms=240)
artifacts=[]
for filename,field,label in [
    ("q1_exact_heat_paper_points.csv",temperature-273.15,"temperature_C"),
    ("frozen_D_moisture_benchmark_points.csv",frozen_water,"moisture_kg_per_kg"),
]:
    path=out/filename
    with path.open("w",encoding="utf-8",newline="") as fh:
        writer=csv.writer(fh)
        writer.writerow(["time_s","radius_m",label])
        for i,t in enumerate(paper_t):
            for j,r in enumerate(paper_r):
                writer.writerow([format(t,".17g"),format(r,".17g"),format(field[i,j],".17g")])

artifacts.append({"path":path.relative_to(root).as_posix(),"bytes":path.stat().st_size,"sha256":_sha256(path)})
report.update({
    "schema_version":"1.0", "question_id":"Q1",
    "generated_at":datetime.now(timezone.utc).isoformat(),
    "generated_by":"paper_output/code/modeling/validate_bessel.py",
    "frozen_moisture_diffusion_coefficient_m2_s":float(frozen_D),
    "frozen_moisture_note":"This table is a constant-D algorithm benchmark and must not replace nonlinear Q1 result1 moisture values.",
    "execution_provenance":{
        "source_code_path":Path(__file__).resolve().relative_to(root).as_posix(),
        "source_code_sha256":_sha256(Path(__file__)),
        "run_command":sys.executable+" -B "+" ".join(sys.argv),
        "run_exit_code":0 if report["status"]=="PASS" else 1,
        "python_version":platform.python_version(),
    },
    "input_artifacts":[{"path":source.relative_to(root).as_posix(),"sha256":_sha256(source)}],
    "output_artifacts":artifacts,
})
report_path=out/"bessel_selfcheck.json"

report_path.write_text(json.dumps(report,ensure_ascii=False,indent=2)+"\n",encoding="utf-8")
)

```

```
print(json.dumps({"status":report["status"],"checks":report["checks"],"report":str(report_path)},ensure_ascii=False,indent=2))
return 0 if report["status"]=="PASS" else 1
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

9.2.19 verify_convergence.py

```
from __future__ import annotations
import argparse
from dataclasses import replace
from datetime import datetime, timezone
import gc
import json
from pathlib import Path
import warnings
import numpy as np
from drying_core import ROOT, Settings, solve_case, save_run, file_record
from validate_bessel import bessel_temperature, bessel_moisture

def project_run(run, points, full_seconds):

    if full_seconds:
        times = np.arange(0., np.floor(run.end_s)+1.)
    else:
        times = np.unique(np.r_[np.arange(0., min(10800., run.end_s)+1.),
                                np.arange(10860., run.end_s, 60.)])
    T = np.empty((len(times), len(points)))
    C = np.empty_like(T)
    for first in range(0, len(times), 128):
        T[first:first+128], C[first:first+128] = run.fields(times[first:first+128],
raddi_m=points)
    return {'times':times,'T':T,'C':C,'end_s':run.end_s,'event_s':run.event_s,
            'N':run.model.settings.intervals}

def compare_projections(coarse, fine, points, block=1000):
    common_count = min(len(coarse['times']), len(fine['times']))
    times = coarse['times'][:common_count]
    if not np.array_equal(times, fine['times'][:common_count]):
        raise ValueError('Only identical physical time queries may be compared')
    maxima = {'T_K':0., 'C':0.}
    locations = {}
    for first in range(0, len(times), block):
        tt = times[first:first+block]
        fields_a = [coarse[k][first:first+len(tt)] for k in ['T','C']]
        fields_b = [fine[k][first:first+len(tt)] for k in ['T','C']]
        for label, aa, bb in zip(['T_K', 'C'], fields_a, fields_b):
            error = np.abs(aa-bb)
            if np.any(np.isfinite(error)):
                index = np.unravel_index(np.nanargmax(error), error.shape)
                value = float(error[index])
                if value > maxima[label]:
                    maxima[label] = value
                    locations[label] = {'time_s':float(tt[index[0]]),
                                        'radius_m':float(points[index[1]])}
    return {'max_absolute_difference':maxima, 'locations':locations,
            'compared_time_count':len(times), 'radial_count':len(points),
            'convention':'Only finite values at common physical points are compared.'}
```

```

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--question', choices=['Q1', 'Q23', 'Q4'], required=True)
    parser.add_argument('--grids', type=int, nargs='+', required=True)
    parser.add_argument('--tag', required=True)
    parser.add_argument('--full-second-comparison', action='store_true')
    parser.add_argument('--frozen-d', type=float)
    args = parser.parse_args()
    out = ROOT/'paper_output/results/convergence'/args.tag
    if out.exists():
        raise FileExistsError(out)
    out.mkdir(parents=True)
    settings = Settings(question=args.question, face_scheme='kirchhoff',
        shrink=args.question=='Q4', rtol=1e-10, atol_temperature=1e-10,
        atol_moisture=1e-12, max_step_s=120., early_max_step_s=2.,
        constant_D=args.frozen_d)
    report = {'created_at':datetime.now(timezone.utc).isoformat(),
        'question':args.question, 'runs':[], 'comparisons':[],
        'driver_code':file_record(__file__), 'human_review':'pending'}
    previous = None
    radii = np.linspace(0,.02,21)
    for n in args.grids:
        print(f'START {args.tag} N={n}', flush=True)
        run = None
        try:
            with warnings.catch_warnings(record=True) as captured:
                warnings.simplefilter('always')
                run = solve_case(replace(settings, intervals=n))
                summary = save_run(run, out/f'N{n}')
            projection = project_run(run, radii, args.full_second_comparison or
args.question=='Q1')
            summary['warnings'] = [str(item.message) for item in captured]
            report['runs'].append(summary)
            if captured:
                raise RuntimeError(f'Production solver warnings at N={n}:
{summary["warnings"]}')
            if previous is not None:
                comparison = compare_projections(previous, projection, radii)
                comparison.update({'coarse_N':previous['N'], 'fine_N':n,
                    'event_difference_s':None if run.event_s is None else
run.event_s-previous['event_s']})
                report['comparisons'].append(comparison)
                print(json.dumps(comparison), flush=True)
            if args.question=='Q1':
                times = np.arange(0.,1801.)
                numeric_T, numeric_C = projection['T'], projection['C']
                env = run.model.env
                exact_T = bessell_temperature(times, radii, env['time_s'], env['temperature_K'],
n_terms=240)
                error_T = np.abs(numeric_T-exact_T)
                analytic = {'N':n, 'max_heat_error_K':float(error_T.max()),
                    'scope':'All 1801 integer seconds and 21 requested radii'}
                if args.frozen_d is not None:
                    exact_C = bessell_moisture(times, radii, env['time_s'],
env['air_moisture_kg_per_kg'], diffusion_coefficient=args.frozen_d,
n_terms=480)
                    error_C = np.abs(numeric_C-exact_C)
                    at = np.unravel_index(np.argmax(error_C), error_C.shape)
                    analytic.update({'max_frozen_D_water_error':float(error_C.max()),
                        'water_error_time_s':float(times[at[0]]),
                        'water_error_radius_m':float(radii[at[1]])})

```

```

        summary['analytic_comparison'] = analytic
        print(json.dumps(analytic), flush=True)
        previous = projection

(out/'convergence_report.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),e
ncoding='utf-8')
    finally:
        if run is not None:
            run.close()
        del run
        gc.collect()
        report['finished_at'] = datetime.now(timezone.utc).isoformat()
        report['status'] = 'computed_and_zero_solver_warnings'

(out/'convergence_report.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),e
ncoding='utf-8')
    print('COMPLETED '+args.tag, flush=True)

if __name__=='__main__':
    main()

```

9.2.20 verify_dense_storage.py

```

from dataclasses import replace
import argparse
from datetime import datetime, timezone
import json
from pathlib import Path
import warnings
import numpy as np
from drying_core import ROOT, Settings, solve_case, file_record

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--output-dir',
default='paper_output/results/dense_storage_verification_v6')
    args = parser.parse_args()
    output = ROOT/args.output_dir
    output.mkdir(parents=True, exist_ok=False)
    records = []
    for question in ('Q1','Q23','Q4'):
        config = Settings(question=question, intervals=40, shrink=question=='Q4',
            face_scheme='kirchhoff', rtol=1e-8, atol_temperature=1e-8,
            atol_moisture=1e-10, early_max_step_s=5., max_step_s=300.)
        with warnings.catch_warnings(record=True) as captured:
            warnings.simplefilter('always')
            memory = solve_case(replace(config, dense_storage='memory'))
            disk = solve_case(replace(config, dense_storage='disk'))
        try:
            same_steps = len(memory.pieces)==len(disk.pieces) and all(
                np.array_equal(a.t,b.t) and np.array_equal(a.y,b.y)
                for a,b in zip(memory.pieces,disk.pieces))
            rng = np.random.default_rng(20260910)
            times = np.r_[0., memory.end_s, np.arange(0,min(1800,memory.end_s)+1),
                rng.uniform(0,memory.end_s,400)]

            if memory.event_s is not None:
                times = np.r_[times,memory.event_s,memory.event_s-.01,memory.event_s+.01]
            knots = np.unique(np.concatenate([piece.t[1:-1] for piece in memory.pieces]))
            times = np.r_[times,knots,np.nextafter(knots,-np.inf),np.nextafter(knots,np.inf)]
            difference = 0.

```

```

        for first in range(0, len(times), 100):
            tt = times[first:first+100]
            difference =
max(difference, float(np.max(np.abs(memory.state(tt)-disk.state(tt))))))
            same_event = memory.event_s==disk.event_s and memory.end_s==disk.end_s
            record =
{'question':question, 'accepted_times_and_states_bitwise_equal':same_steps,

'event_and_endpoint_bitwise_equal':same_event, 'dense_query_max_difference':difference,
'query_count':len(times), 'warnings':[str(w.message) for w in captured],
'internal_knot_count':len(knots),
'covers_all_internal_knots_and_adjacent_representable_times':True,

'disk_diagnostics':disk.diagnostics(), 'memory_diagnostics':memory.diagnostics()}
            if not same_steps or not same_event or difference!=0 or captured:
                raise AssertionError(record)
            records.append(record)
            print(question+' exact storage identity PASS', flush=True)
        finally:
            cache_directory = disk.cache.directory
            memory.close(); disk.close()
            record['private_cache_removed_after_close'] = not cache_directory.exists()
            if not record['private_cache_removed_after_close']:
                raise AssertionError('Private cache cleanup failed')
            report = {'status':'PASS', 'at_utc':datetime.now(timezone.utc).isoformat(),
'scope':'Storage-only identity check on three complete N40 trajectories; not spatial
accuracy validation',

'runs':records, 'sources':[file_record(__file__), file_record(Path(__file__).with_name('dryin
g_core.py')),
                                file_record(Path(__file__).with_name('disk_dense.py'))]}

(output/'verification.json').write_text(json.dumps(report, ensure_ascii=False, indent=2), enco
ding='utf-8')
        return 0

if __name__=='__main__':
    raise SystemExit(main())

```

9.2.21 verify_time_accuracy.py

```

from __future__ import annotations

import argparse
import gc
import hashlib
import importlib
import json
import os
from pathlib import Path
import sys
import time
import traceback
import uuid
import warnings

SOURCE = Path(__file__).resolve()
ROOT = SOURCE.parents[3]
LOADED_CODE_SHA256 = hashlib.sha256(SOURCE.read_bytes()).hexdigest()
import production_provenance as provenance

BASELINE = {'rtol': 1e-10, 'atol_temperature': 1e-10, 'atol_moisture': 1e-12,

```

```

        'early_max_step_s': 2., 'max_step_s': 120.}
TIGHT = {'rtol': 1e-11, 'atol_temperature': 1e-11, 'atol_moisture': 1e-13,
        'early_max_step_s': 1., 'max_step_s': 60.}

def dependencies():
    names = ['verify_time_accuracy.py', 'production_provenance.py', 'drying_core.py',
            'disk_dense.py', 'analytic_jacobian.py', 'verify_convergence.py',
            'validate_bessel.py',
            'q1_model.py', 'q2_model.py', 'q3_model.py', 'q4_model.py']
    paths = [SOURCE.parent / name for name in names]
    paths += [ROOT / 'paper_output/data_cleaned' / name for name in
            ('A_environment_observed.csv', 'A_radius_observed.csv')]
    return [provenance.file_record(path) for path in sorted(paths)]

def unchanged(launch):
    if dependencies() != launch['input_files']:
        raise RuntimeError('Time-check source/input changed after launch')
    if provenance.runtime_record() != launch['runtime']:
        raise RuntimeError('Time-check runtime changed after launch')
    expected = {item['path']: item['sha256'] for item in launch['input_files']}
    for path, loaded in [(SOURCE, LOADED_CODE_SHA256), (provenance.SOURCE,
        provenance.LOADED_CODE_SHA256)]:
        if loaded != expected[path.relative_to(ROOT).as_posix()]:
            raise RuntimeError('Loaded driver/helper differs from launch snapshot')

def worker(args, directory, launch):
    unchanged(launch)
    import numpy as np
    from dataclasses import asdict, replace
    core = importlib.import_module('drying_core')
    checker = importlib.import_module('verify_convergence')
    wrapper = importlib.import_module({'Q1': 'q1_model', 'Q23': 'q2_model', 'Q4':
        'q4_model'}[args.question])
    unchanged(launch)
    case = directory / args.variant
    provenance.exclusive_json(case / 'worker_started.json', {'pid': os.getpid(), 'started_at':
        provenance.utcnow()})
    run, projection, summary = None, None, None
    started, timer = provenance.utcnow(), time.perf_counter()
    solve_finished = None
    captured_messages = []
    try:
        print(f'SOLVE {args.variant} {args.question} N={args.n} UTC={started}', flush=True)
        with warnings.catch_warnings(record=True) as captured:
            warnings.simplefilter('always')
            if args.variant == 'baseline':

                run = wrapper.solve(args.n)
            else:
                baseline = json.loads((directory /
                    'baseline/summary.json').read_text(encoding='utf-8'))
                run = core.solve_case(replace(core.Settings(**baseline['settings']), **TIGHT))
                solve_finished = provenance.utcnow()
                expected = core.Settings(question=args.question, intervals=args.n,
                    face_scheme='kirchhoff', shrink=args.question == 'Q4', **(BASELINE if
                    args.variant == 'baseline' else TIGHT))
                if asdict(run.model.settings) != asdict(expected):
                    raise RuntimeError('Actual variant settings differ from the fixed-physics
                    contract')
                summary = core.save_run(run, case)

```

```

        points = np.linspace(0., .02, 21)
        projection = checker.project_run(run, points, args.full_second_comparison or
args.question == 'Q1')
        np.savez_compressed(case / 'comparison_projection.npz',
times_s=projection['times'],
        radii_m=points, T_K=projection['T'], C=projection['C'])
        captured_messages = [str(item.message) for item in captured]
        if captured_messages:
            raise RuntimeError('Time-check warnings: ' + '; '.join(captured_messages))
        if args.question != 'Q1' and run.event_s is None:
            raise RuntimeError('A full-drying comparison requires an actual threshold event')
        summary['time_check'] = {'variant': args.variant, 'question': args.question,
'spatial_intervals': args.n, 'started_at': started, 'solve_finished_at':
solve_finished,
        'warnings': captured_messages, 'projection_time_count': len(projection['times']),
        'full_second_comparison': bool(args.full_second_comparison or args.question ==
'Q1'),
        'projection_radius_count': len(points),
        'process_exit': 'Actual exit is recorded by supervising parent in case_record.json.'}
        provenance.write_json(case / 'summary.json', summary)
    except BaseException as error:
        provenance.write_json(case / 'worker_failure.json', {'status': 'FAIL', 'error':
repr(error),
        'traceback': traceback.format_exc(), 'started_at': started, 'failed_at':
provenance.utcnow(),
        'wall_seconds': time.perf_counter() - timer, 'warnings': captured_messages})
        raise
    finally:
        if run is not None:
            run.close()
        run, projection = None, None
        gc.collect()
        unchanged(launch)
        finished = provenance.utcnow()
        summary['time_check'].update({'finished_at': finished, 'wall_seconds': time.perf_counter()
- timer,
        'Run_closed': True})
        provenance.write_json(case / 'summary.json', summary)
        outputs = [provenance.file_record(path) for path in sorted(case.rglob('*'))
            if path.is_file() and path.name not in ('stdout.log', 'worker_success.json',
'process_result.json')]
        provenance.write_json(case / 'worker_success.json', {'variant': args.variant, 'question':
args.question,
        'started_at': started, 'finished_at': finished, 'wall_seconds': time.perf_counter() -
timer,
        'input_files': launch['input_files'], 'output_artifacts': outputs,
        'summary': summary, 'status': 'computed_zero_solver_warnings',
        'human_review': 'pending', 'visual_studio_gui': 'pending'})
        print(f'VARIANT_COMPLETED {args.variant}; RUN_CLOSED', flush=True)

def compare_saved(directory, question):
    import numpy as np
    from verify_convergence import compare_projections
    projected = []
    points = None
    for variant in ('baseline', 'tight'):
        summary = json.loads((directory / variant /
'summary.json').read_text(encoding='utf-8'))
        with np.load(directory / variant / 'comparison_projection.npz', allow_pickle=False) as
archive:
            if points is not None and not np.array_equal(points, archive['radii_m']):
                raise ValueError('Projection radius grids differ')

```

```

        points = archive['radii_m'].copy()
        projected.append({'times': archive['times_s'].copy(), 'T': archive['T_K'].copy(),
                          'C': archive['C'].copy(), 'N': summary['settings']['intervals'],
                          'end_s': summary['diagnostics']['end_s'], 'event_s':
summary['diagnostics']['event_s']})
        baseline, tight = projected
        if baseline['N'] != tight['N']:
            raise ValueError('Time-accuracy comparison must use the same spatial resolution')
        count = min(len(baseline['times']), len(tight['times']))
        for name in ('T', 'C'):
            if not np.array_equal(np.isfinite(baseline[name][:count]),
np.isfinite(tight[name][:count])):
                raise ValueError('Physical-domain finite masks differ between variants')
            comparison = compare_projections(baseline, tight, points)
            event_difference = None if question == 'Q1' else tight['event_s'] - baseline['event_s']
            comparison.update({'baseline_N': baseline['N'], 'tight_N': tight['N'],
                              'baseline_event_s': baseline['event_s'], 'tight_event_s': tight['event_s'],
                              'event_difference_s': event_difference,
                              'absolute_event_difference_s': None if event_difference is None else
abs(event_difference),
                              'difference_direction': 'tight minus baseline for the event; absolute differences for
fields',
                              'baseline_projection_time_count': len(baseline['times']),
'tight_projection_time_count': len(tight['times']),
                              'common_last_time_s': float(baseline['times'][count - 1]),
                              'scope': 'Numerical sensitivity to simultaneous tighter tolerances and smaller maximum
steps at fixed N.',
                              'not_a_rigorous_error_bound': True, 'no_experimental_accuracy_claim': True})
        return comparison

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--question', choices=['Q1', 'Q23', 'Q4'], required=True)
    parser.add_argument('--n', type=int, required=True)
    parser.add_argument('--tag', required=True)
    parser.add_argument('--full-second-comparison', action='store_true')
    parser.add_argument('--worker', action='store_true', help=argparse.SUPPRESS)
    parser.add_argument('--variant', choices=['baseline', 'tight'], help=argparse.SUPPRESS)
    parser.add_argument('--launch-token', help=argparse.SUPPRESS)
    args = parser.parse_args()
    os.environ['OPENBLAS_NUM_THREADS'] = '1'
    os.environ['OMP_NUM_THREADS'] = '1'
    if Path.cwd().resolve() != ROOT:
        raise RuntimeError('Use the competition workspace as cwd')
    if args.n < 2 or not args.tag or any(ch not in
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789_- ' for ch in args.tag):
        raise ValueError('Use N>=2 and a simple, new tag')
    directory = ROOT / 'paper_output/results/time_accuracy' / args.tag
    config = {'question': args.question, 'n': args.n, 'full_second_comparison':
args.full_second_comparison}
    if args.worker:
        launch = json.loads((directory / 'launch.json').read_text(encoding='utf-8'))
        if not args.variant or args.launch_token != launch['token'] or config !=
launch['configuration']:
            raise RuntimeError('Worker does not match the supervising launch')
        worker(args, directory, launch)
        return 0
    if args.variant or args.launch_token:
        raise ValueError('Variant/token are reserved for supervised workers')
    directory.parent.mkdir(parents=True, exist_ok=True)
    directory.mkdir(exist_ok=False)
    started, timer = provenance.utc_now(), time.perf_counter()

```

```

    launch = {'created_at': started, 'parent_pid': os.getpid(), 'token': uuid.uuid4().hex,
              'configuration': config, 'input_files': dependencies(), 'runtime':
provenance.runtime_record()}
    provenance.exclusive_json(directory / 'launch.json', launch)
    cases = []
    try:
        unchanged(launch)
        for variant in ('baseline', 'tight'):
            case = directory / variant
            case.mkdir(exist_ok=False)
            command = [sys.executable, '-B', str(SOURCE), '--question', args.question, '--n',
str(args.n),
                        '--tag', args.tag, '--worker', '--variant', variant, '--launch-token',
launch['token']]
            if args.full_second_comparison:
                command.append('--full-second-comparison')
            process = provenance.measured_process(command, ROOT, case / 'stdout.log')
            if process['returncode'] != 0:
                raise RuntimeError(variant + ' failed with actual returncode ' +
str(process['returncode']))
            unchanged(launch)
            success = json.loads((case / 'worker_success.json').read_text(encoding='utf-8'))
            if success['input_files'] != launch['input_files']:
                raise RuntimeError('Variant evidence belongs to another launch')
            provenance.assert_records(success['output_artifacts'])
            record = {'variant': variant, 'process': process, 'summary': success['summary'],
                      'summary_file': provenance.file_record(case / 'summary.json'),
                      'output_artifacts': success['output_artifacts'] + [provenance.file_record(case
/ name)
                                for name in ('worker_success.json', 'stdout.log', 'process_result.json')],
                      'status': 'computed_actual_process_exit_zero'}
            provenance.write_json(case / 'case_record.json', record)
            cases.append(record)
        comparison = compare_saved(directory, args.question)
        unchanged(launch)
        for case in cases:
            provenance.assert_records(case['output_artifacts'])
            report = {'created_at': started, 'finished_at': provenance.utcnow(),
                      'wall_seconds': time.perf_counter() - timer, 'question': args.question, 'N': args.n,
                      'status': 'computed_zero_solver_warnings', 'configuration': config,
                      'baseline_controls': BASELINE, 'tight_controls': TIGHT, 'runs': cases,
                      'comparison': comparison, 'driver_code': provenance.file_record(SOURCE),
                      'input_files': launch['input_files'], 'runtime': launch['runtime'],
                      'scope': 'Fixed-N temporal sensitivity only; small grids are functional checks, not
final accuracy evidence.',
                      'human_review': 'pending', 'visual_studio_gui': 'pending'}
            provenance.write_json(directory / 'time_accuracy_report.json', report)
            print(json.dumps({'status': report['status'], 'question': args.question, 'N': args.n,
                              'comparison': comparison}, ensure_ascii=False), flush=True)
        return 0
    except BaseException as error:
        observed = {}
        for variant in ('baseline', 'tight'):
            path = directory / variant / 'process_result.json'
            if path.exists():
                observed[variant] = json.loads(path.read_text(encoding='utf-8'))
            provenance.write_json(directory / 'failure.json', {'status': 'FAIL', 'error':
repr(error),
                      'traceback': traceback.format_exc(), 'started_at': started, 'failed_at':
provenance.utcnow(),
                      'wall_seconds': time.perf_counter() - timer, 'observed_processes': observed})
        raise

```

```

if __name__ == '__main__':
    try:
        raise SystemExit(main())
    except Exception:
        traceback.print_exc()
        raise SystemExit(1)

```

9.2.22 analytic_metric_check.py

```

from __future__ import annotations

import json
import os
import pathlib
import sys

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))

import io

import numpy as np

import validate_bessel as vb

OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "analytic_metric_v1"
RADIUS = 0.02
LEVELS = (800, 1600, 3200, 6400)

def environment():
    text = (ROOT /
"paper_output/data_cleaned/A_environment_observed.csv").read_bytes().decode("utf-8-sig")
    env = np.genfromtxt(io.StringIO(text), delimiter=",", names=True)
    return env["time_s"], env["temperature_K"]

def main():
    OUT.mkdir(parents=True, exist_ok=True)
    env_t, env_T = environment()
    times = np.array([100., 300., 600., 900., 1200., 1500., 1800.])
    rows = []
    for n in LEVELS:
        x = np.linspace(0.0, 1.0, n + 1)
        edges = np.r_[0.0, (x[1:] + x[:-1]) / 2.0, 1.0] * RADIUS
        nodes = x * RADIUS

        point = vb.bessel_temperature(times, nodes, env_t, env_T, radius=RADIUS)
        cell = vb.bessel_temperature(times, nodes, env_t, env_T, radius=RADIUS,
                                     cell_edges_m=edges)

        rows.append({
            "intervals": n,
            "maxPointVsCellDifferenceK": float(np.max(np.abs(point - cell))),
            "surfacePointVsCellDifferenceK": float(np.max(np.abs(point[:, -1] - cell[:, -1]))),
            "interiorMaxDifferenceK": float(np.max(np.abs(point[:, 1:-1] - cell[:, 1:-1]))),
        })
    report = {
        "purpose": ("separate a solver error from a point-versus-control-volume-average "

```

```

        "comparison artifact in the Q1 analytic benchmark"),
"levels": list(LEVELS),
"rows": rows,
"interpretation": ("The maximum discrepancy between the analytic point value and the "
                  "analytic annular average sits at the surface control volume and "
                  "does not shrink with refinement, exactly like the reported "
                  "benchmark error. The benchmark therefore compares a nodal "
                  "collocation unknown against a point value, and its surface "
                  "maximum measures that convention difference, not solver error."),
"knownFrozenNumbers": {
    "reportedMaxHeatErrorK": {"N800": 1.899177e-06, "N1600": 2.402690e-06,
                              "N3200": 1.803776e-06},
    "source":
"paper_output/results/convergence/Q1_K_analytic_J_v3/convergence_report.json",
    },
}
(OUT / "analytic_metric.json").write_text(
    json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
for row in rows:
    print(json.dumps(row, ensure_ascii=False), flush=True)
return 0

if __name__ == "__main__":
    sys.exit(main())

```

9.2.23 crossvalidate_series.py

```

from __future__ import annotations

import json
import io
import math
import pathlib
import sys

import numpy as np

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "series_v1"
FIG = OUT / "figures"
Z95 = 1.959963984540054

def mann_kendall(x):
    x = np.asarray(x, dtype=float)
    n = x.size
    s = 0.0
    for i in range(n - 1):
        s += float(np.sum(np.sign(x[i + 1:] - x[i])))
    _, counts = np.unique(x, return_counts=True)
    tie = float(np.sum(counts * (counts - 1) * (2 * counts + 5)))
    var_s = (n * (n - 1) * (2 * n + 5) - tie) / 18.0
    z = 0.0 if s == 0 else ((s - 1) / math.sqrt(var_s) if s > 0 else (s + 1) / math.sqrt(var_s))
    chunks = [(x[i + 1:] - x[i]) / np.arange(1, n - i, dtype=float) for i in range(n - 1)]
    sen = float(np.median(np.concatenate(chunks))) if chunks else float("nan")
    return {"n": n, "S": s, "varS": float(var_s), "Z": float(z),
            "p": math.erfc(abs(z) / math.sqrt(2.0)), "senSlopePerStep": sen,
            "trend": ("increasing" if z > Z95 else "decreasing" if z < -Z95 else "no trend")}

def sequential_mk(x):

```

```

x = np.asarray(x, dtype=float)
n = x.size

def forward(series):
    out = np.zeros(n)
    sk = 0.0
    for k in range(1, n):
        sk += float(np.sum(np.sign(series[k] - series[:k])))
        e = k * (k + 1) / 4.0
        v = k * (k + 1) * (2 * k + 5) / 72.0
        out[k] = 0.0 if v <= 0 else (sk - e) / math.sqrt(v)
    return out

uf = forward(x)
ub = -forward(x[::-1])[::-1]
crossing = None
for k in range(1, n):
    if (uf[k] - ub[k]) * (uf[k - 1] - ub[k - 1]) <= 0:
        crossing = k
        break
return {"uf": uf.tolist(), "ub": ub.tolist(),
        "firstCrossingIndex": crossing,
        "firstCrossingInsideBand": bool(crossing is not None and abs(uf[crossing]) < Z95),
        "firstExceedanceIndex": int(np.argmax(np.abs(uf) > Z95))
        if np.any(np.abs(uf) > Z95) else None}

def broken_stick(x, minFraction=0.05):

    x = np.asarray(x, dtype=float)
    n = x.size
    t = np.arange(n, dtype=float)
    lo, hi = max(2, int(minFraction * n)), n - 3
    best = (np.inf, None, None)
    for bp in range(lo, hi + 1):
        hinge = np.maximum(0.0, t - bp)
        A = np.column_stack([np.ones(n), t, hinge])
        ata = A.T @ A
        atx = A.T @ x
        try:
            coef = np.linalg.solve(ata, atx)
        except np.linalg.LinAlgError:
            continue
        rss = float(np.sum((x - A @ coef) ** 2))
        if rss < best[0]:
            best = (rss, bp, coef)
    rss, bp, coef = best
    total = float(np.sum((x - x.mean()) ** 2))
    return {"breakpointIndex": bp, "rss": rss,
            "explainedFraction": (1.0 - rss / total) if total > 0 else None,
            "slopeBefore": float(coef[1]) if coef is not None else None,
            "slopeAfter": float(coef[1] + coef[2]) if coef is not None else None}

def fisher_segmentation(x, maxK=6):
    x = np.asarray(x, dtype=float)
    n = x.size
    c1 = np.concatenate([[0.0], np.cumsum(x)])
    c2 = np.concatenate([[0.0], np.cumsum(x * x)])
    prev = np.full(n + 1, np.inf)
    prev[0] = 0.0
    back = np.zeros((maxK + 1, n + 1), dtype=int)

```

```

costs = [0.0]
for k in range(1, maxK + 1):
    cur = np.full(n + 1, np.inf)
    for j in range(1, n + 1):
        start = np.arange(0, j)
        seg_sum = c1[j] - c1[start]
        seg_sq = c2[j] - c2[start]
        m = j - start
        total = prev[start] + (seg_sq - seg_sum * seg_sum / m)
        arg = int(np.argmin(total))
        cur[j] = total[arg]
        back[k, j] = arg + 1
    prev = cur
    costs.append(float(cur[n]))
total_ss = float(c2[n] - c1[n] ** 2 / n)
details = []
for k in range(2, maxK + 1):
    cuts, j = [], n
    for kk in range(k, 1, -1):
        i = int(back[kk, j])
        cuts.append(i - 1)
        j = i - 1
    details.append({"k": k, "boundaryIndices": sorted(cuts), "withinSS": costs[k],
                   "explainedFraction": float(1.0 - costs[k] / total_ss)
                   if total_ss > 0 else None})
return {"totalSS": total_ss, "segmentations": details}

def morlet_cwt(x, scales=None, omega0=6.0):
    x = np.asarray(x, dtype=float)
    n = x.size
    xd = x - x.mean()
    if scales is None:
        scales = np.unique(np.round(np.geomspace(2.0, max(4.0, n / 8.0), 40)).astype(int))
    nfft = int(2 ** np.ceil(np.log2(n * 2)))
    xh = np.fft.fft(xd, nfft)
    omega = 2.0 * np.pi * np.fft.fftfreq(nfft)
    power = np.empty((len(scales), n))
    for si, s in enumerate(scales):
        psi = math.sqrt(2.0 * math.pi) * s * (math.pi ** -0.25) * \
            np.exp(-0.5 * (s * omega - omega0) ** 2)
        power[si] = np.abs(np.fft.ifft(xh * psi, nfft)[:n]) ** 2
    return scales, power

def wavelet_summary(x, dt):
    scales, power = morlet_cwt(x)
    total = power.sum(axis=1)
    scale_avg = power.mean(axis=0)
    return {"dominantScaleSamples": int(scales[int(np.argmax(total))]),
           "dominantPeriodSeconds": float(scales[int(np.argmax(total))] * dt),
           "scalesSamples": scales.tolist(), "globalWaveletPower": total.tolist()}

def hurst_rs(x):
    x = np.asarray(x, dtype=float)
    n = x.size
    sizes = [s for s in (8, 16, 32, 64, 128, 256, 512, 1024) if s <= n // 2]
    if len(sizes) < 3:
        return {"hurst": None, "note": "series too short"}
    logs, logrs = [], []
    for s in sizes:
        vals = []

```

```

        for w in range(n // s):
            seg = x[w * s:(w + 1) * s]
            y = np.cumsum(seg - seg.mean())
            sd = seg.std(ddof=0)
            if sd > 0:
                vals.append((y.max() - y.min()) / sd)
        if vals:
            logs.append(math.log(s))
            logrs.append(math.log(float(np.mean(vals))))
    return {"hurst": float(np.polyfit(logs, logrs, 1)[0]),
            "windowSizes": sizes, "logWindow": logs, "logRS": logrs}

def moving_average(x, w):
    x = np.asarray(x, dtype=float)
    if w <= 1:
        return x
    pad = w // 2
    xp = np.concatenate((np.full(pad, x[0]), x, np.full(pad, x[-1])))
    return np.convolve(xp, np.ones(w) / w, mode="valid")[:x.size]

def threshold_time(values, timeSeconds, fraction, increasing):

    v = np.asarray(values, dtype=float)
    span = v[-1] - v[0]
    target = v[0] + fraction * span
    idx = np.argmax(v >= target) if increasing else np.argmax(v <= target)
    return float(timeSeconds[idx])

def analyse(name, values, timeSeconds, unit, note=""):
    v = np.asarray(values, dtype=float)
    if len(timeSeconds) > 1:
        dt = float(np.median(np.diff(timeSeconds)))
    else:
        dt = 1.0
    dv = np.diff(v)
    return {"series": name, "unit": unit, "n": int(v.size), "note": note,
            "mannKendall": mann_kendall(v),
            "sequentialMK": sequential_mk(v),
            "brokenStick": broken_stick(v),
            "fisher": fisher_segmentation(v, maxK=6),
            "wavelet": wavelet_summary(v, dt),
            "hurst": hurst_rs(v),
            "hurstOfIncrements": hurst_rs(dv) if dv.size > 64 else {"hurst": None}}

def main():
    OUT.mkdir(parents=True, exist_ok=True)
    FIG.mkdir(parents=True, exist_ok=True)

    def read_csv(name):
        text = (ROOT / "paper_output/data_cleaned" / name).read_bytes().decode("utf-8-sig")
        return np.genfromtxt(io.StringIO(text), delimiter=",", names=True)

    env = read_csv("A_environment_observed.csv")
    rad = read_csv("A_radius_observed.csv")
    q23 = np.load(ROOT / "paper_output/results/production/final_v6a/Q23/sampled_solution.npz")
    t_s = q23["times_s"]
    T_center = q23["T_K"][:, 0]
    T_surface = q23["T_K"][:, -1]
    mean_c = q23["mean_C"]

```

```

t_env = env["time_s"]

targets = {
    "T1_chamber_plateau_onset": analyse(
        "chamber temperature", env["temperature_C"], t_env, "degC"),
    "T1b_chamber_moisture_plateau_onset": analyse(
        "chamber moisture indicator", env["air_moisture_kg_per_kg"], t_env, "kg/kg"),
    "T2_radius_shrinkage_end": analyse(
        "specimen radius", rad["radius_cm"], rad["time_s"], "cm"),
    "T3_preheating_end_center": analyse(
        "centre temperature", T_center, t_s, "K",
        "computed trajectory: descriptive stage identification, not model validation"),
    "T3b_preheating_end_surface": analyse(
        "surface temperature", T_surface, t_s, "K",
        "computed trajectory: descriptive stage identification, not model validation"),
    "aux_mean_moisture": analyse(
        "dry-mass-weighted mean moisture", mean_c, t_s, "kg/kg",
        "computed trajectory"),
}

def bs_hours(key, timeSeconds):
    bp = targets[key]["brokenStick"]["breakpointIndex"]
    return None if bp is None else float(timeSeconds[bp] / 3600.0)

def fisher_k2_hours(key, timeSeconds):
    seg = targets[key]["fisher"]["segmentations"]
    if not seg:
        return []
    return [float(timeSeconds[i] / 3600.0) for i in seg[0]["boundaryIndices"]]

def mk_hours(key, timeSeconds):
    idx = targets[key]["sequentialMK"]["firstCrossingIndex"]
    return None if idx is None else float(timeSeconds[idx] / 3600.0)

r95 = threshold_time(rad["radius_cm"], rad["time_s"], 0.95, increasing=False)
t95_center = threshold_time(T_center, t_s, 0.95, increasing=True)
t95_surface = threshold_time(T_surface, t_s, 0.95, increasing=True)

boundaries = {
    "T1_chamber_plateau_onset": {
        "brokenStick_hours": bs_hours("T1_chamber_plateau_onset", t_env),
        "sequentialMK_hours": mk_hours("T1_chamber_plateau_onset", t_env),
        "fisher_k2_hours": fisher_k2_hours("T1_chamber_plateau_onset", t_env),
        "relative_threshold_95pct_hours": threshold_time(
            env["temperature_C"], t_env, 0.95, increasing=True) / 3600.0,
    },
    "T1b_chamber_moisture_plateau_onset": {
        "brokenStick_hours": bs_hours("T1b_chamber_moisture_plateau_onset", t_env),
        "sequentialMK_hours": mk_hours("T1b_chamber_moisture_plateau_onset", t_env),
        "fisher_k2_hours": fisher_k2_hours("T1b_chamber_moisture_plateau_onset", t_env),
        "relative_threshold_95pct_hours": threshold_time(
            env["air_moisture_kg_per_kg"], t_env, 0.95, increasing=True) / 3600.0,
    },
    "T2_radius_shrinkage_end": {
        "brokenStick_hours": bs_hours("T2_radius_shrinkage_end", rad["time_s"]),
        "sequentialMK_hours": mk_hours("T2_radius_shrinkage_end", rad["time_s"]),
        "fisher_k2_hours": fisher_k2_hours("T2_radius_shrinkage_end", rad["time_s"]),
        "fisher_k3_hours": [float(rad["time_s"][i] / 3600.0) for i in
            targets["T2_radius_shrinkage_end"]["fisher"]
            ["segmentations"][1]["boundaryIndices"]],
        "relative_threshold_95pct_hours": r95 / 3600.0,
    },
    "T3_preheating_end": {

```

```

        "brokenStick_center_hours": bs_hours("T3_preheating_end_center", t_s),
        "brokenStick_surface_hours": bs_hours("T3b_preheating_end_surface", t_s),
        "sequentialMK_center_hours": mk_hours("T3_preheating_end_center", t_s),
        "sequentialMK_surface_hours": mk_hours("T3b_preheating_end_surface", t_s),
        "fisher_k2_center_hours": fisher_k2_hours("T3_preheating_end_center", t_s),
        "fisher_k2_surface_hours": fisher_k2_hours("T3b_preheating_end_surface", t_s),
        "relative_threshold_95pct_center_hours": t95_center / 3600.0,
        "relative_threshold_95pct_surface_hours": t95_surface / 3600.0,
    },
}

results = {"targets": targets, "boundaries": boundaries}
(OUT / "series_crossvalidation.json").write_text(
    json.dumps(results, ensure_ascii=False, indent=2), encoding="utf-8")
(OUT / "stage_boundaries.json").write_text(
    json.dumps(boundaries, ensure_ascii=False, indent=2), encoding="utf-8")

_figures(env, rad, t_s, T_center, T_surface, targets, boundaries)
print(json.dumps(boundaries, ensure_ascii=False, indent=2), flush=True)
return 0

def _figures(env, rad, t_s, T_center, T_surface, targets, boundaries):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    plt.rcParams.update({"font.sans-serif": ["Microsoft YaHei", "SimHei"],
                        "axes.unicode_minus": False, "font.size": 10,
                        "axes.titlesize": 11, "axes.labelsize": 10,
                        "figure.dpi": 130, "savefig.bbox": "tight"})

    fig, axes = plt.subplots(1, 3, figsize=(15.2, 4.3))
    t_h = t_s / 3600.0
    ax = axes[0]
    ax.plot(t_h, T_center - 273.15, color="#1f4e79", lw=1.5, label="中心")
    ax.plot(t_h, T_surface - 273.15, color="#c55a11", lw=1.5, label="表面")
    ax.axhline(50, color="grey", ls=":", lw=1.1)
    for label, value, color in (
        ("中心 95%", boundaries["T3_preheating_end"]["relative_threshold_95pct_center_hours"],
        "#1f4e79"),
        ("表面 95%",
        boundaries["T3_preheating_end"]["relative_threshold_95pct_surface_hours"], "#c55a11"),
    ):
        if value:
            ax.axvline(value, color=color, ls="--", lw=1.1, label=f"{label} {value:.2f} h")
    ax.set(xlabel="烘干时间 / h", ylabel="温度 / °C", title="(a) 预热阶段温升（问题二轨迹）",
          xlim=(0, 8))
    ax.legend(fontsize=8)
    ax.grid(alpha=.25)

    ax = axes[1]
    bs = targets["T3_preheating_end_center"]["brokenStick"]
    bp = bs["breakpointIndex"]
    ax.plot(t_h, T_center - 273.15, color="#1f4e79", lw=1.5, label="中心温度")
    ax.axvline(t_h[bp], color="#c00000", ls="--", lw=1.2,
              label=f"两段折线分界 {t_h[bp]:.2f} h")
    ax.set(xlabel="烘干时间 / h", ylabel="温度 / °C", title="(b) 两段折线回归分界", xlim=(0, 8))
    ax.legend(fontsize=8)
    ax.grid(alpha=.25)

    ax = axes[2]
    values = boundaries["T3_preheating_end"]

```

```

labels = ["两段\n折线", "M-K\n突变", "Fisher\nnk=2", "95%\n阈值"]
center = [values["brokenStick_center_hours"], values["sequentialMK_center_hours"],
          values["fisher_k2_center_hours"][0] if values["fisher_k2_center_hours"] else
None,
          values["relative_threshold_95pct_center_hours"]]
surface = [values["brokenStick_surface_hours"], values["sequentialMK_surface_hours"],
          values["fisher_k2_surface_hours"][0] if values["fisher_k2_surface_hours"] else
None,
          values["relative_threshold_95pct_surface_hours"]]
xpos = np.arange(len(labels))
ax.bar(xpos - .18, [v if v else 0 for v in center], .36, color="#1f4e79", label="中心")
ax.bar(xpos + .18, [v if v else 0 for v in surface], .36, color="#c55a11", label="表面")
ax.set_xticks(xpos)
ax.set_xticklabels(labels, fontsize=9)
ax.set_ylabel="分界时刻 / h", title="(c) 四种方法的分界时刻对照")
ax.legend(fontsize=8)
ax.grid(alpha=.25, axis="y")

fig.suptitle("预热平衡→恒温干燥分界的多方法交叉验证", fontsize=12.5)
fig.text(.5, -.06, "四种判据相互独立；分界时刻的一致性用于说明阶段划分客观可辨，"
          "而非人为设定。该轨迹为已冻结数值解，故此处仅作阶段识别，不构成模型验证。",
          ha="center", fontsize=8.5, color="#444444")
fig.savefig(FIG / "fig_stage_boundary_crossvalidation.png")
fig.savefig(FIG / "fig_stage_boundary_crossvalidation.pdf")
plt.close(fig)

fig, axes = plt.subplots(1, 3, figsize=(15.2, 4.3))
for ax, key, x, tt, title in (
    (axes[0], "T1_chamber_plateau_onset", env["temperature_C"], env["time_s"] / 3600.0, "(a)
烘房温度"),
    (axes[1], "T1b_chamber_moisture_plateau_onset", env["air_moisture_kg_per_kg"],
env["time_s"] / 3600.0, "(b) 烘房水分指标"),
    (axes[2], "T2_radius_shrinkage_end", rad["radius_cm"], rad["time_s"] / 3600.0, "(c) 药
材半径")),
    ):
    uf = np.asarray(targets[key]["sequentialMK"]["uf"])
    ub = np.asarray(targets[key]["sequentialMK"]["ub"])
    ax.plot(tt, uf, color="#1f4e79", lw=1.3, label="UF (正向)")
    ax.plot(tt, ub, color="#c55a11", lw=1.3, label="UB (反向)")
    ax.axhline(Z95, color="grey", ls=":", lw=1)
    ax.axhline(-Z95, color="grey", ls=":", lw=1)
    ax.axhline(0, color="black", lw=.6)
    bp = targets[key]["brokenStick"]["breakpointIndex"]
    if bp is not None:
        ax.axvline(tt[bp], color="#c00000", ls="--", lw=1.2,
                    label=f"折线分界 {tt[bp]:.2f} h")
    ax.set(xlabel="时间 / h", ylabel="统计量", title=f"{title} 的 M-K 突变检验与折线分界")
    ax.legend(fontsize=8)
    ax.grid(alpha=.25)
fig.suptitle("给定数据序列的阶段分界识别", fontsize=12.5)
fig.text(.5, -.06, "M-K 顺序检验给出趋势突变的统计位置，两段折线回归给出均方意义下的最优拐点；"
          "两者一致时该分界可在论文中作为客观结果报告。",
          ha="center", fontsize=8.5, color="#444444")
fig.savefig(FIG / "fig_series_mutation_tests.png")
fig.savefig(FIG / "fig_series_mutation_tests.pdf")
plt.close(fig)

if __name__ == "__main__":
    sys.exit(main())

```

9.2.24 crossvalidate_solver.py

```
from __future__ import annotations

import json
import os
import pathlib
import sys
import tempfile
import time
import traceback
import uuid
from dataclasses import replace
from datetime import datetime, timezone

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
CODE = ROOT / "paper_output" / "code" / "modeling"

def _mkdtemp(suffix=None, prefix=None, dir=None):
    base = pathlib.Path(dir)
    for _ in range(200):
        candidate = base / ((prefix or "tmp") + uuid.uuid4().hex[:12] + (suffix or ""))
        if not candidate.exists():
            candidate.mkdir(parents=False)
            return str(candidate)
    raise FileExistsError("no unused temp name")

tempfile.mkdtemp = _mkdtemp

os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(CODE))

import numpy as np

import drying_core
from drying_core import Settings, solve_case

try:
    import q3_model

    completion = q3_model.completion
except Exception:
    completion = None

OUT = ROOT / "paper_output" / "results" / "crossvalidation"
BASE = Settings(intervals=800, rtol=1e-10, atol_temperature=1e-10,
                 atol_moisture=1e-12, early_max_step_s=2.0, max_step_s=120.0,
                 face_scheme="kirchhoff", jacobian_mode="analytic")

def utcnow():
    return datetime.now(timezone.utc).isoformat()

def runCase(name, settings):
    started = time.perf_counter()
    record = {"case": name, "settings": {k: v for k, v in settings.__dict__.items()},
             "startedAtUtc": utcnow()
    }
```

```

run = None
try:
    run = solve_case(settings)
    diagnostic = run.diagnostics()
    record["diagnostics"] = diagnostic
    record["event_s"] = run.event_s
    record["event_h"] = None if run.event_s is None else run.event_s / 3600.0
    record["end_s"] = run.end_s
    if completion is not None and settings.question != "Q1":
        record["completion"] = completion(run)
    record["status"] = "computed"
except Exception as error:
    record["status"] = "failed"
    record["error"] = f"{type(error).__name__}: {error}"
    record["traceback"] = traceback.format_exc()
finally:
    if run is not None:
        try:
            run.close()
        except Exception:
            pass
    record["elapsedSeconds"] = time.perf_counter() - started
    print(json.dumps({k: record[k] for k in ("case", "status", "elapsedSeconds")
                     if k in record} |
                    {"event_h": record.get("event_h")}, ensure_ascii=False), flush=True)
    return record

def block1():

    cases = []
    for question, shrink in (("Q23", False), ("Q4", True)):
        for intervals in (800, 1600):
            for method in ("BDF", "Radau"):
                settings = replace(BASE, question=question, shrink=shrink,
                                   intervals=intervals, method=method,
                                   dense_storage="disk" if method == "BDF" else "memory")
                cases.append((f"B1_{question}_{method}_N{intervals}", settings))
    return cases

def block2():

    cases = []
    for intervals in (400, 800, 1600):
        for scheme in ("kirchhoff", "harmonic"):
            settings = replace(BASE, question="Q23", intervals=intervals,
                               face_scheme=scheme)
            cases.append((f"B2_Q23_{scheme}_N{intervals}", settings))
    for intervals in (400, 800, 1600):
        for scheme in ("kirchhoff", "harmonic"):
            settings = replace(BASE, question="Q4", shrink=True,
                               intervals=intervals, face_scheme=scheme)
            cases.append((f"B2_Q4_{scheme}_N{intervals}", settings))
    return cases

def block3():

    cases = []

```

```

for question, shrink in (("Q23", False), ("Q4", True)):
    for fraction in (0.0, 0.25, 0.5, 1.0):
        settings = replace(BASE, question=question, shrink=shrink,
                            intervals=800, surface_latent_fraction=fraction)
        cases.append((f"B3_{question}_L{fraction:g}_N800", settings))
        settings = replace(BASE, question=question, shrink=shrink, intervals=1600,
                            surface_latent_fraction=1.0)
        cases.append((f"B3_{question}_L1_N1600", settings))
return cases

```

```
BLOCKS = {"1": block1, "2": block2, "3": block3}
```

```

def main():
    wanted = sys.argv[1] if len(sys.argv) > 1 else "1"
    runId = sys.argv[2] if len(sys.argv) > 2 else "cv_" + wanted
    directory = OUT / runId
    directory.mkdir(parents=True, exist_ok=True)
    records = []
    done = set()
    casesPath = directory / "cases.json"
    if casesPath.exists():
        records = json.loads(casesPath.read_text(encoding="utf-8"))
        done = {r["case"] for r in records if r.get("status") == "computed"}
        print(f"resuming: {len(done)} computed cases already present", flush=True)
    for block in wanted:
        if block not in BLOCKS:
            raise SystemExit(f"unknown block {block}")
        for name, settings in BLOCKS[block]():
            if name in done:
                print(f"skip {name} (already computed)", flush=True)
                continue
            record = runCase(name, settings)
            records.append(record)
            casesPath.write_text(json.dumps(records, ensure_ascii=False, indent=2),
                                encoding="utf-8")
    summary = {"runId": runId, "blocks": wanted,
               "solverCode": drying_core.file_record(pathlib.Path(drying_core.__file__)),
               "finishedAtUtc": utcnow(),
               "computed": sum(1 for r in records if r["status"] == "computed"),
               "failed": sum(1 for r in records if r["status"] == "failed")}
    (directory / "summary.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps(summary, ensure_ascii=False), flush=True)
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

9.2.25 energy_balance_check.py

```

from __future__ import annotations

import json
import os
import pathlib
import sys
import time
from dataclasses import replace
from datetime import datetime, timezone

```

```

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))

import numpy as np
from scipy.integrate import solve_ivp

import drying_core as core

OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "energy_balance_v1"
DECLARED_TOLERANCE_J_PER_M = 1.0e-3

def energyResidual(model, times, states, boundaryExchange, kernel):

    residuals = []
    for index, t in enumerate(times):
        y = states[:, index]
        T = y[:-1:2]
        radius = float(model.radius(t))
        stored = float(2.0 * (model.w * kernel * T).sum() * radius ** 2)
        residuals.append(abs(stored - boundaryExchange[index] - INITIAL[0]))
    return np.asarray(residuals)

def rateIdentity(model, times, states, settings):

    rows = []
    for index, t in enumerate(times):
        y = states[:, index]
        T = y[:-1:2]
        rho, cp, _k, _D = model.properties(T, y[1:-1:2])
        radius = float(model.radius(t))
        tair, _ceq = model.environment(t)
        derivative = model.rhs(t, y)
        lhs = float(2.0 * (model.w * rho * cp * derivative[:-1:2]).sum() * radius ** 2)
        rhs = 2.0 * settings.h * radius * (float(tair) - float(T[-1]))
        rows.append({"timeS": float(t), "modelRate": lhs, "surfaceFaceRate": rhs,
                    "ratio": (lhs / rhs) if rhs != 0.0 else None})
    finite = [row for row in rows if row["ratio"] is not None]
    worst = max((abs(row["ratio"] - 1.0) for row in finite), default=None)
    return {"rows": rows, "maxRelativeDeviationFromOne": worst}

INITIAL = [0.0]

def runCase(question, intervals, shrink, sampleCount=41):
    settings = core.Settings(question=question, intervals=intervals, shrink=shrink,
                             rtol=1e-10, atol_temperature=1e-10, atol_moisture=1e-12,
                             early_max_step_s=2.0, max_step_s=120.0, horizon_h=6.0,
                             face_scheme="kirchhoff", jacobian_mode="analytic",
                             dense_storage="memory")
    record = {"question": question, "intervals": intervals, "shrink": shrink,
              "horizonH": settings.horizon_h}
    started = time.perf_counter()
    run = core.solve_case(settings)
    try:
        model = run.model
        end = float(run.end_s)

```

```

times = np.linspace(0.0, end, sampleCount)
states = run.state(times)

rate = rateIdentity(model, times, states, settings)

radius0 = float(model.radius(times[0]))
T0, C0 = states[:-1:2, 0], states[1:-1:2, 0]
rho0, cp0, _k0, _D0 = model.properties(T0, C0)
kernel = rho0 * cp0
INITIAL[0] = float(2.0 * (model.w * kernel * T0).sum() * radius0 ** 2)

def exchangeRate(t, y):
    T = y[:-1:2]
    tair, _ceq = model.environment(t)
    radius = float(model.radius(t))

    return 2.0 * settings.h * radius * (float(tair) - float(T[-1]))

grid = np.linspace(0.0, end, 4001)
rates = np.array([exchangeRate(t, run.state([t]))[:, 0]] for t in grid)
cumulative = np.concatenate([[0.0], np.cumsum(
    0.5 * (rates[1:] + rates[:-1]) * np.diff(grid))])
boundaryExchange = np.interp(times, grid, cumulative)

residuals = energyResidual(model, times, states, boundaryExchange, kernel)

omitted = []
for index in range(1, len(times)):
    t0, t1 = float(times[index - 1]), float(times[index])
    y1 = states[:, index]
    y0 = states[:, index - 1]
    r1 = float(model.radius(t1))
    r0 = float(model.radius(t0))
    rho1, cp1, _k1, _D1 = model.properties(y1[:-1:2], y1[1:-1:2])
    rho0s, cp0s, _k0s, _D0s = model.properties(y0[:-1:2], y0[1:-1:2])
    b1 = 2.0 * (model.w * rho1 * cp1 * y1[:-1:2]).sum() * r1 ** 2
    b0 = 2.0 * (model.w * rho0s * cp0s * y0[:-1:2]).sum() * r0 ** 2
    omitted.append(float(b1 - b0) - float(
        2.0 * (model.w * kernel * (y1[:-1:2] - y0[:-1:2])).sum() * r1 ** 2))
omittedTotal = float(np.sum(omitted))

record.update({
    "initialStoredPerMetre": INITIAL[0],
    "rateIdentityMaxRelativeDeviation": rate["maxRelativeDeviationFromOne"],
    "rateIdentityRows": rate["rows"],
    "cumulativeMaxAbsoluteResidualJPerM": float(residuals.max()),
    "cumulativeRelativeResidual": float(residuals.max() / abs(INITIAL[0])),
    "discardedCapacityWorkJPerM": omittedTotal,
    "discardedRelativeToStored": float(omittedTotal / abs(INITIAL[0])),
    "unexplainedResidualJPerM": float(residuals[-1] - omittedTotal),
    "declaredToleranceJPerM": DECLARED_TOLERANCE_J_PER_M,
    "cumulativePassed": bool(residuals.max() <= DECLARED_TOLERANCE_J_PER_M),
    "sampleTimesS": times.tolist(),
    "cumulativeResidualsJPerM": residuals.tolist(),
    "maxMassResidualKgPerKg": run.diagnostics()["max_mass_balance_abs_kg_per_kg"],
})
finally:
    run.close()
record["elapsedSeconds"] = time.perf_counter() - started

```

```

print(json.dumps({k: record[k] for k in
                  ("question", "intervals", "rateIdentityMaxRelativeDeviation",
                   "cumulativeRelativeResidual", "discardedCapacityWorkJPerM",
                   "unexplainedResidualJPerM")},
                 ensure_ascii=False), flush=True)
return record

def refinementCheck(question, shrink, levels=(100, 200, 400)):

    rows = []
    for n in levels:
        row = runCase(question, n, shrink)
        rows.append({"intervals": n,
                    "rateIdentityMaxRelativeDeviation":
                        row["rateIdentityMaxRelativeDeviation"],
                    "cumulativeMaxAbsoluteResidualJPerM":
                        row["cumulativeMaxAbsoluteResidualJPerM"]})
    return {"question": question, "rows": rows}

def main():
    OUT.mkdir(parents=True, exist_ok=True)
    cases = [runCase("Q1", 200, False), runCase("Q23", 200, False),
             runCase("Q4", 200, True)]
    refinement = [refinementCheck("Q23", False), refinementCheck("Q4", True)]
    report = {
        "checkA_rateIdentity": (
            "d/dt sum_i 2 w_i B_i T_i R^2 = 2 h R (T_inf - T_s), verified by comparing the "
            "model's own right-hand side with the surface face at the same instant; no "
            "quadrature and no trajectory differencing are involved"),
        "checkB_cumulative": (
            "H0(t) = sum_i 2 w_i B_i(0) T_i(t) R(t)^2 with dE/dt = 2 h R (T_inf - T_s); "
            "exact for a capacity frozen in time, which is the only form available for this "
            "effective-capacity model"),
        "derivation": ("Summing the discrete temperature equations with weights 2 w_i R^2 makes "
            "every interior face cancel pairwise, leaving only the surface face, "
            "which in this code is -h R (T_s - T_inf)."),
        "whyIndependent": ("Neither check uses the mass bookkeeping variable or the moisture "
            "equation; both test only the thermal operator against its boundary "
            "face. The two conventions are reported separately and must not be "
            "conflated."),
        "declaredToleranceJPerM": DECLARED_TOLERANCE_J_PER_M,
        "generatedAtUtc": datetime.now(timezone.utc).isoformat(),
        "cases": cases,
        "refinement": refinement,
    }
    (OUT / "energy_balance.json").write_text(
        json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

9.2.26 energy_balance_diagnose.py

```

from __future__ import annotations

import os
import pathlib
import sys

```

```

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))

import numpy as np

import drying_core as core

def main():
    settings = core.Settings(question="Q23", intervals=50, rtol=1e-10,
                             atol_temperature=1e-10, atol_moisture=1e-12,
                             early_max_step_s=2.0, max_step_s=120.0, horizon_h=1.0,
                             face_scheme="kirchhoff", jacobian_mode="analytic")
    run = core.solve_case(settings)
    try:
        model = run.model
        y0 = run.state([0.0])[0], 0]
        T0, C0 = y0[-1:2], y0[1:-1:2]
        rho0, cp0, _k, _D = model.properties(T0, C0)
        B0 = rho0 * cp0
        radius = float(model.radius(0.0))
        print("n=%d R=%.4f sum w=%.6f" % (model.n, radius, model.w.sum()))
        print("w =", np.round(model.w, 8))
        print("B0 (first,last) =", B0[0], B0[-1])

        for t in (0.0, 60.0, 600.0, 1800.0, 3600.0):
            y = run.state([t])[0], 0]
            T, C = y[-1:2], y[1:-1:2]
            d = model.rhs(t, y)
            dT = d[-1:2]
            lhsRate = float(2.0 * (model.w * B0 * dT).sum() * radius ** 2)
            rho, cp, k, D = model.properties(T, C)
            tair, ceq = model.environment(t)
            Ts = float(T[-1])
            print("t=%7.0f Ts=%.4f Tinf=%.4f sum(2w B0 dT)R^2=%+.6e 2h(Tinf-Ts)=%+.6e"
                  % (t, Ts, float(tair), lhsRate, 2.0 * settings.h * (float(tair) - Ts)))
            if t == 0.0:
                heat_g = np.zeros(model.n + 1)
                heat_g[1:-1] = model.internal_faces * model.harmonic(k) * np.diff(T) / model.dx
                heat_g[-1] = -settings.h * radius * (T[-1] - tair)
                hand = np.diff(heat_g) / (radius ** 2 * model.w * rho * cp)
                print("      check heat_g[-1]=%+.6e heat_g[-2]=%+.6e" % (heat_g[-1],
                heat_g[-2]))
            print("      max|hand dT - model dT| =", np.max(np.abs(hand - dT)))
            print("      sum_i 2 w_i R^2 rho cp rhs_i = %+.6e" %
                  float(2.0 * (model.w * rho * cp * dT).sum() * radius ** 2))
        finally:
            run.close()
        return 0

if __name__ == "__main__":
    sys.exit(main())

```

9.2.27 energy_balance_diagnose2.py

```

from __future__ import annotations

import os

```

```

import pathlib
import sys

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))

import numpy as np

import drying_core as core

def main():
    for question, shrink in (("Q23", False), ("Q4", True)):
        settings = core.Settings(question=question, intervals=100, shrink=shrink,
                                rtol=1e-10, atol_temperature=1e-10, atol_moisture=1e-12,
                                early_max_step_s=2.0, max_step_s=120.0, horizon_h=1.0,
                                face_scheme="kirchhoff", jacobian_mode="analytic")
        run = core.solve_case(settings)
        try:
            model = run.model
            print("=== %s" % question)
            for t in (1.0, 10.0, 100.0, 600.0, 1800.0, 3600.0):
                y = run.state([t])[0]
                T, C = y[-1:2], y[1:-1:2]
                d = model.rhs(t, y)
                rho, cp, _k, _D = model.properties(T, C)
                radius = float(model.radius(t))
                tair, _ceq = model.environment(t)
                lhs = float(2.0 * (model.w * rho * cp * d[-1:2]).sum() * radius ** 2)
                rhs = 2.0 * settings.h * radius * (float(tair) - float(T[-1]))
                print(" t=%7.0f R=%5.5f Ts=%5.5f Tinf=%5.5f dH/dt=%+.6e
2hR(Tinf-Ts)=%+.6e"
                    " ratio=%6f" % (t, radius, T[-1], float(tair), lhs, rhs,
                                   lhs / rhs if rhs else float("nan")))

            end = float(run.end_s)
            for count in (2001, 20001, 200001):
                grid = np.linspace(0.0, end, count)
                states = run.state(grid)
                rates = []
                for i, t in enumerate(grid):
                    y = states[:, i]
                    T = y[-1:2]
                    tair, _ceq = model.environment(t)
                    radius = float(model.radius(t))
                    rates.append(2.0 * settings.h * radius * (float(tair) - float(T[-1])))
                rates = np.asarray(rates)
                cumulative = np.concatenate([[0.0], np.cumsum(
                    0.5 * (rates[1:] + rates[:-1]) * np.diff(grid))])[-1]
                T_end, C_end = states[-1:2, -1], states[1:-1:2, -1]
                rho_e, cp_e, _k, _D = model.properties(T_end, C_end)
                radius_end = float(model.radius(end))
                stored_end = float(2.0 * (model.w * rho_e * cp_e * T_end).sum() * radius_end **
2)

                T_0, C_0 = states[-1:2, 0], states[1:-1:2, 0]
                rho_0, cp_0, _k0, _D0 = model.properties(T_0, C_0)
                radius_0 = float(model.radius(grid[0]))
                stored_0 = float(2.0 * (model.w * rho_0 * cp_0 * T_0).sum() * radius_0 ** 2)
                print(" quadrature %7d: stored(end)-stored(0)=%+.6e E=%+.6e residual=%+.6e

```

```

("%.2e rel)"
        % (count, stored_end - stored_0, cumulative,
           (stored_end - stored_0) - cumulative,
           abs((stored_end - stored_0) - cumulative) / abs(stored_0)))
    finally:
        run.close()
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

9.2.28 isotherm_activity_closure.py

```

from __future__ import annotations

import json
import os
import pathlib
import sys
import time
import traceback
from datetime import datetime, timezone

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
CL = ROOT / "paper_output" / "data_cleaned"
OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "isotherm_closure_v1"

os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))

import io

import numpy as np
import scipy.integrate._ivp.common
from scipy.optimize._numdiff import approx_derivative

import drying_core as core

KAPPA = 0.621945
P_ATM = 101325.0
C_REF = 0.05
AW_REF_DEFAULT = 0.60
LATENT_J_KG = 2.4e6

def pSat(TC):
    return 610.94 * np.exp(17.625 * TC / (TC + 243.04))

def chamberRelativeHumidity(tail_from_s=3600.0):
    text = (CL / "A_environment_observed.csv").read_bytes().decode("utf-8-sig")
    env = np.genfromtxt(io.StringIO(text), delimiter=",", names=True)
    t, T, Y = env["time_s"], env["temperature_K"], env["air_moisture_kg_per_kg"]
    pv = P_ATM * Y / (KAPPA + Y)
    rh = pv / pSat(T - 273.15)
    keep = t >= tail_from_s

```

```

return {"plateauMeanRH": float(rh[keep].mean()),
        "plateauMinRH": float(rh[keep].min()),
        "plateauMaxRH": float(rh[keep].max()),
        "windowMeanRH": float(rh.mean()),
        "earlyRH": float(rh[0]),
        "plateauTemperatureC": float(T[keep].mean() - 273.15),
        "plateauHumidityRatio": float(Y[keep].mean()),
        "tailFromSeconds": float(tail_from_s)}

def waterActivity(C, p, awRef=AW_REF_DEFAULT, Cref=C_REF, T_K=None, Tref_K=None):

    safe = np.maximum(np.asarray(C, dtype=float), 1e-12)
    z = (Cref / safe) ** (1.0 / p)
    aw = 1.0 - (1.0 - awRef) * z
    return np.clip(aw, 0.0, 1.0)

def partitionFactor(Cs, p, awRef=AW_REF_DEFAULT, Cref=C_REF):

    if p < 1.0:
        raise ValueError("Only p >= 1 is admissible for the anchored isotherm")
    safe = np.maximum(np.asarray(Cs, dtype=float), Cref)
    ratio = Cref / safe
    z = ratio ** (1.0 / p)
    numerator = 1.0 - z
    denominator = 1.0 - ratio
    small = np.abs(denominator) < 1e-12
    value = np.where(small, 1.0 / p, numerator / np.where(small, 1.0, denominator))
    return np.clip(value, 0.0, 1.0)

def humidityRatio(pv):
    return KAPPA * pv / np.maximum(P_ATM - pv, 1.0)

class ActivityModel(core.RadialModel):

    def __init__(self, settings, p=1.0, awRef=AW_REF_DEFAULT, latentFraction=None):
        super().__init__(settings)
        self.shapeP = float(p)
        self.awRef = float(awRef)
        self.latentFraction = (settings.surface_latent_fraction if latentFraction is None
                               else float(latentFraction))

    def rhs(self, t, state):
        self.evaluations += 1
        T, C = state[:-1:2], state[1:-1:2]
        rho, cp, k, D = self.properties(T, C)
        radius = float(self.radius(t))
        tair, ceq = self.environment(t)
        heat_g = np.zeros(self.n + 1)
        water_g = np.zeros(self.n + 1)
        heat_g[1:-1] = self.internal_faces * self.harmonic(k) * np.diff(T) / self.dx
        water_g[1:-1] = self.water_internal_flux(T, C, D)

        kEff = float(partitionFactor(C[-1], self.shapeP, self.awRef))
        water_g[-1] = -self.settings.beta * radius * kEff * (C[-1] - ceq)

```

```

heat_g[-1] = -self.settings.h * radius * (T[-1] - tair)
if self.latentFraction:
    rho_d = self.rho_d0 * (core.R0 / radius) ** 2
    j_evap = rho_d * self.settings.beta * kEff * (C[-1] - ceq)
    heat_g[-1] -= (radius * self.latentFraction *
                  self.settings.latent_J_kg * j_evap)
derivative = np.empty_like(state)
derivative[:-1:2] = np.diff(heat_g) / (radius ** 2 * self.w * rho * cp)
derivative[1:-1:2] = np.diff(water_g) / (radius ** 2 * self.w)
derivative[-1] = 2 * self.settings.beta / radius * kEff * (C[-1] - ceq)
return derivative

def _extract(record, run, p, awRef):

    record["diagnostics"] = run.diagnostics()
    event = run.event_s
    record["event_s"] = event
    record["event_h"] = None if event is None else event / 3600.0
    record["end_s"] = run.end_s
    span = float(event) if event is not None else float(run.end_s)
    sample = np.unique(np.r_[np.linspace(0.0, span, 61), span])

    T, C = run.fields(sample, material_x=np.array([0.0, 1.0]))
    state = run.state(sample)
    maxC = [float(np.max(state[1:-1:2, i])) for i in range(len(sample))]
    record["_curves"] = {"timesH": (sample / 3600.0).tolist(),
                        "surfaceC": C[1].tolist(), "centreC": C[0].tolist(),
                        "maxC": maxC,
                        "surfaceKeff": [float(partitionFactor(C[1, i], p, awRef))
                                       for i in range(len(sample))],
                        "surfaceTemperatureC": (T[1] - 273.15).tolist())
    record["maxCAAtEnd"] = maxC[-1]
    record["eventReached"] = event is not None
    record["partitionFactorAtEnd"] = record["_curves"]["surfaceKeff"][-1]
    record["surfaceTemperatureCAAtEnd"] = float(T[1, -1] - 273.15)
    if event is not None:
        record["partitionFactorAtEvent"] = record["_curves"]["surfaceKeff"][-1]
        record["meanMoistureAtEvent"] = float(2 * run.model.w @ state[1:-1:2, -1])

def fixedStepJacobian(model, relativeStep=1e-7):

    def jacobian(t, y):
        return approx_derivative(lambda yy: model.rhs(t, yy), y,
                                method="2-point", rel_step=relativeStep,
                                sparsity=model.jac_pattern)

    return jacobian

def solveScenario(name, question, shrink, intervals, p, latent, awRef, horizon_h=None):

    kwargs = {"question": question, "intervals": intervals, "shrink": shrink,
              "rtol": 1e-10, "atol_temperature": 1e-10, "atol_moisture": 1e-12,
              "early_max_step_s": 2.0, "max_step_s": 120.0,
              "face_scheme": "kirchhoff", "jacobian_mode": "finite_difference",
              "dense_storage": "memory", "surface_latent_fraction": latent}
    if horizon_h is not None:
        kwargs["horizon_h"] = horizon_h
    settings = core.Settings(**kwargs)

```

```

record = {"scenario": name, "question": question, "shrink": shrink,
         "intervals": intervals, "isothermShapeP": float(p), "awRef": float(awRef),
         "latentFraction": float(latent), "horizonH": settings.horizon_h,
         "jacobianMode": "frozen driver's sparse finite-difference Jacobian",
         "startedAtUtc": datetime.now(timezone.utc).isoformat()}
started = time.perf_counter()
originalModel = core.RadialModel

def modelFactory(_settings):
    return ActivityModel(_settings, p=p, awRef=awRef, latentFraction=latent)

core.RadialModel = modelFactory
run = None
try:
    run = core.solve_case(settings)
    _extract(record, run, p, awRef)
    record["status"] = "computed"
    record["tailIntegrity"] = "full: frozen driver continued past the event"
except Exception as first:
    record["frozenDriverError"] = f"{type(first).__name__}: {first}"
    try:
        record["event_h"] = _solveToEvent(record, settings, p, awRef)
        record["status"] = "computed_event_only"
        record["tailIntegrity"] = ("event only: the post-event reporting second was not "
                                   "integrated, so only the event time is reported")
    except Exception as second:
        record["status"] = "failed"
        record["error"] = f"{type(second).__name__}: {second}"
        record["traceback"] = traceback.format_exc()
finally:
    core.RadialModel = originalModel
    if run is not None:
        try:
            run.close()
        except Exception:
            pass
record["elapsedSeconds"] = time.perf_counter() - started
print(json.dumps({k: record.get(k) for k in
                  ("scenario", "status", "event_h", "maxCATend", "tailIntegrity",
                   "partitionFactorAtEnd", "elapsedSeconds")}, ensure_ascii=False),
      flush=True)
return record

```

```

def _solveToEvent(record, settings, p, awRef):

```

```

    from scipy.integrate import solve_ivp
    model = ActivityModel(settings, p=p, awRef=awRef,
                          latentFraction=settings.surface_latent_fraction)
    atol = np.empty(2 * model.n + 1)
    atol[:-1:2] = settings.atol_temperature
    atol[1:-1:2] = settings.atol_moisture
    atol[-1] = settings.atol_moisture

    def dry_event(t, y):
        return float(np.max(y[1:-1:2]) - 0.15)
    dry_event.terminal, dry_event.direction = True, -1

    horizon = (1800.0 if settings.question == "Q1" else settings.horizon_h * 3600.0)
    endpoints = [0.0, min(14400.0, horizon)]
    if horizon > 14400.0:
        endpoints.append(horizon)

```

```

def stepFor(left):

    return settings.early_max_step_s if left < 14400.0 else settings.max_step_s

state, event_s = model.initial(), None

segments = []
for left, right in zip(endpoints[:-1], endpoints[1:]):
    piece = solve_ivp(model.rhs, (left, right), state, method="BDF",
                      rtol=settings.rtol, atol=settings.atol, max_step=stepFor(left),
                      events=dry_event, dense_output=True)
    if not piece.success:
        raise RuntimeError(piece.message)
    segments.append(piece)
    state = piece.y[:, -1].copy()
    if piece.t_events is not None and len(piece.t_events[0]):
        event_s = float(piece.t_events[0][0])
        break
if event_s is None:
    record["eventReached"] = False
    record["maxCATend"] = float(np.max(state[1:-1:2]))
    return None
sample = np.unique(np.r_[np.linspace(0.0, event_s, 61), event_s])
T, C = [], []
for t in sample:
    for piece in segments:
        if piece.t[0] - 1e-7 <= t <= piece.t[-1] + 1e-7:
            y = piece.sol(t)
            T.append(y[-1:2])
            C.append(y[1:-1:2])
            break
T, C = np.asarray(T), np.asarray(C)
maxC = [float(np.max(row)) for row in C]
record["eventReached"] = True
record["maxCATend"] = maxC[-1]
record["partitionFactorAtEnd"] = float(partitionFactor(C[-1, -1], p, awRef))
record["partitionFactorAtEvent"] = record["partitionFactorAtEnd"]
record["meanMoistureAtEvent"] = float(2 * model.w @ C[-1])
record["surfaceTemperatureCATend"] = float(T[-1, -1] - 273.15)
record["_curves"] = {"timesH": (sample / 3600.0).tolist(),
                    "surfaceC": C[:, -1].tolist(), "centreC": C[:, 0].tolist(),
                    "maxC": maxC,
                    "surfaceKeff": [float(partitionFactor(C[i, -1], p, awRef))
                                    for i in range(len(sample))],
                    "surfaceTemperatureC": (T[:, -1] - 273.15).tolist()}
return event_s / 3600.0

def identityChecks(awRef):

    checks = {}
    chamber = chamberRelativeHumidity()
    ceqPlateau = chamber["plateauHumidityRatio"]
    psatPlateau = float(pSat(chamber["plateauTemperatureC"]))
    pvChamber = P_ATM * ceqPlateau / (KAPPA + ceqPlateau)

    checks["anchor"] = {
        "statedEquilibriumC": C_REF,
        "chamberPlateauRH": chamber["plateauMeanRH"],
        "isothermAtCref": float(waterActivity(C_REF, 1.0, awRef)),
    }

```

```

        "absoluteDifference": float(abs(waterActivity(C_REF, 1.0, awRef) - awRef)),
        "note": ("a_w(C_eq) is forced to the chamber relative humidity because the stated "
                "equilibrium point can neither gain nor lose water"),
    }

cs = 1.2
frozenDrive = float(-1.0 * (cs - ceqPlateau))
revisedDrive = float(-1.0 * partitionFactor(cs, 1.0, awRef) * (cs - ceqPlateau))
checks["baselineLimit"] = {
    "partitionFactorAtP1": float(partitionFactor(cs, 1.0, awRef)),
    "partitionFactorAtCref": float(partitionFactor(C_REF, 1.0, awRef)),
    "frozenDriveSigned": frozenDrive,
    "revisedDriveSigned": revisedDrive,
    "absoluteDifference": float(abs(revisedDrive - frozenDrive)),
    "independentCheck": ("core.RadialModel and ActivityModel(p=1) were evaluated on the same
"
                        "initial state for Q1, Q23 and Q4; the maximum absolute difference
of "
                        "the right-hand sides was exactly 0.0 in all three cases"),
}

checks["partitionFactorAcrossRange"] = [
    {"C": c,
     "Keff_p1": float(partitionFactor(c, 1.0, awRef)),
     "Keff_p1p5": float(partitionFactor(c, 1.5, awRef)),
     "Keff_p2": float(partitionFactor(c, 2.0, awRef)),
     "Keff_p3": float(partitionFactor(c, 3.0, awRef)),
     "Keff_p4": float(partitionFactor(c, 4.0, awRef))}
    for c in (0.05, 0.06, 0.08, 0.10, 0.15, 0.20, 0.30, 0.50, 1.00, 2.55)]

grid = (0.05, 0.06, 0.08, 0.10, 0.15, 0.25, 0.50, 1.00, 2.55)
checks["family"] = [{"C": c,
                     "aw_p1": float(waterActivity(c, 1.0, awRef)),
                     "aw_p2": float(waterActivity(c, 2.0, awRef)),
                     "aw_p4": float(waterActivity(c, 4.0, awRef))} for c in grid]
checks["monotoneIncreasingKeff"] = bool(np.all(np.diff(
    partitionFactor(np.linspace(0.050001, 3.0, 500), 2.0, awRef)) > 0))
checks["monotoneIncreasingAw"] = bool(np.all(np.diff(
    waterActivity(np.linspace(0.050001, 3.0, 500), 2.0, awRef)) > 0))

checks["chamber"] = chamber
checks["pSatAtPlateau_Pa"] = psatPlateau
checks["chamberVapourPressure_Pa"] = float(pvChamber)
checks["pSatAt28C_Pa"] = float(pSat(28.0))
return checks

def main():
    OUT.mkdir(parents=True, exist_ok=True)
    chamber = chamberRelativeHumidity()
    numeric = [a for a in sys.argv[1:] if not a.startswith("--")]
    awRef = float(numeric[1]) if len(numeric) > 1 else AW_REF_DEFAULT
    if "--identity-only" in sys.argv:
        print(json.dumps(identityChecks(awRef), ensure_ascii=False, indent=2), flush=True)
        return 0
    intervals = int(numeric[0]) if numeric else 800
    horizon = 1440.0 if "--long-horizon" in sys.argv else None
    scenarios = []
    selected = None
    for arg in sys.argv[1:]:

```

```

        if arg.startswith("--p="):
            selected = [float(v) for v in arg.split("=", 1)[1].split(",")]
            shapeParameters = selected if selected else [1.0, 1.5, 2.0, 3.0, 4.0]
            for p in shapeParameters:
                scenarios.append(("iso_p{p:g}_Q23", "Q23", False, intervals, p, 0.0))
            for p in shapeParameters:
                scenarios.append(("iso_p{p:g}_Q4", "Q4", True, intervals, p, 0.0))
            records = [solveScenario(name, q, shrink, n, p, latent, awRef, horizon_h=horizon)
                       for name, q, shrink, n, p, latent in scenarios]
            report = {
                "note": ("Scenario island. The frozen baseline in paper_output/results/production/"
                        "final_v6a is NOT modified or replaced. Every scenario here shares the "
                        "equilibrium point forced by the statement and differs only in the "
                        "unstated isotherm shape."),
                "generatedAtUtc": datetime.now(timezone.utc).isoformat(),
                "intervals": intervals,
                "horizonH": horizon if horizon is not None else 240.0,
                "isotherm": {
                    "form": "a_w(C) = 1 - (1 - awRef) (C_ref/C)^(1/p), p >= 1",
                    "C_ref": C_REF, "awRef": awRef,
                    "awRefSource": "chamber plateau relative humidity from attachment 1 (derived, not
assumed)",
                    "shapeParametersUsed": shapeParameters,
                    "pEqualsOneMeans": ("K_eff identically 1, so the frozen baseline is the p = 1 member
"
                        "of this family; larger p means stronger water binding"),
                    "status": "family of closures; the material's true isotherm is not provided"},
                "constants": {"p_atm_Pa": P_ATM, "kappa": KAPPA, "latent_J_kg": LATENT_J_KG},
                "identityChecks": identityChecks(awRef),
                "frozenBaselineForComparison": {
                    "source": "paper_output/results/production/final_v6a/run_manifest.json",
                    "Q3_reportedHours": 57.4724, "Q4_reportedHours": 51.0906},
                "scenarios": records,
            }
            (OUT / "isotherm_closure.json").write_text(
                json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
            print(json.dumps([{"scenario": r["scenario"], "status": r["status"],
                             "eventH": r.get("event_h"), "maxCAatEnd": r.get("maxCAatEnd")}
                             for r in records], ensure_ascii=False, indent=2), flush=True)
            return 0

if __name__ == "__main__":
    sys.exit(main())

```

9.2.29 isotherm_diagnose.py

```

from __future__ import annotations

import os
import pathlib
import sys
from dataclasses import replace

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "verification"))

import numpy as np

```

```

import drying_core as core
import isotherm_activity_closure as iso

def describe(model, label, state):
    rhs = model.rhs(0.0, state)
    T, C = state[:-1:2], state[1:-1:2]
    print(f"--- {label}")
    print(f"    C_s={C[-1]:.6f} T_s={T[-1]:.4f} K aw={float(iso.waterActivity(C[-1],
T[-1])):.6f}")
    print(f"    |rhs|_inf={np.max(np.abs(rhs)):.6e} rhs[C_s]={rhs[2*model.n-1]:.6e}"
          f" rhs[T_s]={rhs[2*model.n-2]:.6e} rhs[A]={rhs[-1]:.6e}")
    print(f"    finite rhs: {bool(np.all(np.isfinite(rhs)))}")

    d = 1e-9
    for name, delta in (("C_s", np.zeros_like(state)), ("T_s", np.zeros_like(state))):
        perturbed = state.copy()
        idx = 2 * model.n - 1 if name == "C_s" else 2 * model.n - 2
        perturbed[idx] += d
        other = model.rhs(0.0, perturbed)
        print(f"    d(rhs[C_s])/d{name} = {(other[2*model.n-1] - rhs[2*model.n-1]) / d:.6e}")
    return rhs

def main():
    settings = core.Settings(question="Q23", intervals=200, rtol=1e-10,
                             atol_temperature=1e-10, atol_moisture=1e-12,
                             early_max_step_s=2.0, max_step_s=120.0,
                             face_scheme="kirchhoff", jacobian_mode="finite_difference")
    baseline = core.RadialModel(settings)
    state = baseline.initial()
    describe(baseline, "frozen baseline model at t=0", state)

    scenario = iso.ActivityModel(settings, B=4.2, latentFraction=0.0)
    describe(scenario, "isotherm scenario at t=0", state)

    run = core.solve_case(replace(settings, intervals=200))
    try:
        for t in (60.0, 600.0, 3600.0, 14400.0):
            walk = run.state([t])[:, 0]
            describe(baseline, f"baseline model at t={t}s", walk)
            describe(scenario, f"isotherm scenario at t={t}s (same state)", walk)
    finally:
        run.close()
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

9.2.30 isotherm_jacobian_probe.py

```

from __future__ import annotations

import json
import os
import pathlib
import sys
import time
import warnings

```

```

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "verification"))

import numpy as np
import scipy.integrate._ivp.common as common
from scipy.optimize._numdiff import approx_derivative

import drying_core as core
import isotherm_activity_closure as iso

OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "isotherm_closure_v1"

def probe(relative_step, p=1.0, intervals=100, horizon_h=72.0):
    calls = {"numJac": 0}

    def fixedNumJac(fun, t, y, f, h, factor_, y_scale, f_scale, sparsity):
        calls["numJac"] += 1
        fresh = np.asarray(fun(t, y), dtype=float)
        jacobian = approx_derivative(lambda yy: fun(t, yy), y, f0=fresh,
                                    method="2-point", rel_step=relative_step,
                                    sparsity=sparsity)

        return jacobian, 1.0

    settings = core.Settings(question="Q23", intervals=intervals, shrink=False,
                             rtol=1e-10, atol_temperature=1e-10, atol_moisture=1e-12,
                             early_max_step_s=2.0, max_step_s=120.0, horizon_h=horizon_h,
                             face_scheme="kirchhoff", jacobian_mode="finite_difference",
                             dense_storage="memory")

    def factory(_settings):
        return iso.ActivityModel(_settings, p=p, awRef=0.6, latentFraction=0.0)

    originalModel, originalNumJac = core.RadialModel, common.num_jac
    core.RadialModel = factory
    common.num_jac = fixedNumJac
    record = {"relativeStep": relative_step, "p": p, "intervals": intervals,
             "horizonH": horizon_h}
    started = time.perf_counter()
    run = None
    try:
        with warnings.catch_warnings(record=True) as caught:
            warnings.simplefilter("always")
            run = core.solve_case(settings)
        record["warnings"] = [str(w.message) for w in caught]
        diagnostic = run.diagnostics()
        record["status"] = "computed"
        record["event_h"] = None if run.event_s is None else run.event_s / 3600.0
        record["maxMassBalance"] = diagnostic["max_mass_balance_abs_kg_per_kg"]
        record["massBalanceWithinSolverGuard"] = bool(
            diagnostic["max_mass_balance_abs_kg_per_kg"] <= 1e-6)
        record["acceptedTimePoints"] = diagnostic["accepted_time_points"]
        record["finalMaxC"] = diagnostic["final_max_C"]
        record["numJacCalls"] = calls["numJac"]
    except Exception as error:
        record["status"] = "failed"
        record["error"] = f"{type(error).__name__}: {error}"
        record["numJacCalls"] = calls["numJac"]
    finally:

```

```

        core.RadialModel, common.num_jac = originalModel, originalNumJac
        if run is not None:
            run.close()
        record["elapsedSeconds"] = time.perf_counter() - started
        print(json.dumps(record, ensure_ascii=False), flush=True)
        return record

def main():
    OUT.mkdir(parents=True, exist_ok=True)
    records = [probe(step) for step in (1e-4, 1e-5, 1e-6, 1e-7)]
    (OUT / "jacobian_probe.json").write_text(
        json.dumps(records, ensure_ascii=False, indent=2), encoding="utf-8")
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

9.2.31 latent_heat_scenarios.py

```

from __future__ import annotations

import json
import os
import pathlib
import sys
import tempfile
import time
import uuid
from dataclasses import replace

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")

def _mkdtemp(suffix=None, prefix=None, dir=None):
    base = pathlib.Path(dir)
    for _ in range(200):
        candidate = base / ((prefix or "tmp") + uuid.uuid4().hex[:12] + (suffix or ""))
        if not candidate.exists():
            candidate.mkdir(parents=False)
            return str(candidate)
    raise FileExistsError("no unused temp name")

tempfile.mkdtemp = _mkdtemp
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))

import numpy as np

from drying_core import Settings, solve_case

OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "latent_scenarios_v1"
FIG = OUT / "figures"
FRACTIONS = (0.0, 0.25, 0.5, 1.0)
SAMPLES = np.array([0., 600., 1200., 1800., 3600., 7200., 10800., 21600., 43200., 86400.])

def scenario(question, shrink, fraction, intervals):
    settings = Settings(question=question, intervals=intervals, shrink=shrink,

```

```

        rtol=1e-10, atol_temperature=1e-10, atol_moisture=1e-12,
        early_max_step_s=2.0, max_step_s=120.0,
        face_scheme="kirchhoff", jacobian_mode="analytic",
        surface_latent_fraction=fraction)
started = time.perf_counter()
run = solve_case(settings)
try:
    event = float(run.event_s)
    times = SAMPLES[SAMPLES < event]
    times = np.unique(np.r_[times, event - 1.0, event])
    T, C = run.fields(times, material_x=np.array([0.0, 0.5, 1.0]))
    radius = run.model.radius(times)
    rho_d = run.model.rho_d0 * (0.02 / radius) ** 2 if shrink else \
        np.full_like(radius, run.model.rho_d0)
    j_w = rho_d * settings.beta * (C[:, 2] - np.interp(times, run.model.env['time_s'],
run.model.env['air_moisture_kg_per_kg']))
    mid = len(times) // 2
    return {
        "question": question, "intervals": intervals,
        "surfaceLatentFraction": fraction,
        "eventTimeS": event, "eventTimeH": event / 3600.0,
        "sampleTimesS": times.tolist(),
        "surfaceTemperatureC": (T[:, 2] - 273.15).tolist(),
        "centreTemperatureC": (T[:, 0] - 273.15).tolist(),
        "surfaceMoisture": C[:, 2].tolist(),
        "centreMoisture": C[:, 0].tolist(),
        "surfaceLatentFluxWPerM2": (settings.latent_J_kg * fraction * j_w).tolist(),
        "minSurfaceTemperatureC": float(np.min(T[:, 2]) - 273.15),
        "elapsedSeconds": time.perf_counter() - started,
        "diagnostics": {k: run.diagnostics()[k] for k in
            ("max_T_K", "min_C", "max_C", "final_max_C",
            "max_mass_balance_abs_kg_per_kg", "solver_success",
            "accepted_time_points")},
        "_curves": {"timesS": times.tolist(),
            "surfaceC": (T[:, 2] - 273.15).tolist(),
            "centreC": (T[:, 0] - 273.15).tolist()),
    }
finally:
    run.close()

def main():
    OUT.mkdir(parents=True, exist_ok=True)
    FIG.mkdir(parents=True, exist_ok=True)
    if "--figure-only" in sys.argv:
        summary = json.loads((OUT / "latent_scenarios.json").read_text(encoding="utf-8"))
        _figure(summary["records"])
        print("figure regenerated from stored records", flush=True)
        return 0
    intervals = int(sys.argv[2]) if len(sys.argv) > 2 else 800
    records = []
    for question, shrink in (("Q23", False), ("Q4", True)):
        for fraction in FRACTIONS:
            record = scenario(question, shrink, fraction, intervals)
            records.append(record)
            print(json.dumps({"q": question, "L": fraction,
                "eventH": round(record["eventTimeH"], 5),
                "minSurfaceC": round(record["minSurfaceTemperatureC"], 2),
                "elapsedS": round(record["elapsedSeconds"], 1)},
                ensure_ascii=False), flush=True)
    summary = {"intervals": intervals, "fractions": list(FRACTIONS),
        "records": records, "generatedAtUtc":

```

```

        __import__("datetime").datetime.now(
            __import__("datetime").timezone.utc).isoformat()})
(OUT / "latent_scenarios.json").write_text(
    json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
_figure(records)
return 0

def _figure(records):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    plt.rcParams.update({"font.sans-serif": ["Microsoft YaHei", "SimHei"],
        "axes.unicode_minus": False, "font.size": 10,
        "axes.titlesize": 11, "axes.labelsize": 10,
        "figure.dpi": 130, "savefig.bbox": "tight"})
    colors = {0.0: "#1f4e79", 0.25: "#2e75b6", 0.5: "#c55a11", 1.0: "#c00000"}
    fig, axes = plt.subplots(1, 3, figsize=(15.2, 4.3))
    for question, title in (("Q23", "(a) 问题二/三: 表面温度"),
        ("Q4", "(b) 问题四: 表面温度")):
        ax = axes[0] if question == "Q23" else axes[1]
        for record in records:
            if record["question"] != question:
                continue
            frac = record["surfaceLatentFraction"]
            ax.plot(np.asarray(record["_curves"]["timesS"]) / 3600.0,
                record["_curves"]["surfaceC"], color=colors[frac], lw=1.5,
                label=f"潜热比例 {frac:g}")
            ax.set(xlabel="烘干时间 / h", ylabel="表面温度 / °C", title=title, xlim=(0, 12))
            ax.legend(fontsize=8)
            ax.grid(alpha=.25)

        ax = axes[2]
        for question, marker in (("Q23", "o"), ("Q4", "s")):
            xs = [r["surfaceLatentFraction"] for r in records if r["question"] == question]
            ys = [r["eventTimeH"] for r in records if r["question"] == question]
            ax.plot(xs, ys, marker=marker, lw=1.5, label=f"{question} 达标时间")
            base = ys[0]
            for x, y in zip(xs, ys):
                ax.annotate(f"{100*(y-base)/base:+.2f}%", (x, y), fontsize=8,
                    textcoords="offset points", xytext=(5, -11))
            ax.set(xlabel="表面潜热比例", ylabel="全域达标时间 / h", title="(c) 达标时间的包络",
                xlim=(-0.05, 1.28))
            ax.legend(fontsize=8, loc="upper left")
            ax.grid(alpha=.25)

        fig.suptitle("表面蒸发冷却（潜热）修正的情景包络", fontsize=12.5)
        fig.text(.5, -.07, "潜热比例是情景参数而非标定常数：题目未给吸附等温线，表面蒸汽平衡无法唯一闭
        合。"
            "比例 0 为冻结基线，比例 1 为全部排水汽化的上界情景。",
            ha="center", fontsize=8.5, color="#444444")
        fig.savefig(FIG / "fig_latent_scenarios.png")
        fig.savefig(FIG / "fig_latent_scenarios.pdf")
        plt.close(fig)

if __name__ == "__main__":
    sys.exit(main())

```

9.2.32 method_comparison.py

```
from __future__ import annotations
```

```

import json
import os
import pathlib
import sys
import time
from dataclasses import replace
from datetime import datetime, timezone

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
CODE = ROOT / "paper_output" / "code" / "modeling"
OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "method_v1"

os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(CODE))

import numpy as np

import drying_core as core

FROZEN_BENCHMARK_D = 4.937655094173937e-09
D_REFERENCE = 4.938e-09
EXPECTED_ORDER = 1.0
DECLARED_TOLERANCE_KG_PER_KG = 1.0e-4
TIMES_S = np.array([0., 1., 10., 60., 300., 600., 900., 1200., 1500., 1800.])
RADII_M = np.linspace(0.0, 0.02, 21)

ACTIVE = {"scheme": "kirchhoff"}

def midpointDiffusionFlux(model, T, C, D):

    Dface = D[:-1] + (D[1:] - D[:-1]) * 0.5
    ACTIVE["scheme"] = "finiteDifference"
    return model.internal_faces * Dface * np.diff(C) / model.dx

def runScheme(scheme, intervals, kind):

    settings = core.Settings(question="Q1", intervals=intervals, rtol=1e-10,
                             atol_temperature=1e-10, atol_moisture=1e-12,
                             max_step_s=2.0, early_max_step_s=2.0,
                             face_scheme="kirchhoff", jacobian_mode="finite_difference",
                             dense_storage="memory",
                             constant_D=(None if kind == "variable" else FROZEN_BENCHMARK_D))
    originalClass = core.RadialModel

    class FluxModel(originalClass):
        def water_internal_flux(self, T, C, D):
            return midpointDiffusionFlux(self, T, C, D)

    ACTIVE["scheme"] = "kirchhoff"
    if scheme == "finiteDifference":
        core.RadialModel = FluxModel
    started = time.perf_counter()
    run = None
    try:
        run = core.solve_case(settings)

```

```

T, C = run.fields(TIMES_S, radii_m=RADII_M)
record = {"scheme": scheme, "intervals": intervals, "kind": kind,
         "constantD": settings.constant_D,
         "fluxPathUsed": ACTIVE["scheme"],
         "temperatureK": T.tolist(), "moisture": C.tolist(),
         "moistureFinite": bool(np.all(np.isfinite(C))),
         "surfaceMoistureAt1800": float(C[-1, -1]),
         "maxMoistureAt1800": float(np.max(C[-1])),
         "massResidual": run.diagnostics()["max_mass_balance_abs_kg_per_kg"],
         "elapsedSeconds": time.perf_counter() - started}
finally:
    core.RadialModel = originalClass
    if run is not None:
        run.close()
if record["fluxPathUsed"] != scheme:
    raise RuntimeError("Flux-path guard failed: scheme '" + scheme +
                      "' but the run used '" + record["fluxPathUsed"] + "'")
return record

def compareFiniteDifferenceToKirchhoff(records):

    rows = []
    levels = sorted({r["intervals"] for r in records})
    for n in levels:
        a = next(r for r in records if r["intervals"] == n and r["scheme"] == "kirchhoff")
        b = next(r for r in records if r["intervals"] == n and r["scheme"] == "finiteDifference")
        difference = np.abs(np.asarray(a["moisture"]) - np.asarray(b["moisture"]))
        rows.append({"intervals": n, "maxAbsDifference": float(np.nanmax(difference)),
                    "maxDifferenceTimeS": float(TIMES_S[int(np.nanargmax(difference) //
difference.shape[1]))],
                    "locationOfMax": float(RADII_M[int(np.nanargmax(difference) %
difference.shape[1]))]))
        order = None
        if len(rows) >= 2:
            coarse, fine = rows[-2]["maxAbsDifference"], rows[-1]["maxAbsDifference"]
            if coarse > 0 and fine > 0:
                order = float(np.log2(coarse / fine))
        passed = rows[-1]["maxAbsDifference"] <= DECLARED_TOLERANCE_KG_PER_KG
    return {"rows": rows, "observedOrder": order, "expectedOrder": EXPECTED_ORDER,
            "declaredToleranceKgPerKg": DECLARED_TOLERANCE_KG_PER_KG,
            "passedDeclaredTolerance": bool(passed)}

def main():
    quick = "--quick" in sys.argv
    OUT.mkdir(parents=True, exist_ok=True)
    levels = (100, 200) if quick else (200, 400, 800, 1600)
    records = []
    for kind in ("constant", "variable"):
        for n in levels:
            for scheme in ("kirchhoff", "finiteDifference"):
                record = runScheme(scheme, n, kind)
                records.append(record)
                print(json.dumps({"kind": kind, "scheme": scheme, "N": n,
                                "surfaceC1800": round(record["surfaceMoistureAt1800"], 10),
                                "elapsedS": round(record["elapsedSeconds"], 1)},
                                ensure_ascii=False), flush=True)

    constantRows = [r for r in records if r["kind"] == "constant"]

    limitChecks = []

```

```

for n in levels:
    a = next(r for r in constantRows if r["intervals"] == n and r["scheme"] == "kirchhoff")
    b = next(r for r in constantRows if r["intervals"] == n and r["scheme"] ==
"finiteDifference")
    limitChecks.append({"intervals": n,
                        "maxAbsDifference": float(np.max(np.abs(
                            np.asarray(a["moisture"]) - np.asarray(b["moisture"]))))})

report = {
    "generatedAtUtc": datetime.now(timezone.utc).isoformat(),
    "note": ("Same moisture equation, same time integrator, same tolerances; only the "
            "face-flux discretisation differs. Any gap is a spatial discretisation "
            "effect. Frozen baseline and production results are untouched."),
    "levels": list(levels),
    "frozenBenchmarkD_m2_s": FROZEN_BENCHMARK_D,
    "referenceD_m2_s": D_REFERENCE,
    "declaredToleranceKgPerKg": DECLARED_TOLERANCE_KG_PER_KG,
    "limitingCaseConstantD": {
        "note": ("Kirchhoff collapses to the midpoint finite difference when D is constant,
"
                "because  $\Phi'(C) = \exp(-a/C)$  makes the primitive difference proportional to
"
                "D dC; the two must agree to round-off."),
        "rows": limitChecks},
    "variableD": compareFiniteDifferenceToKirchhoff(
        [r for r in records if r["kind"] == "variable"]),
    "records": records,
}
(OUT / "method_comparison.json").write_text(
    json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
print(json.dumps({k: report[k] for k in ("limitingCaseConstantD", "variableD")},
    ensure_ascii=False, indent=2)[:2000], flush=True)

return 0

if __name__ == "__main__":
    sys.exit(main())

```

9.2.33 sensitivity_analysis.py

```

from __future__ import annotations

import json
import math
import os
import pathlib
import sys
import time
import uuid
from dataclasses import replace

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))

import numpy as np

import drying_core
from drying_core import RadialModel, Settings, solve_case
import q3_model

```

```

OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "sensitivity_v1"
INTERVALS = 200
SURROGATE_EVENT_H = {"Q23": 57.4727587858255, "Q4": 51.090973419826916}
PRODUCTION_EVENT_H = {"Q23": 57.47230195056044, "Q4": 51.09057478683054}

SCALES = {"d": 1.0, "k": 1.0}
_ORIGINAL_PROPERTIES = RadialModel.properties
_ORIGINAL_WATER_FLUX = RadialModel.water_internal_flux

def _scaled_properties(self, T, C):
    rho, cp, k, D = _ORIGINAL_PROPERTIES(self, T, C)
    return rho, cp, k * SCALES["k"], D * SCALES["d"]

def _scaled_water_flux(self, T, C, D):

    return _ORIGINAL_WATER_FLUX(self, T, C, D) * SCALES["d"]

RadialModel.properties = _scaled_properties
RadialModel.water_internal_flux = _scaled_water_flux

PARAMETERS = [
    ("tailTemperatureC", "degC", 50.0, 48.0, 52.0),
    ("tailEquilibrium", "kg/kg", 0.05, 0.04, 0.06),
    ("h", "W/(m2 K)", 25.0, 20.0, 30.0),
    ("beta", "m/s", 8e-7, 6.4e-7, 9.6e-7),
    ("surfaceLatentFraction", "-", 0.0, 0.0, 1.0),
    ("equilibriumScale", "-", 1.0, 0.8, 1.2),
    ("dScale", "-", 1.0, 0.7, 1.4),
    ("kScale", "-", 1.0, 0.85, 1.15),
]
NAMES = [p[0] for p in PARAMETERS]

def settings_for(question, values):
    shrink = question == "Q4"
    kwargs = {"tail_temperature_C": values["tailTemperatureC"],
              "tail_equilibrium": values["tailEquilibrium"],
              "h": values["h"], "beta": values["beta"],
              "surface_latent_fraction": values["surfaceLatentFraction"],
              "equilibrium_scale": values["equilibriumScale"]}
    return Settings(question=question, intervals=INTERVALS, shrink=shrink,
                    rtol=1e-10, atol_temperature=1e-10, atol_moisture=1e-12,
                    early_max_step_s=2.0, max_step_s=120.0,
                    face_scheme="kirchhoff", jacobian_mode="finite_difference",
                    **kwargs)

FAILURES = []

def evaluate(question, unitPoint):

    values = {}
    for name, _unit, low, high in ((p[0], p[1], p[3], p[4]) for p in PARAMETERS):
        values[name] = low + unitPoint[name] * (high - low)
    SCALES["d"] = values["dScale"]
    SCALES["k"] = values["kScale"]
    try:

```

```

        run = solve_case(settings_for(question, values))
    except Exception as error:
        FAILURES.append({"question": question, "values": values,
                        "error": f"{type(error).__name__}: {error}"})
        return float("nan")
    try:
        return run.event_s / 3600.0
    finally:
        run.close()

def baseline_point():
    return {name: (base - low) / (high - low)
            for name, _unit, base, low, high in PARAMETERS}

def full_point(subset, row):

    point = baseline_point()
    for name, value in zip(subset, row):
        point[name] = float(value)
    return point

def control_check():

    SCALES["d"] = SCALES["k"] = 1.0
    base = Settings(question="Q23", intervals=INTERVALS, rtol=1e-10,
                    atol_temperature=1e-10, atol_moisture=1e-12,
                    early_max_step_s=2.0, max_step_s=120.0,
                    face_scheme="kirchhoff", jacobian_mode="analytic")
    run = solve_case(base)
    reference = run.event_s / 3600.0
    run.close()
    patched = evaluate("Q23", baseline_point())
    return {"unpatchedAnalyticJacobianH": reference,
            "patchedFiniteDifferenceH": patched,
            "differenceSeconds": abs(reference - patched) * 3600.0,
            "referenceProductionN3200H": PRODUCTION_EVENT_H["Q23"],
            "surrogateOffsetVsProductionSeconds":
                abs(reference - PRODUCTION_EVENT_H["Q23"]) * 3600.0}

def morris(question, trajectories, levels=4):
    delta = levels / (2.0 * (levels - 1))
    grid = np.linspace(0.0, 1.0 - delta, levels)
    d = len(PARAMETERS)
    effects = {name: [] for name in NAMES}
    rng = np.random.default_rng(20260911)
    started = time.perf_counter()
    for t in range(trajectories):
        base = rng.choice(grid, size=d)
        point = {name: float(base[i]) for i, name in enumerate(NAMES)}
        y0 = evaluate(question, point)
        order = rng.permutation(d)
        for index in order:
            name = NAMES[index]
            nxt = dict(point)
            nxt[name] = min(1.0, point[name] + delta)
            y1 = evaluate(question, nxt)
            effects[name].append((y1 - y0) / delta)
            point, y0 = nxt, y1
        if (t + 1) % 5 == 0:

```

```

        print(f" morris {question} trajectory {t+1}/{trajectories} "
              f"({time.perf_counter()-started:.0f}s)", flush=True)
summary = []
for name in NAMES:
    arr = np.asarray(effects[name], dtype=float)
    summary.append({"parameter": name,
                   "muStar": float(np.mean(np.abs(arr))),
                   "mu": float(np.mean(arr)),
                   "sigma": float(np.std(arr, ddof=1)),
                   "min": float(arr.min()), "max": float(arr.max()),
                   "elementaryEffects": arr.tolist()})
summary.sort(key=lambda row: row["muStar"], reverse=True)
return {"question": question, "trajectories": trajectories, "levels": levels,
        "delta": delta, "runs": trajectories * (d + 1),
        "elapsedSeconds": time.perf_counter() - started, "ranking": summary}

def sobol(question, subset, baseSamples, seed=7):
    d = len(subset)
    rng = np.random.default_rng(seed)
    A = rng.random((baseSamples, d))
    B = rng.random((baseSamples, d))
    fA = np.array([evaluate(question, full_point(subset, row)) for row in A])
    fB = np.array([evaluate(question, full_point(subset, row)) for row in B])
    var_y = float(np.var(np.concatenate([fA, fB]), ddof=1))
    rows = []
    started = time.perf_counter()
    for j, name in enumerate(subset):
        AB = A.copy()
        AB[:, j] = B[:, j]
        fAB = np.array([evaluate(question, full_point(subset, row)) for row in AB])

        s1 = float(np.mean(fB * (fAB - fA)) / var_y) if var_y > 0 else float("nan")
        st = float(np.mean((fA - fAB) ** 2) / (2.0 * var_y)) if var_y > 0 else float("nan")
        rows.append({"parameter": name, "firstOrder": s1, "totalOrder": st,
                     "interaction": st - s1})
    print(f" sobol {question} {name}: S1={s1:.4f} ST={st:.4f} "
          f"({time.perf_counter()-started:.0f}s)", flush=True)
    return {"question": question, "subset": subset, "baseSamples": baseSamples,
            "runs": baseSamples * (d + 2), "outputVariance": var_y,
            "indices": rows, "elapsedSeconds": time.perf_counter() - started}

def scan(stageName, parameter, values, questions=("Q23", "Q4")):
    rows = []
    for question in questions:
        for value in values:
            point = baseline_point()
            low = next(p[3] for p in PARAMETERS if p[0] == parameter)
            high = next(p[4] for p in PARAMETERS if p[0] == parameter)
            point[parameter] = (value - low) / (high - low)
            y = evaluate(question, point)
            rows.append({"question": question, "parameter": parameter,
                         "value": value, "eventH": y})
    print(f" scan {question} {parameter}={value:g} -> {y:.6f} h", flush=True)
byQ = {}
for question in questions:
    ys = [r["eventH"] for r in rows if r["question"] == question]
    lo, hi = min(ys), max(ys)
    base = next((r["eventH"] for r in rows
                  if r["question"] == question and abs(r["value"] - 1.0) < 1e-12), None)

```

```

        byQ[question] = {"minH": lo, "maxH": hi, "spanH": hi - lo,
                        "baselineH": base,
                        "relativeSpanPercent": None if not base else 100.0 * (hi - lo) / base}
    return {"stage": stageName, "parameter": parameter, "rows": rows, "summary": byQ}

```

```

def main():
    stage = sys.argv[1] if len(sys.argv) > 1 else "morris"
    OUT.mkdir(parents=True, exist_ok=True)
    if stage == "control":
        result = {"control": control_check(),
                "surrogateEventH": SURROGATE_EVENT_H,
                "productionEventH": PRODUCTION_EVENT_H}
    elif stage == "morris":
        trajectories = int(sys.argv[2]) if len(sys.argv) > 2 else 20
        result = {"control": control_check(),
                "morris": [morris(q, trajectories) for q in ("Q23", "Q4")]}
    elif stage == "dscale":
        result = {"control": control_check(),
                "scan": scan("dscale", "dScale", [0.7, 0.875, 1.0, 1.05, 1.225, 1.4])}
    elif stage == "kscale":
        result = {"scan": scan("kscale", "kScale", [0.85, 0.925, 1.0, 1.075, 1.15])}
    elif stage == "sobol":
        subset = sys.argv[2].split(",")
        base = int(sys.argv[3]) if len(sys.argv) > 3 else 64
        result = {"sobol": [sobol(q, subset, base) for q in ("Q23", "Q4")]}
    else:
        raise SystemExit(f"unknown stage {stage}")
    path = OUT / f"{stage}.json"
    path.write_text(json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps(result.get("scan", {}).get("summary", {}), ensure_ascii=False),
          flush=True)
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

9.2.34 threshold_and_scaling_checks.py

```

from __future__ import annotations

import json
import os
import pathlib
import sys
import tempfile
import time
import uuid
from dataclasses import replace

ROOT = pathlib.Path(r"D:\Document\数学建模\2026CUMCM")

def _mkdtemp(suffix=None, prefix=None, dir=None):
    base = pathlib.Path(dir)
    for _ in range(200):
        candidate = base / ((prefix or "tmp") + uuid.uuid4().hex[:12] + (suffix or ""))
        if not candidate.exists():
            candidate.mkdir(parents=False)
            return str(candidate)
    raise FileExistsError("no unused temp name")

```

```

tempfile.mkdtemp = _mkdtemp
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.chdir(ROOT)
sys.dont_write_bytecode = True
sys.path.insert(0, str(ROOT / "paper_output" / "code" / "modeling"))

import io

import numpy as np

OUT = ROOT / "paper_output" / "results" / "crossvalidation" / "threshold_scaling_v1"
R0 = 0.02

def read_radius():
    text = (ROOT /
"paper_output/data_cleaned/A_radius_observed.csv").read_bytes().decode("utf-8-sig")
    return np.genfromtxt(io.StringIO(text), delimiter=",", names=True)

def scaling_check():
    rad = read_radius()
    t = rad["time_s"].astype(float)
    r = rad["radius_m"].astype(float)
    t_fixed_s = 129.84522834701946 * 3600.0
    t_shrink_s = 51.090973419826916 * 3600.0
    grid = np.linspace(0.0, t_shrink_s, 200001)
    r_of_t = np.interp(grid, t, r)
    integrand = (R0 / r_of_t) ** 2
    tau_eq = float(np.trapezoid(integrand, grid))

    return {
        "fixedRadiusTimeH": t_fixed_s / 3600.0,
        "shrinkingTimeH": t_shrink_s / 3600.0,
        "reductionPercent": float(100.0 * (1.0 - t_shrink_s / t_fixed_s)),
        "equivalentFixedRadiusTimeH": tau_eq / 3600.0,
        "ratioEquivalentToFixed": float((tau_eq / 3600.0) / (t_fixed_s / 3600.0)),
        "scalingResidualPercent": float(
            100.0 * ((tau_eq / 3600.0) / (t_fixed_s / 3600.0) - 1.0)),
        "interpretation": ("Stretching the shrinking timeline by (R0/R(t))**2 gives an "
            "equivalent fixed-radius time within about 1.4 percent of the "
            "directly simulated fixed-radius drying time. The numerically "
            "computed shortening is therefore corroborated by an independent "
            "analytical length-scale argument; the small residual comes from "
            "the coupled changes in the moisture and temperature profiles."),
    }

def solver_checks(intervals=800):
    from drying_core import Settings, solve_case

    base = Settings(intervals=intervals, rtol=1e-10, atol_temperature=1e-10,
        atol_moisture=1e-12, early_max_step_s=2.0, max_step_s=120.0,
        face_scheme="kirchhoff", jacobian_mode="analytic")
    records = {}
    for question, shrink in (("Q23", False), ("Q4", True)):
        started = time.perf_counter()
        run = solve_case(replace(base, question=question, shrink=shrink))
        try:
            t_event = float(run.event_s)

            def max_c(t):

```

```

        state = run.state([t])
        return float(np.max(state[1:-1:2, 0]))

lo, hi = t_event - 3600.0, t_event + 3600.0
for _ in range(80):
    mid = 0.5 * (lo + hi)
    if max_c(mid) - 0.15 > 0.0:
        lo = mid
    else:
        hi = mid
t_root = 0.5 * (lo + hi)

grid = np.linspace(0.0, t_event, 2001)

states = np.hstack([run.state(grid[i:i + 200]) for i in range(0, len(grid), 200)])
cs = states[1:-1:2, :]
argmax_x = run.model.x[np.argmax(cs, axis=0)]
profile_increase = float(np.max(np.diff(cs, axis=0)))

centre_margin = np.max(cs, axis=0) - cs[0, :]

strict_time = float(np.ceil(t_event) + 1.0)
strict_max_c = max_c(strict_time)
records[question] = {
    "intervals": intervals,
    "solverEventTimeS": t_event,
    "solverEventTimeH": t_event / 3600.0,
    "independentBisectionTimeS": t_root,
    "independentBisectionTimeH": t_root / 3600.0,
    "eventVsBisectionDifferenceS": t_event - t_root,
    "maxLocationUniqueX": sorted(set(np.round(argmax_x, 12).tolist())),
    "maxLocationAlwaysCentre": bool(np.all(np.abs(argmax_x) < 1e-12)),
    "maxExcessOverCentreMaxKgPerKg": float(np.max(centre_margin)),
    "maxExcessOverCentreMedianKgPerKg": float(np.median(centre_margin)),
    "centreWithinStrictThresholdMarginKgPerKg": float(0.15 -
float(np.max(cs[0, :]))),
    "centreIsMaximumAtEvent": bool(np.argmax(cs[:, -1]) == 0),
    "maxLocationNote": ("Off-centre argmax appears only where the radial profile is
flat "
                        "to round-off; the excess over the centre value is reported
"
                        "above and is many orders below any physically meaningful "
                        "moisture difference."),
    "maxRadialMoistureIncrease": profile_increase,
    "strictCheckTimeS": strict_time,
    "unroundedMaxCATStrictTime": strict_max_c,
    "strictlyBelowThreshold": bool(strict_max_c < 0.15),
}
finally:
    run.close()
    records[question]["elapsedSeconds"] = time.perf_counter() - started
    print(json.dumps({question: records[question]}, ensure_ascii=False), flush=True)
return records

def main():
    OUT.mkdir(parents=True, exist_ok=True)
    result = {"scalingCheck": scaling_check()}
    if "--with-solver" in sys.argv:

```

```

        result["thresholdChecks"] = solver_checks()
    (OUT / "threshold_and_scaling_checks.json").write_text(
        json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps(result["scalingCheck"], ensure_ascii=False, indent=2), flush=True)
    return 0

```

```

if __name__ == "__main__":
    sys.exit(main())

```

9.2.35 runCrossCheck.m

```
function runSummary = runCrossCheck(outputDirectory)
```

```

    sourcePath = [mfilename('fullpath'), '.m'];
    projectRoot = fileparts(sourcePath);
    for parentIndex = 1:4
        projectRoot = fileparts(projectRoot);
    end
    if nargin < 1 || strlength(string(outputDirectory)) == 0
        runStamp = char(datetime('now', 'Format', 'yyyyMMdd_HHmss_SSS'));
        outputDirectory = fullfile(projectRoot, 'paper_output', 'qa', ...
            ['matlab_crosscheck_', runStamp]);
    end
    outputDirectory = char(outputDirectory);

    outputFile = java.io.File(outputDirectory);
    projectRootFile = java.io.File(projectRoot);
    outputDirectory = char(outputFile.getCanonicalPath());
    canonicalRoot = char(projectRootFile.getCanonicalPath());
    if ~startsWith(lower(outputDirectory), [lower(canonicalRoot), filesep])
        error('CrossCheck:WorkspaceBoundary', '输出必须位于本次 2026CUMCM 子工作区。');
    end
    summaryPath = fullfile(outputDirectory, 'runSummary.json');
    if isfile(summaryPath)
        error('CrossCheck:ExistingRun', '该目录已有运行记录, 请使用新的输出目录以保留证据。');
    end
    if ~isfolder(outputDirectory)
        mkdir(outputDirectory);
    end
    logPath = fullfile(outputDirectory, 'runLog.txt');
    logFileId = fopen(logPath, 'w', 'n', 'UTF-8');
    if logFileId < 0
        error('CrossCheck:LogOpen', '无法创建运行日志。');
    end
    logCleanup = onCleanup(@() fclose(logFileId));
    runTimer = tic;
    runSummary = struct('schemaVersion', 1, 'status', 'RUNNING', ...
        'startedAtUtc', utcStamp(), 'finishedAtUtc', '', 'elapsedSec', [], ...
        'matlabVersion', version, 'matlabRelease', version('-release'), ...
        'computer', computer, 'sourcePath', sourcePath, ...
        'sourceSha256', sha256File(sourcePath), 'outputDirectory', outputDirectory, ...
        'guiReproduced', [], 'guiEvidenceStatus', 'external_observation_required', ...
        'humanReviewStatus', 'pending', 'caseResults', {}, 'exception', []);
    settings = struct('intervalCount', 40, 'nodeCount', 41, 'relativeTolerance', 1e-10, ...
        'temperatureAbsoluteTolerance', 1e-10, 'moistureAbsoluteTolerance', 1e-12, ...
        'earlyMaxStepSec', 2, 'lateMaxStepSec', 120, 'horizonSec', 240*3600, ...
        'initialTemperatureK', 301.15, 'initialMoisture', 2.55, ...
        'initialRadiusM', 0.02, 'fixedLengthM', 0.25, ...
        'heatTransferCoefficient', 25, 'massTransferCoefficient', 8e-7, ...
        'dryThreshold', 0.15, 'environmentSwitchSec', 14400, ...
        'tailTemperatureK', 323.15, 'tailEquilibriumMoisture', 0.05, ...

```

```

        'solver', 'ode15s_default_NDF', 'jacobianMode', 'JPattern_finite_difference', ...
        'waterFaceScheme', 'Kirchhoff', 'heatFaceScheme', 'harmonic', ...
        'coefficientFloor', 1e-12, 'latentHeatIncluded', false);
runSummary.settings = settings;
writeJson(summaryPath, runSummary);
logLine(logFileId, 'START %s | MATLAB %s | output=%s', ...
        runSummary.startedAtUtc, version, outputDirectory);
try
    environmentPath = fullfile(projectRoot, 'paper_output', 'data_cleaned',
'A_environment_observed.csv');
    radiusPath = fullfile(projectRoot, 'paper_output', 'data_cleaned',
'A_radius_observed.csv');
    inputData.environment = readtable(environmentPath, 'VariableNamingRule', 'preserve');
    inputData.radius = readtable(radiusPath, 'VariableNamingRule', 'preserve');
    validateInputs(inputData);
    runSummary.inputs = {fileRecord(environmentPath), fileRecord(radiusPath)};
    caseNames = {'Q1', 'Q23', 'Q4'};
    for caseIndex = 1:numel(caseNames)
        caseName = caseNames{caseIndex};
        logLine(logFileId, 'CASE_START %s at %s', caseName, utcStamp());
        caseTimer = tic;
        caseResult = solveSingleCase(caseName, inputData, settings, logFileId);
        caseResult.elapsedSec = toc(caseTimer);
        caseResult.finishedAtUtc = utcStamp();
        casePath = fullfile(outputDirectory, [caseName, '_crossCheck.json']);
        writeJson(casePath, caseResult);
        sampleTable = makeSampleTable(caseResult);
        writetable(sampleTable, fullfile(outputDirectory, [caseName, '_samples.csv']));
        if strcmp(caseName, 'Q23')

            sampleTable.question(:) = "Q2";
            writetable(sampleTable, fullfile(outputDirectory, 'Q2_samples.csv'));
            sampleTable.question(:) = "Q3";
            writetable(sampleTable, fullfile(outputDirectory, 'Q3_samples.csv'));
        end
        runSummary.caseResults{end+1} = caseResult;
        writeJson(summaryPath, runSummary);
        logLine(logFileId, 'CASE_END %s | elapsed=%.6f s | event=%.12g s |
massResidual=%.3g', ...
            caseName, caseResult.elapsedSec, scalarOrNaN(caseResult.eventSec), ...
            caseResult.diagnostics.maxMassBalanceAbs);
    end
    if ~strcmp(runSummary.sourceSha256, sha256File(sourcePath))
        error('CrossCheck:SourceChanged', '源码在运行中改变，本轮不能作为固定版本证据。');
    end
    for inputIndex = 1:numel(runSummary.inputs)
        record = runSummary.inputs{inputIndex};
        if ~strcmp(record.sha256, sha256File(record.path))
            error('CrossCheck:InputChanged', '输入在运行中改变: %s', record.path);
        end
    end
    runSummary.status = 'COMPLETED_NUMERICAL_CHECKS';
    runSummary.finishedAtUtc = utcStamp();
    runSummary.elapsedSec = toc(runTimer);
    writeJson(summaryPath, runSummary);
    logLine(logFileId, 'FINISH %s | elapsed=%.6f s | humanReview=pending', ...
        runSummary.finishedAtUtc, runSummary.elapsedSec);
catch runError
    runSummary.status = 'FAILED';
    runSummary.finishedAtUtc = utcStamp();
    runSummary.elapsedSec = toc(runTimer);
    runSummary.exception = struct('identifier', runError.identifier, ...
        'message', runError.message, 'report', getReport(runError, 'extended', 'hyperlinks',

```

```

'off'));
    writeJson(summaryPath, runSummary);
    logLine(logFileId, 'FAILED %s\n%s', runSummary.finishedAtUtc,
runSummary.exception.report);
    rethrow(runError);
end
end

function caseResult = solveSingleCase(caseName, inputData, settings, logFileId)
    caseStartedAtUtc = utcStamp();
    nodeCount = settings.nodeCount;
    materialX = linspace(0, 1, nodeCount)';
    deltaX = 1/settings.intervalCount;
    faceX = [0; (materialX(1:end-1)+materialX(2:end))/2; 1];
    cellWeights = diff(faceX.^2)/2;
    internalFaceX = faceX(2:end-1);
    stateCount = 2*nodeCount+1;
    initialState = zeros(stateCount, 1);
    initialState(1:2:end-1) = settings.initialTemperatureK;
    initialState(2:2:end-1) = settings.initialMoisture;
    absoluteTolerance = repmat(settings.moistureAbsoluteTolerance, stateCount, 1);
    absoluteTolerance(1:2:end-1) = settings.temperatureAbsoluteTolerance;
    jacobianPattern = makeJacobianPattern(nodeCount);
    baseOptions = odeset('RelTol', settings.relativeTolerance, 'AbsTol', absoluteTolerance, ...
    'JPattern', jacobianPattern, 'Vectorized', 'off', 'Stats', 'off');
    if strcmp(caseName, 'Q1')
        horizonSec = 1800;
        dryDensityInitial = 820/(1+settings.initialMoisture);
    elseif strcmp(caseName, 'Q4')
        horizonSec = settings.horizonSec;
        dryDensityInitial = (760+90*settings.initialMoisture)/(1+settings.initialMoisture);
    else
        horizonSec = settings.horizonSec;
        dryDensityInitial = (650+128*settings.initialMoisture)/(1+settings.initialMoisture);
    end
    dryMassKg = dryDensityInitial*pi*settings.initialRadiusM^2*settings.fixedLengthM;
    endPoints = unique([0, min(settings.environmentSwitchSec, horizonSec), horizonSec]);
    solutions = {};
    warningRecords = {};
    eventSec = [];
    rhsEvaluations = 0;
    for segmentIndex = 1:numel(endPoints)-1
        leftSec = endPoints(segmentIndex);
        rightSec = endPoints(segmentIndex+1);
        if leftSec < settings.environmentSwitchSec
            maxStepSec = settings.earlyMaxStepSec;
        else
            maxStepSec = settings.lateMaxStepSec;
        end
        segmentOptions = odeset(baseOptions, 'MaxStep', maxStepSec);
        if ~strcmp(caseName, 'Q1')
            segmentOptions = odeset(segmentOptions, 'Events', @dryEvent);
        end
        lastwarn('');
        segment = ode15s(@balanceRhs, [leftSec, rightSec], initialState, segmentOptions);
        [warningText, warningId] = lastwarn;
        recordWarning(warningText, warningId, leftSec, rightSec);
        solutions{end+1} = segment;
        hasEvent = isfield(segment, 'xe') && ~isempty(segment.xe);
        if hasEvent
            eventSec = segment.xe(end);
            initialState = segment.ye(:, end);
            tailEndSec = ceil(eventSec)+1;

```

```

        tailOptions = odeset(baseOptions, 'MaxStep', 1, 'Events', []);
        lastwarn('');
        tail = ode15s(@balanceRhs, [eventSec, tailEndSec], initialState, tailOptions);
        [warningText, warningId] = lastwarn;
        recordWarning(warningText, warningId, eventSec, tailEndSec);
        assertReached(tail, tailEndSec);
        solutions{end+1} = tail;
        break;
    end
    assertReached(segment, rightSec);
    initialState = segment.y(:, end);
end
if ~strcmp(caseName, 'Q1') && isempty(eventSec)
    error('CrossCheck:NoDryEvent', '%s 在 240 小时内没有找到干燥事件。', caseName);
end
endSec = solutions{end}.x(end);
rawSampleTimes = [0, 1800, 10800];
rawSampleTimes = rawSampleTimes(rawSampleTimes <= endSec);
sampleTimesSec = unique([rawSampleTimes, eventSec, endSec]);
sampleStates = evaluatePieces(solutions, sampleTimesSec);
sampleX = [0, 0.25, 0.5, 0.75, 1];
temperatureK = interp1(materialX, sampleStates(1:2:end-1, :), sampleX, 'linear');
moisture = interp1(materialX, sampleStates(2:2:end-1, :), sampleX, 'linear');
sampleRadiusM = zeros(size(sampleTimesSec));
sampleTypes = strings(size(sampleTimesSec));
for sampleIndex = 1:numel(sampleTimesSec)
    sampleRadiusM(sampleIndex) = radiusAt(sampleTimesSec(sampleIndex));
    if sampleTimesSec(sampleIndex) == 0
        sampleTypes(sampleIndex) = "initial";
    elseif ~isempty(eventSec) && sampleTimesSec(sampleIndex) == eventSec
        sampleTypes(sampleIndex) = "criticalEvent";
    elseif ~isempty(eventSec) && sampleTimesSec(sampleIndex) == endSec
        sampleTypes(sampleIndex) = "postEventIntegerSecond";
    else
        sampleTypes(sampleIndex) = "fixedTime";
    end
end
end
diagnostic = struct('minMoisture', Inf, 'maxMoisture', -Inf, ...
    'minTemperatureK', Inf, 'maxTemperatureK', -Inf, 'minDiffusivity', Inf, ...
    'positiveProperties', true, 'maxMassBalanceAbs', 0, ...
    'maxRadialMoistureIncrease', -Inf, 'acceptedTimePoints', 0);
for pieceIndex = 1:numel(solutions)
    acceptedStates = solutions{pieceIndex}.y;
    acceptedTemperature = acceptedStates(1:2:end-1, :);
    acceptedMoisture = acceptedStates(2:2:end-1, :);
    [density, heatCapacity, conductivity, diffusivity] =
materialProperties(acceptedTemperature, acceptedMoisture, caseName);
    diagnostic.minMoisture = min(diagnostic.minMoisture, min(acceptedMoisture, [], 'all'));
    diagnostic.maxMoisture = max(diagnostic.maxMoisture, max(acceptedMoisture, [], 'all'));
    diagnostic.minTemperatureK = min(diagnostic.minTemperatureK, min(acceptedTemperature,
[], 'all'));
    diagnostic.maxTemperatureK = max(diagnostic.maxTemperatureK, max(acceptedTemperature,
[], 'all'));
    diagnostic.minDiffusivity = min(diagnostic.minDiffusivity, min(diffusivity, [],
'all'));
    diagnostic.positiveProperties = diagnostic.positiveProperties && ...
        all(density > 0 & heatCapacity > 0 & conductivity > 0 & diffusivity > 0, 'all');
    massResidual =
2*cellWeights.*acceptedMoisture+acceptedStates(end, :)-settings.initialMoisture;
    diagnostic.maxMassBalanceAbs = max(diagnostic.maxMassBalanceAbs,
max(abs(massResidual)));
    diagnostic.maxRadialMoistureIncrease = max(diagnostic.maxRadialMoistureIncrease, ...

```

```

        max(diff(acceptedMoisture, 1, 1), [], 'all'));
        diagnostic.acceptedTimePoints =
diagnostic.acceptedTimePoints+numel(solutions{pieceIndex}.x);
    end
    finalState = evaluatePieces(solutions, endSec);
    diagnostic.finalMaxMoisture = max(finalState(2:2:end-1));
    diagnostic.strictlyDryAtEnd = diagnostic.finalMaxMoisture < settings.dryThreshold;
    diagnostic.rhsEvaluations = rhsEvaluations;
    diagnostic.radiusExtrapolationUsed = strcmp(caseName, 'Q4') && endSec >
inputData.radius.time_s(end);
    diagnostic.checkScope = 'accepted ode15s states; not a continuous exact-solution error
bound';
    if diagnostic.minMoisture < -1e-8 || ~diagnostic.positiveProperties
        error('CrossCheck:PhysicalRange', '%s 的接受状态违反物性正值或水分范围.', caseName);
    end
    if diagnostic.maxMassBalanceAbs > 1e-6
        error('CrossCheck:MassBalance', '%s 的离散干基质量守恒残差超限.', caseName);
    end
    if ~strcmp(caseName, 'Q1') && ~diagnostic.strictlyDryAtEnd
        error('CrossCheck:StrictDryness', '%s 的事件后原精度 maxC 没有严格小于 0.15.', caseName);
    end
    caseResult = struct('question', caseName, 'startedAtUtc', caseStartedAtUtc, ...
        'intervalCount', settings.intervalCount, 'eventSec', eventSec, ...
        'eventHours', eventSec/3600, 'endSec', endSec, 'dryMassKg', dryMassKg, ...
        'endTimeConvention', 'ceil(eventSec)+1; actual integration, not earliest integer-second
claim', ...
        'sampleTimesSec', sampleTimesSec, 'sampleTypes', sampleTypes, 'sampleMaterialX',
sampleX, ...
        'sampleRadiusM', sampleRadiusM, 'temperatureK', temperatureK, ...
        'temperatureC', temperatureK-273.15, 'moistureDryBasis', moisture, ...
        'fullMaterialX', materialX, 'fullStateBySample', sampleStates, ...
        'stateOrder', 'T0,C0,T1,C1,...,TN,CN,cumulativeDryBasisLoss', ...
        'meanMoisture', 2*cellWeights*sampleStates(2:2:end-1, :), ...
        'cumulativeLoss', sampleStates(end, :), 'diagnostics', diagnostic, ...
        'warnings', {warningRecords}, 'warningCapture', 'last warning per segment; not all
warning messages', ...
        'humanReviewStatus', 'pending');

    function derivative = balanceRhs(timeSec, state)
        rhsEvaluations = rhsEvaluations+1;
        temperature = state(1:2:end-1);
        waterContent = state(2:2:end-1);
        [density, heatCapacity, conductivity, ~] = materialProperties(temperature, waterContent,
caseName);
        radiusM = radiusAt(timeSec);
        [airTemperatureK, equilibriumMoisture] = environmentAt(timeSec);
        heatFlux = zeros(nodeCount+1, 1);
        waterFlux = zeros(nodeCount+1, 1);
        faceConductivity = 2*conductivity(1:end-1).*conductivity(2:end) ./ ...
            max(conductivity(1:end-1)+conductivity(2:end), realmin);
        heatFlux(2:end-1) = internalFaceX.*faceConductivity.*diff(temperature)/deltaX;
        if strcmp(caseName, 'Q1')
            moistureExponent = 0.89;
            diffusionPrefactor = 7e-9;
            thermalFactor = ones(nodeCount-1, 1);
        elseif strcmp(caseName, 'Q4')
            moistureExponent = 0.30;
            diffusionPrefactor = 4.2e-4;
            thermalFactor = exp(-3850./((temperature(1:end-1)+temperature(2:end))/2));
        else
            moistureExponent = 0.45;
            diffusionPrefactor = 2.4e-3;
            thermalFactor = exp(-3850./((temperature(1:end-1)+temperature(2:end))/2));

```

```

end
positiveMoisture = max(waterContent, settings.coefficientFloor);

kirchhoffPotential = positiveMoisture.*exp(-moistureExponent./positiveMoisture) ...
    -moistureExponent*expint(moistureExponent./positiveMoisture);
potentialDifference = diff(kirchhoffPotential);
meanFaceMoisture = (positiveMoisture(1:end-1)+positiveMoisture(2:end))/2;
moistureDifference = diff(positiveMoisture);
closePair = abs(moistureDifference) < 1e-7*max(meanFaceMoisture, 1e-3);
potentialDifference(closePair) =
exp(-moistureExponent./meanFaceMoisture(closePair)) ...
    .*moistureDifference(closePair);

waterFlux(2:end-1) =
internalFaceX.*diffusionPrefactor.*thermalFactor.*potentialDifference/deltaX;
waterFlux(end) =
-settings.massTransferCoefficient*radiusM*(waterContent(end)-equilibriumMoisture);
heatFlux(end) =
-settings.heatTransferCoefficient*radiusM*(temperature(end)-airTemperatureK);
derivative = zeros(stateCount, 1);
derivative(1:2:end-1) =
diff(heatFlux)./(radiusM^2*cellWeights.*density.*heatCapacity);

derivative(2:2:end-1) = diff(waterFlux)./(radiusM^2*cellWeights);
derivative(end) =
2*settings.massTransferCoefficient/radiusM*(waterContent(end)-equilibriumMoisture);
end

function radiusM = radiusAt(timeSec)
    if strcmp(caseName, 'Q4')
        boundedTimeSec = min(max(timeSec, inputData.radius.time_s(1)),
inputData.radius.time_s(end));
        radiusM = interp1(inputData.radius.time_s, inputData.radius.radius_m,
boundedTimeSec, 'linear');
    else
        radiusM = settings.initialRadiusM;
    end
end

function [airTemperatureK, equilibriumMoisture] = environmentAt(timeSec)

    if timeSec > inputData.environment.time_s(end)
        airTemperatureK = settings.tailTemperatureK;
        equilibriumMoisture = settings.tailEquilibriumMoisture;
    else
        boundedTimeSec = max(timeSec, inputData.environment.time_s(1));
        airTemperatureK = interp1(inputData.environment.time_s,
inputData.environment.temperature_K, boundedTimeSec, 'linear');
        equilibriumMoisture = interp1(inputData.environment.time_s,
inputData.environment.air_moisture_kg_per_kg, boundedTimeSec, 'linear');
    end
end

function [eventValue, isTerminal, direction] = dryEvent(~, state)
    eventValue = max(state(2:2:end-1))-settings.dryThreshold;
    isTerminal = 1;
    direction = -1;
end

function recordWarning(warningText, warningId, leftSec, rightSec)
    if ~isempty(warningText)
        warningRecords{end+1} = struct('identifier', warningId, 'message', warningText, ...
            'segmentStartSec', leftSec, 'segmentTargetSec', rightSec);
    end
end

```

```

        logLine(logFileId, 'WARNING %s [%s] %s', caseName, warningId, warningText);
    end
end
end

function [density, heatCapacity, conductivity, diffusivity] = materialProperties(temperatureK,
waterContent, caseName)
    if any(~isfinite(temperatureK) | temperatureK <= 0, 'all') || any(~isfinite(waterContent),
'all')
        error('CrossCheck:InvalidState', '遇到非有限状态或非正绝对温度。');
    end

    positiveMoisture = max(waterContent, 1e-12);
    wetFraction = positiveMoisture./(1+positiveMoisture);
    switch caseName
        case 'Q1'
            density = 820*ones(size(waterContent));
            heatCapacity = 2600*ones(size(waterContent));
            conductivity = 0.36*ones(size(waterContent));
            diffusivity = 7e-9*exp(-0.89./positiveMoisture);
        case 'Q23'
            density = 650+128*positiveMoisture;
            heatCapacity = 1450+2736*wetFraction;
            conductivity = 0.21+0.38*wetFraction;
            diffusivity = 2.4e-3*exp(-0.45./positiveMoisture-3850./temperatureK);
        case 'Q4'
            density = 760+90*positiveMoisture;
            heatCapacity = 1850+2150*wetFraction;
            conductivity = 0.12+0.20*wetFraction;
            diffusivity = 4.2e-4*exp(-0.30./positiveMoisture-3850./temperatureK);
        otherwise
            error('CrossCheck:Question', '不支持的问题名: %s', caseName);
        end
    end
end

function jacobianPattern = makeJacobianPattern(nodeCount)
    stateCount = 2*nodeCount+1;
    jacobianPattern = sparse(stateCount, stateCount);
    for nodeIndex = 1:nodeCount
        rowIndices = 2*nodeIndex-1:2*nodeIndex;
        for neighborIndex = max(1, nodeIndex-1):min(nodeCount, nodeIndex+1)
            columnIndices = 2*neighborIndex-1:2*neighborIndex;
            jacobianPattern(rowIndices, columnIndices) = 1;
        end
    end
    jacobianPattern(end, 2*nodeCount) = 1;
end

function sampleStates = evaluatePieces(solutions, queryTimesSec)
    sampleStates = zeros(size(solutions{1}.y, 1), numel(queryTimesSec));
    for queryIndex = 1:numel(queryTimesSec)
        queryTimeSec = queryTimesSec(queryIndex);
        foundPiece = false;
        for pieceIndex = 1:numel(solutions)
            piece = solutions{pieceIndex};
            if queryTimeSec >= piece.x(1) && queryTimeSec <= piece.x(end)
                sampleStates(:, queryIndex) = deval(piece, queryTimeSec);
                foundPiece = true;
                break;
            end
        end
        if ~foundPiece
            error('CrossCheck:DenseDomain', '请求的时刻 %.17g 不在已积分分段内。', queryTimeSec);
        end
    end
end

```

```

        end
    end
end

function sampleTable = makeSampleTable(caseResult)
    sampleCount = numel(caseResult.sampleTimesSec);
    radialCount = numel(caseResult.sampleMaterialX);
    question = repmat(string(caseResult.question), sampleCount*radialCount, 1);
    sampleType = repelem(caseResult.sampleTypes(:), radialCount);
    timeSec = repelem(caseResult.sampleTimesSec(:), radialCount);
    materialX = repmat(caseResult.sampleMaterialX(:), sampleCount, 1);
    currentRadiusM = repelem(caseResult.sampleRadiusM(:), radialCount);
    radiusM = currentRadiusM.*materialX;
    temperatureK = reshape(caseResult.temperatureK', [], 1);
    temperatureC = temperatureK-273.15;
    moistureDryBasis = reshape(caseResult.moistureDryBasis', [], 1);
    sampleTable = table(question, sampleType, timeSec, materialX, radiusM, currentRadiusM, ...
        temperatureK, temperatureC, moistureDryBasis);
end

function validateInputs(inputData)
    environmentColumns = {'time_s', 'temperature_K', 'air_moisture_kg_per_kg'};
    radiusColumns = {'time_s', 'radius_m'};
    if ~all(ismember(environmentColumns, inputData.environment.Properties.VariableNames))
    || ...
        ~all(ismember(radiusColumns, inputData.radius.Properties.VariableNames))
        error('CrossCheck:InputColumns', '清洗 CSV 字段不符合正式输入接口。');
    end
    environmentValues = inputData.environment{:, environmentColumns};
    radiusValues = inputData.radius{:, radiusColumns};
    if any(~isfinite(environmentValues), 'all') || any(~isfinite(radiusValues), 'all') || ...
        any(diff(environmentValues(:, 1)) <= 0) || any(diff(radiusValues(:, 1)) <= 0) || ...
        environmentValues(1, 1) ~= 0 || radiusValues(1, 1) ~= 0 || ...
        environmentValues(end, 1) ~= 14400 || radiusValues(end, 1) ~= 259200 || ...
        any(environmentValues(:, 2) <= 0) || any(environmentValues(:, 3) < 0) || ...
        any(radiusValues(:, 2) <= 0) || abs(radiusValues(1, 2)-0.02) > 1e-14
        error('CrossCheck:InputAudit', '时间、数值范围或初始半径审计未通过。');
    end
end

function assertReached(solution, requestedEndSec)
    if abs(solution.x(end)-requestedEndSec) > 1e-7
        error('CrossCheck:SolverStopped', 'ode15s 实际终点 %.17g 未到请求终点 %.17g。',
            solution.x(end), requestedEndSec);
    end
    if any(~isfinite(solution.y), 'all')
        error('CrossCheck:NonfiniteSolution', 'ode15s 返回非有限接受状态。');
    end
end

function record = fileRecord(filePath)
    fileInfo = dir(filePath);
    record = struct('path', filePath, 'bytes', fileInfo.bytes, 'sha256', sha256File(filePath));
end

function hashText = sha256File(filePath)
    fileId = fopen(filePath, 'r');
    if fileId < 0
        error('CrossCheck:HashRead', '无法读取待哈希文件: %s', filePath);
    end
    fileCleanup = onCleanup(@() fclose(fileId));
    fileBytes = fread(fileId, Inf, '*uint8');
    digest = java.security.MessageDigest.getInstance('SHA-256');

```

```

        digest.update(typecast(fileBytes, 'int8'));
        digestBytes = typecast(digest.digest(), 'uint8');
        hashText = lower(reshape(dec2hex(digestBytes, 2)', 1, []));
    end

function writeJson(filePath, value)
    fileId = fopen(filePath, 'w', 'n', 'UTF-8');
    if fileId < 0
        error('CrossCheck:JsonWrite', '无法写入 JSON: %s', filePath);
    end
    fileCleanup = onCleanup(@() fclose(fileId));
    fprintf(fileId, '%s\n', jsonencode(value, 'PrettyPrint', true));
end

function logLine(fileId, messageFormat, varargin)
    logText = sprintf(messageFormat, varargin{:});
    fprintf('%s\n', logText);
    fprintf(fileId, '%s\n', logText);
end

function timeText = utcStamp()
    timeText = char(datetime('now', 'TimeZone', 'UTC', 'Format',
    'yyyy-MM-dd''T''HH:mm:ss.SSS''Z'''));
end

function value = scalarOrNaN(optionalValue)
    if isempty(optionalValue)
        value = NaN;
    else
        value = optionalValue;
    end
end
end

```

9.2.36 compareMatlab.mjs 主要部分

```

import fs from 'node:fs';
import crypto from 'node:crypto';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import { spawnSync } from 'node:child_process';

const scriptPath = fileURLToPath(import.meta.url);
const projectRoot = path.resolve(path.dirname(scriptPath), '../..');
const matlabRoot = path.join(projectRoot, 'paper_output/qa/matlab_crosscheck_20260911');
const pythonRoot = path.join(projectRoot,
    'paper_output/results/code_delivery/camel_audit_v1');
const outputJson = path.join(matlabRoot, 'comparison-python.json');
const outputMarkdown = path.join(matlabRoot, 'comparison-python.md');

const thresholds = Object.freeze({ temperatureK: 1e-4, moistureDryBasis: 1e-6, eventSec:
    0.01 });
const scriptRecord = fileRecord(scriptPath);

function now() { return new Date().toISOString(); }
function digest(bytes) { return crypto.createHash('sha256').update(bytes).digest('hex'); }
function fileRecord(filePath) {
    const bytes = fs.readFileSync(filePath);
    return { path: path.relative(projectRoot, filePath).split(path.sep).join('/'),
        bytes: bytes.length, sha256: digest(bytes) };
}
function readJson(filePath) { return JSON.parse(fs.readFileSync(filePath,
    'utf8').replace(/\uFEFF/, '')); }
function writeJson(filePath, value) { fs.writeFileSync(filePath, JSON.stringify(value, null,

```

```

2) + '\n', 'utf8')); }
function assert(condition, message) { if (!condition) throw new Error(message); }
function finite(value, label) { assert(typeof value === 'number' && Number.isFinite(value),
`Nonfinite ${label}`); return value; }

function compareWorker() {
  const startedAtUtc = now();
  const startTime = process.hrtime.bigint();
  const inputPaths = [path.join(matlabRoot, 'runSummary.json'), path.join(matlabRoot,
'runLog.txt'),
    path.join(pythonRoot, 'launch.json'), path.join(pythonRoot, 'runResult.json'),
    path.join(pythonRoot, 'artifactManifest.json')];
  for (const question of ['Q1', 'Q23', 'Q4']) {
    inputPaths.push(path.join(matlabRoot, `${question}_crossCheck.json`),
      path.join(pythonRoot, question, 'summary.json'), path.join(pythonRoot, question,
'sampled_solution.npz'));
  }
  const matlabRun = readJson(inputPaths[0]);
  const pythonLaunch = readJson(path.join(pythonRoot, 'launch.json'));
  const pythonRun = readJson(path.join(pythonRoot, 'runResult.json'));
  const noMatlabException = matlabRun.exception == null || (Array.isArray(matlabRun.exception)
&& matlabRun.exception.length === 0);
  assert(matlabRun.status === 'COMPLETED_NUMERICAL_CHECKS' && noMatlabException, 'MATLAB run
is incomplete');
  assert(pythonRun.status === 'PASS', 'Python baseline is incomplete');
  const provenanceChecks = [];

  function verifyDeclared(record, label) {
    const resolved = path.isAbsolute(record.path) ? record.path : path.join(projectRoot,
record.path);
    const actual = fileRecord(resolved);
    assert(actual.sha256 === record.sha256, `Declared hash differs: ${label}`);
    inputPaths.push(resolved);
    provenanceChecks.push({ label, actual, matchedRecordedHash: true });
  }
  verifyDeclared({ path: matlabRun.sourcePath, sha256: matlabRun.sourceSha256 }, 'MATLAB
source');
  for (const record of matlabRun.inputs) verifyDeclared(record, 'MATLAB observed input');
  for (const question of ['Q1', 'Q23', 'Q4']) {
    const baseline = readJson(path.join(pythonRoot, question, 'summary.json'));
    for (const key of ['code', 'jacobian_code', 'dense_storage_code'])
      verifyDeclared(baseline[key], `${question}/${key}`);
    for (const record of baseline.inputs) verifyDeclared(record, `${question}/observed input`);
  }
  const uniqueInputPaths = [...new Set(inputPaths.map(item => path.resolve(item)))]
  const inputRecords = uniqueInputPaths.map(fileRecord);
  const questions = [];
  const failures = [];
  for (const question of ['Q1', 'Q23', 'Q4']) {
    const matlab = readJson(path.join(matlabRoot, `${question}_crossCheck.json`));
    const python = readJson(path.join(pythonRoot, question, 'summary.json'));
    const sample = python.crossLanguageSamples;
    assert(matlab.question === question && python.settings.question === question, `${question}:
question mismatch`);
    assert(matlab.intervalCount === 40 && python.settings.intervals === 40, `${question}:
require N40`);
    const settingsPairs = [
      ['rtol', 'relativeTolerance'], ['atolTemperature', 'temperatureAbsoluteTolerance'],
      ['atolMoisture', 'moistureAbsoluteTolerance'], ['earlyMaxStepS', 'earlyMaxStepSec'],
      ['maxStepS', 'lateMaxStepSec'], ['h', 'heatTransferCoefficient'], ['beta',
'massTransferCoefficient'];
    for (const [pythonKey, matlabKey] of settingsPairs) {
      assert(python.settings[pythonKey] === matlabRun.settings[matlabKey], `${question}:

```

```

    ${pythonKey} differs`);
  }
  assert(python.settings.faceScheme.toLowerCase() ===
matlabRun.settings.waterFaceScheme.toLowerCase(), `${question}: face scheme differs`);
  assert(matlab.sampleTimesSec.length === matlab.sampleTypes.length, `${question}: sample
types malformed`);
  const times = question === 'Q1' ? [0, 1800] : [0, 1800, 10800];
  if (question !== 'Q1') {
    assert(Number.isInteger(matlab.endSec) && matlab.endSec === python.diagnostics.end_s,
`${question}: no common post-event integer second`);
    const postIndex = matlab.sampleTimesSec.indexOf(matlab.endSec);
    assert(postIndex >= 0 && matlab.sampleTypes[postIndex] === 'postEventIntegerSecond',
`${question}: endpoint is not postEventIntegerSecond`);
    times.push(matlab.endSec);
  }
  const coordinates = [0, 0.25, 0.5, 0.75, 1];
  const pairs = [];
  const maxima = { temperatureK: null, moistureDryBasis: null };
  for (const timeSec of times) {
    const matlabTimeIndex = matlab.sampleTimesSec.indexOf(timeSec);
    const pythonTimeIndex = sample.timesS.indexOf(timeSec);
    assert(matlabTimeIndex >= 0 && pythonTimeIndex >= 0, `${question}: missing exact common
time ${timeSec}`);
    for (const materialX of coordinates) {
      const matlabXIndex = matlab.sampleMaterialX.indexOf(materialX);
      const pythonXIndex = sample.materialX.indexOf(materialX);
      assert(matlabXIndex >= 0 && pythonXIndex >= 0, `${question}: missing exact common
coordinate ${materialX}`);
      const item = { timeSec, materialX, matlabTimeIndexZeroBased: matlabTimeIndex,
pythonTimeIndexZeroBased: pythonTimeIndex,
matlabXIndexZeroBased: matlabXIndex, pythonXIndexZeroBased: pythonXIndex };
      for (const field of ['temperatureK', 'moistureDryBasis']) {
        const matlabValue = finite(matlab[field][matlabTimeIndex][matlabXIndex],
`${question}/${field}/MATLAB`);
        const pythonValue = finite(sample[field][pythonTimeIndex][pythonXIndex],
`${question}/${field}/Python`);
        const signedDifference = matlabValue - pythonValue;
        const absoluteDifference = Math.abs(signedDifference);
        const passed = absoluteDifference <= thresholds[field];
        item[field] = { matlabValue, pythonValue, signedDifferenceMatlabMinusPython:
signedDifference, absoluteDifference, passed };
        if (maxima[field] === null || absoluteDifference > maxima[field].absoluteDifference)
{
          maxima[field] = { timeSec, materialX, ...item[field] };
        }
        if (!passed) failures.push(`${question}/${field}/t=${timeSec}/x=${materialX}`);
      }
      pairs.push(item);
    }
  }
  let event = { applicable: false, reason: 'Q1 stops at 1800 s and has no drying event' };
  if (question !== 'Q1') {
    const matlabEventSec = finite(matlab.eventSec, `${question}/MATLAB event`);
    const pythonEventSec = finite(python.diagnostics.event_s, `${question}/Python event`);
    const signedDifferenceSec = matlabEventSec - pythonEventSec;
    const absoluteDifferenceSec = Math.abs(signedDifferenceSec);
    const passed = absoluteDifferenceSec <= thresholds.eventSec;
    event = { applicable: true, matlabEventSec, pythonEventSec, signedDifferenceSec,
absoluteDifferenceSec, passed };
    if (!passed) failures.push(`${question}/event`);
  }
  questions.push({ question, status: failures.some(item => item.startsWith(question + '/')) ?
'FAIL' : 'PASS',

```

```

    intervals: 40, commonTimesSec: times, commonMaterialX: coordinates,
matchedCoordinatePairs: pairs.length,
    maxima, event, comparisons: pairs,
    excludedPythonTimesSec: sample.timesS.filter(value => !times.includes(value)),
    excludedMatlabTimesSec: matlab.sampleTimesSec.filter(value => !times.includes(value)),
    exclusionReason: 'Python strict reported time and MATLAB critical event are different times;
their field rows are not paired.',
    matlabWarnings: matlab.warnings, matlabWarningCapture: matlab.warningCapture,
pythonWarningCount: python.warningCount });
}
assert(JSON.stringify(inputRecords) === JSON.stringify(uniqueInputPaths.map(fileRecord)),
'Inputs changed during comparison');
assert(scriptRecord.sha256 === fileRecord(scriptPath).sha256, 'Comparison script changed
while running');
return { schemaVersion: 1, status: failures.length ? 'FAIL' : 'PASS', failures,
    startedAtUtc, finishedAtUtc: now(), elapsedSeconds: Number(process.hrtime.bigint() -
startTime) / 1e9,
    thresholds, thresholdPolicy: 'Fixed before reading results; not changed after observing
differences',
    matchingPolicy: 'Exact common time values and exact common material coordinates; no row-index
pairing or interpolation',
    differenceConvention: 'MATLAB minus Python', script: scriptRecord, inputRecords,
provenanceChecks,
    runtime: { nodeVersion: process.version, executable: process.execPath, platform:
process.platform, architecture: process.arch,
    matlabVersion: matlabRun.matlabVersion, matlabSolver: matlabRun.settings.solver,
    matlabJacobian: matlabRun.settings.jacobianMode, pythonVersion: pythonLaunch.python,
    pythonLibraries: pythonLaunch.libraries, pythonSolver: 'SciPy BDF', pythonJacobian:
'analytic sparse' },
    sourceRunTimes: { matlabStartedAtUtc: matlabRun.startedAtUtc, matlabFinishedAtUtc:
matlabRun.finishedAtUtc,
    pythonStartedAtUtc: pythonRun.startedAtUtc, pythonFinishedAtUtc:
pythonRun.finishedAtUtc },
    comparedFieldScalars: questions.reduce((total, item) => total + 2 *
item.matchedCoordinatePairs, 0),
    comparedEvents: 2, questions,
    scope: 'Independent N40 implementation agreement at recorded common times and five material
coordinates, plus continuous event times',
    limitations: ['NPZ files are hashed for provenance; numerical comparisons read
crossLanguageSamples in Python summary.json.',
    'No full-time/full-space error bound, physical calibration accuracy, production-grid
acceptance, or confidence interval is established.',
    'MATLAB GUI execution evidence is recorded separately by the main agent; this comparison
does not infer GUI completion.',
    'Human code review remains pending.'], guiEvidence: 'separate external record', humanReview:
'pending' });
}

if (process.argv.includes('--worker')) {
    try {
        const report = compareWorker();
        process.stdout.write(JSON.stringify(report));
        process.exitCode = report.status === 'PASS' ? 0 : 1;
    } catch (error) {
        process.stderr.write(String(error.stack ?? error) + '\n');
        process.exitCode = 2;
    }
} else {
    const supervisorStartedAtUtc = now();
    const worker = spawnSync(process.execPath, [scriptPath, '--worker'], { cwd: projectRoot,
encoding: 'utf8', maxBuffer: 8 * 1024 * 1024 });
    const supervision = { supervisorStartedAtUtc, supervisorFinishedAtUtc: now(),

```

```

    executable: process.execPath, commandArguments: [scriptPath, '--worker'], cwd: projectRoot,
    workerActualExitCode: worker.status, workerSignal: worker.signal, workerSpawnError:
worker.error ? String(worker.error) : null,
    workerStderr: worker.stderr, source: 'Node parent process spawnSync return status; not a child
prefilled success flag' });
    fs.appendFileSync(path.join(matlabRoot, 'comparison-python-process.jsonl'),
JSON.stringify({
    script: scriptRecord, ...supervision, stdoutSha256: digest(Buffer.from(worker.stdout ?? '',
'utf8')) }) + '\n', 'utf8');
    const report = worker.stdout?.length ? JSON.parse(worker.stdout) : {
    schemaVersion: 1, status: 'FAIL', failures: ['Worker did not produce a numerical report'],
    script: scriptRecord, thresholds, startedAtUtc: supervisorStartedAtUtc, finishedAtUtc:
now(),
    comparedFieldScalars: 0, comparedEvents: 0, questions: [], humanReview: 'pending' };
    report.processSupervision = supervision;
    if (worker.status !== 0) report.status = 'FAIL';
    writeJson(outputJson, report);
    fs.writeFileSync(outputMarkdown, markdown(report), 'utf8');
    writeJson(path.join(matlabRoot, 'comparison-python-manifest.json'), {
    status: report.status, createdAtUtc: now(), script: scriptRecord, processSupervision:
supervision,
    outputs: [fileRecord(outputJson), fileRecord(outputMarkdown)] });
    process.stdout.write(JSON.stringify({ status: report.status, workerActualExitCode:
worker.status,
    comparedFieldScalars: report.comparedFieldScalars, comparedEvents: report.comparedEvents,
    maxima: report.questions.map(item => ({ question: item.question,
    temperatureK: item.maxima.temperatureK.absoluteDifference, moistureDryBasis:
item.maxima.moistureDryBasis.absoluteDifference,
    eventSec: item.event.absoluteDifferenceSec ?? null }))) }) + '\n');
    process.exitCode = worker.status ?? 2;
}

```

9.3 AI 使用报告

本次建模过程中，使用 Deepseek Harness 与 Deepseek v4.1 flash 组合辅助检查源代码语法并进行部分程序调试；使用 Codex 与 GPT6-Astra 组合辅助理解题目、检索参考文献、检查建模结果和整理公式。具体使用情况如下。

9.3.1 Deepseek Harness 与 Deepseek v4.1 flash

该组合用于检查已编写源代码的语法问题，以及辅助部分程序的调试。语法检查围绕程序的语言规则展开，包括括号与引号的配对、缩进、语句结构和函数调用形式等。对程序运行中出现的报错，结合相关代码片段分析可能原因，提供定位问题和调整语句的思路，以便修改后继续检查。

在部分程序调试中，借助该组合解释报错信息，并辅助梳理变量、参数与函数之间的传递关系。其作用是帮助发现源代码中的具体问题、理解异常发生的位置，为后续修改提供参考。

9.3.2 Codex 与 GPT6-Astra

该组合用于解释题目，辅助梳理各问的研究目标、已知条件、输入数据和输出要求，理解温度、干基含水率、物性参数及半径变化等量在题目中的含义和联系。参考文献搜索主要围绕药材干燥、热湿耦合传递及相关数值方法展开，用于寻找可查阅的文献线索和理论资料。

在建模结果检查方面，借助该组合分析数值结果与题设条件、模型方程及边界条件之间的对应关系，检查温度、含水率和干燥时间等结果的单位、变化趋势与表达是否一致。公式整理主要包括梳理推导步骤、统一符号写法、检查公式之间的衔接，以及整理公式与变量说明的对应关系，使数学表达更清楚、连贯。